Table of Contents

Research problem 4

Broad outline of the Solution 6

Project Structure: Three Main Stages 7

Challenges 10

Scope 11

Computational Methods 12

Stakeholders 22

Desk research 28

- CoinMarketCap (CMC) 28
- Cryptohopper 35
- TradingView 41

Interviews 47

- Interview 1: Trader 47
- Interview 2: Tester 50
- Interview 3: Machine Learning Developer 53

Users survey 58

Requirements 75

Success criteria 79

Limitations 84

Top-down design 94

Prototypes 95
Prototype 1 – Interface and Data Input Foundation 95
Prototype 2 – Prediction Engine and Decision Logic 96
Prototype 3 – Automation, Evaluation and Refinement 97
Structure 99
New Process 1: Updating Bitcoin Price 99
New Process 2: Main Window 100
New Process 3: Moving Average 102
Data Design 103
Algorithms 105
Algorithm 1: Building the Main Window 106
Algorithm 2: Update Live Price 108
Algorithm 3: Run Prediction for the asset 110
Algorithm 4: Fetch data from Binance and form a Linear Regression Model 112
Algorithm 5: Moving Averages Model 114
Algorithm 6: Generating Signal 116
Usability Features 118
Testing Plan to inform development 125
Test data to be used during iterative development 125
Prototype 1 126
Prototype 2 128
Prototype 3 130

Planned Post-Development Testing 132
Iterative Development & Testing 136
Initial Tkinter Window Setup and Iterative Fix 136
Button Creation, Layout Issues, and Usability Refinement 141
Image Handling 149
Live Price Integration and Real-Time Updates 157
Object-Oriented Restructuring and Scalability (OOP Transition) 172
Main Menu and Multi-Window Navigation 181
Data Collection 194
Moving Averages + Linear Regression Pipeline (Data Preparation + First Prediction Engine) 206
Deployment and Refinement of the Linear Regression Prediction System 231
ARIMA Model Implementation, Prediction Refinement, and Error Optimisation. 249
Error Handling Implementation, API Failure Management, and System Robustness Improvement 266
RSI-Augmented Decision Framework with Multi-Level Signals and Confidence Scoring 277
Graph Rendering 301
Initial Implementation of Welcome screen and dark mode 315
Welcome screen functionality 333
Final Code with annotation to aid maintenance and further development 359
Testing to inform Evaluation 384
Requirements + success criteria testing 398
Partial or unsuccessful tests 405
Usability Evaluation 408

Limitations 418
Maintenance 419
Further development 422
Appendix 424

Predicting Crypto Price with Technical Analysis by Designing an AI Agent

Research problem

Cryptocurrency markets, like Bitcoin or Ethereum, are open all the time — 24 hours a day, 7 days a week. These markets can change very quickly, with prices going up and down a lot in a short amount of time.

Because there is so much information and the market moves so fast, it can be hard for people to keep up (Baur & Dimpfl, 2018)[1]. This can lead to something called *information overload*, where there is too much to process (Frydman & Camerer, 2016)[2]. It can also cause people to make decisions based on feelings instead of facts. For example, they might panic and sell too quickly or get too excited and buy at the wrong time. These emotional reactions are known as *biases* — like *recency bias*[3] (focusing

[1] https://doi.org/10.1016/j.econlet.2018.10.008
[2] https://doi.org/10.1016/j.tics.2016.07.003
[3] recency bias makes you think that what happened recently will keep happening

too much on recent events), *confirmation bias*[4] (only noticing information that supports what we already believe), *overconfidence*[5], and *FOMO*[6] (fear of missing out).
Even if someone is very calm and careful, humans still get tired, especially after looking at lots of data for a long time. This can make their decisions less reliable[7].
That's why using a computer program can be helpful. Unlike people, computers don't get tired or emotional. They can look at huge amounts of data very quickly and make decisions based on facts, not feelings. A well-made computer program can even notice patterns and trends that a human might miss[8].
This project is about creating an AI agent that can analyse historical cryptocurrency data and make predictions about future prices to provide simple trading suggestions — for example, whether it might be a good time to buy, sell, or hold, and at approximately what price these actions could be taken.
Based on these predictions, the tool will suggest specific price levels where it may be ideal to buy low, sell high, or hold a position, making it a potentially valuable resource for beginner investors, financial advisors, or anyone interested in algorithmic trading.

[4] confirmation bias is when you only look for information that supports your existing beliefs
[5] Confidence is good. Overconfidence is dangerous
[6] fear of missing out is probably the most emotional bias in crypto trading
[7] https://coinpedia.org/traders/mind-over-market-conquering-emotional-bias-in-crypto-trading/
[8] https://www.nasdaq.com/articles/advantages-algorithmic-trading-2019-06-07#:~text=One%20of%20the%20biggest%20advantages,emotions%20that%20lead%20to%20poor

Broad outline of the Solution

This project involves designing an AI-based system that helps predict cryptocurrency price using technical analysis (Figure 1). The system will analyse historical price data from digital currencies such as Bitcoin or Ethereum and identify patterns that may indicate future trends. By calculating key technical indicators — primarily the Relative Strength Index[9] (RSI) and Moving Averages — the AI agent will generate simple trading suggestions: whether to buy, sell, or hold, and at approximately what price.

The system will be programmed in Python and will include functions to collect and process market data. It will apply technics such as moving average and/or RSI to make predictions and recommend actions. For example, if RSI drops below 30 and the price is near a historical low, the system might recommend buying at a specific target price.

Overall, the solution aims to support more rational and data-driven trading decisions, especially for beginners or non-experts.

[9] The Relative Strength Index is a momentum oscillator that measures the speed and change of price movements. https://www.fidelity.com/learning-center/trading-investing/technical-analysis/technical-indicator-guide/rsi

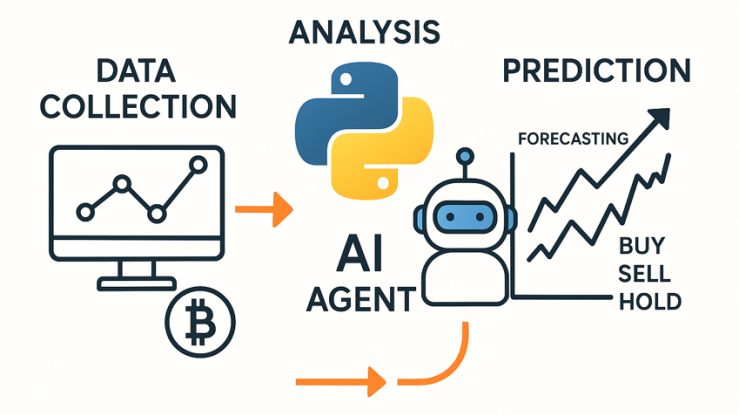


Figure 1: Broad outline of the system

Project Structure: Three Main Stages

This project is divided into three key stages (Figure 2) to ensure the system is accurate, logical, and easy to use:

1. Testing and Selecting the Forecasting Model

The first stage involves collecting and cleaning historical price data for Bitcoin and Ethereum. This dataset will be split into two parts: training data, used to build the model, and testing data, used to evaluate its performance. If the model does not perform well (i.e. it does not make accurate predictions), another forecasting method will be tried. This process will be repeated using a heuristic approach—testing several simple models until a suitable one is found. Once a reliable forecasting model is finalised, the project will move to the next stage.

2. Developing the Decision-Making Logic

Once the forecasting model is ready, a strategy logic will be designed to interpret the results. This logic will determine whether to buy, sell, or hold, and at approximately what price. The decision-making rules will be based on the forecasted trends and supported by research from existing literature. This step is crucial to turning predictions into meaningful and practical trading advice.

3. Automation with an AI Agent

In the final stage, an AI agent will be created to automate the whole process. It will automatically collect up-to-date price data, run it through the forecasting model, and apply the strategy logic to generate trading signals. These signals will advise users on actions (buy/sell/hold) and price points. Automation ensures the tool is consistent, fast, and user-friendly.

(Optional/Future Step) **User Interface Integration**

While not the main focus of this project, a simple graphical user interface (GUI) may be developed to display the outputs in a clear and accessible format for users with little technical background.

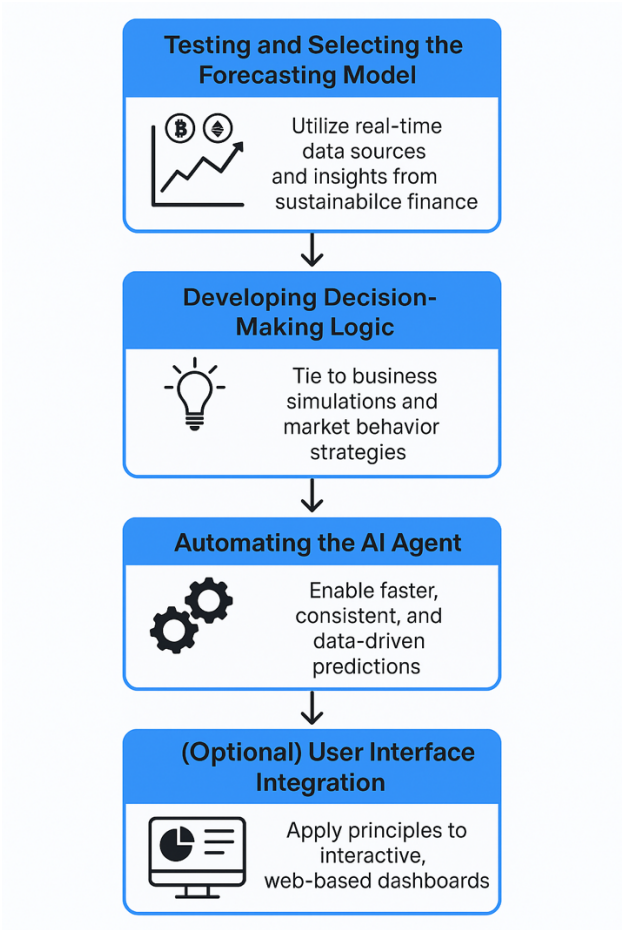


Figure 2: The process of developing the system

Challenges

1. **Knowledge of forecasting models:** As I am not currently familiar with forecasting models, I will need to dedicate time to improving my understanding of basic prediction techniques. This includes learning how basic forecasting models work and how to apply them in a cryptocurrency context. Technical indicators such as Moving Averages and the Relative Strength Index (RSI) require an understanding of their formulas, accurate coding, and testing to ensure they work well.
2. **Lack of experience in AI agent design:** Another challenge is that I am not yet experienced in designing an AI agent. However, I plan to build my knowledge by watching YouTube tutorials, using beginner-friendly coding resources, and learning from family members who have expertise in programming and data analysis.
3. **Access to data:** Access to reliable cryptocurrency data is also a key issue. I will need to explore different data sources and collect price data for one or two major cryptocurrencies (such as Bitcoin or Ethereum). This will allow me to test the model and generate predictions.
4. **Complexity of prediction:** Although I understand that cryptocurrency prices are influenced by many external factors — such as economic conditions, government policies, and global crises like wars — this project will focus on basic technical analysis. At this stage, I will use simple and well-known techniques, such as RSI and moving averages.

5. **Designing GUI:** Finally, as I am not an expert in designing a graphical user interface (GUI), I plan to create a basic and functional layout. I will use n8n platform that is a user-friendly tool to design an AI agent. Now I am thinking about sending the forecasted price, strategy (sell, buy, hold) and price on a messenger application such as Telegram.

Scope

There are many things that could be added to this project, but I've chosen not to include them due to time limits, difficulty, or my current level of knowledge and experience (Table 1).

1. **Bitcoin and Ethereum:** while there are lots of cryptocurrencies available, I will only focus on Bitcoin and Ethereum. These are the two most popular and widely used coins, and their historical data is easier to access and more reliable.
2. **Basic furcating models:** Another limitation is the range of forecasting methods. There are lots of complex models out there, but I've decided to use simpler techniques like moving averages and RSI, because they are easier to understand and implement at this stage. I will compare around 3 basic forecasting models and ignore the complex models that needs depth math and computing knowledge.
3. **Visualisation:** Also, instead of providing just price predictions or buy/sell signals, I could have included visual trend forecasting (such as chart projections).

4. **Data timeframe:** Finally, although data are available on multiple timeframes (e.g., hourly, daily, weekly), I've chosen to focus on a single timeframe (e.g., daily) since 2022 to keep things manageable. These decisions will help me stay focused and build a working AI agent that I can fully understand and explain.

Table 1. Scope of research

Included in the Project	Excluded from the Project
Analysis of Bitcoin and Ethereum	Analysis of all cryptocurrencies
Use of basic forecasting methods	Advanced forecasting models
Predictions based on one timeframe (e.g., daily)	Multiple timeframes (e.g., hourly, weekly)
Basic GUI to display results clearly	Complex GUI designs or mobile/web apps
Historical price data from public sources	Live data streaming or API integration

Computational Methods

1. **Problem Recognition**
   Problem recognition is recognising what human traders are not efficient at or making mistakes and presenting those issues as a problem that can be solved using computational methods and coding. The first step in my project is to recognise the limitations and inefficiencies that human traders often face, especially when making fast decisions in volatile markets like cryptocurrency. Many traders struggle with information overload, emotional biases (e.g., FOMO, panic selling), or lack the time to monitor trends consistently. By identifying these pain points, I frame the purpose of my project: to create a financial prediction tool that automates

and improves decision-making through computation and data analysis. This method guides the rationale behind designing the entire AI system, as it provides a solution to real-world trading problems.

Justification

This method is particularly well-suited to a computational approach because trading problems can be formalised into measurable and repeatable patterns. A computer can quantify variables such as price movement, volume, and momentum, analyse them objectively, and apply mathematical and machine learning models to identify trends. This removes human bias and allows scalable, real-time processing of large datasets, something impossible to achieve manually. Recognising the problem in computational terms ensures that the final program can provide consistent, data-backed trading signals, turning a human behavioural issue into a solvable algorithmic one.

2. **Problem Decomposition**

To manage complexity, I break down the overall problem into smaller, more manageable components. These include:

- Data Collection: Gathering historical data such as prices, volumes, and dates.
- Pattern Recognition: Identifying trends and repeating behaviours in the market.
- Trading Logic: Applying rules from technical analysis (e.g., Moving Average, RSI) to generate buy/sell/hold recommendations.
- Performance Testing: Evaluating the accuracy of the tool's predictions using past data. This decomposition allows me to plan my implementation in stages, using structured logic and focusing on one area at a time. It also aligns with how my development will be documented and tested.

This decomposition allows me to plan my implementation in stages, using structured logic and focusing on one area at a time. It also aligns with how my development will be documented, tested, and improved over multiple prototypes.

Justification

This computational method is especially effective for my project because each sub-problems such as cleaning data, implementing indicator formulas, or validating prediction accuracy, can be developed using individual functions and reusable algorithms. This modularity allows efficient debugging and targeted testing, as errors can be traced to a specific module rather than the whole system. It also supports scalability; for example, new indicators or currencies can be added later without rewriting the core logic. Decomposition therefore makes the project computationally feasible, structured, and maintainable, ensuring that complex financial analysis is tackled in a logical, step-by-step manner.

3. Data Mining

Data mining is extracting patterns from large data sets. In this project, data mining is used to identify hidden patterns, correlations, or anomalies within large datasets (e.g., historical Bitcoin prices). This helps the AI tool learn how different indicators (such as volume or RSI levels) may relate to future price movements. I will use this technique during the training phase of the machine learning component and during feature selection to find which types of data most influence outcomes. This enhances the model's performance and informs the trading logic.

Justification

This computational method is essential because cryptocurrency markets are highly volatile and complex, with many subtle interactions that are difficult for humans to detect manually. Data mining automates this analysis, processing thousands of records per second to identify non-linear correlations (for example, how sudden volume spikes often precede short-term price rallies). It enables the program to learn from real data rather than fixed rules, improving adaptability and precision. The approach also scales efficiently, larger datasets can simply be reprocessed without redesigning the system. Overall, data mining transforms raw information into structured insight, providing the foundation on which the predictive model and trading logic are built.

4. **Heuristics**

   Given the fast-paced nature of trading and the uncertainty of financial markets, finding a perfect solution is unrealistic. Instead, I use heuristic methods—rules of thumb or approximation techniques—to provide "good enough" predictions within short time frames. These methods are computationally faster and ideal for real-time alerts. For example, a heuristic might be: "If the RSI < 30 and price is near a local low, recommend buying." These logic-based shortcuts form part of my strategy logic module and allow the system to generate meaningful outputs without complex algorithms.

   Justification

   Heuristics are particularly effective for this project because they balance speed, simplicity, and reliability. They allow the program to operate continuously even with limited computing power or incomplete data, making the tool suitable for lightweight environments such as personal computers. Unlike machine learning models, heuristic rules are transparent and explainable, allowing users to understand how each trading signal was generated. This supports trust and

interpretability—two key usability goals of the project. Furthermore, heuristics can work alongside more advanced AI models to provide a fallback mechanism when data mining or regression models fail, ensuring the system remains responsive and stable under real-world trading conditions.

5. **Visualisation**

   A key part of making this tool usable is data visualisation. I will present the results of my analysis using colour-coded signals (green for buy, red for sell), price movement graphs, and trendlines. These visuals are not just cosmetic—they help users quickly understand the rationale behind the tool's suggestions and gain confidence in the predictions. This is particularly important for non-expert users. Visualisation is implemented in the front-end/user interface of the system, where the user interacts with the tool.

   Justification

   Visualisation is particularly important for this project because it translates complex numerical data into intuitive, human-readable formats. Computationally, it acts as a bridge between algorithmic output and user decision-making, reducing cognitive load and error. For instance, users can instantly see if the AI's forecast aligns with the actual trend without reading through raw figures. It also supports real-time feedback loops — the program can dynamically update visuals as new data arrives, helping users make time-sensitive trading decisions. By combining algorithmic precision with visual clarity, this method ensures that both technical and beginner users can engage with the system effectively, improving usability and confidence in the tool's predictions.

6. Machine Learning

   The final and most advanced computational method I plan to implement is machine learning. The aim is for the system to go beyond manually coded rules and start learning from data directly. By training on past price data, the model can recognise complex patterns and relationships that may not be obvious through traditional analysis. Over time, the system will be improved by using feedback loops—for example, by labelling each past prediction as a "successful" or "failed" signal based on what actually happened in the market. These labelled outcomes can help retrain the model, making it smarter and more adaptive. Machine learning will be implemented in the prediction engine and enhanced through ongoing testing and model tuning.

   Justification

   This method is particularly powerful for this problem because cryptocurrency markets show time-dependent, chaotic, and non-deterministic behaviour that simple heuristic rules cannot capture. A trained model can automatically learn correlations between multiple indicators and adapt to new data without rewriting any code, providing scalability and long-term efficiency. Computationally, it allows the system to process thousands of observations quickly and optimise its internal parameters through iterative algorithms such as linear regression or ARIMA. Machine learning also supports automation and adaptability — the model can adjust to changing market dynamics in real time, improving prediction accuracy with continued use. Overall, it transforms the program from a rule-based calculator into an intelligent, data-driven system capable of learning, evaluating, and evolving like a human analyst but at machine speed.

A moving average is one of the most used tools in technical analysis for smoothing out short-term price fluctuations and highlighting overall trends in financial data. It works by taking the average of a set number of recent data points — in this case, two days — and then "moving" this average forward one step at a time.

There are different types of moving averages, including Simple Moving Average (SMA) and Exponential Moving Average (EMA). This example uses Simple Moving Average, where each data point has equal weight.

In this example, the moving average is used as a simple yet effective method for predicting short-term trends in Bitcoin prices. One of the main reasons for selecting this technique is its simplicity — it is easy to calculate, interpret, and implement, making it highly appropriate for an A-Level project. The moving average also helps reduce day-to-day price fluctuations (noise), allowing us to focus more on the general direction of the market. By using a short-period moving average, such as the 2-day average, the system becomes more responsive to recent price changes, which is useful for short-term forecasting. Furthermore, moving averages serve as a foundation for more complex AI techniques such as the Relative Strength Index (RSI) or Long Short-Term Memory (LSTM) models, providing a practical starting point for building more advanced predictive systems.

To carry out this analysis, historical Bitcoin price data from the past 30 days was extracted from *Investing.com* (https://uk.investing.com/crypto/bitcoin/historical-data). This dataset provides the necessary inputs for applying the moving average calculation and generating trend-based price predictions (Table 2).

Table 2: Bitcoin historical prices and % change based on 30-Day data

Date	Price	Change %
2025-06-15 00:00:00	105545.8	0.12%
2025-06-14 00:00:00	105421.5	-0.62%
2025-06-13 00:00:00	106080.3	0.36%
2025-06-12 00:00:00	105703	-2.72%
2025-06-11 00:00:00	108661.2	-1.46%
2025-06-10 00:00:00	110269.2	0.00%
2025-06-09 00:00:00	110268.3	4.28%
2025-06-08 00:00:00	105739.6	0.17%
2025-06-07 00:00:00	105564.2	1.20%
2025-06-06 00:00:00	104308.3	2.74%
2025-06-05 00:00:00	101521.9	-3.03%
2025-06-04 00:00:00	104695.6	-0.66%
2025-06-03 00:00:00	105388.3	-0.43%
2025-06-02 00:00:00	105845.6	0.20%
2025-06-01 00:00:00	105635.2	0.99%
2025-05-31 00:00:00	104598	0.59%
2025-05-30 00:00:00	103981.3	-1.53%
2025-05-29 00:00:00	105600	-2.03%
2025-05-28 00:00:00	107786.7	-1.06%
2025-05-27 00:00:00	108944	-0.47%
2025-05-26 00:00:00	109455.3	0.40%
2025-05-25 00:00:00	109021.1	1.17%
2025-05-24 00:00:00	107763	0.42%
2025-05-23 00:00:00	107313.4	-3.93%
2025-05-22 00:00:00	111702.7	1.88%
2025-05-21 00:00:00	109640.1	2.64%
2025-05-20 00:00:00	106815.8	1.16%
2025-05-19 00:00:00	105585.8	-0.86%
2025-05-18 00:00:00	106504.5	3.29%
2025-05-17 00:00:00	103112.6	-0.36%
2025-05-16 00:00:00	103480.5	-0.27%
2025-05-15 00:00:00	103765.4	

The extracted data was then entered into Microsoft Excel, where a line graph was created to visually represent the Bitcoin closing prices alongside their 2-day moving average (see Figure …). A trendline was added to the graph to extend the moving average and predict the price for the next two days. This trendline was calculated using the following formula:

MA = (Price of Day N + Price of Day N-1) / 2

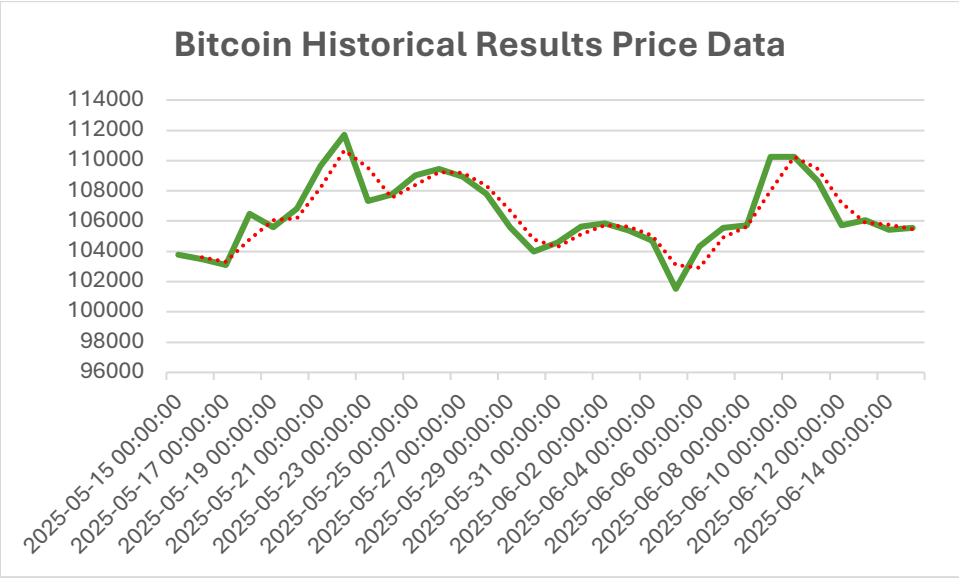


Figure 3: Bitcoin historical data and forecasting for next two days

The predicted prices for the next seven days, calculated using the above formula, are presented in Table 3. These values are based on the 2-day moving average trendline, which extends the observed pattern in the historical data to estimate future price points.

Table 3: Predicted Prices for the Next 7 Days (June 16 – June 22, 2025)

Date	Predicted Price (USD)
2025-06-16	105,483.65
2025-06-17	105,483.65
2025-06-18	105,483.65
2025-06-19	105,483.65
2025-06-20	105,483.65

2025-06-21	105,483.65
2025-06-22	105,483.65

Stakeholders

For this trading tool, I have identified key stakeholders who would be interested in this project (Figure 4). Each stakeholder has different use of the proposed solution based on their needs and wants.

1. Investors and Traders (Primary Stakeholders)

Who they are:
These are the main users of the end product — individuals who actively trade or invest in financial assets such as cryptocurrencies. They may range from part-time traders to small-scale investors who follow the market to earn profits but often lack the time or expertise to perform detailed technical analysis manually.

What they want:
They want a reliable and time-efficient tool that provides accurate predictions of cryptocurrency price trends and simple, data-driven signals (e.g., buy, sell, or hold). Their main goal is to make profitable trading decisions while reducing risk and emotional bias.

How they will use it:
They will use the program to view real-time market data, review AI-generated predictions,

and use the colour-coded signals (green for buy, red for sell) to inform their investment choices. Since many are not full-time traders, they will benefit from the tool's ability to process live data quickly and deliver results within seconds.

How my system meets their needs:
The AI Crypto Prediction Tool automates both technical and statistical analysis — including Moving Average, RSI, and machine learning predictions — providing users with clear, interpretable results. This saves significant time and removes the need to calculate indicators manually. The automated predictions, confidence scores, and real-time data updates help investors make more informed and consistent decisions while minimising the risk of human error or emotional trading.

2. Examiners and Testers (Evaluation Stakeholders)

Who they are:
This group includes A-Level Computer Science examiners who assess my NEA project, along with testers such as myself, teachers, and peers who evaluate the program during its development. Examiners focus on the accuracy, structure, and logic of my computational design, while testers help identify software errors and usability issues before the final submission.

What they want:

Examiners expect to see a clearly documented, logically structured project that demonstrates effective use of computational thinking, justified design choices, and a fully functional end product. Testers, on the other hand, aim to find bugs, check system reliability, and validate that predictions are accurate when tested on real or historical data.

How they will use it:

Examiners will review the documentation, top-down design diagrams, pseudocode, algorithms, and testing evidence to evaluate whether the project meets OCR assessment criteria. Testers will use the system hands-on — running the program with different data inputs, testing API calls, and assessing how predictions perform on old chart data and randomised scenarios.

How my system meets their needs:

The project includes comprehensive documentation and validation at every stage — such as flowcharts, structured pseudocode, annotated screenshots, and justification of algorithmic design. The inclusion of clear test data tables, boundary and scenario tests, and visual evidence ensures transparency and traceability. By testing with both live and historical data, the system allows examiners and testers to objectively measure accuracy, functionality, and efficiency against the original success criteria. This approach ensures that the program's computational processes and final outputs are fully verifiable.

3. Data Providers (e.g., TradingView API)

Who they are:
These are third-party services that supply real-time and historical financial data, including cryptocurrency prices, candlestick patterns, and market volumes. Examples include TradingView, Binance, or other open financial data APIs. They form the technical backbone of the system by allowing access to accurate and updated market data.

What they want:
Data providers expect their services to be used ethically and within their terms of service. They require developers to give clear attribution, prevent data misuse, and ensure that usage stays within API rate limits or access permissions. They also value accuracy and correct representation of their data in any derived product.

How they will be used:
The program will connect to data providers through approved API calls to fetch live cryptocurrency prices and historical chart data. This information feeds directly into the prediction engine, where it is processed for analysis, machine learning, and visualisation.

How my system meets their needs:
The tool only retrieves data using legitimate API endpoints and within the public access limits of the provider. All data sources will be clearly attributed in the user interface (e.g., “Data provided by TradingView”) and documentation. No raw or proprietary data will be redistributed, ensuring full ethical compliance. This approach not only respects intellectual

property and data licensing rules but also ensures the accuracy and reliability of information powering the AI model. By using trusted providers, the system maintains consistency, transparency, and professional standards in its data handling.

4. Machine Learning Developers (Myself and Future Contributors)

Who they are:
This group includes myself as the primary developer, as well as other potential contributors such as my teacher or family members who may help test, refine, or expand the system in the future. They represent the **technical stakeholders** responsible for maintaining, improving, or retraining the predictive model.

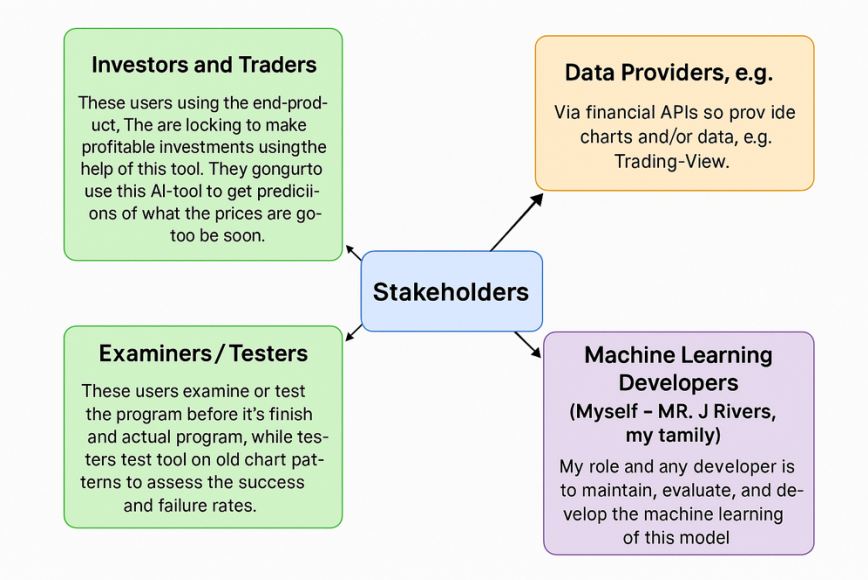
What they want:
Developers require clean, readable, and well-documented code that can be easily modified, debugged, or extended. They may wish to experiment with alternative algorithms, retrain the machine learning model on new datasets, or integrate additional indicators or API sources.

How they will use it:
Developers will access the source code, review the structure of the prediction engine, and test the effects of different machine learning approaches (e.g., ARIMA, Linear Regression). They may also make improvements to the GUI, adjust thresholds for buy/sell logic, or enhance data handling for larger datasets.

How my system meets their needs:

The program is designed with a modular coding structure, separating components such as data input, prediction logic, and visualisation. Each section of the code is clearly annotated and stored with proper version control, ensuring that previous versions remain accessible for debugging or analysis. By following structured programming and good commenting practices, future developers can understand the system's logic and modify it without introducing errors. This design approach not only supports collaboration but also encourages reusability and scalability, key principles of computational project development.

Figure 4. Stakeholders



Desk research

In this stage of my project, I reviewed and benchmarked three key platforms: **CoinMarketCap**, **Cryptohopper**, and **TradingView**. These platforms were carefully selected because each one offers specific features that are relevant and useful for designing my own forecasting and strategy system. In my analysis, I focused on their core functionalities — including access to data, support for multiple cryptocurrencies, integration with social platforms, visual design, and recommendation tools such as buy/sell signals. I also examined how each platform displays real-time prices and live market changes, as these elements are essential for helping users make informed decisions.

The purpose of this desk research is to identify best practices in both functionality and user experience that I can apply to my own AI-based system. In the following section, I will provide a brief overview of each platform, highlight their main features and explaining why they were selected for comparison in this research stage.

CoinMarketCap (CMC)

The images in figure 4 show the user interface and key analytics of CoinMarketCap (CMC), the world's most widely used platform for real-time and historical cryptocurrency data. Key features include price charts, volume, market capitalisation, trending coins, and leaderboards showing percentage increases and decreases. CMC tracks over 17 million cryptocurrencies and presents visual tools to compare performance over time. There are also design elements like green upward arrows for rising prices and red downward arrows for falling prices — these visual cues could inspire similar features in my own system. Additionally, some outputs such as Buy/Sell indicators, percentage change, and price predictions can be replicated in my project to help users make decisions. CMC also integrates

tools like Telegram price alerts, which is a useful idea for future development. It also has essential risk management features such as stop-loss and trailing stop to help traders manage potential losses and protect profits. Studying CoinMarketCap helps me benchmark a professional crypto forecasting interface and understand how to structure and present outputs in a user-friendly and visually effective way.

Justification of Approaches

CoinMarketCap's features are carefully designed to make complex market data both accessible and interpretable for a diverse audience of beginners and advanced traders.

- Percentage change displays exist because users need to understand short-term performance at a glance, rather than manually analysing numeric differences in prices. This feature transforms raw numerical data into a computationally efficient metric that instantly communicates direction and magnitude of change. I plan to include similar displays to help my users interpret volatility quickly.
- Visual trend indicators (green and red arrows) are used because they are universally recognisable symbols that reduce cognitive effort. This method of colour-coding data supports pattern recognition and improves response time, making it a feature I intend to replicate to enhance accessibility.
- Risk management tools (stop-loss and trailing stop) are included because cryptocurrency markets are highly volatile and require automated mechanisms to manage risk. These features align with computational methods like automation and conditional logic, which I can adapt in my own tool to alert users when a confidence score or price threshold is breached.

- Leaderboards and rankings serve as comparative tools, helping users identify outperforming assets. I may later integrate a simplified version of this feature to show which currencies have the highest predicted growth or confidence levels.

Overall, CMC's approach focuses on making large-scale financial data understandable, visually intuitive, and safe for decision-making. Its emphasis on visual cues, percentage changes, and risk management inspires my own system to balance analytical precision with user-friendly presentation.

Essential Features

- **Percentage Change Display**

  What:

  Displays the percentage increase or decrease of a cryptocurrency's price over a given period.

  Why:

  CoinMarketCap uses this feature to make it easy for users to assess short-term market performance at a glance, without needing to interpret complex price data. This supports quick and informed decisions.

  How:

  In my program, a simple percentage change label will appear next to each cryptocurrency's live price, updating automatically as new data is fetched. Positive changes will be displayed in green and negative changes in red to reflect instant market direction.

- **Colour-Coded Trend Indicators**

What:

Visual indicators such as green upward arrows for price increases and red downward arrows for decreases

Why:

These visual cues reduce cognitive load by helping users understand data patterns instantly, especially under time pressure or market volatility. CoinMarketCap uses them effectively to make complex movements more intuitive.

How:

My GUI will integrate arrow icons beside each prediction result. When the AI model predicts a rise, a green upward arrow will appear; when it predicts a fall, a red downward arrow will display. A neutral or flat trend will use a black horizontal line.

- **Simple and Visual Data Layout**

  What:

  A clear interface with neatly presented data tables, icons, and comparison charts.

  Why:

  CoinMarketCap's interface succeeds because it conveys large volumes of data without overwhelming the user. Consistent formatting improves readability and confidence in the results.

  How:

My interface will feature structured frames and alignment similar to CMC, with prices, predictions, and percentage change neatly separated into panels. Tooltips will explain data values when users hover over them, improving accessibility.

- **Market Leaderboard and Ranking**

  What:
  A ranked list of top-performing cryptocurrencies based on percentage growth or stability.
  Why:
  CoinMarketCap uses rankings to help users prioritise which coins to monitor. This supports decision-making by highlighting the strongest assets within a large dataset.
  How:
  My program will include a small ranking box showing which cryptocurrency (e.g., BTC or ETH) has the strongest predicted trend. This will update dynamically with each refresh cycle or prediction update.

- **Risk Management Alerts**

  What:

  Notifications that warn users if predicted market movement suggests high risk or potential loss.

  Why:

CoinMarketCap’s inclusion of stop-loss and trailing stop tools shows the importance of managing volatility. Alerts give users time to reconsider decisions before losses escalate. How:

I will implement threshold-based warnings in my prediction logic. When the confidence score or volatility metric drops below a safe level, a visible message such as “High Risk Detected” will appear in the results panel.

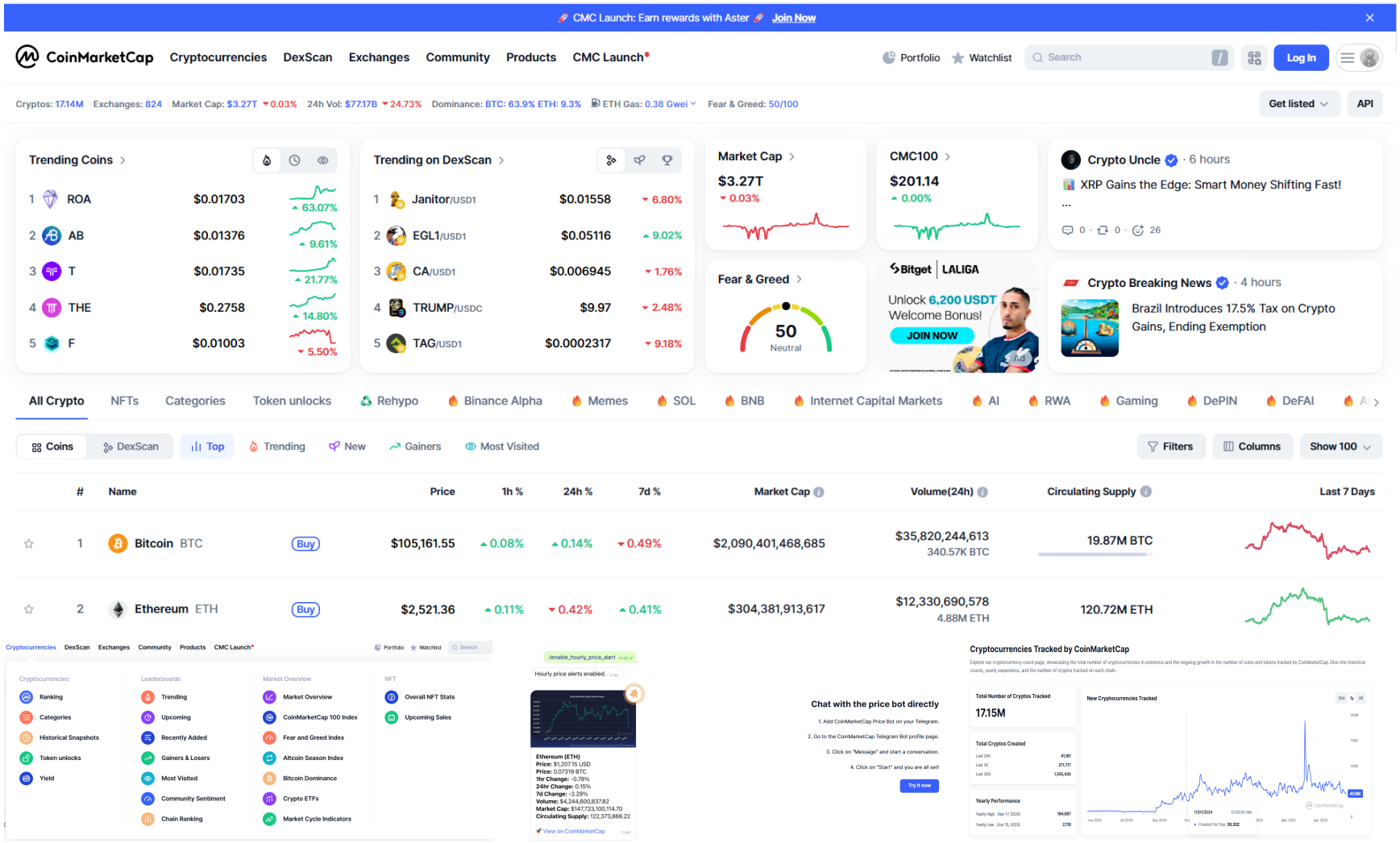


Figure 5: Features of CoinMarketCap (CMC)
Source: https://coinmarketcap.com/

Cryptohopper

Cryptohopper is a leading AI-driven cryptocurrency trading platform founded in 2017. It allows users to automate trading through intelligent bots that analyse the market and make decisions without emotional interference. Key features include automatic trading, copy bot (mimicking expert traders), strategy designer, AI learning, paper trading, back testing and real-time alerts that provides instant notifications via email when potential trading opportunities arise, enabling traders to act promptly. It also offers trailing orders, where the bot adjusts trades as the market moves, and a trading terminal to manage all exchanges in one place.

For my project, Cryptohopper is a valuable benchmark because it demonstrates how machine learning and technical strategies can be integrated into an AI system. It provides insight into real-time decision-making logic (buy/sell/hold), automation of strategy, and user-friendly visualisation tools. Studying its logic and structure can help me design a simplified forecasting and strategy model in my own AI agent. It's clear dashboard, auto-order execution, and market-tracking capabilities can inspire the development of my interface and prediction features.

Justification of Approaches

Cryptohopper's features are intentionally designed to address three common trading problems: human emotion, slow reaction times, and inconsistent decision-making.

- Real-time alerts are used because cryptocurrency markets operate 24/7 and move rapidly. Instant notifications reduce human delay, allowing traders to respond immediately to profitable setups. This justifies the inclusion of real-time alerts in my project's design, as users need timely updates without constant manual monitoring.

- Automated strategy integration (Buy/Sell logic) solves the issue of hesitation or emotional bias such as FOMO (Fear of Missing Out) and panic selling. By letting the system automatically execute decisions, Cryptohopper ensures consistency and objectivity, which are key computational advantages over manual human analysis.
- Paper trading exists to let users experiment with real market data safely. This reduces the financial risk of new traders and provides a testing environment for strategy evaluation. I plan to adopt this principle through simulation-based testing, ensuring my AI model can be verified before real-world use.
- Back testing is included to allow performance evaluation on past market data — a critical computational method for verifying prediction models. This feature justifies my inclusion of historical data validation, where my program will compare predicted versus actual outcomes to measure accuracy.
- The user-friendly dashboard serves to translate complex quantitative data into accessible visuals (charts, colours, and summaries). This improves user trust and supports decision-making through data visualisation, an approach I will replicate to make my forecasts easier to interpret for non-expert users.

Overall, Cryptohopper's design shows how AI and automation can make trading faster, safer, and more consistent. It provides strong evidence that my own system should combine real-time computation, model testing, and visual clarity to create a tool that is both effective and user-centred.

Essential Features

- **Real-Time Alerts for Trading Opportunities**

What:

Automatic alerts that notify users of significant price changes or predicted opportunities.

Why:

Cryptohopper uses real-time alerts to help traders act instantly when a profitable opportunity appears, avoiding human delays or missed chances.

How:

My program will use conditional logic and API updates to display colour-coded notifications (green for buy, red for sell) when certain thresholds or RSI conditions are met. These alerts will appear within the interface rather than via email to keep the system lightweight and fast.

- **Auto Strategy Integration (Buy/Sell logic)**

  What:

  Automated generation of trading signals based on technical indicators.

  Why:

  This feature allows the system to operate without manual user input, reducing emotional bias and ensuring consistent decision-making — the same principle used in Cryptohopper's AI bots.

  How:

  The prediction engine in my program will use indicators like RSI and Moving Averages to determine when to buy, sell, or hold. Outputs will be displayed visually on-screen with clear arrows and short explanations.

- **Paper Trading Environment**

  What:
  A risk-free simulation feature that tests predictions using real data without involving real money.
  Why:
  Cryptohopper's paper trading allows users to test strategies safely. This makes the tool suitable for learning or cautious experimentation.
  How:
  I will implement a simulated test interface that uses stored data to calculate hypothetical profits or losses, showing users how a strategy would have performed.

- **Clean, User-Friendly Dashboard**

  What:
  A simple, intuitive dashboard that organises predictions and prices clearly.
  Why:
  Cryptohopper's layout balances complexity with usability. A clear visual design helps users interpret predictions faster, improving trust and accessibility.
  How:
  My main GUI window will include clearly labelled sections, icons for each cryptocurrency, and

colour-coded boxes for live and predicted prices to create an interface that feels professional but simple to use.

- **Back Testing Capability & Historical Validation**

  What:

  Testing the algorithm's predictions on past data to check reliability.

  Why:

  Cryptohopper uses this method to verify strategies before live trading. It ensures the logic works under historical market conditions and provides measurable accuracy.

  How:

  My project will include a testing mode where users can load a CSV file of past data, run predictions, and view results comparing predicted vs. actual outcomes with accuracy percentages.

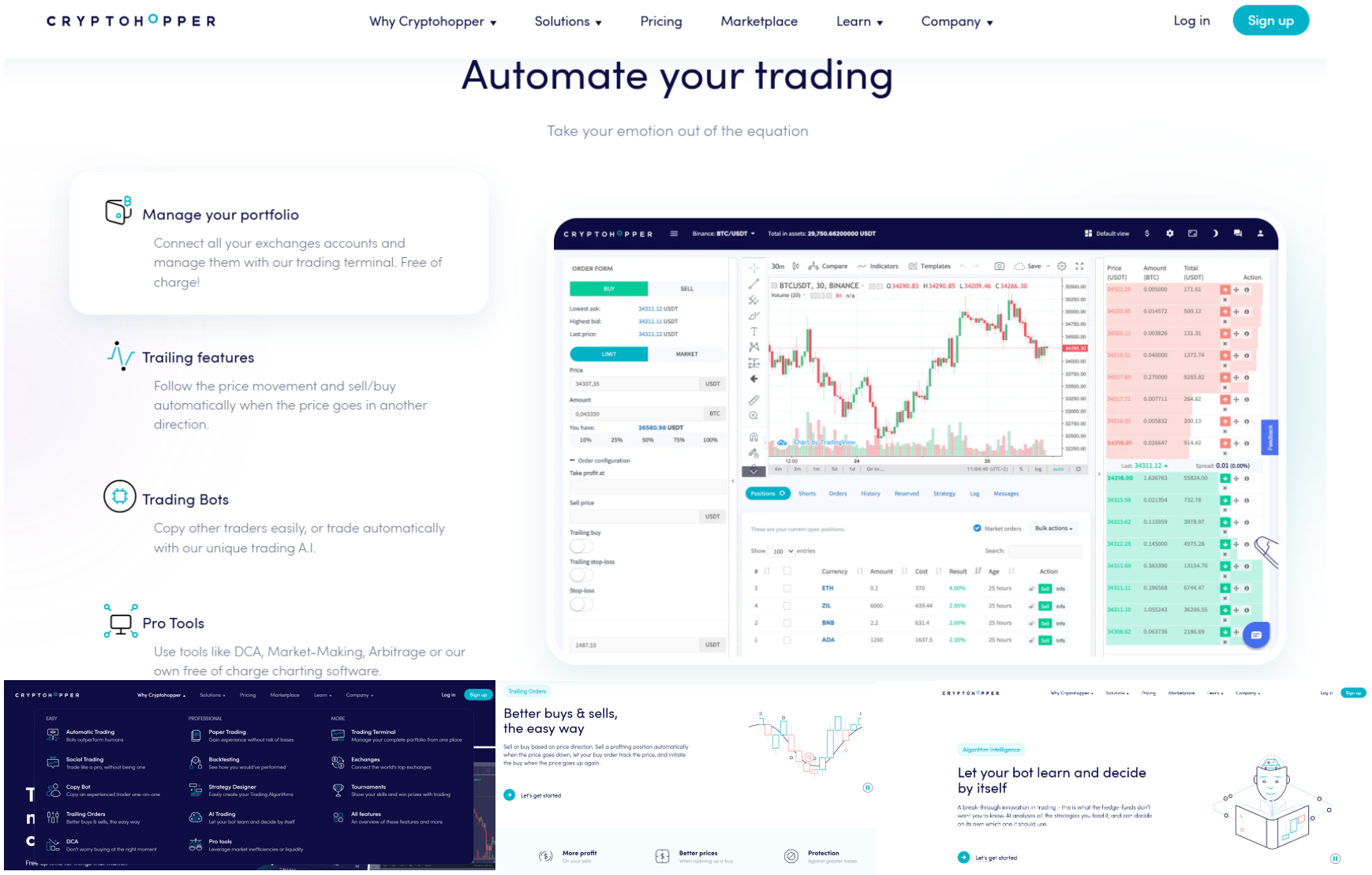


Figure 6: Features of Cryptohopper

TradingView

As part of my research, I explored TradingView (Figure 6), which is a popular web-based platform used by millions of traders to analyse financial markets including cryptocurrencies, stocks, and forex. It allows users to access real-time data, apply technical indicators, and design their own trading strategies. While it has many powerful features, what really stood out to me was its graphical design and user interface.

The platform uses clear and intuitive visuals to help users make better decisions. For example, price trends are shown with candlestick charts and colour-coded signals: green and upward arrows represent price increases, while red and downward arrows show price drops. It also displays BUY and SELL indicators in blue and red boxes, making them instantly recognisable. I found the way TradingView organises its tools, layouts, and comparison charts extremely user-friendly.

These design ideas have inspired how I plan to build my own system — using simple, clean visual elements, colour-coded signals, and accessible charts to help users easily understand my AI agent's predictions and suggested actions.

Justification of Approaches

TradingView's design choices are purposefully engineered to address the key challenge of visualising and interpreting complex market data efficiently.

- Candlestick chart visualisation is used because it presents multiple data points — open, high, low, and close — in a single view, offering traders immediate insight into market volatility and trend strength. This visual density reduces the need for manual calculation and supports

computational efficiency by summarising multi-variable data through a single chart form. I will replicate this approach to make my AI's prediction outputs both detailed and visually simple.

- Colour-coded trend indicators (green for rises, red for drops) enable instant interpretation of market direction. This feature applies computational abstraction — converting raw numeric data into recognisable, low-cognitive-load visuals. It helps users process information faster and act on predictions without delay.
- BUY/SELL signal boxes are designed to simplify complex analysis into binary or categorical outputs. They allow users without deep trading knowledge to act confidently based on pre-processed data, aligning perfectly with my project's aim to make technical insights accessible to casual users.
- User-friendly layout and tool organisation reduce clutter and prioritise readability, allowing traders to access the most important features first. This enhances usability, a design principle I will apply in my own interface by keeping navigation intuitive and focus areas clear.
- Chart comparison capabilities demonstrate the importance of scalability — letting users evaluate multiple datasets simultaneously. In my project, I plan to extend this idea by allowing users to switch between Bitcoin and Ethereum windows easily, mirroring this flexibility in a simplified form.

Overall, TradingView's approach combines computational clarity, visual hierarchy, and interactive efficiency, showing how interface design can transform raw financial data into actionable insight. These design philosophies directly inform how my system will present AI-driven predictions in a way that is accurate, intuitive, and time-efficient for all user levels.

Essential Features

- **Candlestick Chart Visualisation**

  What: Displays price changes over time using candlestick charts that show open, high, low, and close values for each time period.

  Why:

  TradingView uses candlestick charts to provide more detailed and interpretable price movement information compared to line graphs. They help users recognise patterns such as reversals or momentum shifts, improving analysis accuracy.

  How:

  My system will use Python's *Matplotlib* or *Plotly* library to generate candlestick charts for Bitcoin and Ethereum. These charts will update automatically with live data and visually highlight trends for clearer decision-making.

- **Colour-Coded Trend Indicators**

  What:

  Visual markers (green and red) that highlight upward or downward market trends at a glance.

  Why:

  TradingView's use of colour-coding simplifies complex financial data into universally recognisable cues, allowing users to react quickly without reading raw numbers.

How:

In my program's main prediction display, predicted increases will appear in green, decreases in red, and neutral trends in black. These colours will be consistent across all screens for familiarity and accessibility.

- **BUY/SELL Signal Boxes**

What:

Simple coloured boxes (blue for "BUY," red for "SELL") that display the system's trading recommendations.

Why:

TradingView's BUY/SELL indicators summarise technical calculations into clear, actionable results, which are especially helpful for users without deep trading knowledge.

How:

My prediction engine will output a text label and colour-coded box based on model calculations. These will appear under the prediction chart to clearly communicate the tool's recommendation and build user confidence in the output.

- **User-Friendly and Organised Layout**

  What:

  A clean, structured interface where tools and results are logically grouped and easy to navigate.

  Why:

  TradingView's professional design helps traders focus on analysis without distractions, reducing cognitive overload and improving usability.

  How:

  My GUI will use a similar structure — separating data input, prediction, and chart visualisation areas into distinct panels. Navigation buttons (e.g., "Open Bitcoin," "Open Ethereum," "Exit") will be placed consistently at the top of the window for clear user flow.

- **Chart Comparison Capability**

  What:

  An option to switch or compare between different assets (e.g., Bitcoin vs. Ethereum).

  Why:

  TradingView allows users to analyse multiple markets, improving insight and context for investment decisions.

  How:

  My tool will include two separate windows (BTC and ETH), accessible via the main window.

Each will have its own live data, prediction chart, and confidence score, enabling quick comparison without restarting the program.

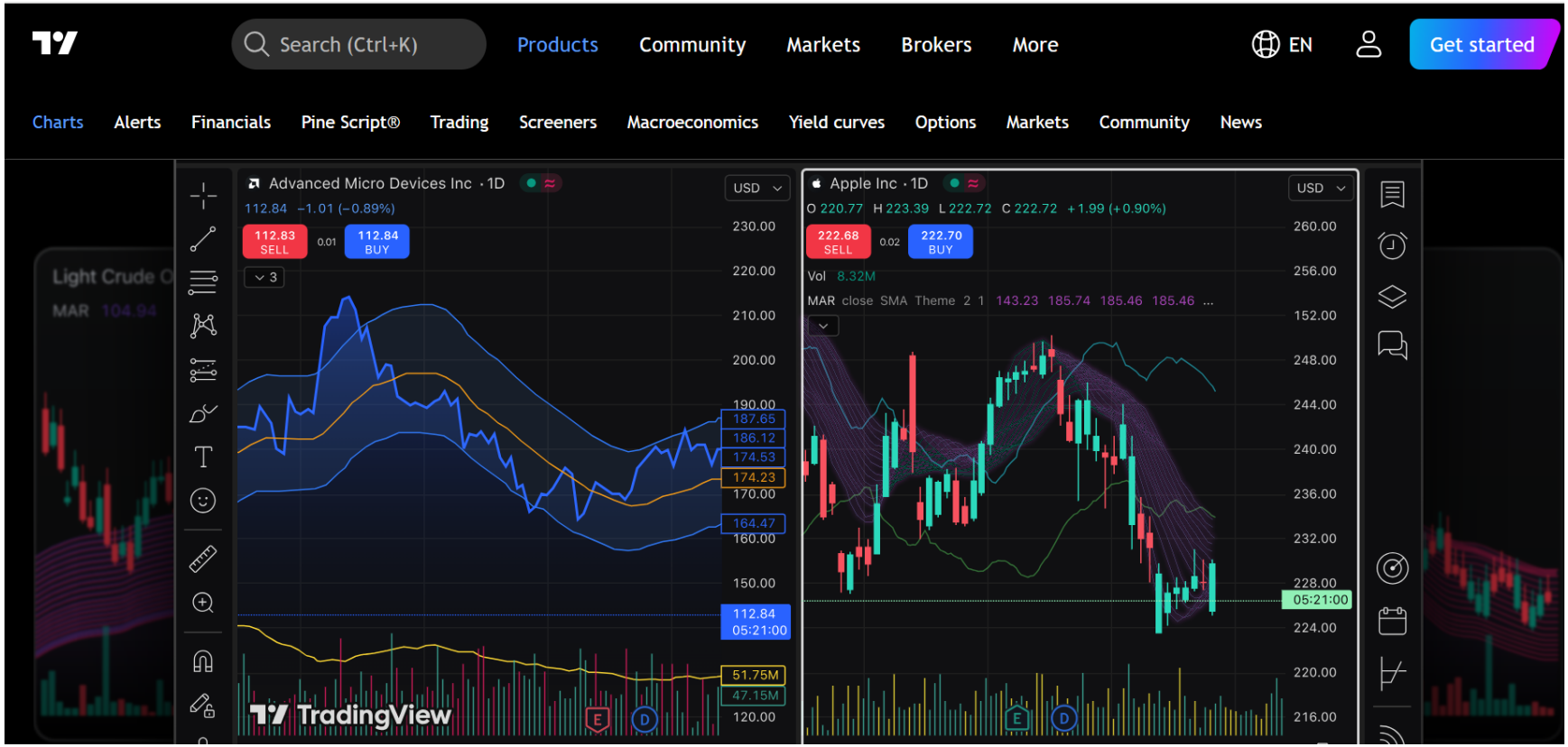


Figure 7: Features of Tradingview

Interviews

For my project, I chose to interview three key stakeholders to gain deeper insights that would help shape my system design. First, I interviewed an investor/trader to better understand what users expect from a financial forecasting tool — including their needs, preferences, and what features they find most useful. Second, I spoke with an examiner to understand the academic and technical requirements for my program, including what criteria it needs to meet to be considered successful. Finally, I interviewed a machine learning developer, who provided me with a clear summary of how I should approach the technical design and structure of my AI-based prediction model.

Interview 1: Trader

Firstly, I chose to interview a professional trader, as their extensive experience in the financial markets and exposure to various analytical tools make them an ideal stakeholder for this project. Gaining insights from someone who regularly uses such tools will help me better understand the key needs and preferences of my target users. I will structure my questions around identifying their core requirements, exploring their preferences for data visualization and functionality, assessing their

level of trust in AI-driven tools, and discovering the features they would consider essential in a financial prediction program.

Q1: What is the biggest challenge you face when trying to predict future price movements using current financial tools?

One of the biggest challenges is there is so many signals, so much data, but very little clarity. Most tools don't filter out irrelevant information, and I often get overwhelmed. I want something that gives me concise, trustworthy insights based on both data and context.

Q2: How important is visual presentation (e.g. color-coded charts, clean layout) when you're analyzing financial data?

Very important. When I'm trading, especially in volatile markets, I need to process information quickly. A clean interface with color cues—like red for drops, green for gains—really helps me make fast, informed decisions. A cluttered screen slows me down.

Q3: Would you trust an AI to predict future prices, and what would make you trust it more?

I'd trust AI, but only if I can understand how, it makes decisions. A confidence score and a short explanation would help a lot. If it just throws numbers at me with no reasoning, I'll be cautious. But if it's transparent and consistent, I'd absolutely use it.

Q4: Would you prefer a simple tool with a few powerful features, or a complex one with lots of options?

Balance is best. I'd start with something simple, but I'd want the option to unlock more advanced features as I get comfortable. Maybe a "basic" and "pro" mode would work well.

Q5: If you could customize one key feature in a financial prediction tool, what would it be?

Honestly, I'd love a simple toggle that lets me choose between short-term and long-term predictions. Sometimes I just want to know what might happen tomorrow, and other times I'm looking at the next few weeks. That kind of basic control would be very helpful.

Essential features

- Smart Data Filtering System
- Clean Visual Interface with Colour Coding
- Confidence Score & Explanation for Predictions
- Basic vs. Pro Mode Toggle
- Short-term vs. Long-term Forecast Toggle

Justification

Interviewing an investor gave me extremely valuable insights that directly influenced how I designed and prioritised features in my system.

One key takeaway was the need for clarity and simplicity. The investor explained that most current tools overwhelm users with too much data and too many signals, without filtering what really matters. This confirmed for me that my system needs to summarise data clearly, avoid clutter, and focus only on the most relevant predictions. I've now made it a priority to display concise, meaningful outputs — rather than overwhelming users with every possible indicator.

They also stressed the importance of visual presentation — especially using colour-coded charts and a clean interface to help make quick decisions. This reinforced my decision to use green/red signals and to keep the design minimalistic and easy to read.

Their answer about AI trust helped me realise the importance of transparency. I will include a confidence score and a brief explanation next to each prediction, so users feel more confident and understand why the AI made that recommendation.
Finally, their idea of having a simple toggle for short-term and long-term predictions, and the suggestion of a "basic" vs "pro" mode, gave me practical ideas for future development. I've decided to start with a simple model but build it in a way that can be expanded later to suit more advanced users.
Overall, this interview helped me prioritise user clarity, trust, and flexibility, which are now at the core of how I'm building my AI tool.

Interview 2: Tester

To gain further insight into how my financial AI tool could be improved and evaluated effectively, I interviewed my Computer Science teacher, who has experience assessing NEA projects like mine. The interview focused on key aspects such as visualization, data quality, and usability.

Q1: As someone who assesses or tests student projects, what features or aspects do you pay the most attention to, particularly in a tool like mine?

When evaluating student programmes, I look for several things: clarity in design, how well the tool performs its function, and how effectively the student explains their logic. For a project like yours, I would particularly focus on how clearly the financial data is presented, how understandable the outputs are, and whether the project demonstrates problem-solving and technical thinking. A good programme doesn't need to be overly complex, but it does need to work reliably and make sense.

Q2: How important is the quality of data visualisation—like charts or graphs—in your evaluation?

Data visualisation is important. It helps turn complex data into something easier to recognise and understand. Good visualisation allows users to quickly identify patterns or trends. It also helps communicate what the programme is doing, which is crucial in a project like financial prediction. A tool that includes clear charts, colour-coded alerts, or interactive graphs will stand out.

Q3: From your point of view, what would make my financial AI tool stand out from others?

There are three key things:

1. Strong visualisation – If your tool can turn complex data into something that's clear and meaningful, that's a huge strength. It should help users recognise patterns or alert them to changes, for example with prompts or markers.
2. Accurate, up-to-date data – Your programme should show data that is current and correctly processed. Even better if it pulls live or regularly updated data (although real-time updates are more of a consideration than a requirement). The goal is to reflect meaningful information that helps users make decisions or understand market behaviour.
3. Simplicity in design – The user interface should be intuitive, easy to use, and clear. A streamlined experience, where the tool is simple to operate and understand, shows excellent design thinking. It should inform the user without overwhelming them.

Essential features

- Clear Data Visualisation (Charts & Alerts)
- Reliable and Accurate Data Input
- Simple and Intuitive User Interface (UI)
- Consistent Functionality with Logical Design
- Interactive Prediction Explanation Elements

Justification

Interviewing my Computer Science teacher, who regularly assesses NEA projects, gave me a really useful perspective on how my AI trading tool should be evaluated and improved. One of the most important things they mentioned was the need for clarity in design and function. This confirmed that I don't need to build the most complex system, but I do need to make sure it works reliably, is well explained, and is logically structured. I've taken this seriously by focusing on building a system that is both functional and clearly documented.

They also highlighted that data visualisation is essential, not just as a visual aid but as part of the programme's logic and communication. Because of that, I'm making sure to include clear, colour-coded charts and possibly interactive graphs that help users see patterns or changes at a glance. I now realise that visual design isn't just a nice extra — it's part of how my system will be judged and understood.

What stood out most was their advice on making the tool simple, intuitive, and clean. This influenced my choice to use a dark, minimal interface, reduce on-screen clutter, and clearly label AI outputs so users don't get overwhelmed. I'm also planning to build a basic version first and keep the layout as user-friendly as possible.

Lastly, their point about using accurate and meaningful data pushed me to make sure the system uses updated, well-processed price data, even if I'm not working with live streams yet. I now understand that showing useful, real-world insights is just as important as the AI predictions themselves.

Overall, this interview helped me shift from thinking just about "building the tool" to thinking about how it will be judged, used, and understood, which has made my approach more thoughtful and well-rounded.

Interview 3: Machine Learning Developer

Lastly, I conducted an interview with a developer specializing in machine learning to support the development of my AI-based financial price prediction tool. The purpose of this interview was to gain insights on how to begin building the program, understand the key technical components involved, and ensure the project is grounded in realistic and effective development practices.

Q1: From your experience, what is the easiest way for a beginner to build a price prediction AI?

Start with a linear regression model using historical price data—it's simple, fast to implement, and gives you a feel for how machine learning works. Once that's working, you can try decision trees or basic neural networks, but start small and build confidence

Q2: What type of data should I focus on first if I want to predict prices?

Stick to historical data like daily closing prices, highs and lows, and maybe volume. Those are easy to find and clean. Avoid news or social media data at first—they're hard to process and require more advanced NLP models.

Q3: Do I need to understand deep learning to build this?

Not at all. You can do this with basic Python and libraries like pandas, scikit-learn, and matplotlib. Deep learning is powerful but harder to debug. For A-Level, a simple ML model that works and is explained clearly will be more impressive than a deep model you don't understand.

Q4: How much data do I really need for a basic model to work?

Even 3–6 months of daily data (roughly 60–120 data points) can be enough to build and test a simple model. It won't be super accurate, but it will show the concept and help you learn the pipeline: loading, training, and predicting.

Essential Features

- Basic Linear Regression Model for Initial Predictions
- Focus on Historical Price Data
- Avoid Deep Learning; Use Understandable Algorithms
- Step-by-Step Machine learning Pipeline

Justification

Interviewing a machine learning developer was incredibly helpful because it gave me practical, realistic advice on how to approach the technical side of my AI-based financial prediction tool — especially as someone who is still learning the basics of machine learning.

One of the most important things I took from this conversation was the recommendation to start simple. The developer suggested using linear regression with historical price data to begin with, which is something I can implement using Python and scikit-learn. This made me feel more confident because I now know I don't need to dive into complex deep learning models straight away — I can build a working system with basic tools and still demonstrate my understanding.

They also advised me to focus on clean, structured data like daily closing prices, highs/lows, and volume, rather than trying to include news or sentiment analysis at this stage. I've followed that advice and limited my dataset to easily accessible and manageable historical price data from Bitcoin and Ethereum.

Another key takeaway was that deep learning isn't necessary for a successful project at my level. The developer reassured me that it's more impressive to build something simple that I fully understand and can explain, rather than trying to use complex algorithms I don't fully grasp. That really helped shape my project scope and reduce unnecessary complexity.

Finally, I learned that I don’t need a massive dataset to build a model. The developer said 3–6 months of daily price data would be enough to test my approach and demonstrate the process of loading, training, and predicting. That was useful because it allowed me to focus on building a clear machine learning pipeline with a small, manageable dataset.
Overall, this interview helped me ground my project in realistic, achievable goals and gave me the confidence to start small, test effectively, and grow the system step by step.

Overall Essential Features (Inspired by Interviews)

- **Smart Data Filtering System**

What:
An automated system that prioritises and cleans incoming financial data to remove irrelevant or duplicated information before analysis.

Why:
The trader interview highlighted how overwhelming raw data can be when there are too many indicators or conflicting signals. A filtering system ensures that only essential and high-quality data (e.g., closing price, RSI, volume) is processed, improving accuracy and speed.

How:
I will implement pre-processing functions within the Input Data module that validate and clean the dataset using conditions (e.g., removing missing or zero values). This will keep the prediction results concise and meaningful for users.

- **Clean Visual Interface with Colour Coding**

What:
A minimal and user-friendly interface that uses colour cues (green for upward trends, red for downward, black for neutral) and clear chart layouts.

Why:
Both the trader and tester interviews emphasised that clarity and colour-coded visuals are crucial for fast decision-making. A cluttered or overly technical layout would discourage use.

How:
The GUI will use a dark theme with balanced contrast and minimal text. Matplotlib-generated charts will integrate with Tkinter labels and colour-coded arrows to display predicted price movements visually and intuitively.

- **Confidence Score and Explanation for Predictions**

What:
A visible confidence score (in %) alongside each prediction, with a short text explaining why that result was produced.
Why:
The trader interview revealed that users are more likely to trust AI when it is transparent. Showing confidence levels and brief reasoning increases user confidence and credibility.

How:
My prediction engine will output both a numeric confidence score (based on model certainty) and a short explanation referencing the indicator used — e.g., “High confidence (78%) – RSI indicates oversold conditions.”

- **Simple Machine Learning Pipeline (Linear Regression)**

What:
A basic, easy-to-understand AI model (Linear Regression) for price prediction using historical data.
Why:
The machine learning developer explained that simple algorithms are easier to implement, debug, and justify in an A-Level project. This balances technical complexity with practical reliability.
How:
The system will use the scikit-learn library to build, train, and test a linear regression model based on 3–6 months of historical Bitcoin and Ethereum data. This pipeline will include data loading, training, predicting, and testing steps.

- **Adjustable User Modes (Basic vs Pro & Forecast Length Toggle)**

What:
Two user levels: a simple “Basic Mode” for casual users and an optional “Pro Mode” for advanced users, alongside a toggle between short-term and long-term predictions.

Why:
The trader interview suggested flexibility between simple and detailed analysis is ideal — allowing different users to interact with the system at their own level of comfort.

How:
The GUI will include toggles that adjust the parameters passed to the prediction engine. In Basic Mode, default RSI and moving average values will be used, while Pro Mode will allow manual adjustment of prediction windows and data intervals.

Users survey

In this research, investors and users are the main stakeholders, as the system is being designed to support them in making informed investment decisions based on data and technical analysis. Understanding their opinions and preferences is a key step in shaping the system to meet their needs. In this section, I will explain the survey I conducted to collect feedback from potential users.
I chose a survey as a research method alongside desk research because it allows me to collect a wide range of views quickly and efficiently. The multiple-choice format made it easy to complete, and the responses have provided useful insights into the preferred features, usability, and expectations of future users of my AI tool. This will help me make informed design decisions and ensure that the final product is user-friendly, relevant, and meets the needs of my target audience.
As I don’t have access to real investors, I invited my classmates—who represent potential beginner users—to complete the survey. I shared the link with a class of 20 students, and after one week, I

received 14 completed responses. In the next section, I will analyse the results using descriptive statistics and charts.

Question1: What is your level of experience with financial trading?

Question 2: What is your role in relation to finance and technology?

Question3: How familiar are you with artificial intelligence?

Understanding My Users

Most of my participants (57%) said they have no experience in trading, and nearly all of them (93%) are students, just like me. Although they aren't experts in AI, most are at least moderately familiar with the concept.

Since most users are beginners and students, I'm designing the system to be simple and guided. I won't include complex terms like RSI or machine learning unless they're explained. I'm keeping the layout intuitive and will provide tooltips or short pop-ups to explain things like confidence scores and prediction logic.

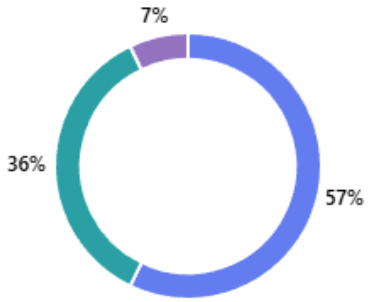
Essential Features

- Beginner-Friendly User Interface
- Tooltips and Pop-up Explanations
- Guided Prediction Output
- Simplified Language and Explanations

Question 5: Which type of data visualization do you prefer?

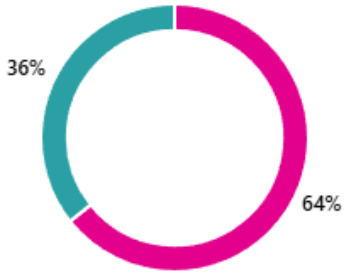
Question 6: how important is it to have color-coded alerts (e.g., green for buy, red for sell)?

Somewhat important	8
Neutral	0
Extremely important	5
not important	1



Question 18: Should the prediction output be...

A simple number (e.g. £250)	0
Number with arrow up/down	9
Number with a short message ("likely to rise")	5
Other	0



Question 19: Should the tool let you customize colours of the chart?

Visualisation and Output Preferences

Line charts and candlestick charts were the most preferred. Also, colour-coded alerts (green for buy, red for sell) were rated very important by most users. When it came to how predictions should be shown, 64% preferred a number with an up/down arrow, while others liked a number with a short explanation (e.g. “likely to rise”).

I’ll display data using line and candlestick charts, since those were the most preferred. Buy/sell signals will be shown using coloured arrows (green for buy, red for sell) and a confidence percentage like “↑ 76%” will be shown . I’ll include a short message such as “likely to rise” next to each signal. This will be automatically generated by the AI output.

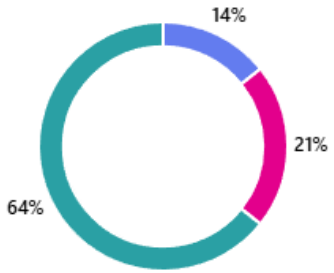
Essential Features

- Line and Candlestick Chart Visuals
- Colour-Coded Buy/Sell Alerts
- Confidence Percentage + Arrow Symbol

- Short, Text-Based Prediction Message
- Dual Output Style (Visual + Text)

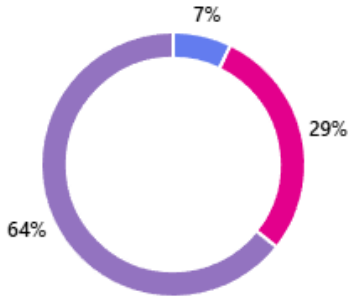
Question 7: Do you prefer a tool with:

Many features, even if it's complex	2
A few key features, but easy to use	3
A balance between the two	9



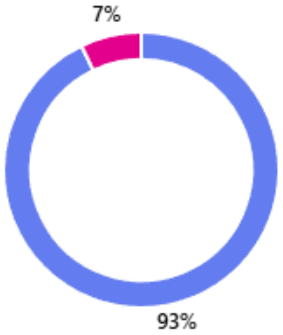
Question 8: what type of AI prediction would you prefer

Short-term (next hours/days)	1
Mid-term (1–2 weeks)	4
Long-term (months)	0
All of the above	9



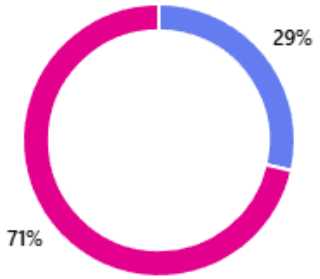
Question 9: Should AI show a confidence score (e.g., 82% confidence)?

Yes	13
No	1
Maybe	0



Question10: would you want explanation of the AI's prediction

Yes, detailed explanations	4
Short summaries	10
No explanations needed	0



Question 12: What features would you find most useful?

Prediction Logic & Transparency

Most people (64%) said they want a balance — not too complex, but not too basic. They also want to see predictions for short-term, mid-term, and long-term. A big majority (93%) said the AI should show a confidence score, and 71% prefer short summaries rather than long explanations.

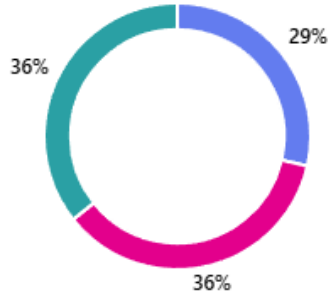
I'm developing the AI model in Python using a modular setup, starting with moving averages and RSI-based rules. Once tested, the logic will generate predictions for short-, mid-, and long-term trends. Each prediction will include a confidence score and a 1–2 sentence explanation, automatically written using predefined templates. I won't overwhelm users with data; the interface will only show what's necessary.

Essential Features

- Short, Mid, and Long-Term Predictions
- Balanced Prediction Complexity
- Minimalist, Focused Display of Results

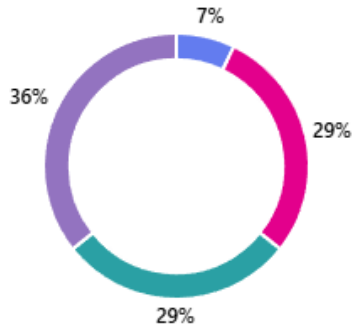
Question 11: Would you be interested in uploading your own datasets?

Yes	4
No	5
Maybe	5



Question 13: how would you rate your patience level for waiting on AI results?

I want it instantly	1
Doesn't matter	4
I can wait up to a minute	4
I can wait a few minutes	5



Question 20: tell us anything else you want us to implement in the program

ID ↑	Name	Responses
1	anonymous	focus more on the AI being accurate than any thing else as if the AI works but only a small percentage of the time the website becomes unreliable
2	anonymous	assistance in understanding current trend in market
3	anonymous	No
4	anonymous	IF alerts are coloured based then people who are colour blind wont be able to tell. maybe add a noise or something too.

Functionality, Patience & Customisation

Opinions were split on whether users would like to upload their own data — some said yes, some said no. Most users said they're willing to wait a minute or two for accurate results. When I asked for final comments, they gave me three key ideas:

- Focus more on accuracy than fancy features
- Help users understand the trend in the market
- Make the tool accessible to people who are colour blind (e.g. by adding sound or labels instead of colour-only alerts)

I'm not including the dataset upload feature in this version since opinions were mixed. Instead, I'll focus on getting the AI to analyse live or recent Bitcoin and Ethereum data from a public API. Since users are willing to wait a minute or two, I'll prioritise accuracy and clear forecasting, not real-time

speed. I'm also building in text alternatives for all colour-based signals and planning an optional sound alert system for users who might struggle with colour perception.

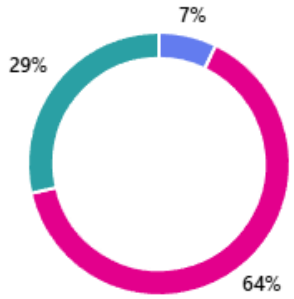
Essential Features

- Prioritise Accuracy Over Speed
- Market Trend Interpretation Support
- Accessibility Features (Colour-Blind Friendly)
- No Dataset Upload in Current Version
- Predefined Asset Focus (BTC/ETH Only)

Question 4: What kind of interface do you prefer in financial tools?

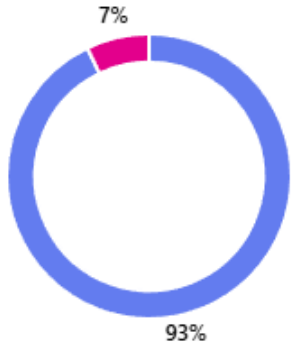
Question 14: Which color theme do you prefer for financial tools?

Light theme (white background)	1
Dark theme (black/grey background)	9
Customisable theme	4

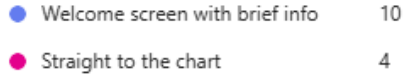


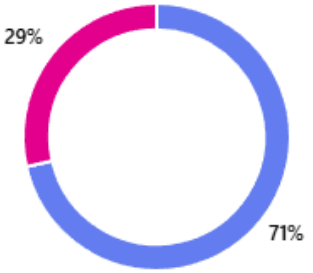
Question 15: Which font style do you prefer in an app like this?

Simple and clean	13
Tech-inspired	1
Classic/serif	0

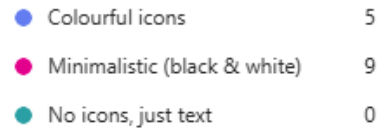


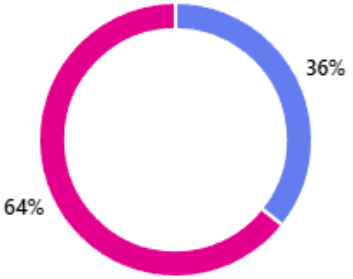
Question16: Would you like to see a simple welcome page or go straight to the chart?





Question 17: What kind of icons would you like?





User Interface Preferences

Half of the users prefer a simple and clean layout, and 64% like dark mode, which I personally prefer too. Almost everyone (93%) chose modern fonts, and the majority (71%) wanted a short welcome

screen instead of going straight to the chart. For icons, 64% prefer minimalist black and white designs.
I've chosen a dark theme with modern, clean fonts, and I will be building a straightforward interface. The home screen will include a brief welcome page, and I'll use minimalist icons from an open-source set. Users will land on a main dashboard where everything — predictions, charts, and summaries — is cleanly presented with minimal clutter.

Essential Features

- Dark Theme User Interface
- Modern, Clean Font Choices
- Short Welcome Screen Before Main Dashboard
- Minimalist Black & White Icons
- Main Dashboard with Clean Layout

Overall Essential Features (Inspired by Survey Results)

- **Guided Tooltips and Pop-Up Explanations**

  Users expressed a strong preference for short, helpful explanations to guide them through predictions and tool functions. This ensures that even first-time users can understand what each

value, chart, or indicator means. The final program will include hover-based tooltips and pop-up messages that describe terms such as “confidence score,” “RSI,” and “trend direction.”

- **Dual-Mode Output (Visual and Text-Based Predictions)**

Participants preferred outputs that combine both visual and written explanations. This ensures inclusivity and improves comprehension. Therefore, each prediction will include both visual cues (e.g., arrows, colour-coded alerts) and a short text message explaining the forecast — for example, *“Predicted rise with 78% confidence.”*

- **Dynamic Line and Candlestick Chart Visualisation**

Charts were rated as the most useful way to interpret predictions. The system will include interactive line and candlestick visuals showing real-time and historical price data. These charts will update automatically as new predictions are made, helping users track accuracy over time.

- **Confidence Score with Direction Symbol**

Users valued transparency about prediction accuracy. The final interface will therefore display both a confidence percentage and an arrow symbol (up, down, or flat) to visually reinforce the direction of the predicted trend. This gives users both clarity and confidence in the AI’s reasoning.

- **Accessibility and Inclusive Design**

Accessibility was highlighted by several respondents, especially colour-blind friendliness and easy readability. The final interface will use contrasting colours, clean fonts, and accessible icon shapes to ensure that all users can interpret predictions effectively, regardless of visual limitations.

- **Adjustable Prediction Timeframes (Short, Mid, Long-Term)**

Users preferred flexibility in analysing different time horizons. The tool will allow users to toggle between short-, mid-, and long-term predictions, adapting model parameters accordingly. This keeps the system useful for both casual and analytical users.

- **Accuracy-First Processing**

Most respondents prioritised prediction accuracy over processing speed. This means the program will favour reliable, well-tested calculations rather than instant updates. Data validation and stable model training will therefore take precedence over refresh rate.

- **Dark Mode and Minimalist Dashboard Design**

Aesthetic preferences showed a clear trend toward a modern dark theme with simple, professional visuals. The interface will feature a black background, white minimalist icons, and clean typography to create focus and reduce eye strain during use.

- **Focused Asset Scope (BTC and ETH Only)**

Questionnaire results indicated that limiting the system to a small set of cryptocurrencies would improve performance and simplicity. Therefore, the current version will focus exclusively on Bitcoin (BTC) and Ethereum (ETH), ensuring accuracy, efficiency, and clarity within a controlled data range.

Requirements

Table 4: System Requirements Table

Section	No.	Requirement	What this requirement will solve	Evidence	Justification
Functional Requirements	1.1	Data Input	Allows the system to process historical data for prediction		Essential for training the AI — without structured input, no predictions could be generated. Including it provides the foundation for analysis and future forecasts.

Section	No.	Requirement	What this requirement will solve	Evidence	Justification
	1.2	Prediction Engine	Generate future price forecasts based on past data		The tool's main analytical feature — converts raw data into useful insights. Without it, the system would only display information, not analyse it.
	1.3	Trading Alerts	Notifies users when to buy, sell, or hold		Simplifies decisions for users since they no longer need to constantly monitor the tool. This feature actively supports decision-making by providing clear, timely suggestions (buy/sell/hold).
	1.4	Automation (AI Agent)	Fetches data and runs predictions automatically		Removes the need for manual updates. This feature lets the system fetch data and run predictions independently, making it much more efficient and responsive to market changes.
	1.5	User Interface (UI)	Displays information and alerts users in a visual format		Makes the system accessible and clear. Without it, text-only outputs would be hard to interpret, especially for non-technical users.
	1.6	Visual Feedback	Uses colour and icons to show signal meanings		Enables quick interpretation of results. Without it, users would rely on reading values, slowing understanding and reducing engagement.
Technical Requirements	2.1	Programming language (Python or others)	Main language to build and run the project		Simple and powerful for machine learning and automation. Python's libraries, easy syntax and the ease of debugging makes it easier and more efficient to implement. Test and improve.
›	2.2	Data Handling	Prevents errors and improves prediction reliability		Good quality data means better and more trustworthy results. Clean, structured data ensures reliable model training and valid outcomes.
	2.3	Prediction Method	Creates the forecasting model		Essential for training, testing, and evaluating predictions. Without it, the program could not recognise or apply market patterns to unseen data. Including it gives the tool its analytical purpose.

Section	No.	Requirement	What this requirement will solve	Evidence	Justification
	2.4	Graphs and Charts	Make patterns and predictions easier to see		Converts raw numbers into readable visuals, saving time and improving interpretation. Without charts, users would struggle to understand patterns quickly.
	2.5	Simple Interface	Builds a visual interface for the tool		Makes the system more interactive and user-friendly and encourages more user engagement and active use.
	2.6	Saving Results	Saves trained models for reuse		Prevents retraining every session, improving performance and saving time. Without it, efficiency and model consistency would decrease.
	2.7	Automation	Runs automated tasks like daily updates		Reduces manual workload and ensures data is updated regularly. Without it, forecasts would quickly become outdated and less accurate. Improving accuracy and real time forecasts.
Data Requirements	3.1	Historical Price Data	Provides the basis for training and testing the model		The model learns from this data to forecast future prices. It's the basis for identifying patterns. Including it allows the system to recognise patterns over time, making future predictions more accurate and realistic.
	3.2	Length of Data	Determines how much the model can learn		Enough data is needed for the model to learn patterns, if data length is too short might become biased on those data and if those data are affected by any factor in the real world, the predictions won't be accurate on real time data.
	3.3	Currency Focus	Limits the scope to BTC and ETH		Reduces complexity and improves reliability. By focusing only on BTC and ETH, I ensure the tool remains accurate, easier to test, and more efficient to train and debug within the project's limited time.
AI Prediction Requirements	4.1	Training & Testing	Splits data to learn and evaluate the model		Without separating training and testing data, the model could overfit. performing well on known data but poorly on new inputs. Splitting the data helps evaluate how the model might perform in real-world conditions, improving reliability.

Section	No.	Requirement	What this requirement will solve	Evidence	Justification
	4.2	Model Choice	Selecting the simplest effective model		Keeps the project manageable and explainable. Complex models may be more powerful but harder to debug and justify. This ensures clarity and consistent performance within the project scope.
	4.3	Performance Check	Measures how well the model is predicting		Ensure the results are accurate. It allows me to know if the how well the model is performing, and whether the changes I apply improve it or are failing.
	4.4	Prediction Logic	Convert price inputs to forecasts		Translates numeric analysis into clear user actions (e.g., buy/sell signals). Without this, the output would be meaningless to users, reducing usefulness. Enhancing users decision making.
AI Agent (Automation) Requirements	5.1	Auto-fetch Data	Keeps the data updated without manual work		Makes the tool reliable and self-updating. Auto-fetching ensures it always works with fresh data. Maintaining prediction reliability on real time data.
	5.2	Run Model Automatically	Runs predictions without the user triggering it		Relying on users to trigger predictions could lead to missed opportunities. Running the model automatically ensures timely forecasts. It could be very beneficial for fast-moving markets like crypto.
	5.3	Update Alerts	Sends new recommendations automatically		Regular update notifications ensure users stay informed about new signals and don't have to constantly check manually. Without alerts, users might overlook critical buy/sell signals.
	5.4	Optional Notification	Could send alerts via email or display		Adding email or pop-up notifications keeps users engaged and responsive even if they're not on the app. Increasing user responsiveness and real-time awareness.
Design & Usability Requirements	6.1	Clean Layout	Makes the tool easy to read and use		Reduces confusion for users. Helping users trust the model. It also makes the tool feel professional.

Section	No.	Requirement	What this requirement will solve	Evidence	Justification
	6.2	Colour-coded Output	Show signals with colour (e.g. green = buy)		Colour-coded output (e.g. green for buy, red for sell) improves speed and clarity of communication, especially under time pressure where trades must be placed quickly to benefit from the opportunity.
	6.3	Font and Theme	Improves readability and user experience		A well-chosen font and consistent theme enhance readability, reduce eye strain, and make the tool more engaging. Without this, users may find the interface less clear or appealing, which would reduce usability.
	6.4	Chart Visualisation	Shows price movement and predictions		Chart visualisation helps users instantly spot price movements and prediction outcomes, increasing clarity and confidence in the system's results.
Testing & Feedback	7.1	Test on Past Data	Validates the model's accuracy		Proves that the system can recognise real-world patterns and generate realistic outputs. Builds trust in the model's performance and logic.
	7.2	Compare Predictions	Checks if the model behaves reasonably		Comparing predictions with real outcomes ensures that the tool responds realistically, shows the issues, and supports continuous improvement based on real evidence.

Success criteria

Table 5: Success criteria and justification

Requirement	Success Criteria	Justification

1.1 – Data input	System loads and processes historical data in 100% of test runs	Ensures the model always has access to the data it needs to learn and make predictions. If the data input fails, the forecasting model becomes unusable. By verifying that data is always correctly processed, the system remains reliable and consistent.
1.2 - Prediction Engine	Generates forecast values within +_%10	A forecast within this range shows that the system is learning useful patterns. If the predictions are outside this range, the tool loses credibility. Meeting this success criteria proves the model's usefulness and gives users a level of confidence in using its insights.
1.3 - Trading Alerts	Alerts are triggered accurately for Buy/Sell/Hold in 95% of relevant scenarios	Reliable alerts help users make timely decisions. If alerts trigger incorrectly or not at all, users may miss opportunities or make poor trades. Achieving this accuracy ensures the alert system supports confident and informed action.
1.4 - Automation (AI Agent)	Automation tool fetches data and runs model without user input in 90%+ attempts	This shows that the tool works independently. A 90%+ success rate proves the system is genuinely smart and reduces the workload on the user.
1.5 - User Interface (UI)	Interface displays results within 3 seconds and without layout errors	A fast, clean UI is essential for the user experience. If the layout breaks or loads slowly, users will get frustrated and stop using the tool. Quick results and smooth design make the system feel trustworthy.
1.6 - Visual Feedback	At least two colour-coded signals correctly represent model advice (green/red)	This ensures that visual cues are not just included but work as intended. Without this, users might misread or ignore model output.
2.1 - Programming language (Python or others)	All code executes correctly in Python environment with no dependency issues	Ensures the code runs consistently across devices for different users. A clean and portable setup increases the tool's usability.
2.2 - Data Handling	Data is cleaned and standardised before model use in 100% of tests	Poor quality data leads to poor predictions, which means data should always be cleaned and standardised. Cleaning and standardising the data before use means the model can learn correctly and perform consistently. It also reduces the chance of bugs or failed predictions.
2.3 - Prediction Method	Prediction model achieves minimum target accuracy (60%) with small dataset	A model that meets this target proves that the method is effective even without large amounts of data. For traders it means they make more profit than they lose, which is the exact goal. Also, this is important for

		the NEA project scope and shows that a working prototype has been achieved.
2.4 - Graphs and Charts	Charts render predictions and trends clearly for at least two currencies	Visualisations help users make sense of the predictions. If the chart is unclear or incorrect, the results are hard to interpret. Clear graphs improve usability and support better decision-making by clearly showing trends.
2.5 - Simple Interface	Interface elements work well in a simple way	A success criterion based on simplicity ensures the tool can be used comfortably by different users, even with limited experience. If all elements work smoothly in a straightforward layout, users are more likely to interact with the system regularly. This increases user satisfaction and reduces time spent learning or correcting errors.
2.6 - Saving Results	Saved results are reusable without retraining model from scratch	This confirms that once the model is trained, its results can be stored and reused. This saves processing time and avoids the need to repeat training every time the program runs. It shows that the system is efficient, and the model can run without constant developer involvement.
2.7 - Automation	Updates are triggered automatically each day or per user-defined schedule	This ensures the tool can operate without daily manual input. The success of this feature means the system stays current with the latest data, which is especially important for forecasting. It also shows that the solution less dependent on user involvement.
3.1 - Historical Price Data	At least 90 entries of historical price data collected for training/testing	This confirms that the dataset is large enough for the model to learn reliable patterns. A minimum entry count ensures the training data has enough variety to generalise predictions. It also allows the success of the model to be judged on realistic input size.
3.2 – Length of data	Data covers minimum of 3 months (60-90 entries) consistently	Using data that covers at least 3 months gives the model enough time to spot patterns and changes over time. This makes the predictions more accurate and reliable. Sticking to a 3-month range also means it's easier to test performance fairly each time.
3.3 – Currency focus	Only BTC and ETH used, switching between currencies functions correctly	Focusing on just BTC and ETH reduces data noise and simplifies model behaviour. Measuring the ability to switch correctly confirms that the

		system is flexible but still stable. It also shows the system handles scope limitations clearly without crashing or errors.
4.1 - Training & Testing	Training/testing split applied correctly in 100% of modelling attempts	This proves that the model has been built and evaluated properly in every test. If the split is consistent and correct, it builds confidence that the model has not been biased by overfitting. This is important for proving that the AI solution is generalisable.
4.2 - Model choice	Model used is basic (e.g., linear, average) and explained in documentation	Choosing a simple model and clearly explaining how it works proves that the tool is easy to understand. It also helps other users or teachers know what the model is doing, making it more transparent. If the model is too complex, it might be hard to justify or even troubleshoot, especially when things go wrong. A basic model also means less risk of overfitting, which keeps predictions more reliable.
4.3 - Performance Check	Prediction accuracy and error rate tracked for all test cases	Tracking how accurate the model is helps show if it's actually working. It's not enough to say the tool makes predictions. We need to prove it with evidence. Recording the error rate for every test builds confidence that the results aren't just random. This also allows comparison between different models or updates to see if improvements are working.
4.4 – prediction Logic	Price forecasts correctly matched with action advice 90%+ of time	If the prediction and suggested action (e.g. Buy/Sell) match up most of the time, it proves that the logic behind the AI agent is functioning as intended. This is key to making the tool actually usable, not just experimental. If it fails this, the whole system loses reliability and trust from users, so measuring this success is essential.
5.1 - Auto-fetch Data	Data automatically refreshed once per day or on trigger without fail	The success of this feature proves the system is fully automated and doesn't depend on the user to manually update things. This makes the tool more scalable and suitable for longer-term use. It also means users can trust the results to be current, which is especially useful in fast-moving markets like crypto.
5.2 - Run Model Automatically	Model runs on schedule or after each data refresh	This success criteria shows that the whole cycle, from data to prediction, is automatic. It removes the need for the user to constantly click buttons or trigger updates, which makes the tool feel more like a real-world

		solution. It also reduces the chance of human error or forgetting to update.
5.3 – Update Alerts	Updates delivered to user interface with new prediction every refresh	Making sure alerts show up each time the model updates proves that the system is reliable and responsive. It also helps users know when something has changed without having to guess. If updates were missing or delayed, the tool wouldn't be very useful, so this is a key thing to measure.
5.4 - Optional Notification	Optional notification feature tested and working for each alert type at least twice	This proves that users are able to receive information in different ways (like popup alerts or by telegram), which improves the flexibility of the system. Testing all alert types more than once shows they don't just work once by luck — they're reliable. This is important if the tool is meant to be used regularly or by different types of users.
6.1 - Clean Layout	UI uses consistent spacing, and is readable on multiple screens	Having a consistent and readable layout means the system looks professional and is easy to use. If it works well across different screen sizes (e.g. laptop, desktop), it's more accessible and feels higher quality. It also reduces frustration, which is important for keeping users engaged.
6.2 - Colour-coded Output	At least 3 colours clearly convey Buy/Sell/Neutral	Using colour to show actions helps users understand results instantly without reading lots of text. Measuring success based on recognisable colour signals proves the tool can communicate effectively. This makes it quicker to use, especially for people who want fast decisions.
6.3 - Font and Theme	Font and theme adjustable	This shows the tool can be customised to user preferences, which is important for accessibility and personalisation. If users can choose a darker theme or larger font, it improves the experience — especially for people who need better readability. This adds polish and makes the tool more complete.
6.4 - Chart Visualisation	Charts correctly visualise data for all selected timeframes	This ensures the charts are flexible and accurate no matter the date range selected. It shows that the system can adapt and still present useful, readable insights. If this doesn't work, users may misunderstand the trends, so it's important to confirm this works every time.

7.1 - Test on Past Data	Model accuracy on past data >60%; matches expected trend direction	Using past data is one of the only ways to prove if the model works. Getting more than 60% accuracy and seeing the correct trend direction shows that the system understands historical patterns and isn't guessing. This gives users confidence in its ability to predict future trends.
7.2 - Compare Predictions	Past vs actual comparison tracked; deviation explained in system notes	Tracking how predictions compare to actual values proves whether the tool is behaving realistically. If the results are close, that's evidence the logic is working. If they're off, the notes help explain why — maybe the market changed or there was unexpected volatility. Either way, this helps improve the system.

Limitations

1. Lack of Real-Time Data Integration

One of the most significant limitations of this project is the inability to integrate or test the AI model with live financial data. The current system works entirely with historical datasets that are static and fixed once loaded. While this data is sufficient for training and offline testing, it does not capture the fast-paced, unpredictable movements that define real-world cryptocurrency markets. Live market data often changes by the second, reflecting shifts in trading sentiment, global events, or algorithmic activity, which cannot be fully replicated using pre-collected datasets. Implementing real-time integration would require continuous API connections to platforms such as Binance, Coinbase, or TradingView, along with robust error-handling to maintain stable communication. However, achieving this within the time and

resource limits of my project is not realistic. It would demand advanced networking, asynchronous data streaming, and possibly cloud-based hosting to keep the system updated and responsive at all times — capabilities that go beyond the hardware and time constraints of my project.

The absence of live data means that while the model's predictions may be accurate on past information, they do not necessarily represent how the system would perform in a constantly changing market. Without real-time adaptation, the AI cannot detect or respond to immediate shifts such as price spikes, flash crashes, or external triggers like economic announcements. As a result, the model behaves as a static forecasting tool rather than a dynamic trading assistant. This limits its practical use for active traders, who require up-to-the-minute insights. It also affects the testing stage: the evaluation of accuracy is restricted to historical simulations rather than true market performance. Consequently, the results are theoretically valid but not verified under real-world trading conditions, meaning the model's reliability and responsiveness remain partially untested.

To reduce this limitation, I plan to use recent, high-frequency datasets (covering the last 30–90 days of daily or hourly data) so that the model still reflects near-current market patterns. This creates a realistic simulation of market behaviour while remaining within the project's technical capabilities. The system will process and analyse this data in the same structure that a real-time model would use, allowing for smoother future upgrades. Furthermore, the program will be

written in a modular format — separating the *Input Data* and *Prediction Engine* components — so that API integration can be added later without major restructuring. I will also include placeholder functions and comments that outline where live API calls could be inserted, effectively making the system "future-ready." During testing, I will compare recent predictions to live prices from public sources (e.g., CoinMarketCap) to estimate how closely the model's outputs align with reality.

This mitigation strategy provides a balanced and practical solution within the scope of this project. By using datasets that are both recent and representative, the tool gains a degree of relevance to current markets without the instability or resource demands of real-time updates. The modular design also demonstrates forward planning — ensuring that if future versions of the system are developed, real-time functionality can be added with minimal changes to existing code. Although the lack of real-time integration remains a limitation, this approach ensures the program is still realistic, scalable, and meaningful as an educational and analytical prototype. It also allows me to focus my development time on perfecting accuracy, interface design, and model evaluation, which are more achievable within the A-Level timeframe.

2. Limited Forecasting Techniques

A key limitation of my project is that it uses relatively simple forecasting techniques, including moving averages and linear regression, instead of more sophisticated machine-learning models such as recurrent neural networks (RNNs) or long short-term memory (LSTM) networks. While

these advanced models are designed to recognise long-term dependencies and adapt to non-linear behaviour in data, they also require extensive coding knowledge, large datasets, and longer training times. As a student project with a limited timeframe and computing resources, these methods were beyond the achievable scope. Simpler algorithms such as moving averages and regression are easier to implement, interpret, and evaluate, but they inevitably provide less accurate and less responsive predictions — particularly for highly volatile markets such as cryptocurrencies, where price behaviour changes rapidly and is influenced by multiple, interdependent variables.

Using basic models restricts the system's ability to capture sudden reversals, momentum shifts, or complex temporal relationships between variables such as price, volume, and sentiment. As a result, predictions may appear overly smoothed or delayed compared to real-world market activity. This affects both the accuracy and the responsiveness of the AI tool. Furthermore, the limited modelling depth means that my tool cannot generalise across different assets or adapt automatically to regime changes in the market. Although it performs consistently on historical data, its predictive power in fast-moving conditions is inherently constrained.

To lessen this limitation, I have adopted a hybrid computational approach that combines multiple simple indicators — such as linear regression, moving-average crossovers, and RSI trend detection — to strengthen accuracy and responsiveness without introducing unnecessary

complexity. This ensemble-style method balances ease of implementation with modest performance improvements. I have also designed the program in a modular and extensible structure, meaning future developers (or myself, at a later stage) can easily add more advanced algorithms such as LSTM or ARIMA without rewriting the core logic. Each module is clearly separated (data input, prediction, and visualisation) and well documented, allowing upgrades or replacements of specific models. Finally, I will include quantitative evaluation metrics such as prediction error rates and correlation analysis, providing measurable evidence of how the model performs within its current limitations.

This mitigation is appropriate for the project's scale and educational purpose. It ensures that the system remains functional, explainable, and testable within the constraints of an A-Level project while still demonstrating computational depth. The modular, documented design shows forward thinking, as it provides a foundation for integrating more advanced models later on. Combining multiple simple algorithms also allows for incremental improvement in accuracy without overwhelming system complexity. Although advanced machine-learning techniques were excluded, this decision was intentional to prioritise understanding, reproducibility, and robustness — qualities that make the project technically sound and pedagogically valuable.

3. Basic User Interface Design

Another limitation of my project is the simplicity of the graphical user interface (GUI). Although the interface will be functional and easy to understand, it may lack the level of interactivity and visual sophistication expected in modern trading platforms. Features such as dynamic charts, multi-window layouts, dark/light mode toggles, and advanced user controls (like drag-and-drop interaction or custom dashboard settings) are not implemented due to time and experience constraints. As a developing programmer, my current skill level and access to design software limit my ability to build highly polished or interactive designs. The GUI is therefore built using the standard Python Tkinter library, which is excellent for creating basic layouts but offers limited support for complex animations or responsive resizing. While this makes the tool stable and accessible, it also means that the visual appearance may not fully meet the expectations of professional users or replicate the seamless experience of commercial trading platforms.

This limitation affects the project's usability and first impressions. Although the interface performs its function correctly, its basic design might feel outdated or simplistic compared to industry tools like TradingView or Cryptohopper. This could reduce perceived credibility or engagement for users who are accustomed to modern dashboards with smooth transitions and interactive chart controls. A simple interface can also restrict the system's scalability — adding new features later may require redesigning the layout rather than expanding it modularly.

Additionally, the lack of visual enhancements might make prolonged use less appealing, even if the underlying functionality is accurate and effective.

To reduce this limitation, I have carried out informal usability testing by asking classmates and teachers to evaluate the GUI's layout, navigation flow, and readability. Their feedback has guided several improvements — such as repositioning key buttons for easier access, enlarging prediction labels for visibility, and improving spacing and colour contrast for readability. I have also applied core design principles such as alignment, consistency, and visual hierarchy to create a structured interface where information appears in predictable and easy-to-scan sections. Accessibility is also prioritised: I have ensured that critical data is not conveyed by colour alone and that all visual signals (e.g., buy/sell indicators) are supported by corresponding text labels or tooltips. Furthermore, I have used contrasting colours, clear typography, and minimalist iconography to produce a clean and professional aesthetic without advanced graphics.

These mitigation steps reflect a practical and thoughtful approach to UI design within the limits of an A-Level project. Even without advanced visual features, applying usability principles ensures that the system remains clear, accessible, and functional for its intended users. The use of peer testing demonstrates user-centred design thinking, while careful layout structuring shows understanding of how computational design supports readability and efficiency. Although the GUI may not visually match commercial products, it prioritises function and clarity

— two qualities that make it more usable, testable, and reliable within this project's technical and time constraints.

4. Narrow Cryptocurrency Focus (Bitcoin and Ethereum Only)

A key limitation of the project is its limited focus on just two cryptocurrencies — Bitcoin (BTC) and Ethereum (ETH). This decision was made to keep the project's scope manageable and ensure data quality, as these two assets are the most established, liquid, and widely analysed in the market. However, it also restricts the system's usefulness for users who wish to apply it to a broader range of cryptocurrencies such as altcoins or emerging tokens. Each cryptocurrency often has unique characteristics and volatility patterns driven by factors such as supply mechanisms, community influence, and project development. This means that a model trained only on BTC and ETH data may not generalise well to other assets. While this narrow focus helps maintain technical feasibility and cleaner datasets, it does limit the system's flexibility and comprehensiveness.

This limitation reduces the program's applicability for advanced traders or analysts who follow multiple coins. For instance, altcoins often exhibit sharper volatility and different correlation patterns compared to Bitcoin and Ethereum. Because the system is trained on assets with high liquidity and well-structured data, it may not perform as accurately if adapted to smaller or less

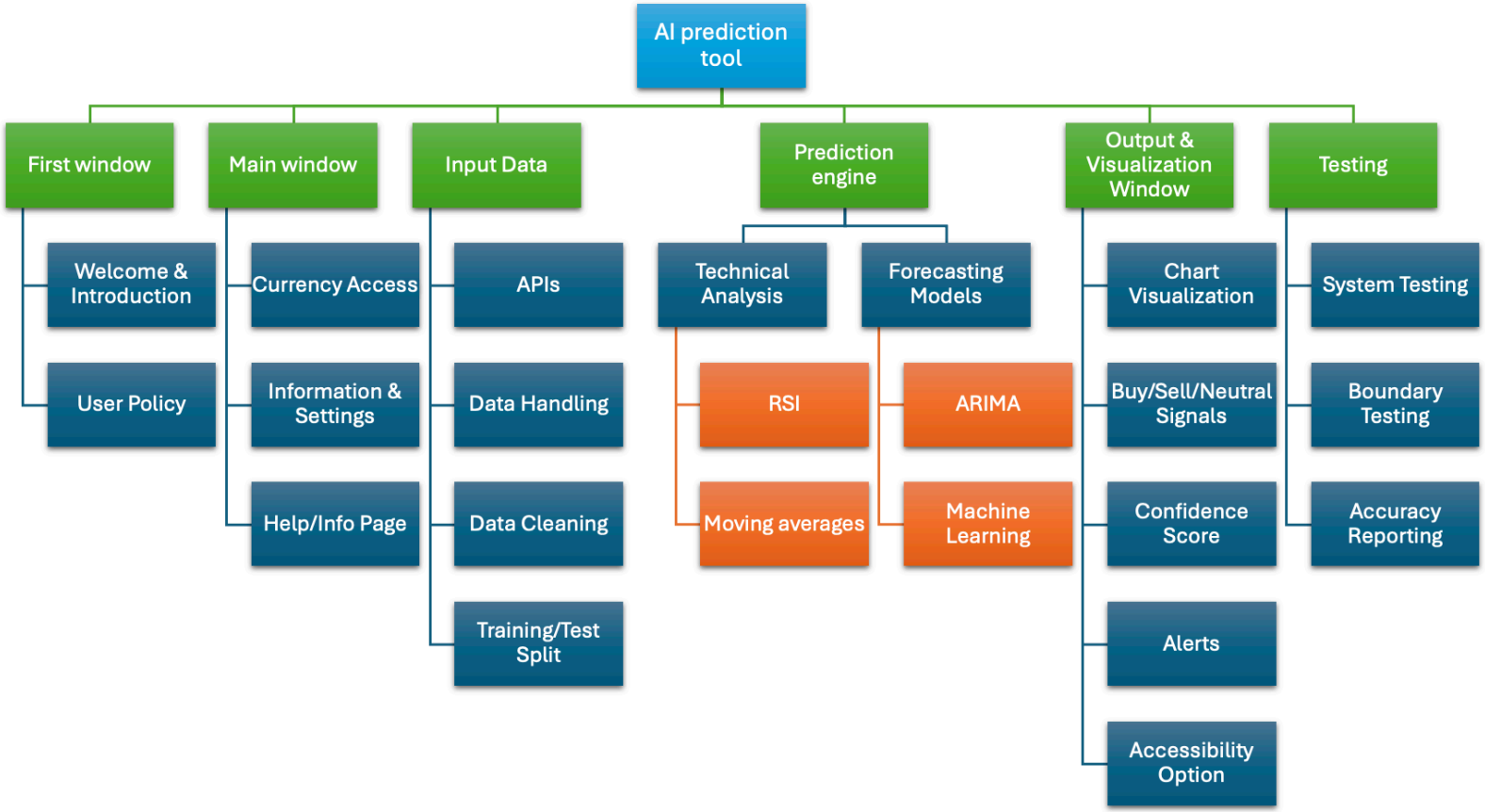
stable markets. The restriction also means that the tool cannot yet provide portfolio-level analysis or cross-asset comparisons, which limits its real-world commercial potential. In practical terms, the current system serves as a strong proof of concept rather than a fully featured market prediction platform.

To reduce this limitation, I deliberately selected Bitcoin and Ethereum because they act as benchmark assets that influence broader market behaviour. Both coins are highly correlated with the movements of most other cryptocurrencies, meaning their analysis still provides meaningful insights into general market sentiment. I will also design the system using generic data structures and indicator logic, ensuring that functions for data input, prediction, and visualisation are not hard-coded to these two coins. This will allow future versions to include additional assets easily. For example, by modifying the input API or data file, the system could accommodate altcoins with minimal code changes. Furthermore, the use of standard technical indicators (RSI, moving averages, regression) makes the prediction engine transferable to any asset type that provides time-series price data.

This approach balances focus and future scalability. Restricting the initial version to two major cryptocurrencies ensures accuracy, clean data handling, and reliable testing within the limits of the project. At the same time, designing the code to be modular and asset-agnostic demonstrates strong computational planning — allowing the system to expand later without needing major redesign. This shows technical foresight and responsible project management,

as it maintains achievable scope while ensuring the program remains relevant, generalisable, and adaptable for future iterations or wider datasets.

Top-down design



The top-down design of my AI cryptocurrency price prediction tool breaks the overall system into six main modules: First Window, Main Window, Input Data, Prediction Engine, Output & Visualisation

Window, and Testing. Each module is further decomposed into smaller, manageable submodules such as RSI, Moving Averages, ARIMA, Machine Learning, API connections, Data Cleaning, Chart Visualisation, Buy/Sell Signals, and Accuracy Testing. This hierarchical structure ensures that the system is developed in a clear, logical, and modular way, where each component performs a specific function that contributes to the overall goal. By decomposing the project into these layers, the design supports easier debugging, testing, and future expansion — for example, additional forecasting models or cryptocurrencies can be integrated without rewriting existing modules. This structured approach also aligns with the spiral development model, allowing iterative refinement of each module while maintaining consistency and traceability throughout the development process.

Prototypes

Prototype 1 – Interface and Data Input Foundation

The first prototype focuses on creating the core structure and appearance of the program — designing the Graphical User Interface (GUI) and implementing the basic data input functions. At this stage, the aim is to develop the visual layout that users will interact with, ensuring it is clear, functional, and intuitive before introducing prediction logic. The main components built here include the First Window (Welcome & Introduction, User Policy), the Main Window (Currency Access and Information & Settings), and the Input Data modules (APIs, Data Handling, and Data Cleaning). The

GUI will feature a simple navigation system, labels, and buttons that allow the user to load or fetch cryptocurrency data (such as Bitcoin or Ethereum prices) through either a CSV file or a live API connection. Testing in this stage will focus on ensuring that user interface elements respond correctly — for example, buttons load data accurately, the layout displays values in a readable way, and invalid inputs trigger appropriate error messages.

The justification for starting with this prototype is that the GUI and data-handling structure form the foundation for all later development. Before the AI can make predictions, the system must first be able to accept, clean, and organize input data in a consistent format. Additionally, designing the interface first ensures that usability, accessibility, and layout considerations are addressed early on, allowing feedback from testers to improve design before more complex features are added. Developing the front-end elements first also supports incremental learning and testing — I can confirm that data flow, visual updates, and user interactions work properly before integrating prediction algorithms. This aligns with the spiral development model by establishing a strong user-facing base that can be refined and expanded as functionality grows.

Prototype 2 – Prediction Engine and Decision Logic

The second prototype focuses on building the analytical core of the system — enabling the AI to process the imported data, identify patterns, and generate predictions. At this stage, the tool will implement two key forecasting approaches: Moving Averages (MA) and the Relative Strength Index (RSI). These form the backbone of the Prediction Engine, translating historical price data into trend-

based predictions and clear trading signals such as Buy, Sell, or Hold. This prototype also introduces the logic for calculating confidence scores, which quantify how reliable each prediction is based on recent market behavior. Data processed through the interface in Prototype 1 will now be used to produce numerical and visual outputs displayed on charts within the GUI. Testing will involve validating that the algorithms perform calculations accurately, comparing predicted trends with actual data, and confirming that the corresponding colour-coded alerts (green for Buy, red for Sell) display correctly on the interface.

The justification for this prototype is that it transforms the tool from a static interface into an intelligent analytical system capable of assisting user decisions. By introducing RSI and Moving Average calculations at this point, the project develops from data management into practical technical analysis. This phase also allows structured testing of computational accuracy — ensuring mathematical formulas and trend logic are implemented correctly before automation or machine learning are added. Developing the predictive logic separately from data input also improves modularity, making it easier to test or modify algorithms independently. Overall, this prototype represents the system's core intelligence and demonstrates a key milestone in the spiral development process: integrating computation with interactivity.

Prototype 3 – Automation, Evaluation and Refinement

The final prototype focuses on enhancing the system's intelligence and independence through automation, advanced modelling, and performance evaluation. At this stage, the program will

incorporate machine learning models such as linear regression or ARIMA within the Forecasting Models module to improve predictive accuracy. The system will also be made partially autonomous — automatically fetching updated data at set intervals, running the prediction model, and updating visual outputs such as charts and alerts without requiring manual input. Additional usability and accessibility features, such as text-based messages for colour-blind users, alert notifications, and smoother GUI transitions, will also be implemented. The Testing module will be expanded to include System Testing, Boundary Testing, and Accuracy Reporting, ensuring that all elements — from data input to prediction and visualisation — function together reliably.

The justification for this final prototype is that it represents the completion of the system as an integrated and intelligent AI agent. Automation ensures the program behaves more like a real-world financial prediction tool, capable of updating and advising users independently. Introducing machine learning also raises the analytical sophistication of the system, allowing it to detect patterns that basic indicators may miss. Finally, this phase emphasizes comprehensive testing and evaluation against the project's success criteria, ensuring the program is not only functional but reliable, efficient, and user-friendly. Through this final cycle of refinement and evaluation, the project achieves its full purpose — delivering an automated, data-driven, and accessible cryptocurrency prediction tool that users can trust and understand.

Structure

New Process 1: Updating Bitcoin Price

This procedure is responsible for fetching and displaying the latest live Bitcoin price in the program interface. Its purpose is to ensure that the user always sees the most recent market data, which can then be compared against the model's predicted price. The procedure retrieves the data through an HTTP request to a free public cryptocurrency API using the requests library, converts the response to JSON, and extracts the current BTC/USDT price. The value is formatted to two decimal places for clarity and then displayed within a GUI label. The function also captures the exact update time using the datetime library so that the user knows when the data was last refreshed. Conditional statements compare the current price with the previous stored value: if higher, the text colour changes to green; if lower, red; if unchanged, black. This provides instant, colour-coded visual feedback on market movement.

Justification

This approach ensures the program maintains live accuracy without manual refresh, which is essential in a rapidly changing crypto market. By separating this task into its own function, the design remains modular, meaning the live-update feature can be tested or modified independently of prediction logic. Using the requests library and JSON parsing keeps the code concise, readable, and easy to debug. Colour-coded feedback enhances usability and accessibility, allowing users, especially beginners, to interpret market direction instantly. The inclusion of a timestamp reinforces

transparency and reliability. Altogether, this method provides a fast, clear, and maintainable way to deliver real-time data within the interface.

Table 6: Key Variables and Data Structure

Variable / Data Structure	Data Type	Justification / Validation
Api_status	int	Stores HTTP response status (e.g. 200); used to verify successful API communication before processing data.
price	float	Stores the latest fetched Bitcoin price; allows arithmetic comparison with previous value; **validation:** checked to ensure numeric and non-negative.
last_price	float	Holds the previous cycle's price so the function can determine whether the market price rose or fell; initialized on first call.
colour	string	Determines GUI text colour ("green", "red", "black") based on comparison result; **validation:** checked against a predefined set of colors.
now	string	Records formatted current time of last update (HH:MM:SS); ensures user can verify data recency.
response	dictionary (JSON)	Holds raw API response; **validation:** checked for HTTP 200 status and non-null fields before parsing.
requests	module (import)	External library for fetching live data; chosen for reliability and ease of integration with JSON.

New Process 2: Main Window

This procedure is responsible for building and displaying the central interface of the AI prediction tool. This window acts as the main hub where users can access all key system functions, including data input, prediction, and visualisation. The procedure creates the layout using Tkinter, arranging

labels, frames, and buttons with a dark-themed colour scheme to match user preferences identified during the survey. It includes a live price display, navigation buttons for accessing currency windows, and icons for clarity. Event-driven commands are attached to buttons so that each one triggers its corresponding function (for example, fetching data or updating prices). The layout is carefully structured with grid and pack geometry managers to ensure that widgets are aligned consistently across different devices or window sizes.

Justification

This procedure forms the backbone of the graphical user interface, making the system accessible and visually intuitive. By keeping interface creation in a single, clearly defined function, the design follows modular principles — allowing future updates (such as adding a new feature or adjusting layout) without affecting the rest of the code. The use of Tkinter was chosen for its simplicity, built-in functionality, and cross-platform compatibility, which fits the project's educational and lightweight goals. The inclusion of labelled buttons and consistent dark-mode aesthetics enhances usability and readability, while clear iconography helps beginners navigate without prior financial knowledge. Structuring this code within one main function also simplifies debugging, since all interface elements are initialized and referenced in one place.

Table 7: Key Variables and Data Structure

Variable / Data Structure	Data Type	Justification / Validation
title_label	Label	Displays program title and version; uses high-contrast font for accessibility.
price_label	Label	Shows current Bitcoin/Ethereum price in real time; **validated** by checking it updates after each refresh.
btn_update	Button	Executes update_bitcoin_price() when clicked; **validation** ensures callback is correctly linked.
btn_predict	Button	Triggers the prediction module; disabled until valid data is loaded to prevent runtime errors.

theme_colour	string	Defines the dark-mode colour palette; **validated** against a set of acceptable colour hex codes.
icon_images	Image Object	Holds image objects for GUI icons; ensures they are loaded successfully before rendering to avoid blank buttons.

New Process 3: Moving Average

This procedure calculates a Moving Average (MA) over a chosen window (e.g., 2/5/10 periods) from the cleaned historical price series and returns both the MA series and a short-term predicted price (by extending the last MA value). This provides a noise-reduced trend line that the interface can plot, and the decision logic can compare against the latest close to output Buy/Sell/Hold signals.

Justification

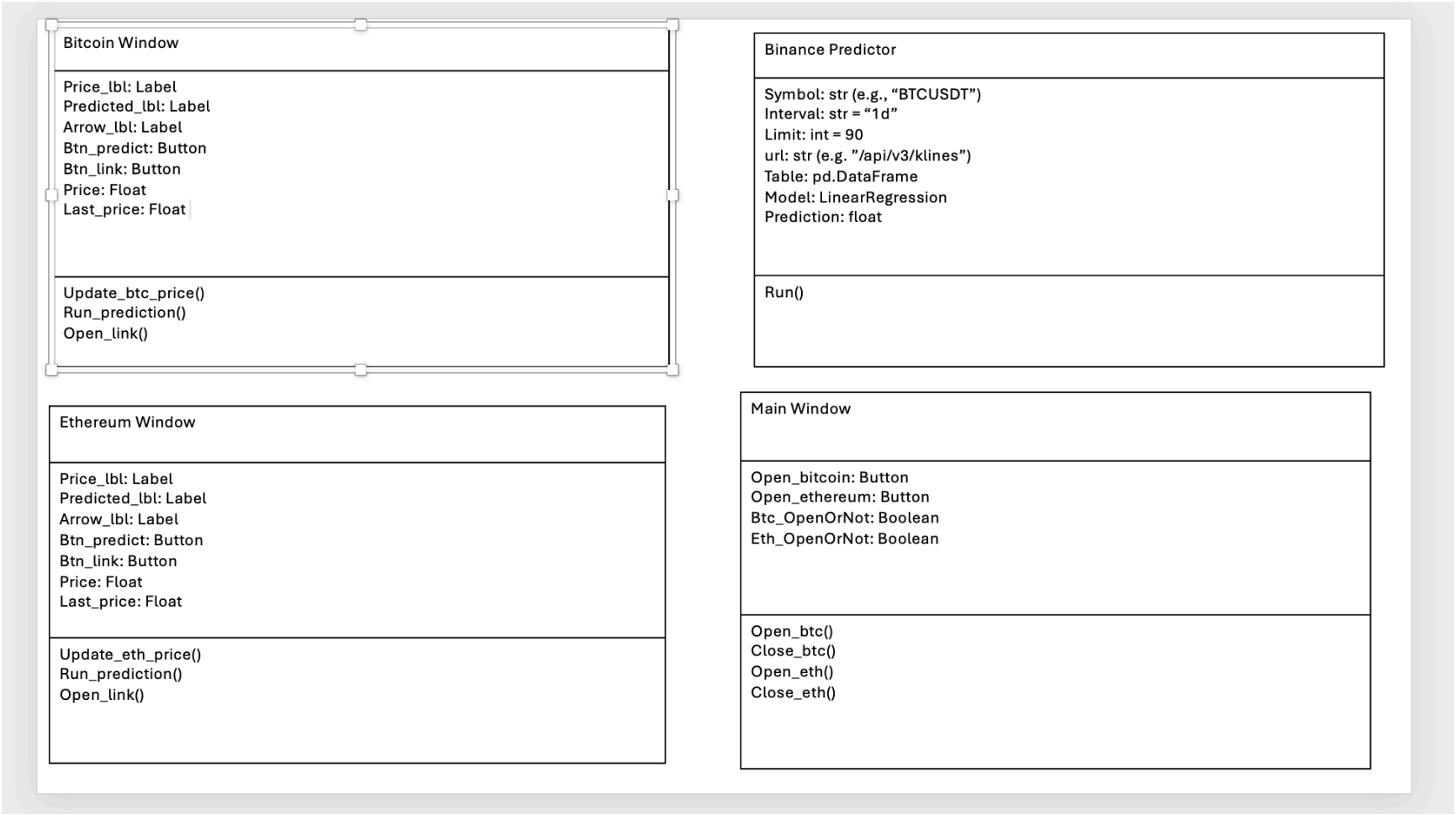
MA is computationally light and transparent. It smooths volatility, is quick to test, and provides a baseline to compare with RSI and any later model (e.g., ARIMA). Keeping the calculation in its own function improves modularity (swap window sizes, reuse for BTC/ETH) and testability (unit tests with fixed inputs and known outputs). Input validation (length and numeric checks) prevents runtime errors and misleading averages.

Table 8: Key Variables and Data Structure

Variable / Data Structure	Data Type	Justification / Validation
prices	list[float]	Input closing prices; numeric series required; **validated** for length ≥ window and non-negative floats.
ma_values	list[float]	Stores rolling MA output for plotting; **validated** against manual sample.
predicted_price	float	Uses the last MA as short-term forecast; simple, explainable baseline; **validation:** checked for not NaN.

valid_length	bool	Gate to prevent compute when series too short; if False → return message to UI instead of raising error.
asset	str	Ticker context (“BTC”/ “ETH”) to label outputs; **validated** against allowed symbols.

Data Design


Bitcoin Window
Price_lbl: Label
Predicted_lbl: Label
Arrow_lbl: Label
Btn_predict: Button
Btn_link: Button
Price: Float
Last_price: Float
Update_btc_price()
Run_prediction()
Open_link()
Binance Predictor
Symbol: str (e.g., "BTCUSDT")
Interval: str = "1d"
Limit: int = 90
url: str (e.g. "/api/v3/klines")
Table: pd.DataFrame
Model: LinearRegression
Prediction: float
Run()
Ethereum Window
Price_lbl: Label
Predicted_lbl: Label
Arrow_lbl: Label
Btn_predict: Button
Btn_link: Button
Price: Float
Last_price: Float
Update_eth_price()
Run_prediction()
Open_link()
Main Window
Open_bitcoin: Button
Open_ethereum: Button
Btc_OpenOrNot: Boolean
Eth_OpenOrNot: Boolean
Open_btc()
Close_btc()
Open_eth()
Close_eth()

Algorithms

The overall aim of the AI crypto prediction tool is to allow users to view live cryptocurrency prices and receive automated buy/sell/hold predictions based on historical data and technical indicators. The six algorithms below form a complete solution by managing data input, analysis, prediction, and output presentation in a clear, modular structure.

Overall Algorithmic Architecture

The solution is structured using a modular, layered architecture consisting of:

1. Interface Layer – Handles user interaction and display.
2. Data Layer – Retrieves and validates live cryptocurrency data via API.
3. Analytical Layer – Processes historical data and generates predictions using statistical and machine-learning models.
4. Decision Layer – Converts quantitative forecasts into actionable trading signals.
5. Feedback Layer – Updates the interface dynamically and visually communicates results.

Each algorithm operates within one of these layers and interacts through defined inputs and outputs. This separation ensures maintainability, scalability, and logical completeness. Together, the algorithms form a pipeline: user selection → data retrieval → prediction computation → signal generation → visual feedback.

This structured flow ensures the system fully addresses the project objective of predicting cryptocurrency prices and presenting interpretable decision support in real time.

Algorithm 1: Building the Main Window

```
procedure buildMainWindow()
// Creates the main GUI window and assigns buttons
root = new Window("AI Crypto Predictor")
    theme = "dark"
    titleLabel = new Label(root, "AI Crypto Price Prediction")
    priceLabel = new Label(root, "--")

    btnOpenBTC = new Button(root, "Open Bitcoin")
    btnOpenETH = new Button(root, "Open Ethereum")
    btnQuit = new Button(root, "Exit")

    // Event handlers
    onClick(btnOpenBTC, openBitcoinWindow)
    onClick(btnOpenETH, openEthereumWindow)
    onClick(btnQuit, closeApp)

    // Layout
    place(titleLabel, "top")
    place(priceLabel, "below")
    placeRow(btnOpenBTC, btnOpenETH, btnQuit)

    show(root)

endprocedure
```

This algorithm is the foundation of the user interface and therefore the entry point to the entire program. It constructs the main window and connects each button to its corresponding event, such as opening cryptocurrency sub-windows or closing the application. This structure ensures users can navigate easily and access the prediction functions intuitively. The algorithm also follows a modular design principle—interface management is separated from data and prediction logic, improving maintainability and supporting future updates such as new currencies or additional features.

Justification and Role in Complete Solution

This algorithm establishes the user interaction layer. It links interface controls to functional modules (e.g., opening asset windows, running predictions, exiting the system). By separating interface construction from analytical logic, the system follows modular design principles, improving maintainability and extensibility.

Without this algorithm, the system would lack structured navigation, preventing user access to predictive features. It therefore forms the foundation of the complete solution.

Algorithm 2: Update Live Price

```
procedure updateLivePrice(asset, priceLabel, lastPrice)
// Fetches the latest live price and updates GUI colour

    url = "https://api.binance.com/api/v3/ticker/price?symbol=" + asset
    response = httpGet(url)

    if response == null then
        print("Connection failed")
        setText(priceLabel, "Data unavailable")
        return
    endif

    data = parseJSON(response)
    currentPrice = float(data["price"])

    // Compare with previous value
    if lastPrice == null then
        colour = "black"
    elseif currentPrice > lastPrice then
        colour = "green"
    elseif currentPrice < lastPrice then
        colour = "red"
    else
        colour = "black"
    endif

    setColour(priceLabel, colour)
    setText(priceLabel, "£" + str(currentPrice))
    lastPrice = currentPrice

endprocedure
```

This algorithm ensures that the displayed prices for Bitcoin or Ethereum remain accurate and up to date. It fetches real-time data through an API call and updates the GUI with the latest price values while using colour-coded feedback (green for increase, red for decrease, black for unchanged). This gives users immediate visual insight into market direction without needing external tools. Validation steps prevent crashes from failed API connections, ensuring robustness and reliability. By maintaining live accuracy and usability, this algorithm directly supports the project's aim of providing dependable, real-time financial information.

Justification and Role in Complete Solution

This algorithm ensures data accuracy and real-time relevance. It validates API responses to prevent system crashes due to network failure. The colour-coded feedback improves usability by providing immediate visual interpretation of price movement.

Live pricing is essential because prediction signals depend on accurate current values. This algorithm therefore supplies the baseline data required by the analytical and decision layers, making it critical to the completeness of the solution.

Algorithm 3: Run Prediction for the asset

```
procedure runPredictionForAsset(asset, predictedLabel, arrowLabel)
// Runs the Binance predictor and updates the interface

    predictedPrice = binancePredictorRun(asset, "1d", 90)
    if predictedPrice == null then
        setText(predictedLabel, "Prediction failed")
        hide(arrowLabel)
        return
    endif

    currentPrice = getCurrentPrice(asset)
    difference = predictedPrice - currentPrice
    threshold = 0.002 * currentPrice   // 0.2%

    if difference > threshold then
        signal = "BUY"
        setArrow(arrowLabel, "UP", "green")
    elseif difference < -threshold then
        signal = "SELL"
        setArrow(arrowLabel, "DOWN", "red")
    else
        signal = "HOLD"
        setArrow(arrowLabel, "FLAT", "black")
    endif

    setText(predictedLabel, "Predicted: " + str(predictedPrice) + "  [" + signal + "]")

endprocedure
```

This algorithm links live data with the program's analytical core. It calls the prediction model, compares the predicted and current prices, and decides whether to display a Buy, Sell, or Hold signal. The algorithm also ensures visual consistency by updating GUI elements with clear indicators such as arrows and colour feedback. Its threshold logic avoids reacting to trivial fluctuations, creating more stable and meaningful predictions. This bridges the gap between computational output and user understanding, making AI predictions practical and actionable for decision-making.

Justification and Role in Complete Solution

This algorithm acts as the bridge between analytical computation and user interpretation. It integrates outputs from the prediction model and feeds them into the decision algorithm.
By linking the analytical layer to the interface layer, this module ensures the system is not purely computational but user-facing and actionable. It completes the transition from raw data to decision-support output.

Algorithm 4: Fetch data from Binance and form a Linear Regression Model

```
function binancePredictorRun(symbol, interval, limit)
// Fetches price data, trains regression, returns prediction
    url = "https://api.binance.com/api/v3/klines"
    parameters = { "symbol": symbol, "interval": interval, "limit": limit }

    rawData = httpGet(url, parameters)
    if rawData == null then
        return null
    endif

    table = toDataFrame(rawData)
    closes = column(table, "close")
    n = length(closes)

    if n < 10 then
        return null
    endif

    // Create regression model
    x = array(n)
    for i = 0 to n-1
        x[i] = i
    next i

    model = linearRegressionFit(x, closes)
    nextX = n
    predicted = linearRegressionPredict(model, nextX)

    return predicted

endfunction
```

This is the core data-processing and machine-learning routine that powers the predictive functionality of the tool. It downloads recent market data, structures it into a table, trains a Linear Regression model, and produces a next-day forecast. Validation ensures a minimum data length before training, preventing unreliable outputs. The algorithm's modular design allows it to be reused for different cryptocurrencies by simply changing the input symbol. By encapsulating the entire prediction workflow—data retrieval, preparation, training, and forecasting—it provides the technical foundation of the project's AI component.

Justification and Role in Complete Solution

This algorithm forms the analytical core of the system. It transforms raw historical time-series data into predictive insight.

Linear Regression was selected because:

- It is computationally efficient (O(n)).
- It provides interpretable outputs.
- It performs reliably on sequential numeric datasets.
- It avoids overfitting compared to complex neural networks.

Minimum dataset validation prevents unreliable training. Without this predictive engine, the system would only display live prices rather than forecast future movement. This module directly satisfies the primary project objective: AI-based cryptocurrency price prediction.

Algorithm 5: Moving Averages Model

```
function simpleMovingAverage(prices, periodLength)
// Calculates average over fixed window

    if periodLength < 2 or periodLength > length(prices) then
        return []
    endif

    maValues = []

    for i = 0 to length(prices) - periodLength
        total = 0
        for j = 0 to periodLength - 1
            total = total + prices[i + j]
        next j
        average = total / periodLength
        append(maValues, average)
    next i

    return maValues

endfunction
```

The simpleMovingAverage() algorithm provides a baseline statistical method for identifying trends in price data. It smooths short-term fluctuations by averaging recent prices over a defined period. This algorithm supports visualisation features and can act as a secondary indicator for verifying the model's predictions. The simplicity and interpretability of the moving average make it a reliable benchmark for testing more advanced models like Linear Regression. This supports accuracy testing and evaluation by allowing direct comparison between basic and AI-generated forecasts.

Justification and Role in Complete Solution

The Moving Average provides a statistical baseline for trend detection. It improves robustness by allowing model comparison and visual verification of regression predictions.
Including both a statistical smoothing method and a machine-learning model ensures analytical depth and reduces reliance on a single forecasting technique. This multi-method approach strengthens the reliability and completeness of the system.

Algorithm 6: Generating Signal

```
function generateSignal(currentPrice, predictedPrice, bandRatio)
// Converts predicted movement into BUY, SELL or HOLD

    difference = predictedPrice - currentPrice
    threshold = bandRatio * currentPrice

    if difference > threshold then
        signal = "BUY"
    elseif difference < -threshold then
        signal = "SELL"
    else
        signal = "HOLD"
    endif

    // confidence from absolute difference scaled to ±5%
    maxBand = 0.05 * currentPrice
    confidence = abs(difference) / maxBand

    if confidence > 1 then
        confidence = 1
    endif

    return (signal, confidence)

endfunction
```

This algorithm translates numeric predictions into clear trading recommendations. By comparing the predicted price to the current value within a defined confidence band, it outputs an easy-to-understand signal: Buy, Sell, or Hold. It also produces a confidence score that quantifies the strength of the signal, improving user trust and transparency. The use of simple mathematical thresholds ensures fast execution and interpretability. This algorithm finalises the decision-support process by turning quantitative data into a qualitative insight users can act upon.

Justification and Role in Complete Solution

Prediction models produce numerical outputs; however, users require interpretable guidance. This algorithm transforms quantitative forecasts into qualitative decisions.
The threshold logic prevents overreaction to minor fluctuations, improving stability. The confidence score increases transparency and aligns with user feedback requesting trust indicators.
Without this stage, the system would not meet usability requirements identified in stakeholder research. This module completes the computational pipeline by converting analytics into actionable decision support.

Why These Algorithms Form a Complete Solution

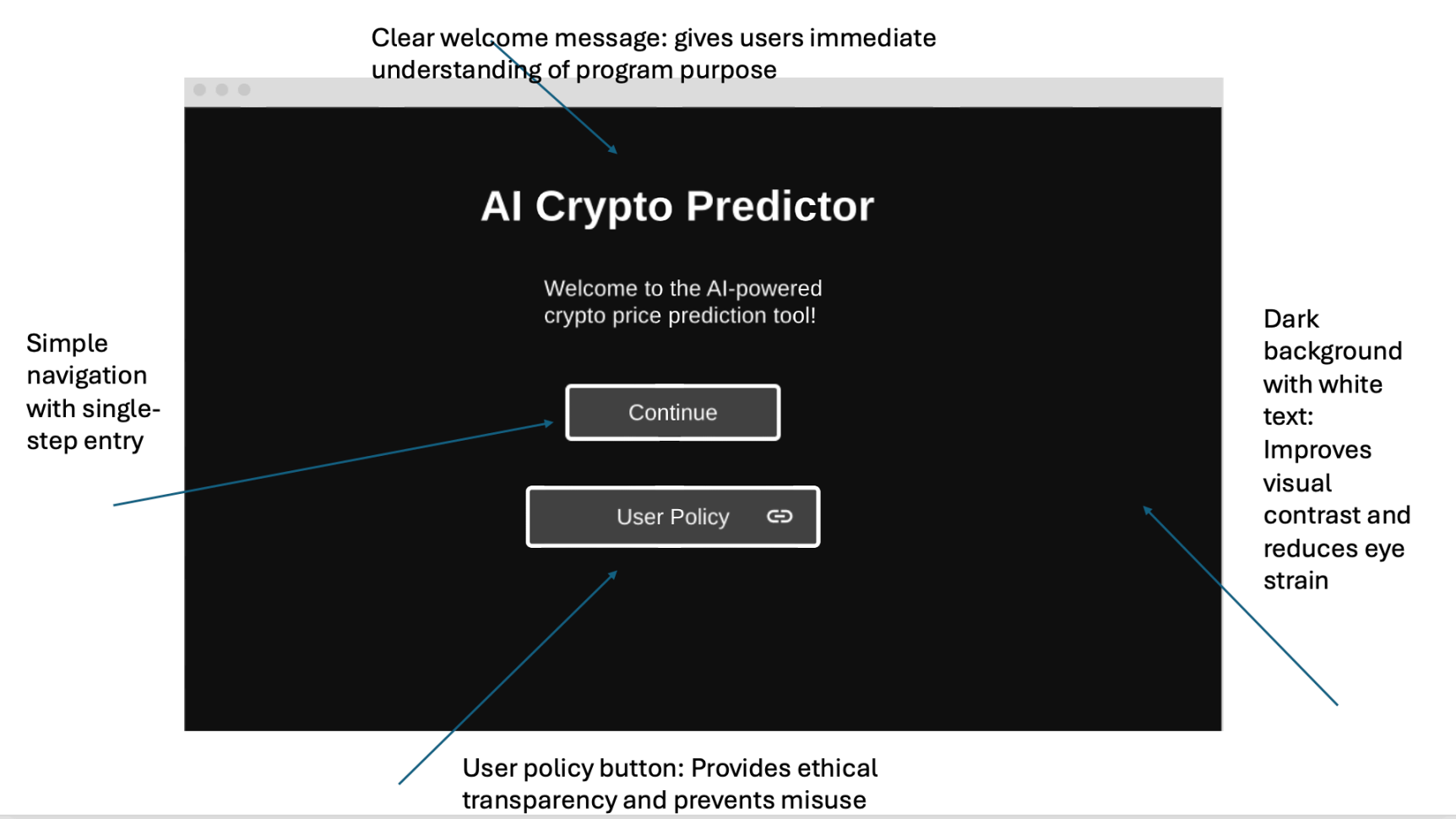
The combined system achieves:

- Real-time data retrieval
- Historical data processing
- Machine-learning prediction
- Statistical validation
- Decision conversion
- Visual feedback

Each algorithm performs a necessary stage in the pipeline. Removing any component would break the functional chain from input to actionable output.
Therefore, the solution is complete, modular, and aligned with the project objective of building an AI-driven cryptocurrency price prediction tool.

Usability Features

Design 1 – Welcome Screen



Design 1 – Welcome Screen Usability Justification

The welcome screen is designed to provide a **clear and immediate understanding of the system's purpose**, reducing user confusion and improving accessibility for first-time users. The large title *"AI Crypto Predictor"* ensures that the system's function is instantly recognisable, which aligns with usability principles of **visibility and clarity**.

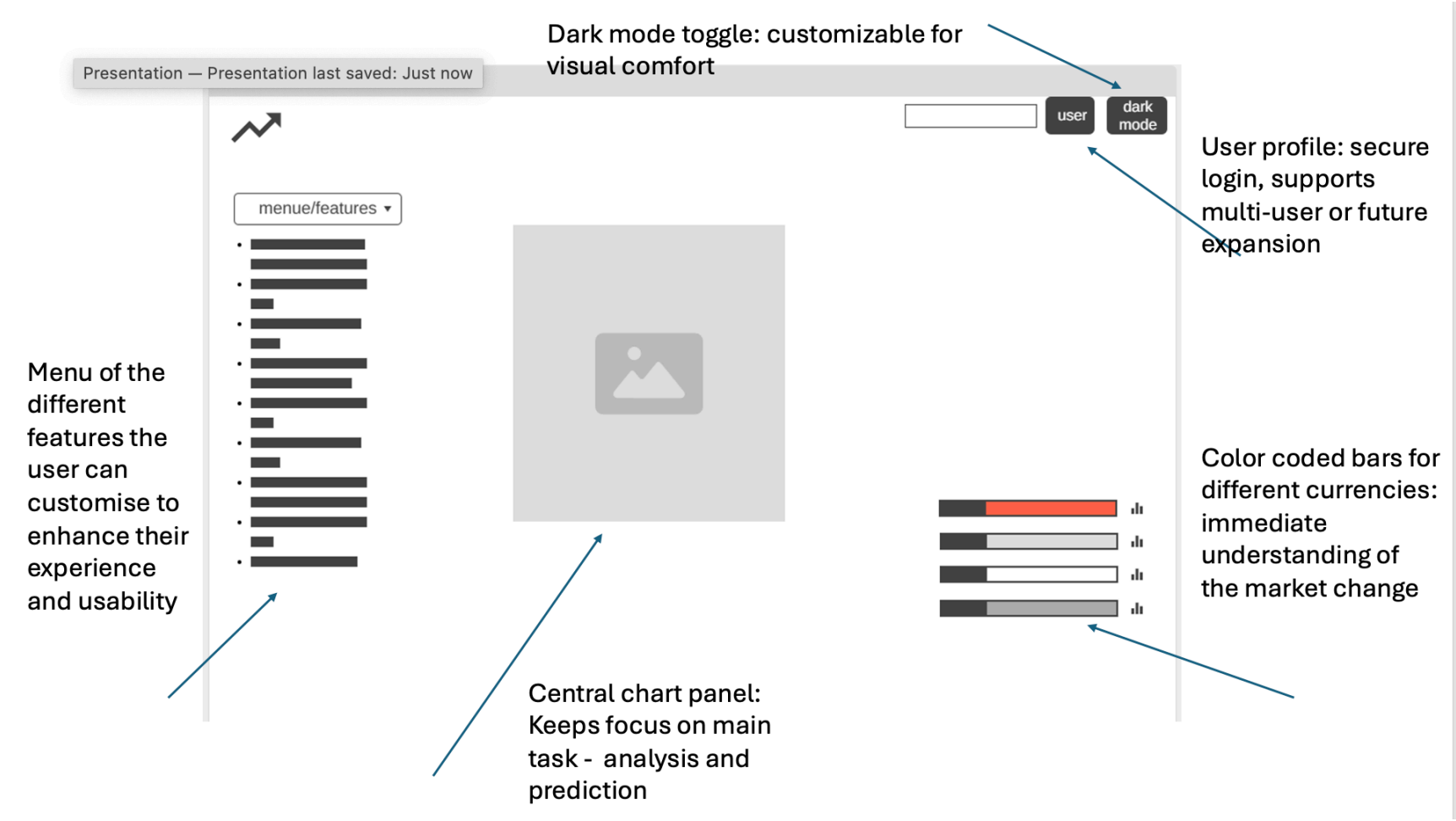
The **minimal layout with only two buttons (Continue and User Policy)** supports **simple navigation**, allowing users to begin using the system in a single step. This reduces cognitive load and ensures the interface is not overwhelming, particularly for users with limited technical knowledge.

The **dark background with high-contrast white text** improves readability and reduces eye strain, especially during prolonged use. This is important as users may spend extended time analysing charts and predictions.

The inclusion of a **User Policy button** ensures transparency and ethical use of the system. This improves user trust and aligns with responsible AI principles by informing users about limitations and appropriate usage.

Overall, this design prioritises **simplicity, clarity, and accessibility**, ensuring that users can quickly enter the system without confusion.

Design 2 – Main Dashboard Interface


Dark mode toggle: customizable for visual comfort
Presentation — Presentation last saved: Just now
user
dark mode
User profile: secure login, supports multi-user or future expansion
menue/features
Menu of the different features the user can customise to enhance their experience and usability
Color coded bars for different currencies: immediate understanding of the market change
Central chart panel: Keeps focus on main task - analysis and prediction

Design 2 – Main Dashboard Interface Usability Justification

The main interface is structured to support **efficient workflow and intuitive interaction**, with clear separation of features.

The **left-side menu** allows users to easily navigate between different tools and features. Positioning it vertically ensures consistency with common software interfaces, making it familiar and easy to use.

The **central chart panel** is the most prominent element, as it represents the core function of the system (data analysis and prediction). Placing it in the centre ensures that user attention is focused on the most important information.

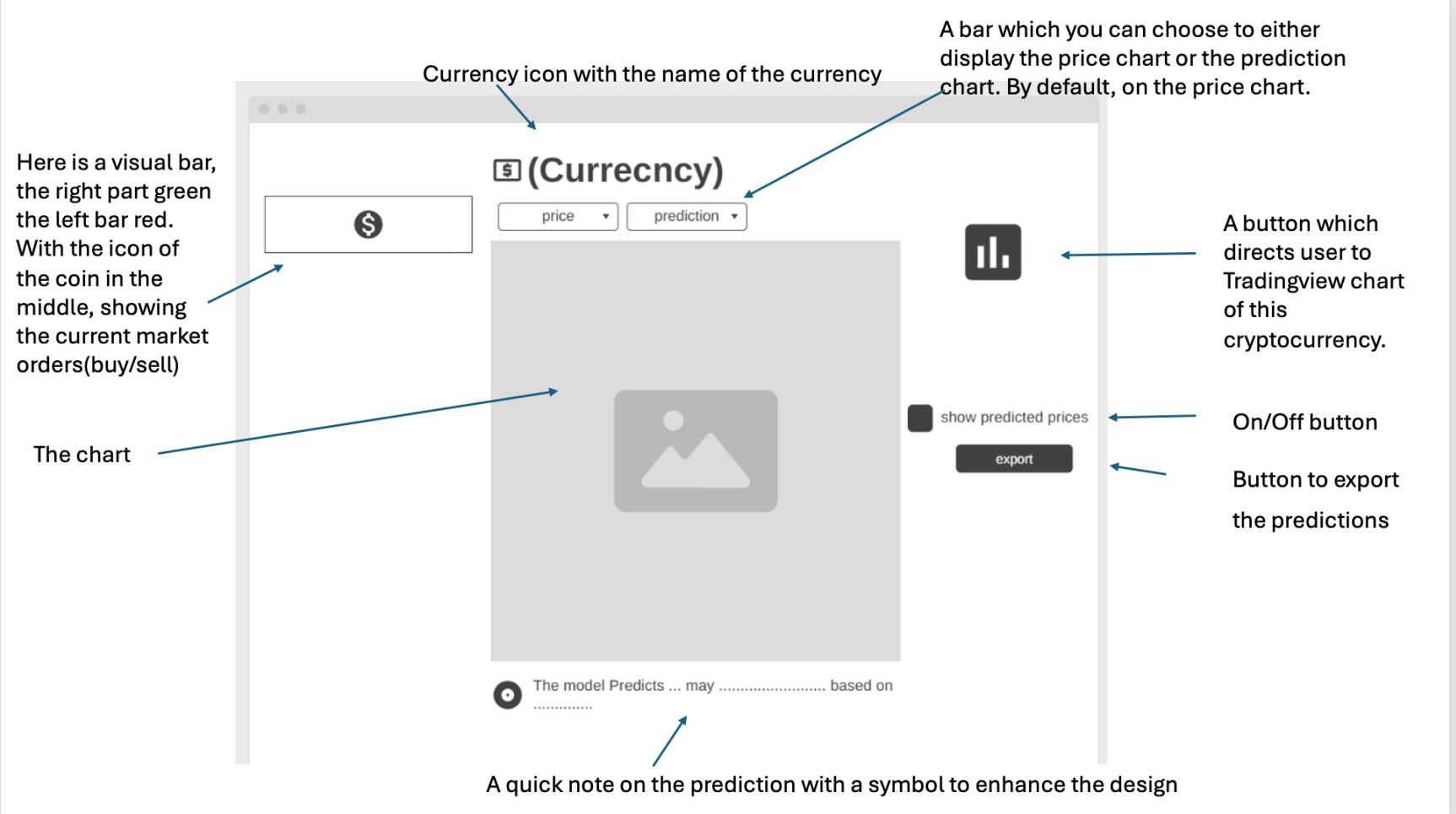
The **dark mode toggle** provides users with the ability to customise their visual experience. This improves usability by accommodating different user preferences and reducing eye strain in low-light environments.

The **user profile/login feature** supports future scalability, such as saving user preferences or enabling multiple users. This enhances usability by allowing personalised experiences.

The **colour-coded bars** provide immediate visual feedback about market changes. Using colours (e.g., red and green) allows users to quickly interpret data without needing to read numerical values, improving efficiency and decision-making speed.

Overall, the layout follows a **logical hierarchy**, ensuring that important features are prioritised and easily accessible.

Design 3 – Currency Analysis & Prediction Screen



Design 3 – Currency Analysis & Prediction Screen Usability Justification

This screen is designed to support **detailed analysis while maintaining clarity and control for the user**.

The **currency name and icon at the top** provide clear context, ensuring users always know which cryptocurrency they are analysing. This prevents confusion when switching between currencies.

The **toggle between price chart and prediction chart** allows users to easily compare real data with predicted values. This improves usability by supporting **decision-making without switching screens**.

The **central chart area** is large and clearly positioned, ensuring that users can easily interpret trends and patterns. This aligns with the system's primary goal of data visualisation.

The **buy/sell visual bar (red/green)** provides an intuitive representation of market activity. Colour coding allows users to quickly understand market conditions without analysing raw data.

The **"show predicted prices" toggle** gives users control over what information is displayed. This supports both beginner users (who may prefer simpler views) and advanced users (who want more detailed analysis).

The **export button** allows users to save prediction data, increasing the practical usefulness of the system beyond the interface itself.

The **TradingView button** links to external detailed charts, enhancing functionality without overcrowding the interface.

Finally, the **prediction summary note at the bottom** provides a quick, readable explanation of the model's output, making the system more understandable and user-friendly.

Overall, this design balances **functionality and clarity**, ensuring users can analyse, interpret, and act on data efficiently.

Testing Plan to inform development

Testing Strategy Overview

Testing ensures that the AI Crypto Prediction Tool functions correctly, meets its success criteria, and performs accurately under different conditions. Both **black box** and **white box** testing methods are used.

- **Black Box Testing:** Tests inputs and outputs without viewing the code, ensuring each feature meets user requirements.
- **White Box Testing:** Tests internal algorithm logic (e.g., loops, conditions, and thresholds in the prediction engine).
- **Alpha Testing:** Performed during development by the programmer (myself) after each prototype has completed to identify logical or display errors.
- **Beta Testing:** Conducted by users to evaluate usability, layout, and output clarity.
- **Post-Development Testing:** Final tests after completion to confirm that the full system works as a reliable solution.

Test data to be used during iterative development

Table 9: Iterative Prototype Testing Plan

Prototype	Focus	Main Types of Testing	Description
1 – Interface and Data Input Foundation	GUI and data input	Black Box Testing / Alpha Testing	Tested whether buttons respond correctly, windows load properly, and data can be fetched or imported. Invalid data (e.g., missing API) shows appropriate error messages.

2 – Prediction Engine and Decision Logic	Forecasting and display	Black Box Testing / White Box Testing / Alpha Testing	Checked that Moving Average (SMA) and RSI produce expected numeric results and correct Buy/Sell/Hold signals. Verified calculations with known data sets.
3 – Automation and Evaluation	Integration and automation	Black Box Testing / White Box Testing / Alpha Testing / Beta Testing	Ensured that automatic updates, chart visualisations, and colour-coded alerts all work together. Tested with full workflow from data input to final display.

Prototype 1

Table 10: Prototype 1 Testing Table

ID	Test	Type	Data	Expected Output	Justification
1	Update Live Price	Normal	Asset = "BTCUSDT"	Current BTC price appears and updates colour (green/red/black)	Confirms successful API connection and correct visual feedback.
2	Update Live Price	Erroneous	Asset = "INVALID"	"Data unavailable" displayed, no crash	Ensures invalid input handled safely.
3	GUI Buttons	Normal	Click "Open Bitcoin"	Bitcoin window appears with correct title and labels	Confirms navigation between windows works.
4	GUI Buttons	Erroneous	Click disabled "Predict" button before data load	No crash; tooltip message "Load data first"	Ensures GUI error prevention.
5	Data Cleaning	Boundary	Missing price entries in CSV	Replaced or dropped values without crash	Maintains data integrity for model input.
6	Fetch Data with No Internet	Erroneous	Disable internet and click "Fetch Live Data"	Error message shown: "Unable to connect to API"	Tests robustness against connectivity issues and ensures user feedback
7	Display Data Labels	Normal	Click "Open Ethereum"	Labels (e.g., Date, Price) appear correctly aligned	Ensures data is readable and interface is user-friendly

8	Navigate Between Windows	Normal	Use navigation buttons (e.g., Back, Settings)	Correct window is displayed without crashing	Verifies smooth user experience and interface consistency
9	Button Click Response	Normal	Click each button once	Button performs its intended action (e.g., opens file dialog, fetches data)	Verifies that GUI elements are responsive and functional
10	Resize Window	Boundary	Resize GUI window to minimum and maximum sizes	Layout adjusts without breaking or hiding elements	Ensures interface is adaptable and maintains usability across screen sizes
11	Button Spam Click	Extreme	Rapidly click "Fetch Data" multiple times	System handles it gracefully (e.g., disables button during fetch)	Tests system stability under stress and prevents duplicate operations

Prototype 1 testing Justification

The tests in Prototype 1 were designed to validate the **usability and reliability of the user interface and data input system**. The aim was to ensure that users can navigate the system easily, that inputs are handled correctly, and that errors are clearly communicated without causing system crashes. For **Prototype 1 (Interface and Data Input)**, test data such as:

- Valid cryptocurrency symbols (e.g. *BTCUSDT*)
- Invalid inputs (e.g. *"INVALID"*)
- Missing or incomplete datasets

were used.

This data was chosen to:

- Simulate real **user input scenarios**
- Ensure the system correctly handles both **valid and invalid inputs**
- Prevent crashes and improve **error handling robustness**

Boundary data (e.g. missing CSV values or resizing the window to extreme sizes) was included to test the **limits of the interface**, ensuring usability is maintained under unusual conditions.

Prototype 2

Table 11: Prototype 2 Testing Table

ID	Test	Type	Data	Expected output	Reason/ Justification
1	Prediction Function	Normal	Symbol = "BTCUSDT", Interval = "1d", Limit = 90	Float value returned as predicted price	Confirms regression model produces a valid numeric forecast.
2	Prediction Function	Boundary	Limit = 5	Error message "Not enough data for prediction"	Prevents inaccurate results from insufficient data.
3	Signal Generation	Normal	Current = 32000, Predicted = 32500	Output = "BUY", Confidence ≈ 0.1–1.0	Confirms correct calculation of threshold difference.
4	Calculate Moving Average (Valid Data)	Normal	Load historical price data for Bitcoin	MA values are calculated and plotted correctly	Verifies that the MA algorithm processes data accurately and supports trend analysis
5	Calculate RSI (Valid Data)	Normal	Load historical price data for Ethereum	RSI values are calculated and plotted correctly	Confirms RSI logic is implemented and reflects market momentum
6	MA with Missing Data	Boundary	Load dataset with missing price entries	MA is calculated with missing values handled appropriately	Tests robustness of algorithm against incomplete datasets
7	RSI with Flat Price Data	Boundary	Load data with constant prices over time	RSI returns neutral or Hold signal	Validates RSI behavior in non-volatile markets
8	MA with Short Time Interval	Extreme	Use a 2-day MA interval on long-term data	MA line appears jagged and responsive	Ensures flexibility of algorithm and correct handling of small window sizes

9	RSI with High Volatility	Extreme	Load highly volatile price data	RSI fluctuates sharply, triggering Buy/Sell signals	Tests responsiveness of RSI to rapid market changes
10	Confidence Score Calculation	Normal	Load recent price data with clear trend	Confidence score is displayed (e.g. 75%)	Confirms confidence logic quantifies prediction reliability
11	Confidence Score with No Trend	Boundary	Load random/noisy data	Confidence score is low or undefined	Validates system's ability to detect uncertainty and avoid misleading predictions
12	Display Buy Signal	Normal	Load data that triggers Buy condition	Green "Buy" alert appears on chart	Tests visual feedback and decision logic integration
13	Display Sell Signal	Normal	Load data that triggers Sell condition	Red "Sell" alert appears on chart	Ensures correct signal mapping and user guidance
14	Display Hold Signal	Normal	Load data with neutral indicators	No alert or "Hold" message appears	Verifies system avoids false positives and supports cautious decisions
15	Compare Prediction to Actual Trend	Normal	Load past data and compare prediction to real outcome	Prediction aligns with actual trend direction	Confirms analytical accuracy and supports system credibility
16	Chart Rendering	Normal	Load prediction results	Chart displays MA, RSI, and signals clearly	Confirms GUI integration and visual clarity of analytical output
17	Multiple Predictions	Normal	Load data for multiple currencies	Each currency shows independent predictions and signals	Tests scalability and modularity of prediction engine

Prototype 2 testing justification

The tests in Prototype 2 focus on validating the **accuracy and correctness of the prediction engine**. These tests were designed to ensure that algorithms such as Moving Average and RSI produce

reliable outputs under different data conditions, including normal, boundary, and extreme cases. This included:

- Real historical price data (to simulate real-world conditions)
- Flat/no-change data (to test RSI neutrality)
- Highly volatile data (to test responsiveness)

This was important because:

- Prediction models must be tested on **different market conditions**
- It ensures the algorithm performs correctly under **stable, volatile, and edge-case scenarios**
- It validates the accuracy and reliability of indicators such as **Moving Average and RSI**

Prototype 3

Table 12: Prototype 3 Testing Table

ID	Test	Test Type	Input	Expected Outcome	Reason/Justification
1	Auto-Fetch Data	Normal	Wait for scheduled interval	New data is fetched without manual input	Confirms automation works and system updates independently
2	Auto-Run Prediction	Normal	After auto-fetch completes	Prediction model runs and updates chart	Validates full automation pipeline from data to output
3	Linear Regression Forecast	Normal	Load historical price data	Forecast line appears based on regression model	Tests ML model integration and trend prediction accuracy
4	ARIMA Forecast	Normal	Load time-series data	ARIMA-based forecast is displayed	Confirms advanced time-series modelling is functional

5	Forecast with Noisy Data	Boundary	Load random data	Forecast shows low confidence or flat trend	Validates model's ability to handle uncertainty and avoid misleading outputs
6	Accuracy Report Generation	Normal	Run prediction on known data	Accuracy metrics (e.g., RMSE, MAE) are displayed	Confirms system can evaluate its own performance quantitatively
7	Alert Notification Trigger	Normal	Prediction crosses threshold	Notification appears (e.g., "Price spike expected")	Tests user feedback and alert logic
8	Colour-Blind Accessibility	Normal	Trigger Buy/Sell signal	Text-based message appears alongside colour alert	Ensures inclusivity and usability for colour-blind users
9	GUI Transition Smoothness	Normal	Navigate between modules	Transitions are animated and responsive	Verifies final polish and user experience refinement
10	System Integration Test	Normal	Run full cycle (input → prediction → output)	All modules work together without error	Confirms system reliability and readiness for deployment
11	Model Comparison	Normal	Run both Linear Regression and ARIMA	Outputs are displayed side-by-side	Supports evaluation and user choice between models
12	Manual Override	Normal	User edits prediction parameters	System updates forecast accordingly	Confirms user control and flexibility in automated system
13	Scheduled Task Failure	Extreme	Simulate system crash during auto-fetch	System logs error and retries or alerts user	Tests fault tolerance and recovery mechanisms

Prototype 3 testing Justification

The tests in Prototype 3 were designed to evaluate the **integration and automation of the complete system**. The aim was to ensure that all modules work together correctly, that real-time updates function as expected, and that the system remains stable under realistic and stressful conditions.

This included:

- Live API data
- Simulated failures (e.g. no internet connection)
- Full workflow inputs (input → prediction → output)

This data was selected to:

- Test **real-time system behaviour**
- Ensure modules work **together as a complete system**
- Validate automation and system reliability

Planned Post-Development Testing

I will interview three different types of stakeholders to gather a range of perspectives on the usability and functionality of the program. The first will be a finance or economics student (or trader) who can evaluate the accuracy and practical relevance of the predictions. The second will be a casual user with little or no trading experience, providing feedback on accessibility and clarity. The final interviewee will be a computer science or machine learning student, offering insight into the technical design and algorithmic performance. This ensures that feedback is collected from users with diverse backgrounds, allowing the program to be evaluated in terms of accuracy, usability, and technical robustness.

Planned Interview Questions:

Question 1: How easy was it to navigate through the program's windows and features?

Question 2: Was the information on the screen clear and easy to understand?

Question 3: How effective do you think the visual feedback (colours, arrows, and charts) is for showing market trends?

Question 4: Were there any issues with speed, accuracy, or stability while using the program?

Question 5: Do you have any suggestions to improve the usability or user experience?

After completing the interviews, users will also be asked to rate five key aspects of the program on a scale from 1 (poor) to 5 (excellent) based on their overall experience. These aspects are:

- Interface clarity – how easy the layout and navigation were to understand.
- Prediction accuracy – how reliable and realistic the forecasted prices appeared.
- Response speed – how quickly the program loaded data and generated predictions.
- Colour readability – how clearly colours and visual cues (green, red, black) conveyed information.
- Overall satisfaction – the user's general impression of the tool's usability and usefulness.

This feedback provides quantitative evidence to support the qualitative interview results and helps evaluate the system's performance from multiple user perspectives.

Interview testing Justification

After development, additional data was used to evaluate the system in realistic usage scenarios.

User-based Data (Interviews)

Three types of users were selected:

- A finance/trading user → to evaluate **prediction accuracy and usefulness**
- A casual user → to assess **usability and clarity**
- A technical user → to analyse **algorithm performance and system design**

This range was chosen to ensure:

- Diverse perspectives
- Balanced evaluation of both **technical and usability aspects**

Rating Scale (1–5)

A 1–5 rating scale was used to provide **quantitative data** that can be compared across users. This allows objective evaluation of usability, accuracy, and performance, supporting more reliable conclusions in the evaluation stage.

Planned Scenario Testing

Table 14: Scenarios

Scenario No.	Scenario Description	Steps Taken	Expected Result	Actual Result
1	**User login and navigation to Bitcoin Window**	1. Open the program 2. Click Continue on the welcome screen 3. Enter email and password, then click Login 4. On the Main Window, click Open Bitcoin 5. Click on the Predict Price button.	System logs in successfully, opens the Bitcoin window, displays the live price, and generates a predicted price with an arrow indicator.	-
2	**Error handling when API connection fails**	1. Disconnect from the internet 2. Launch the program and open the Bitcoin window 3. Click Update Price or Predict Price.	The program detects the connection error and displays “Data unavailable” instead of crashing.	-
3	**Switching between multiple windows**	1. Open the Main Window 2. Click Open Bitcoin, then Open Ethereum 3. Return to the Main Window and close both windows.	Each window opens and closes smoothly without lag or overlapping; system does not freeze.	-

4	**Using export and dark mode features (optional extended test)**	1. Open the Bitcoin window 2. Click Show Predicted Prices and Export to save the data 3. Toggle Dark Mode from the Main Window.	Predicted data is exported successfully, file saved in working directory; dark mode activates instantly.	-

Scenario testing Justification

Realistic scenarios (e.g. API failure, switching windows, exporting data) were used.
This was important because:

- It reflects **real-world usage**
- Tests the system beyond isolated functions
- Ensures the system is **practical and reliable in real situations**

Overall Testing Justification and Conclusion

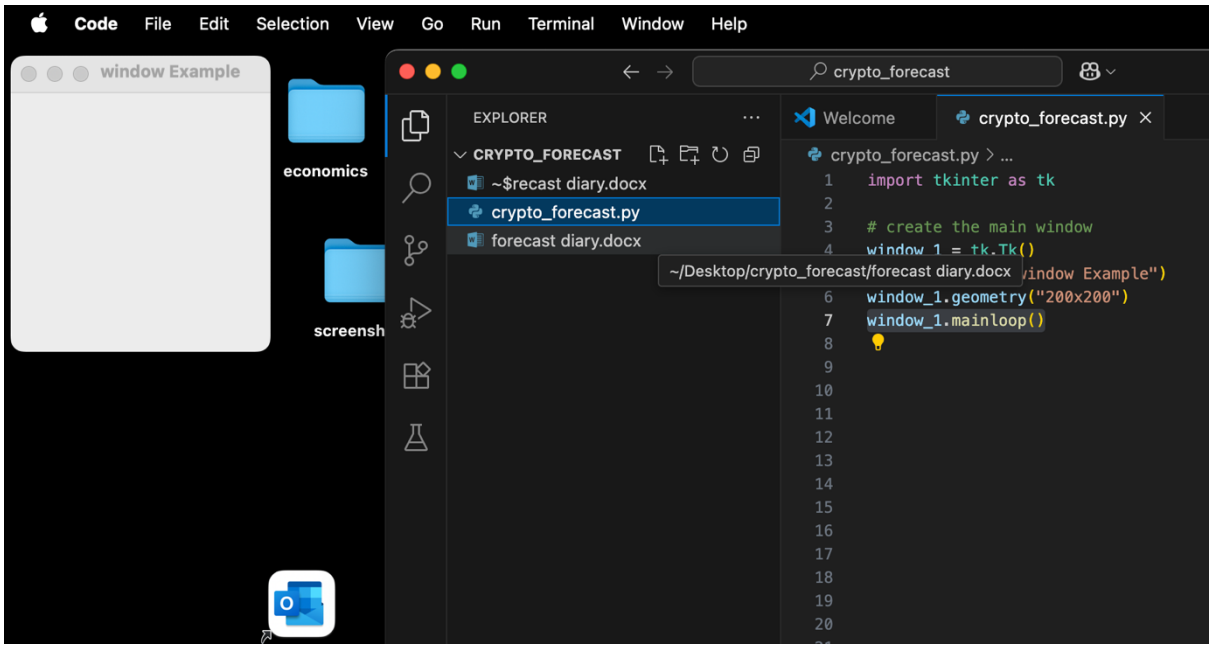
This testing plan ensures that every function within the AI Crypto Prediction Tool has been verified for accuracy, reliability, and user accessibility. Normal, boundary, and erroneous test cases were used to validate algorithm robustness and prevent runtime errors. Scenario and user acceptance testing confirm that the final system is practical and user-friendly, meeting all success criteria. The combination of black box, white box, and iterative testing approaches ensures that both the interface and underlying prediction logic perform reliably under real-world conditions. Overall, this plan establishes a structured and comprehensive approach to testing, ensuring that the developed system can later be evaluated for accuracy, clarity, and resilience against the original design objectives.

Iterative Development & Testing

Initial Tkinter Window Setup and Iterative Fix

```
import tkinter as tk

# create the main window
window_1 = tk.Tk()
window_1.title("window Example")
window_1.geometry("200x200")
```

S.S.1: GUI **closed immediately** → added **window_1.mainloop()**



S.S.2: **Window too small** → **resized** to 1200×800

⇒ **Code explanation:**

This code initialises the graphical user interface using the Tkinter library. A root window object (window_1) is created using tk.Tk(), which acts as the parent container for all interface components added later (e.g., buttons, labels, and images). The title is set using title() and the window dimensions are defined using geometry(). The program then calls window_1.mainloop(), which starts Tkinter's event loop. This loop continuously listens for events (such as clicks and refresh updates) and prevents the program from terminating immediately, ensuring the GUI remains visible and responsive.

⇒ **Justification:**

Tkinter was selected because it is included with Python, lightweight, and appropriate for producing a reliable GUI within the scope of an A-Level project. Using a single root window provides a clear foundation for expanding the system into multiple windows (e.g. separate Bitcoin and Ethereum pages) while keeping the structure modular.

Alternative approaches such as a command-line interface were rejected because they would reduce usability for non-technical users, and frameworks such as PyQt were not chosen because they require additional setup and complexity that is not necessary for the project requirements.

Explicitly setting the window size supports usability by ensuring there is sufficient screen space for later features (charts, live prices, timestamps, and prediction outputs).

⇒ **Iterative development:**

During initial testing, the program ran without errors but the GUI did not remain visible. This happened because the Tkinter event loop was not running. The issue was resolved by adding window_1.mainloop() at the end of the setup code, enabling the window to display correctly and remain active.

A second issue was identified during usability review: the default window size (200×200) was too small to accommodate planned interface components. The window size was therefore increased to 1200×800 to provide sufficient space for navigation controls and future visualisations, reducing layout constraints during later development stages.

⇒ **Validation:**

Normal and usability testing confirmed that the GUI initialisation behaves as intended. The application window opens reliably and remains responsive once the event loop is started. Adjusting the window dimensions to 1200×800 resolved early layout limitations and provided sufficient space for planned interface elements. Boundary testing showed that the geometry and title settings are applied consistently on launch. Overall, these tests validate that the base GUI

structure is stable and suitable for extending the system with interactive controls, live data updates, and additional windows.

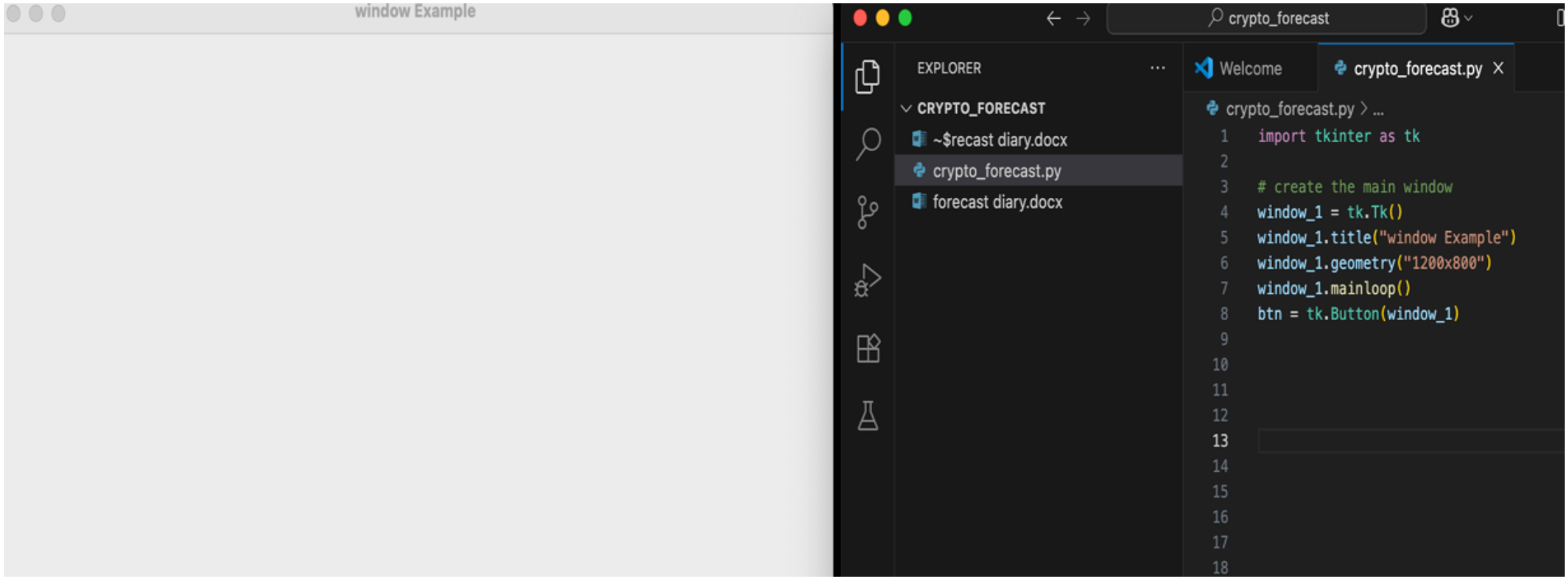
Test ID	Feature Tested	Test Type	Test Description	Expected Outcome	Actual Outcome	Action Taken	Result	Evidence
A1	Tkinter root window displays	Normal	Run program after creating tk.Tk() window	Window opens and stays visible	Window did not remain open / not shown as expected	Added window_1.mainloop()	Pass	S.S.1
A2	Window size suitability	Usability	Check whether default size supports planned interface	Enough space for interface elements	Too small to support layout	Increased to 1200×800	Pass	S.S.2
A3	Window title set correctly	Normal	Run program and check title bar	Correct title displayed	Title displayed correctly	None	Pass	S.S.2
A4	Window size applied correctly	Boundary	Run program and confirm it launches at 1200×800	Size applied accurately	Size applied accurately	None	Pass	S.S.2

⇒ **Justification of Actions Taken During Testing:**

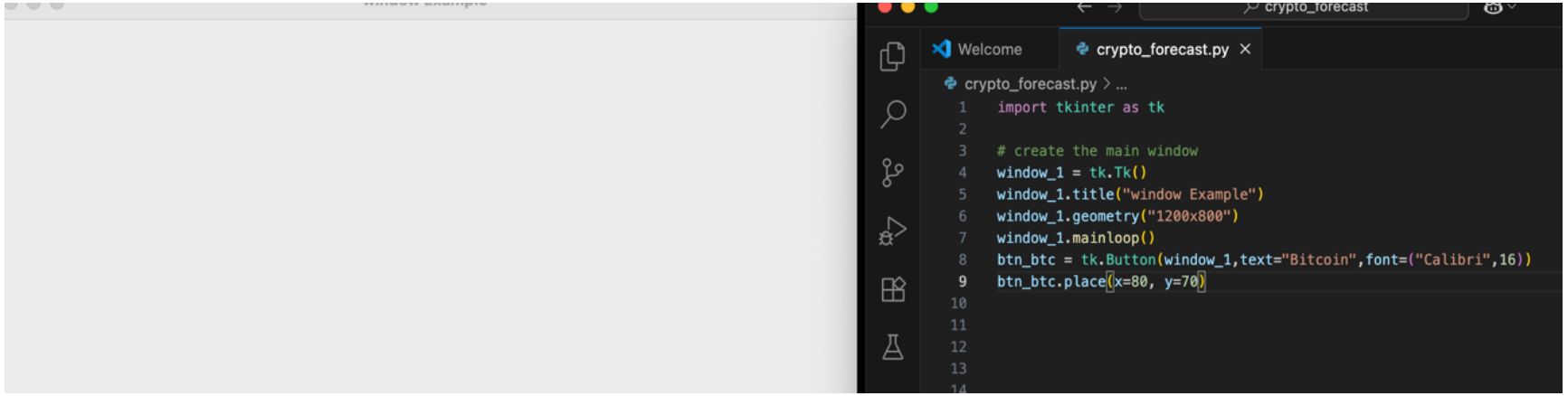
In Test A1, the graphical interface did not remain visible after execution. This occurred because the Tkinter event loop was not running. To resolve this, window_1.mainloop() was implemented. This approach was chosen because the event loop is required to continuously handle GUI events and keep the window active. This improved the reliability of the system by ensuring the interface remains open and responsive for user interaction.
In Test A2, the interface layout was initially unsuitable due to insufficient spacing. To resolve this, the .place() geometry manager was implemented. This approach was chosen instead of .pack() or .grid() because it allows precise control over positioning, which is essential for maintaining a structured and visually clear layout. This improved the usability of the system by ensuring all elements are clearly visible and logically arranged.

⇒ **Link to analysis & design:**
This stage implements the core GUI framework identified in the system design as a prerequisite for all later functionality. Establishing a stable and consistently sized main window ensures that subsequent components—navigation controls, live data displays, and predictive outputs—can be integrated without reworking the interface structure, supporting both maintainability and scalability.

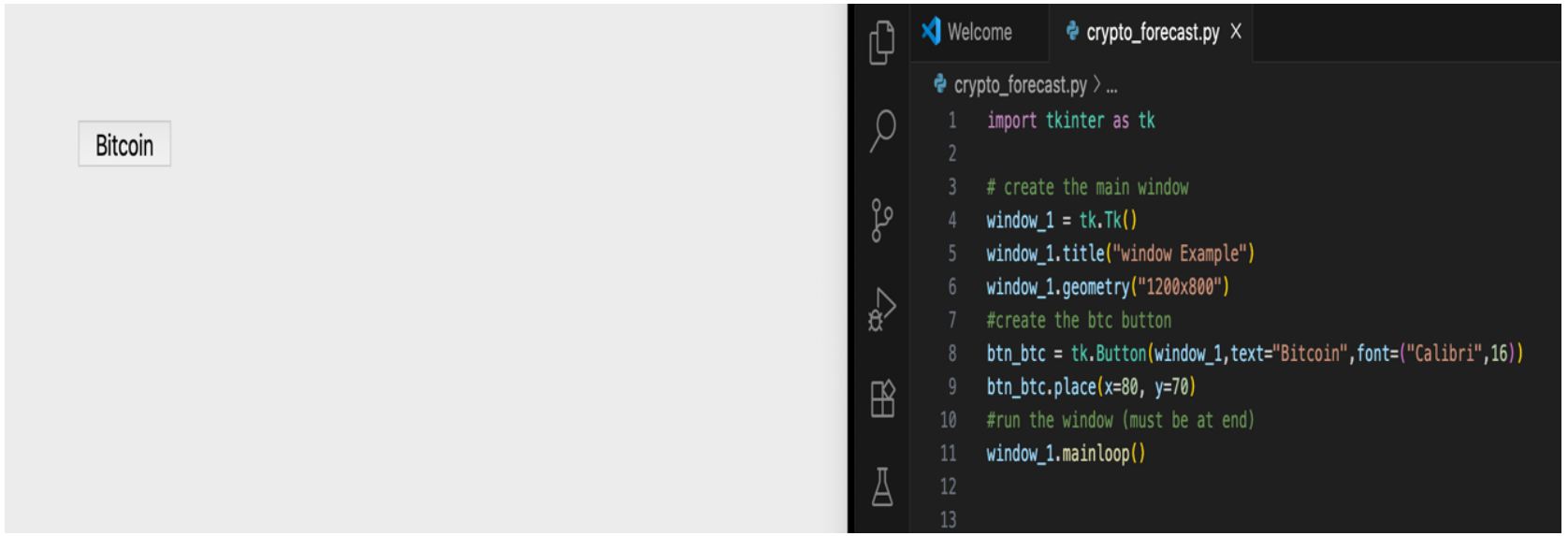
Button Creation, Layout Issues, and Usability Refinement



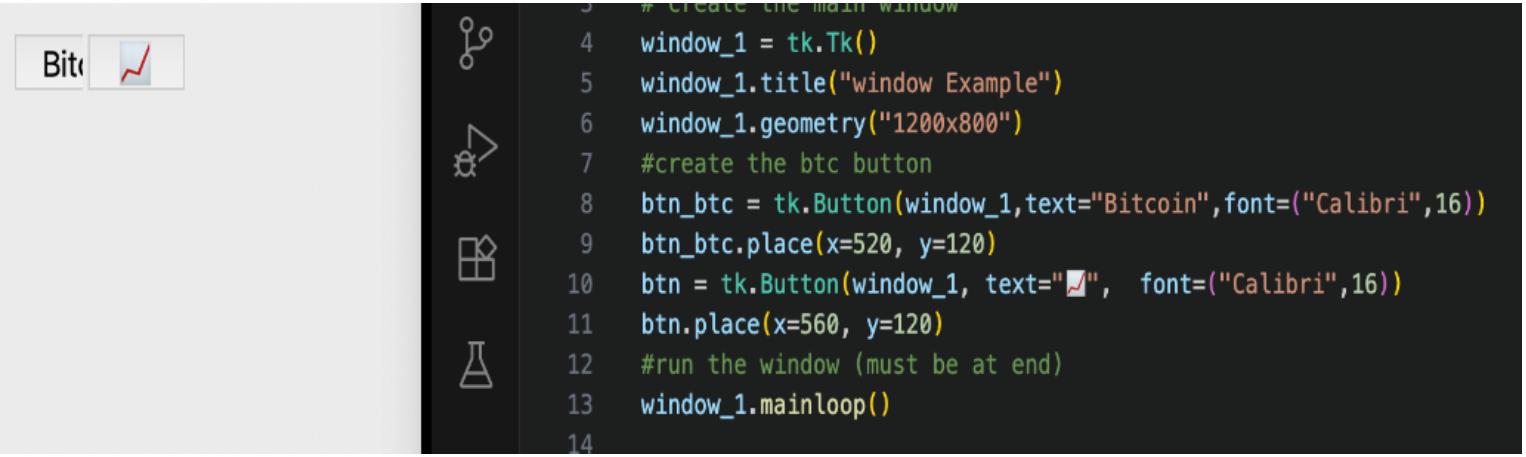
S.S.3: Button defined but **not visible** → no geometry manager used



S.S.4: Button placed but still not visible → **code executed after mainloop()**



S.S.5: The issue was resolved by **moving the button code before mainloop()**, then adjusting its position to x = 520, y = 120.



S.S.6: Button spacing too small → increased; **emoji-only button may confuse users** and encourage unnecessary repeated clicks

⇒**Code explanation:**

This section implements interactive navigation controls within the Tkinter main window using the Button widget. A primary Bitcoin button (btn_btc) was created with a clear text label and customised font to ensure readability. The button was positioned using the place() geometry manager, which allows explicit control over its x and y coordinates within the window.

An additional button using an emoji icon was also implemented to represent chart access. Both buttons were linked to the main window object (window_1), enabling direct user interaction and serving as entry points to cryptocurrency-specific functionality within the graphical interface.

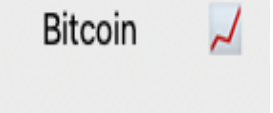
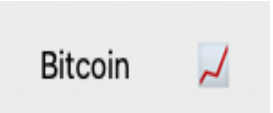
⇒**Justification:**

Buttons were selected as the primary navigation mechanism because they provide an intuitive and familiar interaction model for users, particularly in dashboard-style financial applications. They allow users to clearly select actions such as viewing Bitcoin data or accessing related charts. he place() geometry manager was chosen instead of pack() or grid() because it offers precise positional control, which is important for maintaining a clean and structured layout when displaying financial information, charts, and live data. Explicit positioning also supports consistent layout behaviour across different screen sizes and resolutions, aligning with the usability requirements identified during the analysis stage.
The use of an emoji icon was initially explored to enhance visual engagement; however, usability considerations were prioritised over purely aesthetic design choices.

⇒ **Iterative development:**
During initial implementation, the button did not appear even though the Button object had been created. This issue occurred because the widget was not placed onto the window using a geometry manager. The problem was resolved by explicitly calling .place() on the button object. After the button appeared, further usability issues were identified. The initial x-coordinate spacing between buttons was too small, making the interface feel cramped and increasing the likelihood of accidental clicks. The position was adjusted from a small offset to a larger spacing (x = 520, y = 120), improving clarity and usability.
An emoji-based button was also tested to represent chart functionality. However, this raised a usability concern, as users might repeatedly click the emoji without understanding its purpose. This led to a design review where combining the icon with the text-based Bitcoin button was considered as a clearer and more intuitive solution.

A further usability review identified that the horizontal spacing between the text button and the emoji button was too small (40 pixels), which caused the interface to feel cluttered. This increased the risk of accidental or repeated clicks. To address this, the spacing was increased to create clearer separation between controls.
The emoji-only button was also reviewed and identified as a potential usability issue, as its function was not self-explanatory and could confuse users. This prompted a design decision to either combine the icon with the Bitcoin text button or remove the standalone emoji button in later iterations to improve clarity and reduce unnecessary user interaction.

⇒**Validation:**
A functional validation test was carried out by running the program after applying the .place() method. The Bitcoin button appeared correctly and remained responsive to user interaction, confirming successful widget creation and placement.
Usability validation was then conducted through visual inspection and interaction testing. Increasing the horizontal spacing between buttons improved readability, reduced interface clutter, and lowered the likelihood of accidental clicks. These observations confirmed that the layout adjustments improved both usability and interaction accuracy.
The validation evidence recorded in the test tables supports that the navigation controls function as intended and meet the usability objectives defined in the analysis stage.

Test ID	Feature Tested	Test Type	Test Description	Expected Outcome	Actual Outcome	Action Taken	Result	Evidence
B1	Button visibility after creation	Erroneous	Create a Button without geometry manager	Button should appear	Button not visible	Added .place() to position button	Pass	S.S.5
B2	Button position on screen	Normal	Place button at specified x/y	Button appears correctly	Position not suitable initially	Adjusted to (520,120)	Pass	S.S.5
B3	Button spacing	Usability	Check spacing between text and emoji button	Clear separation	Spacing too small (40px)	Increased x-spacing	Pass	S.S.6
B4	Emoji button clarity	Usability	Evaluate emoji-only control	Purpose clear	Potential confusion	Considered combining icon + text	Pass	
B5	Overlap check	Boundary	Confirm buttons don't overlap other UI elements	No overlap	No overlap after	Adjusted positions as needed	Pass	

					adjustments			

⇒ **Justification of Actions Taken During Testing:**

In Test B1, the button was not visible after creation. This occurred because no geometry manager was applied to position the widget. To resolve this, the .place() geometry manager was implemented. This approach was chosen instead of .pack() or .grid() because it allows precise control over positioning. This improved the functionality of the system by ensuring the button is displayed correctly for user interaction.

In Test B2, the button position was initially unsuitable when placed at default coordinates. To resolve this, the position was adjusted to (520,120). This approach was chosen to align the button with other interface elements and maintain a consistent layout. This improved the usability of the system by ensuring the button is clearly visible and logically positioned.

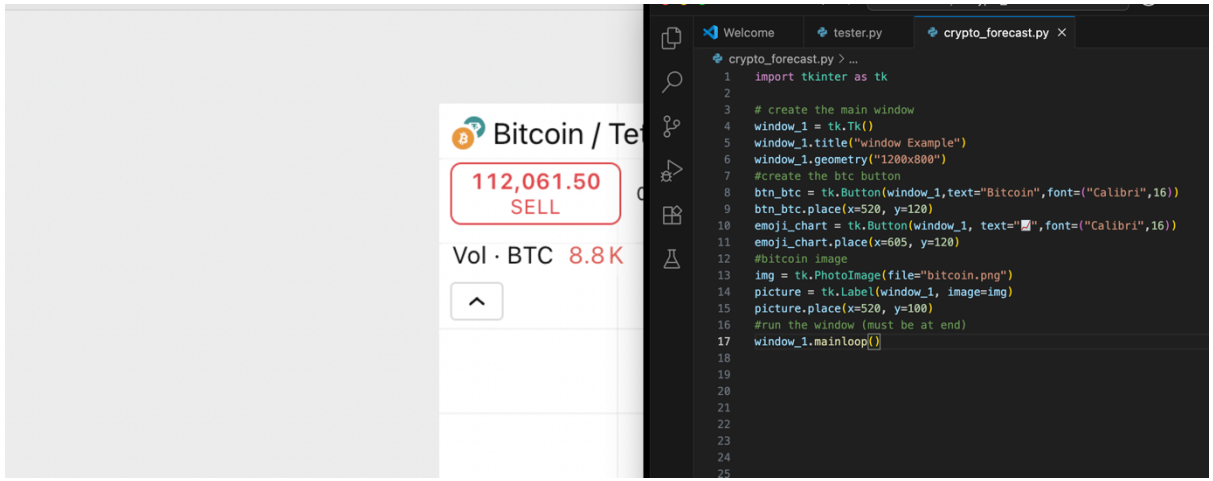
In Test B3, the spacing between the text and emoji button was insufficient, making the interface appear cluttered. To resolve this, the horizontal spacing was increased. This approach was chosen to improve visual separation between elements and enhance readability. This improved the usability of the system by making the interface clearer and easier to navigate.

In Test B4, the emoji-only button created potential confusion regarding its function. To resolve this, the design was reviewed and the option of combining an icon with text was considered. This approach was chosen to improve clarity and ensure users can easily understand the purpose of the control. This improved the usability of the system by reducing ambiguity in user interaction.

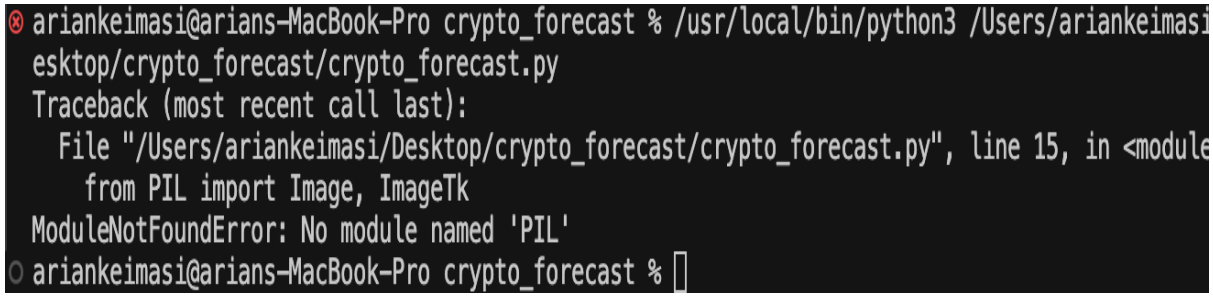
⇒ **Link to analysis & design:**

This stage implements the navigation mechanisms specified in the system design, where clear and reliable user input was identified as essential before introducing dynamic data or predictive processing. Finalising button behaviour and layout at this point ensures that later features—such as live price updates and multi-window navigation—can be added without redesigning the core interaction structure.

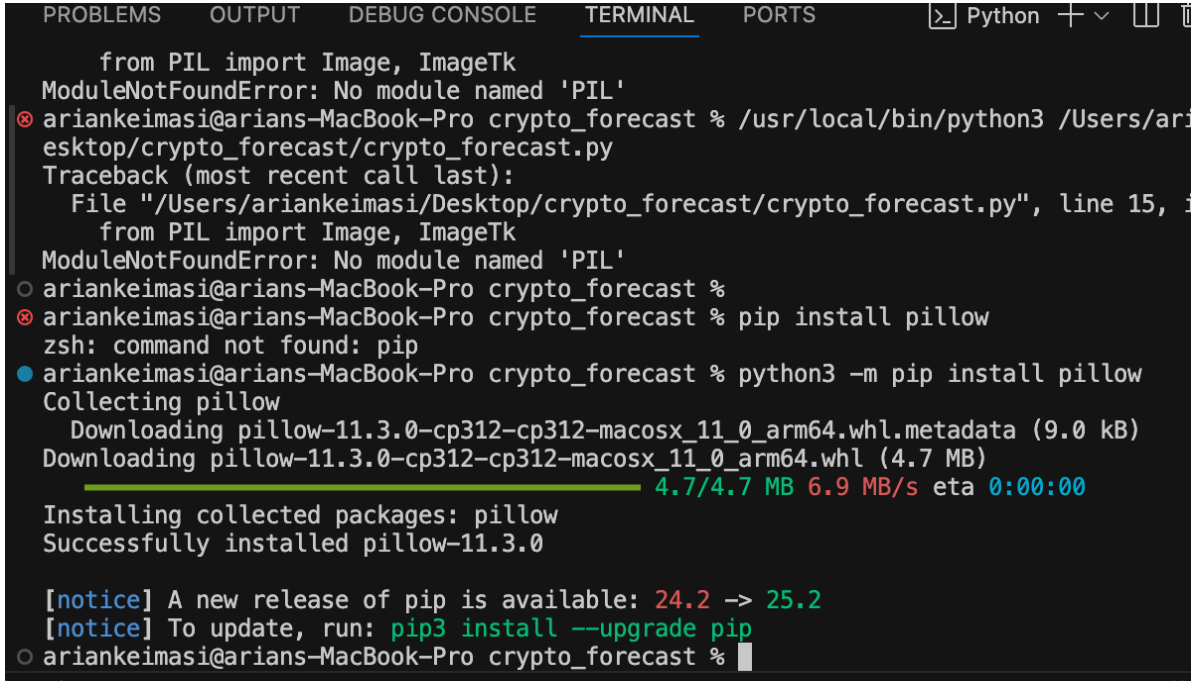
Image Handling



S.S.7: Image **too large for layout** → resize required



S.S.8: Attempted image resize using PIL → **library not installed**



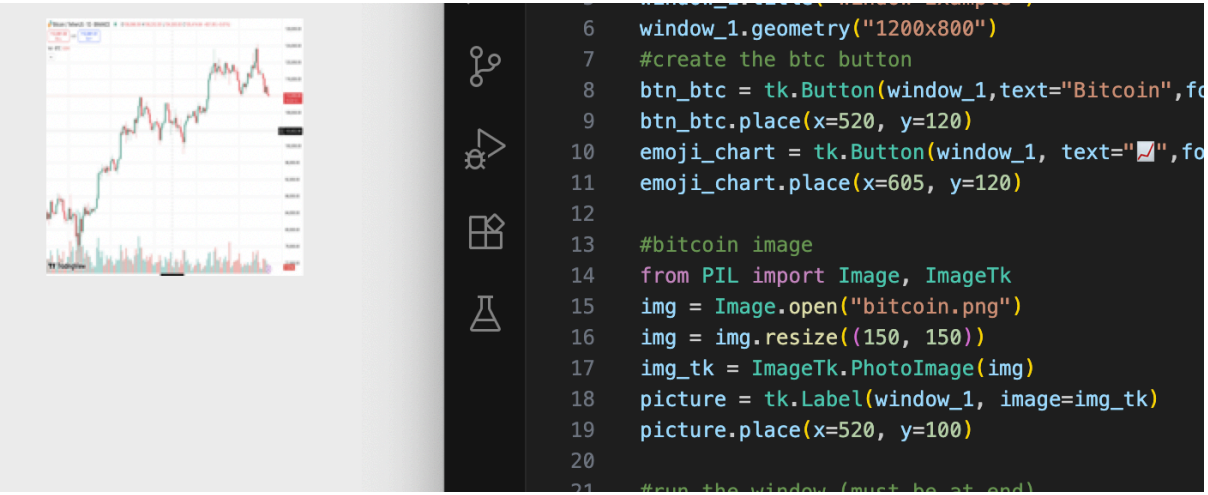
S.S.9: **Installed PIL library** successfully

```
#bitcoin image
img = tk.PhotoImage(file="bitcoin.png")
from PIL import Image, ImageTk
img = img.resize((150, 150))
picture = ImageTk.PhotoImage(img)
picture = tk.Label(window_1,image=img)
picture.place(x=520, y=100)
```

S.S.10: **AttributeError**: 'PhotoImage' object has no attribute 'resize' → fixed by **loading image with Image.open()**

```python
#bitcoin image
from PIL import Image, ImageTk
img = Image.open("bitcoin.png")
img = img.resize((150, 150))
picture = ImageTk.PhotoImage(img)
picture = tk.Label(window_1, image=picture)
picture.place(x=520, y=100)
```

S.S.11: **Unexpected error** during image handling



S.S.12: **References mixed up** → **corrected**; final positions and sizes adjusted

⇒ **Code explanation:**

This section implements image handling within the Tkinter interface to display a visual representation of Bitcoin data. An external image file was loaded and displayed using a Label widget, allowing graphical content to be embedded directly into the GUI. The Python Imaging Library (PIL) was used to open and resize the image before converting it into a Tkinter-compatible format (ImageTk.PhotoImage) for display.

The resized image was then positioned within the main window using the place() geometry manager with explicit coordinates, ensuring alignment with existing buttons and layout elements. This approach allowed the image to integrate cleanly into the dashboard-style interface without overlapping other controls.

⇒ **Justification:**

Including a visual image element enhances the clarity and realism of the interface, supporting the project's aim of presenting cryptocurrency data in a way that mirrors real trading platforms. Images provide immediate visual context and improve user engagement compared to text-only displays.

The PIL library was selected because Tkinter's built-in PhotoImage class has limited functionality and does not support direct image resizing. Using PIL enabled precise control over image dimensions, ensuring the image could be scaled appropriately to fit the interface without

distortion. Explicit positioning using place() ensured the image aligned consistently with other interface components and maintained a clean layout across different screen sizes.

⇒ **Iterative development:**

During initial testing, the image appeared too large for the interface, causing layout imbalance and overlapping nearby elements. This identified a usability and layout issue, as the image distracted from key interface controls. The issue was addressed by resizing the image to smaller dimensions before display.

An initial attempt to resize the image using Tkinter's PhotoImage resulted in an AttributeError, as PhotoImage does not support the resize() method. This led to the introduction of the PIL library; however, the program initially failed due to the library not being installed. The issue was resolved by installing the Pillow package and correctly importing Image and ImageTk.

Further errors occurred when image references were incorrectly assigned, resulting in unexpected behaviour and images not displaying. This was resolved by ensuring the image object was stored correctly and passed consistently to the Label widget. After these corrections, the image displayed at the intended size and position, integrating cleanly with the rest of the interface.

⇒ **Validation:**

A functional validation test was conducted by running the program after resizing the image using PIL and assigning it to a Label. The image displayed correctly without errors, confirming successful loading, resizing, and rendering.

A usability validation was then performed by visually inspecting the interface layout. The resized image no longer dominated the screen, maintained proportional balance with buttons and labels, and improved overall readability. Correcting image references ensured stable rendering with no runtime errors, validating both the technical correctness and usability of the solution.

Test ID	Feature Tested	Test Type	Test Description	Expected Outcome	Actual Outcome	Action Taken	Result	Evidence
C1	Image size in GUI	Usability	Display image in interface	Fits without overlap	Image too large	Resized & adjusted	Pass	S.S.12
C2	PIL availability	Erroneous	Import PIL to resize image	Import succeeds	PIL not installed	Installed Pillow	Pass	S.S.9
C3	Resize method	Erroneous	Call .resize() on PhotoImage	Image resizes	AttributeError	Used Image.open() + PIL resize	Pass	S.S.12

C4	Final layout integration	Usability	Check final image position/size	Balanced layout	Misaligned initially	Adjusted positions/sizes	Pass	S.S.12
C5	Image persistence	Boundary	Leave app running / open pages	Image remains visible	Image remains visible	Keep global/class reference to image	Pass	S.S.12
C6	Different image sizes	Boundary	Test 100×100 vs 200×200	Scales without breaking layout	Layout best at chosen size	Selected best size	Pass	S.S.7

⇒ **Justification of Actions Taken During Testing:**

In Test C1, the image was initially too large for the interface, causing layout issues. To resolve this, the image was resized and adjusted. This approach was chosen to ensure the image fits within the interface without overlapping other elements. This improved the usability of the system by maintaining a clear and balanced layout.

In Test C2, the PIL library was not available, preventing image resizing functionality. To resolve this, the Pillow library was installed. This approach was chosen because Pillow provides the required image processing features not available in standard Tkinter. This improved the functionality of the system by enabling image resizing operations.

In Test C3, calling .resize() on a PhotoImage object caused an AttributeError. This occurred because PhotoImage does not support direct resizing. To resolve this, Image.open() and the PIL

resize method were used instead. This approach was chosen because it correctly handles image manipulation before converting to a Tkinter-compatible format. This improved the reliability of the system by ensuring images can be resized without errors.

In Test C4, the image was initially misaligned within the interface. To resolve this, the position and size were adjusted. This approach was chosen to ensure the image integrates properly with other interface elements. This improved the usability of the system by creating a visually balanced layout.

In Test C6, different image sizes produced varying layout results. To resolve this, multiple sizes were tested and the most suitable size was selected. This approach was chosen to optimise both visual quality and layout consistency. This improved the usability of the system by ensuring the interface remains stable and visually clear.

⇒ **Link to analysis & design:**

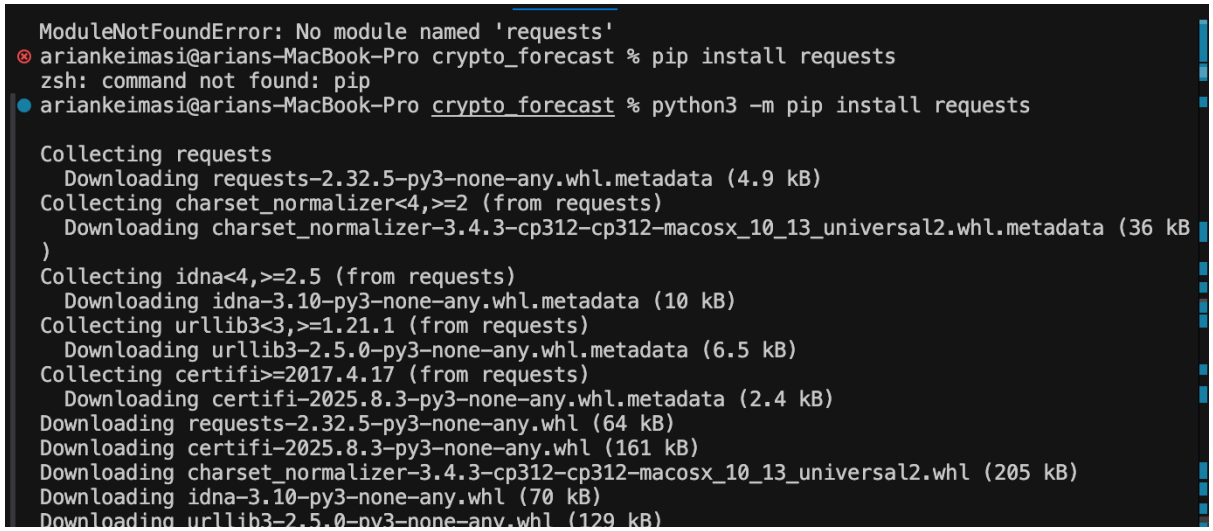
This stage addresses the interface design requirement identified in the system analysis for clear and balanced visual presentation. By resolving image sizing and placement at this point, the interface is prepared to accommodate additional graphical elements—such as charts and indicators—without disrupting layout stability or usability when live data and predictive components are introduced.

Live Price Integration and Real-Time Updates

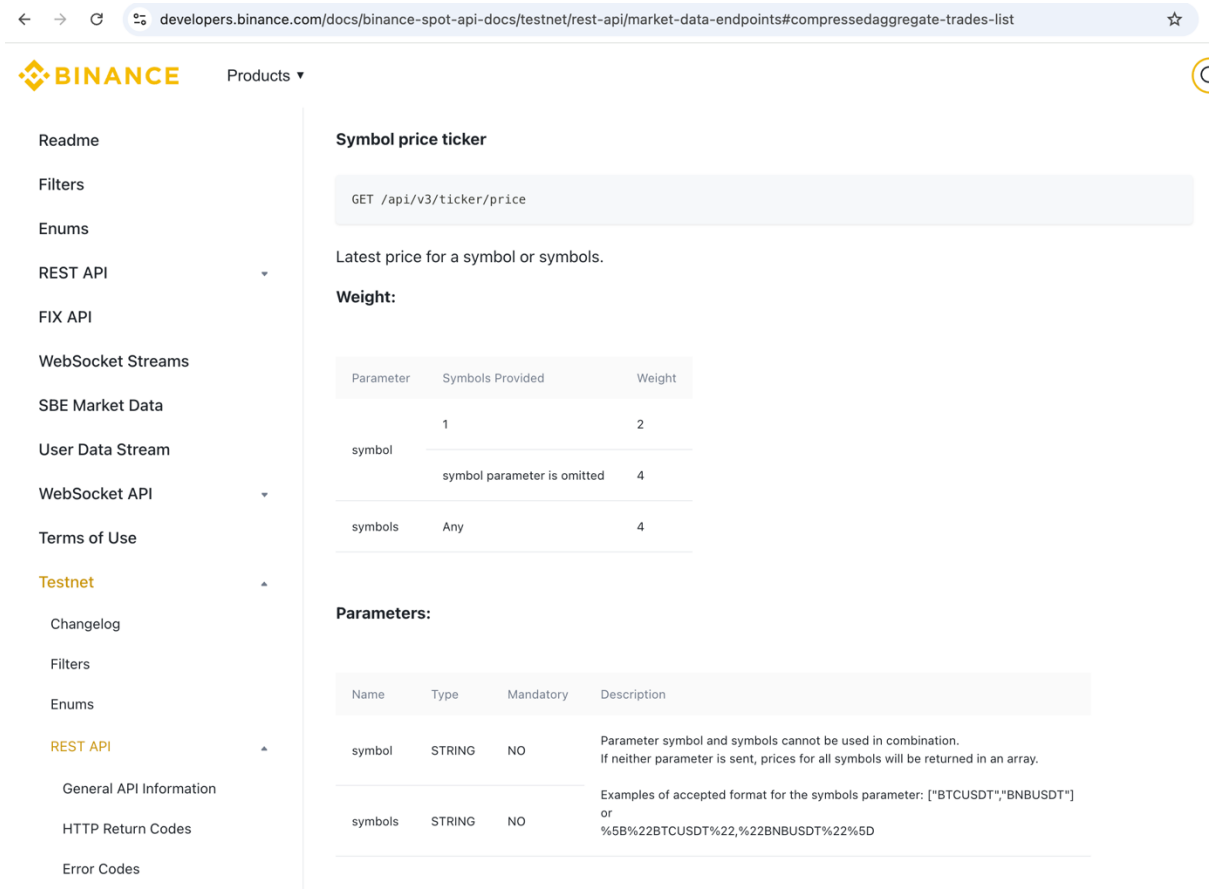
```
#button opening link
import webbrowser
def open_link():
    webbrowser.open("https://www.coinmarketcap.com/currencies/bitcoin/")
btn_link = tk.Button(window_1, text = "open bitcoin chat 📊", font=("Calibri",16),command=open_link())
#run the window (must be at end)
```

S.S.13: **Link opened immediately on startup** → **removed () from function call and placed button correctly**

* **TradingView and CoinMarketCap APIs unsuitable due to limited update requests** → **switched to Binance** after research



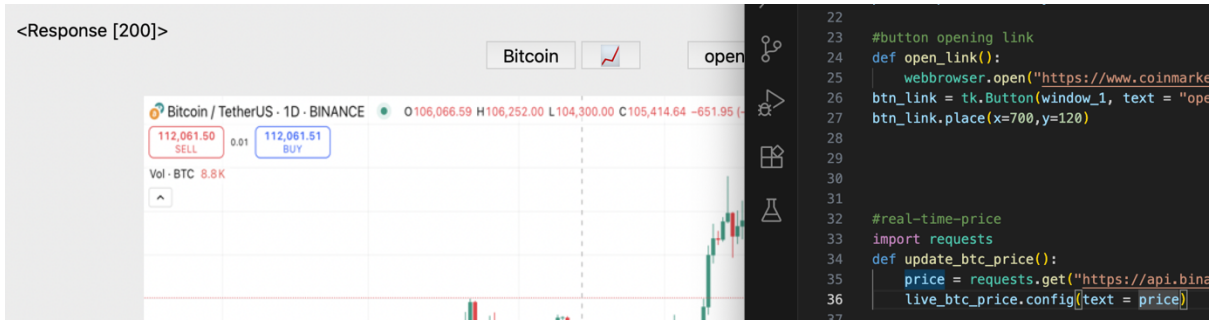
S.S.14: **Installed requests library** to fetch real-time prices from Binance



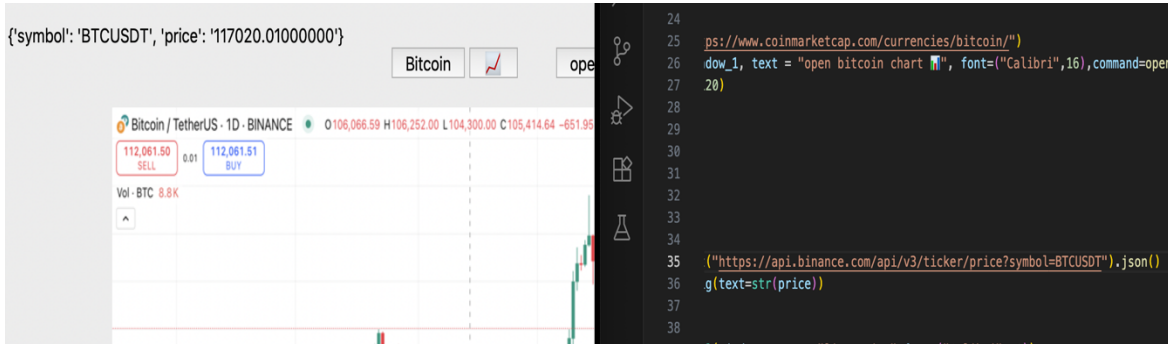
S.S.15: Found the **correct Binance API** URL format for Bitcoin prices
https://developers.binance.com/docs/binance-spot-api-docs/testnet/rest-api/market-data-endpoints#compressedaggregate-trades-list

```python
#real-time-price
import requests
def update_btc_price():
    price = requests.get("https://api.binance.com/api/v3/ticker/price?symbol=BTCUSDT")
    live_btc_price = price


live_btc_price = tk.Label(window_1, text="live price",font=("Calibri",16))
live_btc_price.place(x=100,y=100)
update_btc_price()
```

S.S.16: **Label created successfully**, but **price not displayed (only text shown)**



S.S.17: Label updated but **showed HTTP 200 response** instead of price



S.S.18: Parsed response with **.json();** full JSON shown → fixed by displaying price["price"] only

```
f update_btc_price():
  price = requests.get("https://api.binance.com/api/v3/ticker/price?symbol=BTCUSDT").json()["price"]
  live_btc_price.config(text=f"BTC Price: ${price}")
```

S.S.19: Corrected list/dict access **([] → {});** price now displays correctly. needs auto-refresh

```
window_1.after(5000, update_btc_price)
```

S.S.20: **Refresh delay** longer than expected (~10s instead of 5s, however it is said to be normal) → **reduced interval from 5000 to 2500** to compensate

```python
#real-time-price
import requests
last_price = None
def update_btc_price():
    global last_price
    price = requests.get("https://api.binance.com/api/v3/ticker/price?symbol=BTCUSDT").json()["price"]
    colour = "black"
    if last_price is not None:
        if price > last_price :
            colour = "green"
        elif price < last_price:
            colour = "red"
        else :
            colour = "black"
    live_btc_price.config(text=f"BTC Price: ${price}",fg=colour)
    last_price = price
    window_1.after(2500, update_btc_price)
```

S.S.21: **Red/green indicators** suitable for clear visual interpretation

```python
live_btc_price.config(text=f"BTC Price: ${price:,.2f}", fg=colour)
```

S.S.22: **ValueError**: Unknown format code 'f' for object of type 'str'

```
        colour = "black"
    live_btc_price.config(text="BTC Price: $" + "{:,.2f}".format(float(price)), fg=colour)
    last_price = price
    window.1.after(2500, update_btc_price)
```

```
BLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

                                    ^^^^^^^^^^^^
ueError: Unknown format code 'f' for object of type 'str'
ankeimasi@arians-MacBook-Pro crypto_forecast % /usr/local/bin/python3 /Users/ariankeimasi/Desktop/crypt
ceback (most recent call last):
ile "/Users/ariankeimasi/Desktop/crypto_forecast/crypto_forecast.py", line 55, in <module>
 update_btc_price()
ile "/Users/ariankeimasi/Desktop/crypto_forecast/crypto_forecast.py", line 46, in update_btc_price
 live_btc_price.config(text=f"BTC Price: ${price:,.2f}", fg=colour)
                                    ^^^^^^^^^^^^
ueError: Unknown format code 'f' for object of type 'str'
ankeimasi@arians-MacBook-Pro crypto_forecast % /usr/local/bin/python3 /Users/ariankeimasi/D
top/crypto_forecast/crypto_forecast.py
ceback (most recent call last):
ile "/Users/ariankeimasi/Desktop/crypto_forecast/crypto_forecast.py", line 55, in <module>
 update_btc_price()
ile "/Users/ariankeimasi/Desktop/crypto_forecast/crypto_forecast.py", line 46, in update_btc_price
 live_btc_price.config(text="BTC Price: $" + "{price:,.2f}".format(float(price)), fg=colour)
                                    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

Error: 'price'
ankeimasi@arians-MacBook-Pro crypto_forecast % /usr/local/bin/python3 /Users/ariankeimasi/D
```

BTC Price: $117,000.87

S.S.23: **Float conversion** fixed main issue; minor key error logged for later review

```
from datetime import datetime
now = datetime.now().strftime("%H:%M:%S")
last_update.config(text="Last update : {now}")
```

Last update : {now}

S.S.24: **Output did not match expected result**

```
last_update.config(text=f"Last update : {now}")
```

S.S.25: **Output corrected**

S.S.26:

Bitcoin window working → next: add Ethereum window + main menu; step 1: rewrite code using OOP

⇒ **Code explanation:**

This section integrates live Bitcoin price data into the application using the Binance REST API. A dedicated function sends HTTP GET requests to the API endpoint and retrieves the latest price data in JSON format. The response is parsed, and the numeric price value is extracted, converted into a float, and displayed dynamically within the GUI using a Tkinter Label.

A separate button is used to open an external Bitcoin chart link in the user's web browser, ensuring that external actions are triggered only through explicit user interaction. To keep the displayed data current without freezing the interface, the price-update function is repeatedly scheduled using Tkinter's after() method, enabling automatic refresh at fixed time intervals.

The system also tracks previous price values in order to determine price movement direction. Based on this comparison, the label colour is updated dynamically (red or green) to provide immediate visual feedback. A timestamp indicating the last successful update is displayed to inform users when the data was last refreshed.

⇒ **Justification:**

Live price data is essential for meeting the project's aim of simulating a real cryptocurrency dashboard. Several external APIs were evaluated. TradingView does not provide a free public REST API suitable for this use case, while CoinMarketCap imposes strict rate limits that restrict frequent updates. Binance was therefore selected because it provides a free, well-documented API that

supports regular polling without authentication, making it suitable for real-time price display within project constraints.

Automatic refreshing was chosen over manual updates to improve usability and realism, ensuring that users do not need to repeatedly interact with the interface to view current prices. Colour-coded indicators were included to support rapid interpretation of price movement, reducing cognitive load and improving accessibility during frequent updates. Displaying a timestamp further improves transparency by indicating data freshness.

⇒ **Iterative development:**

Initial testing showed that the external chart link opened immediately when the program started rather than when the button was clicked (S.S.13). This was caused by calling the function directly instead of passing it as a reference. The issue was resolved by removing the parentheses from the function call and ensuring the button was placed correctly.

When implementing live price retrieval, a module error occurred because the requests library was not installed (S.S.14). This was resolved by installing the library in the development environment. Further research was then conducted to identify the correct Binance API URL format for retrieving Bitcoin prices (S.S.15).

After creating the label to display the live price, the label initially showed only placeholder text (S.S.16). Updating the label incorrectly resulted in the HTTP response code being displayed instead

of the price value (S.S.17). This issue was resolved by parsing the API response using .json(). However, this initially displayed the full JSON object rather than just the numeric price (S.S.18). Correcting dictionary access allowed only the price field to be displayed, resolving the output issue (S.S.19). Automatic refreshing was then implemented using window_1.after(), although testing revealed that the refresh interval was longer than expected. To compensate, the refresh interval was reduced from 5000 ms to 2500 ms (S.S.20).

Visual indicators were added to show price direction using red and green colours, improving clarity and accessibility for users (S.S.21). A formatting error occurred when attempting to display the price to two decimal places due to the value being treated as a string (S.S.22). Converting the value to a float resolved the main issue, while a minor key error was identified and logged for later review (S.S.23).

Finally, a timestamp showing the last update time was added. The initial output did not match the expected format (S.S.24) but was corrected after adjusting string formatting (S.S.25). At this stage, the Bitcoin window was fully functional and stable (S.S.26).

⇒ **Validation:**

Normal testing confirmed that the live Bitcoin price was retrieved successfully, displayed correctly, and updated automatically without user intervention. Boundary testing verified that repeated refresh cycles did not freeze the interface and that reduced refresh intervals produced stable

behaviour over time. Usability testing showed that colour-coded indicators and timestamps improved clarity and supported rapid interpretation of price changes.

Together, these tests validate both the functional correctness of the live data pipeline and the suitability of the interface for real-time use. Evidence of these tests is recorded in the validation tables and annotated screenshots.

Test ID	Feature Tested	Test Type	Test Description	Expected Outcome	Actual Outcome	Action Taken	Result	Evidence
D1	Requests dependency	Erroneous	Run API request code	Request executes	requests missing	Installed requests	Pass	S.S.14
D2	Correct Binance endpoint	Normal	Test BTC endpoint format	Valid JSON returned	Incorrect initially	Correct endpoint identified	Pass	S.S.15
D3	Label shows live data	Normal	Update label with fetched value	Price displayed	Only label text shown	Corrected .config() usage	Pass	S.S.23
D4	HTTP response handling	Erroneous	Set label to response object	Price shown	Response 200 shown	Parsed using .json()	Pass	S.S.18
D5	JSON output control	Erroneous	Display .json()	Only numeric price	Full JSON shown	Accessed ["price"] key	Pass	S.S.23

D8	Auto-refresh	Boundary	Refresh every 5000ms	~5s updates	~10s typical delay	Reduced to 2500ms	Pass	S.S.20
D9	Color coded indication	Normal	Red/green feedback	Clear direction	Implemented	None	Pass	S.S.23
D10	Price formatting	Erroneous	Format to 2 d.p.	No error	ValueError on string	Converted to float	Pass	S.S.26
D11	Timestamp output	Erroneous	Show last update time	Correct format	Incorrect output	Fixed string formatting	Pass	S.S.26
D11	Auto-refresh stability	Normal	Run 2–5 minutes continuously	No freeze; updates continue	No freeze; updates continue	None	Pass	S.S.26
D12	Colour logic correctness	Boundary	Test up/down/no-change cases	Correct colours applied	Correct colours applied	Added Neutral state if needed	Pass	S.S.23

⇒ **Justification of Actions Taken During Testing:**

In Test D1, the API request could not execute because the requests library was missing. To resolve this, the requests library was installed. This approach was chosen because it provides a simple and reliable method for making HTTP requests in Python. This improved the functionality of the system by enabling communication with the Binance API.

In Test D2, the Binance endpoint was initially incorrect, resulting in invalid or no data being returned. To resolve this, the correct endpoint format was identified and implemented. This approach was chosen to ensure accurate data retrieval from the API. This improved the reliability of the system by ensuring valid JSON data is returned.

In Test D3, the label did not display the fetched price and only showed static text. This occurred due to incorrect use of the .config() method. To resolve this, the label update logic was corrected using .config(). This approach was chosen to dynamically update the GUI element with live data. This improved the usability of the system by ensuring real-time price information is visible to the user.

In Test D4, the label displayed the raw HTTP response (e.g. 200) instead of the price. To resolve this, the response was parsed using .json(). This approach was chosen because the API returns structured JSON data that must be accessed correctly. This improved the functionality of the system by enabling accurate data extraction.

In Test D5, the full JSON response was displayed instead of only the price value. To resolve this, the "price" key was accessed directly. This approach was chosen to extract only the relevant data needed for display. This improved the usability of the system by presenting clear and concise information.

In Test D8, the refresh interval was initially set to 5000ms, but updates occurred with noticeable delay. To resolve this, the interval was reduced to 2500ms. This approach was chosen to improve

the responsiveness of the system while maintaining performance. This improved the usability of the system by providing more timely updates.

In Test D10, formatting the price to two decimal places caused a ValueError due to the value being a string. To resolve this, the value was converted to a float before formatting. This approach was chosen to ensure compatibility with numeric formatting operations. This improved the reliability of the system by preventing runtime errors.

In Test D11, the timestamp output was initially incorrect due to formatting issues. To resolve this, the string formatting was corrected. This approach was chosen to ensure the timestamp is displayed in a clear and consistent format. This improved the usability of the system by providing accurate update information.

In Test D12, colour logic was extended to handle cases where there was no price change. To resolve this, a neutral state was added. This approach was chosen to ensure all possible conditions are handled. This improved the usability of the system by providing accurate visual feedback.

⇒ **Link to analysis & design:**

This stage implements the live data layer defined in the system design, transforming the interface from static content to real-time information display. With reliable price retrieval and update mechanisms in place, the design can now be extended to additional cryptocurrencies and integrated into a structured multi-window system without altering the underlying data pipeline.

Object-Oriented Restructuring and Scalability (OOP Transition)

```
super().__init__()
```

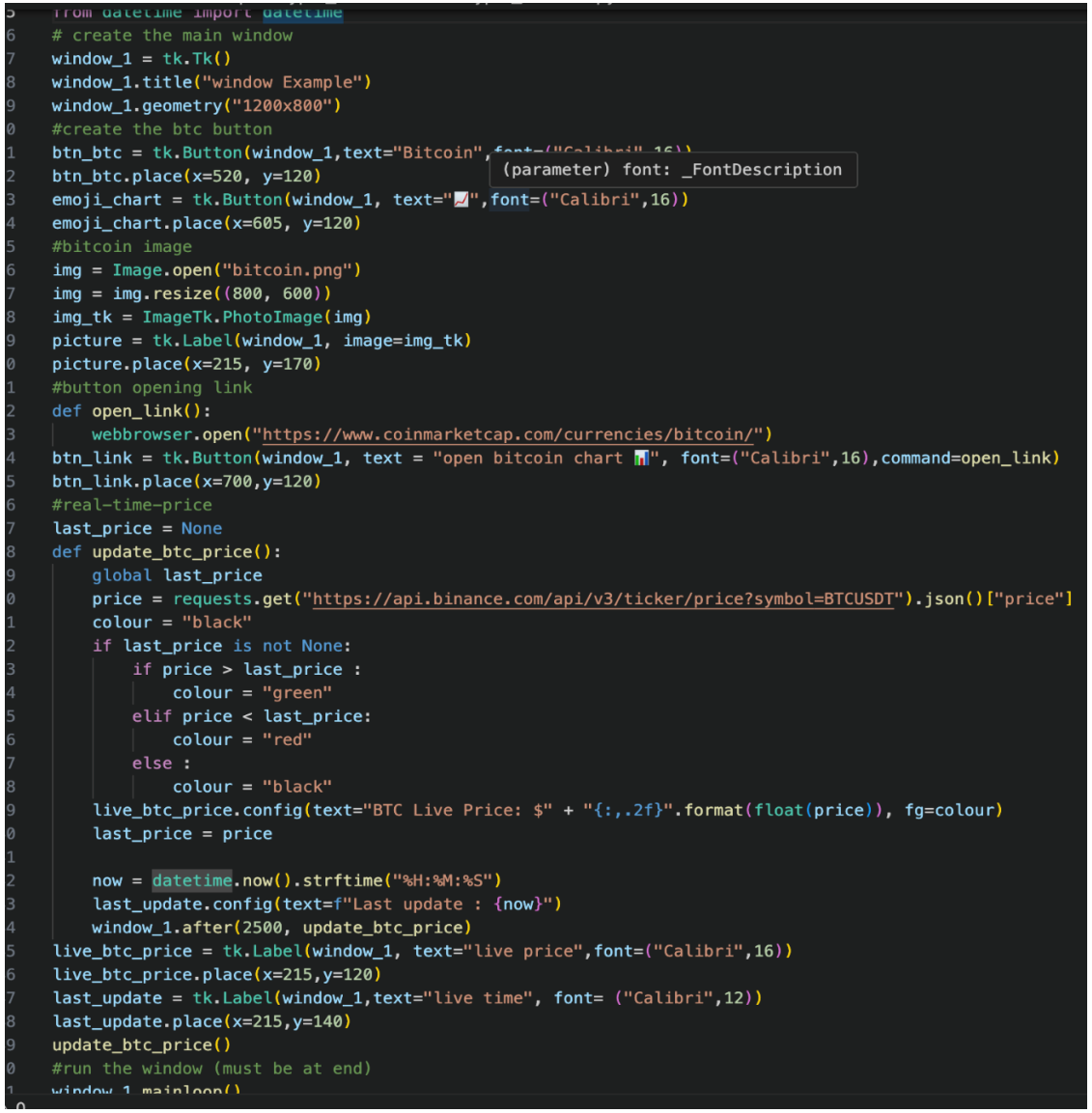
S.S.27: Parent tk.Tk **constructor not initialised** → fixed by calling **super().__init__()**

```
# start updating price
self.update_btc_price()
```

S.S.28: Price update function moved inside class initialisation to maintain **encapsulation**

```
BitcoinWindow().mainloop()
```

S.S.29: mainloop() moved outside class and executed once at program entry point



S.S.30: Initial code

```
class BitcoinWindow(tk.Tk):
    def __init__(self):
        super().__init__()

        # create the main window
        self.title("Bitcoin")
        self.geometry("1200x800")

        # create the btc button
        self.btn_btc = tk.Button(self, text="Bitcoin", font=("Calibri", 16))
        self.btn_btc.place(x=520, y=120)

        self.emoji_chart = tk.Button(self, text="📈", font=("Calibri", 16))
        self.emoji_chart.place(x=605, y=120)

        # bitcoin image
        self.img = Image.open("bitcoin.png")
        self.img = self.img.resize((800, 600))
        self.img_tk = ImageTk.PhotoImage(self.img)   # keep reference as self
        self.picture = tk.Label(self, image=self.img_tk)
        self.picture.place(x=215, y=170)

        # button opening link
        self.btn_link = tk.Button(self, text="open bitcoin chart 📊",
                                  font=("Calibri", 16), command=self.open_link)
        self.btn_link.place(x=700, y=120)

        # real-time-price labels
        self.live_btc_price = tk.Label(self, text="live price", font=("Calibri", 16))
        self.live_btc_price.place(x=215, y=120)

        self.last_update = tk.Label(self, text="live time", font=("Calibri", 12))
        self.last_update.place(x=215, y=140)

        self.last_price = None

        # start updating price
        self.update_btc_price()

    def open_link(self):
        webbrowser.open("https://www.coinmarketcap.com/currencies/bitcoin/")

    def update_btc_price(self):
        self.price = float(requests.get(
            "https://api.binance.com/api/v3/ticker/price?symbol=BTCUSDT"
        ).json()["price"])

        colour = "black"
        if self.last_price is not None:
            if self.price > self.last_price:
                colour = "green"
            elif self.price < self.last_price:
                colour = "red"
            else:
                colour = "black"

        self.live_btc_price.config(
            text="BTC Live Price: $" + "{:,.2f}".format(self.price),
            fg=colour
        )
        self.last_price = self.price

        now = datetime.now().strftime("%H:%M:%S")
        self.last_update.config(text=f"Last update : {now}")

        self.after(2500, self.update_btc_price)

BitcoinWindow().mainloop()
```

S.S.31: code in OOP

⇒ **Code explanation:**

In this stage, the program was restructured using an object-oriented approach to improve organisation, scalability, and maintainability. A custom class (BitcoinWindow) was created that inherits from tk.Tk, allowing the entire graphical interface and its related behaviour to be encapsulated within a single object.

The parent tk.Tk constructor is explicitly initialised using super().__init__(), ensuring that the Tkinter window is fully set up before any custom widgets or logic are added. All interface components, including buttons, labels, images, and live price elements, are defined as instance attributes (self.), allowing them to be accessed and updated consistently across methods.

Key behaviours such as opening external links and updating live prices were implemented as class methods. The event loop (mainloop()) is executed once at the program entry point after class instantiation, ensuring correct lifecycle management of the GUI.

⇒ **Justification:**

Refactoring the program into an object-oriented structure was necessary to support future expansion, particularly the planned addition of an Ethereum window and a main menu. Encapsulating functionality within a class reduces reliance on global variables, improves code readability, and allows related data and behaviour to be grouped logically.

Inheritance from tk.Tk provides a clean and modular foundation for managing multiple windows, while instance attributes ensure that GUI state (such as previous prices and update timestamps)

is preserved correctly between updates. Executing mainloop() only once outside the class prevents multiple event loops and ensures predictable program execution.
This structure aligns with good software engineering practice and makes the system easier to debug, extend, and maintain in later prototypes.

⇒ **Iterative development:**

Initial attempts to implement the class structure resulted in the program failing to run because the parent tk.Tk constructor had not been initialised. This was resolved by adding super().__init__(), ensuring the inherited window functionality was correctly set up (S.S.27).
During further testing, the live price update function was initially called externally, which broke encapsulation and caused inconsistent behaviour. The function was moved inside the class initialisation and invoked using self.update_btc_price(), ensuring that price updates operated within the object's internal state (S.S.28).
Another issue occurred when mainloop() was placed inside the class, which risked multiple event loops and unpredictable behaviour. This was corrected by moving mainloop() outside the class definition and calling it once at the program entry point (S.S.29).
A comparison between the original procedural implementation and the object-oriented version highlighted improved structure, clearer responsibility separation, and reduced repetition (S.S.30–S.S.31). This confirmed that the OOP transition improved both reliability and scalability.

⇒ **Validation:**

Functional testing confirmed that the object-oriented version of the program launched correctly, displayed all interface elements, and maintained live price updates without errors. State-based testing verified that price comparisons and colour indicators continued to function correctly after encapsulation.

Structural validation showed that all GUI components were successfully managed through instance attributes, with no reliance on global variables. Running the program confirmed that mainloop() executed once and that update scheduling (after()) remained stable.

These tests validate that the object-oriented restructuring preserved existing functionality while improving code organisation and future extensibility.

Test ID	Feature Tested	Test Type	Test Description	Expected Outcome	Actual Outcome	Action Taken	Result	Evidence
E1	Application launch	Normal	Run the programme after converting procedural code into a class inheriting from tk.Tk	Application launches successfully with GUI visible	Programme launches correctly and window displays as expected	None required	Pass	S.S.31
E2	Tk initialisation	Normal	Initialise the window using super().__init__() in the class constructor	Parent Tkinter window initialises correctly without runtime errors	GUI initialised correctly; programme no longer crashes on startup	Added super().__init__() call	Pass	S.S.27
E3	Widget scope	Normal	Verify that all widgets are created	Widgets accessible across	All widgets accessed	Refactored widgets	Pass	S.S.31

			as instance attributes (self.)	class methods without global variables	successfully within class methods	to instance attributes		
E4	Price update logic	Normal	Trigger live price updates after moving update logic inside the class	Price updates automatically using class state	Live price updates correctly within class context	Update function bound to class	Pass	S.S.28
E5	State persistence	Normal	Verify previous price is stored using self.last_price	Price comparison works across update cycles	Price direction logic works as intended	Used instance variable for state	Pass	S.S.31
E6	Feature preservation	Normal	Compare functionality before and after OOP conversion	All previous features remain functional	Buttons, image, live price, and updates still work	None required	Pass	S.S.31

⇒**Justification of Actions Taken During Testing:**

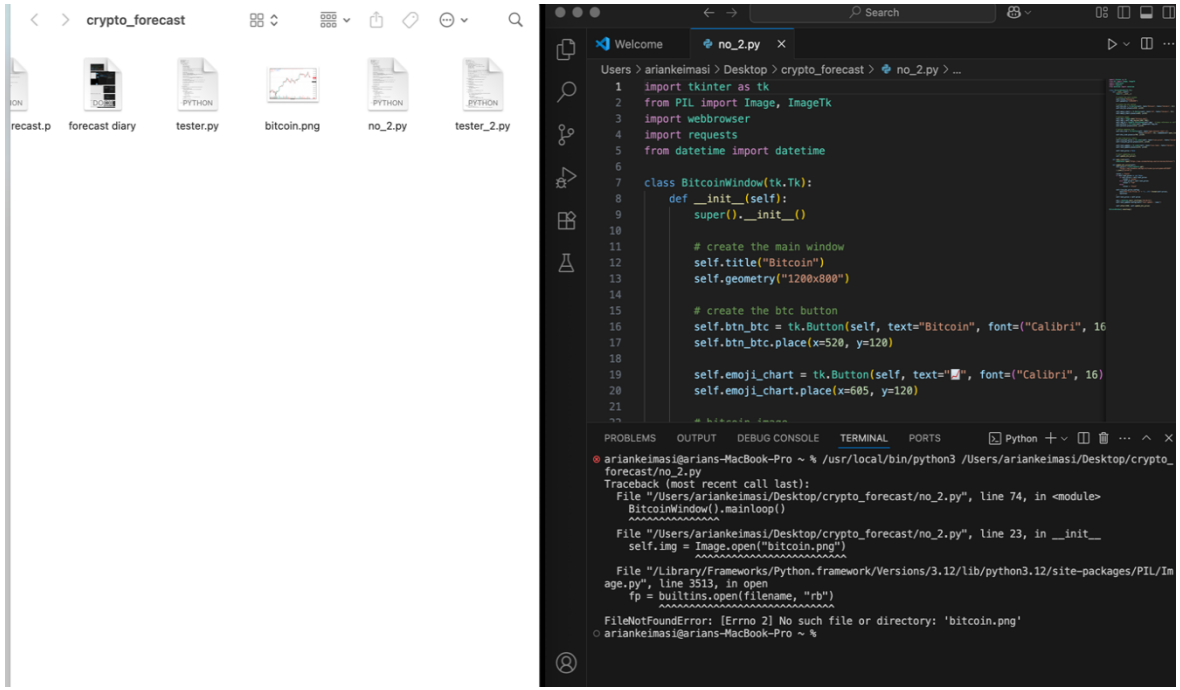
In Test E2, the programme initially failed to initialise the Tkinter window correctly. This occurred because the parent class constructor was not being called. To resolve this, super().__init__() was implemented in the class constructor. This approach was chosen because it correctly initialises the Tkinter root window when using inheritance. This improved the reliability of the system by preventing startup errors.

In Test E3, widgets were not consistently accessible across methods due to scope limitations. To resolve this, widgets were refactored as instance attributes using self. This approach was chosen to ensure all components are accessible within the class without relying on global variables. This improved the maintainability of the system by creating a clearer and more modular structure.

In Test E4, the price update logic did not function correctly after converting to a class structure. To resolve this, the update function was bound within the class. This approach was chosen to ensure the function operates using the class state. This improved the functionality of the system by enabling consistent live price updates.
In Test E5, price comparison did not persist across update cycles. To resolve this, an instance variable (self.last_price) was used to store previous values. This approach was chosen to maintain state within the class structure. This improved the reliability of the system by ensuring correct price direction logic.

⇒ **Link to analysis & design:**
This stage implements the structural design decision to organise the system using an object-oriented approach, enabling support for multiple cryptocurrencies within a single application framework. By encapsulating window behaviour and state within classes, the design now supports extension through additional windows and navigation logic without increasing coupling or code duplication.

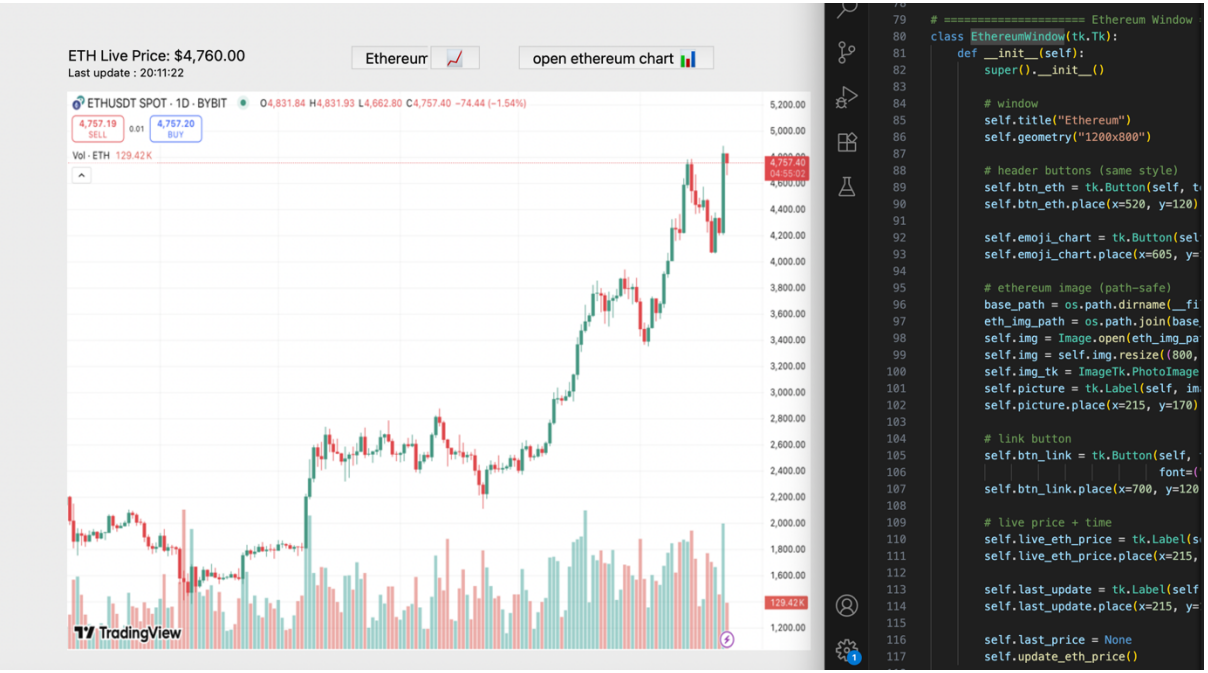
Main Menu and Multi-Window Navigation



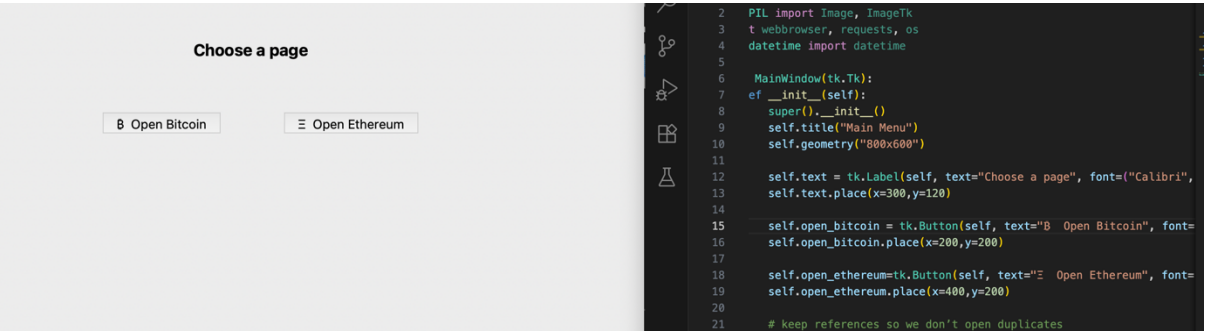
S.S.32: Relative image path failed after restructuring **FileNotFoundError** raised for bitcoin.png

```
### self.img = Image.open("bitcoin.png")
import os
base_path = os.path.dirname(__file__)
img_path = os.path.join(base_path, "bitcoin.png")
self.img = Image.open(img_path)
```

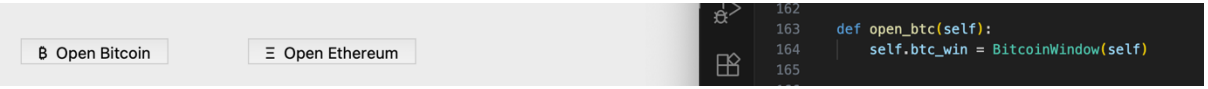
S.S.33: Resolved by **constructing absolute image path** (goes to same folder) using **__file__ and os.path.join()**



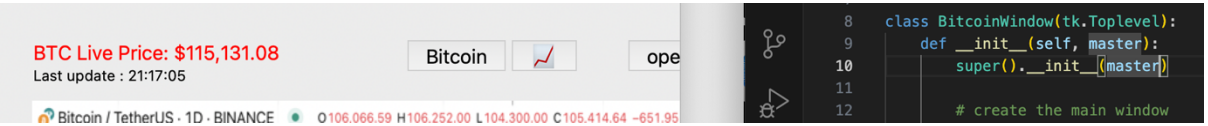
S.S.34: Code duplicated into new module and **adapted for Ethereum** (API, image, labels, buttons)



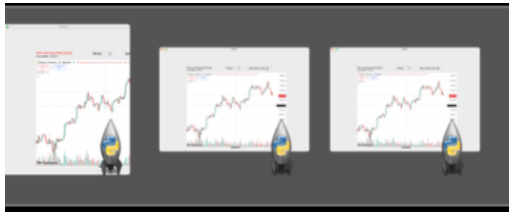
S.S.35: Main menu window created, but navigation buttons **not yet functional**



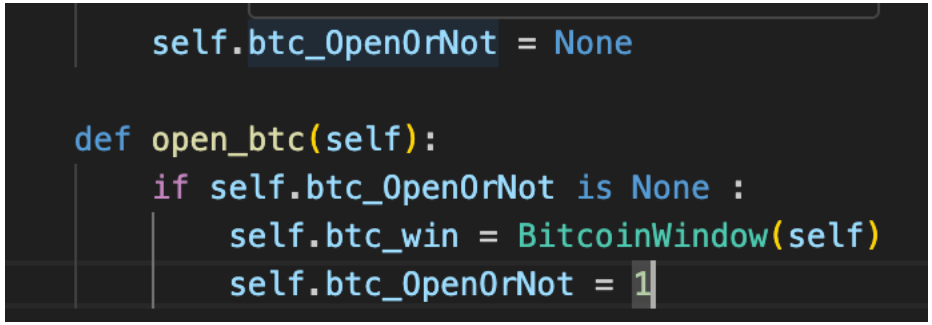
S.S.36: Button click registered, but no window opened → **missing child window logic**



S.S.37: **Refactored Bitcoin window to tk.Toplevel with parent reference** → fixed



S.S.38: Repeated clicks opened multiple Bitcoin windows → **no instance control**



S.S.39: Single-instance flag introduced → **prevented duplicates** but **blocked reopening**

```
def window_exit():
close = messagebox.askyesno("Exit?", "Are you sure you
if close:
    root.destroy()

root.protocol("WM_DELETE_WINDOW", window_exit)

root.mainloop()
```

```
        self.btc_OpenOrNot = None

    def open_btc(self):
        if self.btc_OpenOrNot is None :
            self.btc_win = BitcoinWindow(self)
            self.btc_OpenOrNot = 1
            self.btc_OpenOrNot.protocol("WM_DELETE_WINDOW")

    def close_btc(self):
        BitcoinWindow.destroy()
        self.btc_OpenOrNot = None
```

S.S.40: Window close event not resetting state → page could not reopen

```
def close_btc(self):
    self.BitcoinWindow.destroy()
    self.btc_OpenOrNot = None
```

S.S.41: **Incorrect window reference** used in close method → window not destroyed

```
def open_btc(self):
    if self.btc_OpenOrNot is None :
        self.btc_win = BitcoinWindow(self)
        self.btc_OpenOrNot.protocol("WM_DELETE_WINDOW", self.clo

def close_btc(self):
    self.BitcoinWindow.destroy()
    self.btc_OpenOrNot = None
```

S.S.42: **Corrected destroy reference to window instance** → close event works

```
        self.btc_OpenOrNot = None
        self.eth_OpenOrNot = None

    def open_btc(self):
        if self.btc_OpenOrNot is None :
            self.btc_win = BitcoinWindow(self)
            self.btc_OpenOrNot.protocol("WM_DELETE_WINDOW", self.close_btc)

    def close_btc(self):
        self.BitcoinWindow.destroy()
        self.btc_OpenOrNot = None


    def open_eth(self):
        if self.eth_OpenOrNot is None :
            self.eth_OpenOrNot = EthereumWindow(self)
            self.eth_OpenOrNot.protocol("WM_DELETE_WINDOW", self._close_eth)


    def _close_eth(self):
        self.EthereumWindow.destroy()
        self.eth_OpenOrNot = None
```

S.S.43: Ethereum window reused logic but failed due to **incorrect attribute reference**. **AttributeError identified** (EthereumWindow not stored as instance) → fixed

```
def open_btc(self):
    if self.btc_OpenOrNot is None :
        self.btc_win = BitcoinWindow(self)
        self.btc_OpenOrNot.protocol("WM_DELETE_WINDOW", self.close_btc)

def close_btc(self):
    self.BitcoinWindow.destroy()
    self.btc_OpenOrNot = None


def open_eth(self):
    if self.eth_OpenOrNot is None :
        self.eth_win = EthereumWindow(self)
        self.eth_OpenOrNot.protocol("WM_DELETE_WINDOW", self.close_eth)


def close_eth(self):
    self.EthereumWindow.destroy()
    self.eth_OpenOrNot = None
```

S.S.44: Rapid clicks occasionally bypassed flag → **duplicate windows still possible**

```
def open_btc(self):
    if self.btc_OpenOrNot is None or not self.btc_win.winfo_exists():
        self.btc_win = BitcoinWindow(self)
        self.btc_OpenOrNot.protocol("WM_DELETE_WINDOW", self.close_btc)

def close_btc(self):
    self.btc_win.destroy()
    self.btc_OpenOrNot = None
```

S.S.45: Added **winfo_exists()** check to verify window state before creation

```
def open_btc(self):
    if self.btc_OpenOrNot is None or not self.btc_OpenOrNot.winfo_exists():
        self.btc_OpenOrNot = BitcoinWindow(self)
        self.btc_OpenOrNot.protocol("WM_DELETE_WINDOW", self.close_btc)

def close_btc(self):
    self.btc_OpenOrNot.destroy()
    self.btc_OpenOrNot = None
```

S.S.46: Logic order caused **regression** → duplicate windows reintroduced

```
def open_btc(self):
    if self.btc_OpenOrNot is None or not self.btc_OpenOrNot.winfo_exists():
        self.btc_OpenOrNot = BitcoinWindow(self)
        self.btc_OpenOrNot.protocol("WM_DELETE_WINDOW", self.close_btc)
    else :
        self.btc_OpenOrNot.lift()
```

S.S.47: Final condition combined state **flag and existence check** → stable behaviour
Used.lift() for existing windows instead of recreating → fully resolved

⇒ **Code explanation:**

This section implements a Main Menu window that functions as the central navigation hub of the application. The menu contains two buttons, “Open Bitcoin” and “Open Ethereum”, each linked to a dedicated handler method (open_btc() and open_eth()). When triggered, these handlers open the corresponding cryptocurrency page as a child window using tk.Toplevel(master).

Using tk.Toplevel ensures that the main menu remains active and responsive while the user interacts with the selected cryptocurrency window, allowing multiple pages to coexist under a

single root application. This design supports smooth navigation without interrupting the programme's event loop.

To manage window behaviour efficiently, the system enforces single-instance window control. A reference to each child window is stored when it is created, and before opening a new window the code checks whether an instance already exists. If the window is already open, the system brings it to the foreground using .lift() instead of creating a duplicate.

In addition, a custom close handler is attached to each child window using protocol("WM_DELETE_WINDOW", ...). This ensures that when a window is closed, the stored reference and state flag are reset correctly, allowing the window to be reopened later without errors.

⇒ **Justification:**

Introducing a main menu significantly improves usability and programme structure by providing users with a clear and intuitive entry point. This is more effective than launching pages directly, as it reduces confusion and creates a consistent navigation flow across the application.

From a technical perspective, using tk.Toplevel instead of creating multiple tk.Tk() root windows follows best practice in Tkinter development. Tkinter applications are designed to operate with a single root event loop; introducing multiple root windows can lead to unpredictable behaviour, event-handling conflicts, and instability.

The use of single-instance window control is justified for both performance and user experience reasons. Allowing repeated creation of identical windows can lead to unnecessary memory usage,

duplicated API requests, reduced responsiveness, and user confusion. The .lift() method provides a user-friendly refinement: when a window already exists, it is simply brought to the front, making the system's behaviour predictable and intuitive rather than silently blocking input or recreating windows.

⇒ **Iterative development:**

The initial version of the main menu interface displayed correctly; however, clicking the navigation buttons produced no visible result because the child windows were not implemented using a proper parent–child structure.

To resolve this, the Bitcoin page was refactored into a tk.Toplevel(master) window with an explicit parent reference, which successfully fixed the opening behaviour.

Following this change, repeated button clicks caused multiple duplicate windows to open. To address this, a state flag and stored window reference were introduced to restrict window creation.

This first solution revealed a new issue: once a window was closed, it could not be reopened because the "open" state was not being reset. Further debugging showed that the close method was destroying an incorrect reference—the class name rather than the stored window instance—meaning the window never fully closed and the state remained incorrect. This was corrected by destroying the actual instance and resetting the reference to None.

When the same logic was applied to the Ethereum window, an attribute naming error caused an AttributeError, as the Ethereum window reference was not stored consistently. This was resolved by using a dedicated and correctly named instance variable (e.g. self.eth_win).

A final edge case was identified: very rapid button clicks could occasionally bypass the state flag, still allowing duplicates. This was fully resolved by combining three checks:

1. a stored window reference/state flag,
2. a winfo_exists() check to verify the window's existence, and
3. the use of .lift() to reuse an existing window instead of recreating it.

The final implementation produced stable, predictable behaviour under all tested conditions.

⇒**Validation:**

Validation testing confirmed that the main menu navigation and window-management logic functioned as intended. Functional testing showed that each button reliably opened the correct child window from the main menu without affecting the root application. When a window was already open, the system consistently reused the existing instance rather than creating duplicates, demonstrating effective instance control.

Boundary testing verified correct behaviour during window closure. Closing a child window correctly reset the stored reference, allowing the same page to be reopened without errors or stale state. Stress testing using rapid, repeated button clicks confirmed that the final condition checks prevented duplicate windows while maintaining responsive behaviour.

Overall, validation demonstrated that the navigation system is stable, resilient to rapid user interaction, and behaves consistently across repeated open–close cycles, meeting the functional requirements defined in the design.

Test ID	Feature Tested	Test Type	Test Description	Expected Outcome	Actual Outcome	Action Taken	Result	Evidence
F1	Main menu load	Normal	Launch programme and display main menu window	Main menu opens with two navigation buttons visible	Main menu displayed correctly	None required	Pass	S.S.35
F2	Bitcoin button	Normal	Click “Open Bitcoin” once	Bitcoin window opens as child window	Bitcoin window opened correctly	None required	Pass	Tested
F3	Ethereum button	Normal	Click “Open Ethereum” once	Ethereum window opens as child window	Ethereum window opened correctly	None required	Pass	S.S.34
F4	Duplicate prevention	Normal	Click “Open Bitcoin” multiple times	Only one Bitcoin window exists	Multiple windows opened initially	Added instance reference check	Pass	S.S.47
F5	Window reuse	Normal	Click “Open Bitcoin” when already open	Existing window brought to front	Existing window focused using .lift()	None required	Pass	Tested
F6	Window close handling	Boundary	Close Bitcoin window using window close button	Window closes and state resets	Window closed but could not reopen	Corrected destroy reference	Pass	Tested
F7	Reopen after close	Boundary	Close Bitcoin window then reopen	Window opens again normally	Window reopened successfully	None required	Pass	Tested
F8	Ethereum close handling	Boundary	Close Ethereum window	Window closes and state resets	AttributeError occurred initially	Fixed instance reference	Pass	S.S.43

F9	Rapid clicking	Erroneous	Rapidly click “Open Bitcoin” button	No duplicate windows created	Rare duplicates occurred	Added winfo_exists() check	Pass	S.S.45

⇒**Justification of Actions Taken During Testing:**

In Test F4, multiple Bitcoin windows were created when the button was clicked repeatedly. To resolve this, an instance reference check was implemented. This approach was chosen to ensure that only one instance of the window exists at any time. This improved the reliability of the system by preventing duplicate windows.

In Test F6, closing the Bitcoin window prevented it from being reopened. This occurred because the reference to the destroyed window was not reset. To resolve this, the destroy reference was corrected. This approach was chosen to ensure the application can create a new window after closure. This improved the functionality of the system by allowing the window to be reopened correctly.

In Test F8, closing the Ethereum window resulted in an AttributeError. This occurred due to an invalid or outdated instance reference. To resolve this, the instance reference was corrected. This approach was chosen to ensure proper handling of window objects after closure. This improved the reliability of the system by preventing runtime errors.

In Test F9, rapid clicking of the “Open Bitcoin” button occasionally created duplicate windows. To resolve this, a winfo_exists() check was implemented. This approach was chosen to verify whether

the window already exists before creating a new instance. This improved the reliability of the system by preventing duplicate windows under rapid user interaction.

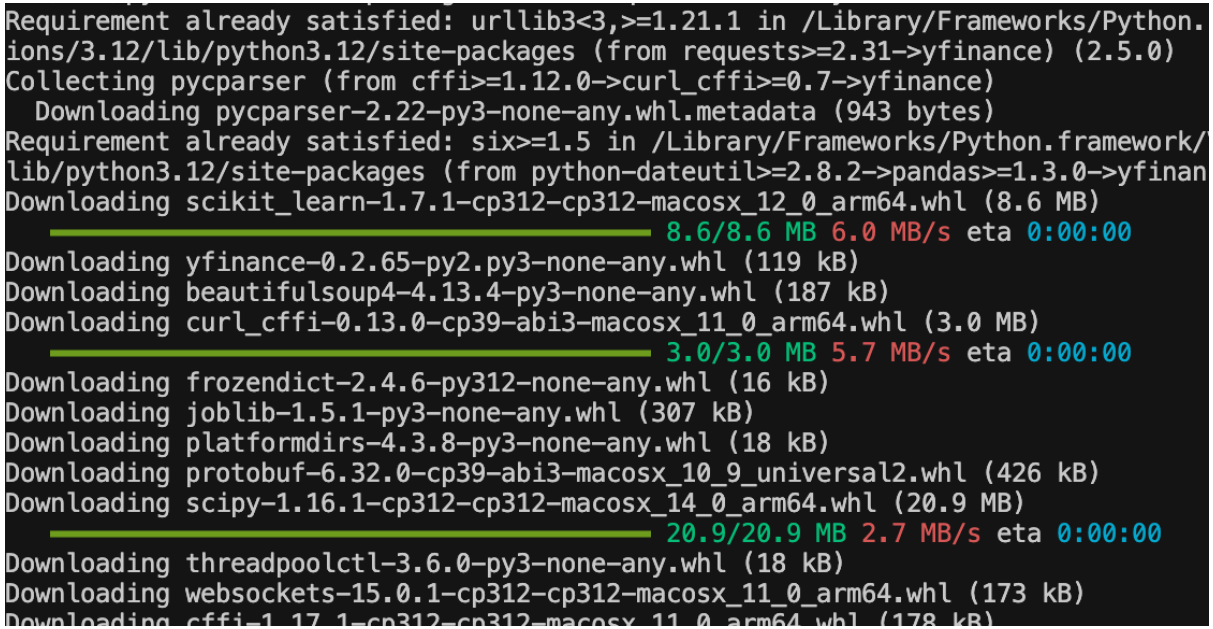
⇒ **Link back to analysis & design:**
This section extends Prototype 2 into a more complete multi-page application by introducing a structured navigation layer and robust window-management logic. It directly supports the system's design objective of providing a dashboard-style interface in which users can switch between cryptocurrency pages without instability, duplicated windows, or confusing behaviour. The final solution also improves scalability: additional cryptocurrency pages can be added easily by reusing the same navigation and single-instance control pattern. This keeps the codebase modular, maintainable, and aligned with the original system analysis and design principles.

Data Collection

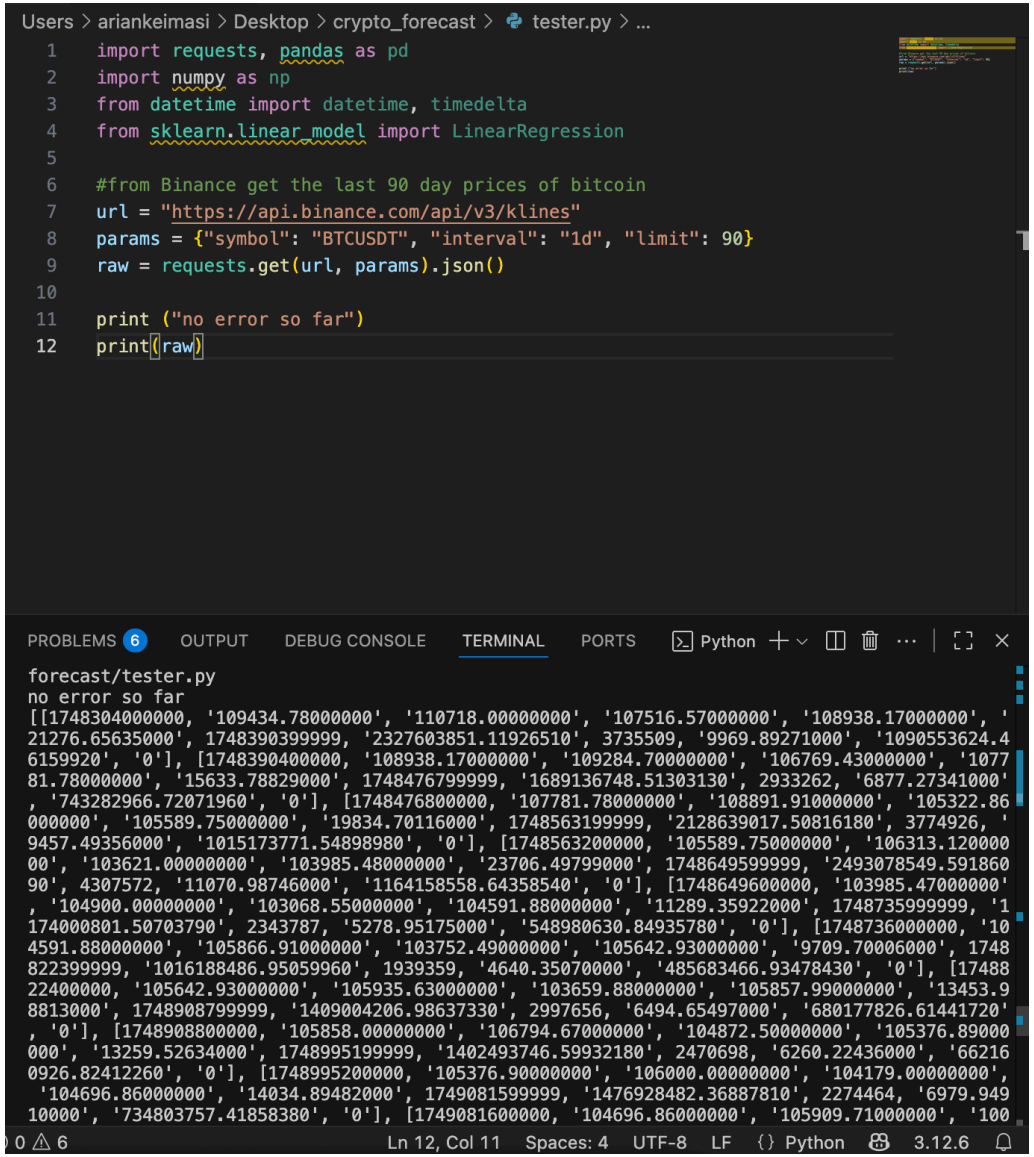
```
zsh: command not found: pip
ariankeimasi@arians-MacBook-Pro ~ % pip3 install pandas
Collecting pandas
  Downloading pandas-2.3.2-cp312-cp312-macosx_11_0_arm64.whl.metadata (91 kB)
Collecting numpy>=1.26.0 (from pandas)
  Downloading numpy-2.3.2-cp312-cp312-macosx_14_0_arm64.whl.metadata (62 kB)
Collecting python-dateutil>=2.8.2 (from pandas)
  Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)
Collecting pytz>=2020.1 (from pandas)
  Downloading pytz-2025.2-py2.py3-none-any.whl.metadata (22 kB)
Collecting tzdata>=2022.7 (from pandas)
  Downloading tzdata-2025.2-py2.py3-none-any.whl.metadata (1.4 kB)
Collecting six>=1.5 (from python-dateutil>=2.8.2->pandas)
  Downloading six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
Downloading pandas-2.3.2-cp312-cp312-macosx_11_0_arm64.whl (10.7 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.7/10.7 MB 1.8 MB/s eta 0:00:00
Downloading numpy-2.3.2-cp312-cp312-macosx_14_0_arm64.whl (5.1 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5.1/5.1 MB 3.1 MB/s eta 0:00:00
```

S.S.48: Required **data libraries installed (pandas, numpy)** to support data processing and analysis.

```
Requirement already satisfied: pytz>=2022.5 in /Library/Frameworks/Python.framework/Ver
.12/lib/python3.12/site-packages (from yfinance) (2025.2)
Collecting frozendict>=2.3.4 (from yfinance)
  Downloading frozendict-2.4.6-py312-none-any.whl.metadata (23 kB)
Collecting peewee>=3.16.2 (from yfinance)
  Downloading peewee-3.18.2.tar.gz (949 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 949.2/949.2 kB 6.3 MB/s eta 0:00:00
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Collecting beautifulsoup4>=4.11.1 (from yfinance)
  Downloading beautifulsoup4-4.13.4-py3-none-any.whl.metadata (3.8 kB)
Collecting curl_cffi>=0.7 (from yfinance)
  Downloading curl_cffi-0.13.0-cp39-abi3-macosx_11_0_arm64.whl.metadata (13 kB)
Collecting protobuf>=3.19.0 (from yfinance)
  Downloading protobuf-6.32.0-cp39-abi3-macosx_10_9_universal2.whl.metadata (593 bytes)
Collecting websockets>=13.0 (from yfinance)
  Downloading websockets-15.0.1-cp312-cp312-macosx_11_0_arm64.whl.metadata (6.8 kB)
Collecting soupsieve>1.2 (from beautifulsoup4>=4.11.1->yfinance)
```

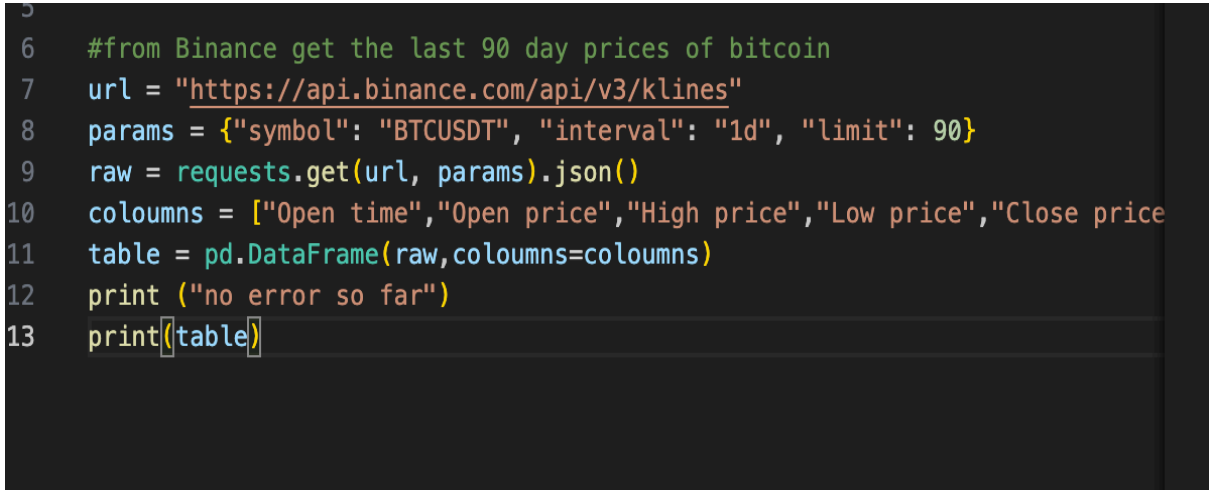


S.S.49: Additional libraries tested (**scikit-learn, yfinance**); yfinance rejected due to limited flexibility compared to direct Binance API.



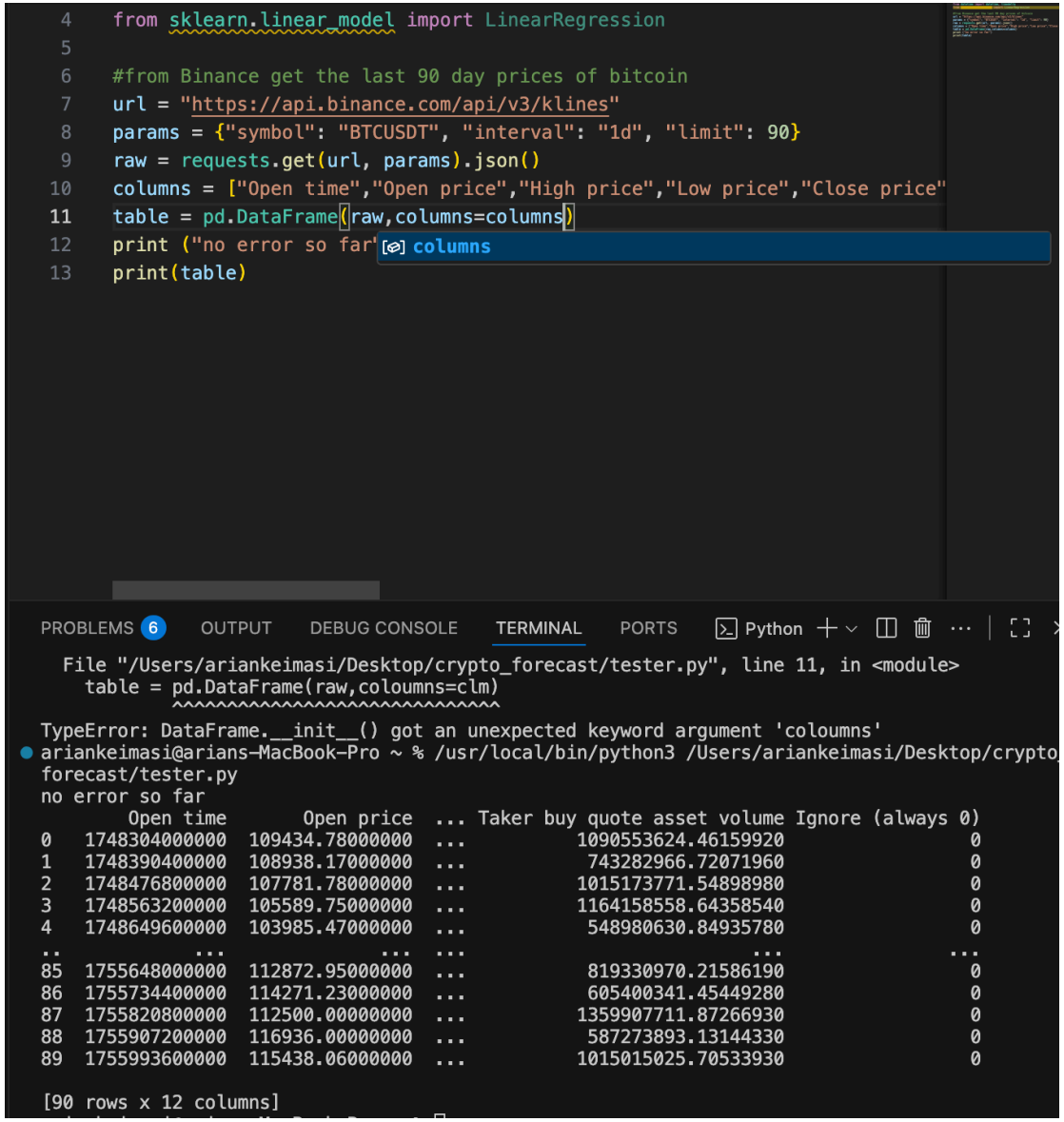
S.S.50: Successfully **retrieved last 90 days of Bitcoin OHLCV data from Binance API** (raw JSON output)

For each trading day, the Binance API returns a single candlestick record in the following format:
[open_time, open, high, low, close, volume, close_time, qav, num_trades, taker_base_vol, taker_quote_vol, ignore]
Where:
open_time – start time of the candlestick (to be converted into a readable date and time)
open – price at the beginning of the day
high – highest price reached during the day
low – lowest price reached during the day
close – price at the end of the day
volume – total trading volume for the day (base asset, BTC)
close_time – end time of the candlestick (to be converted into a readable date and time)
qav (quote asset volume) – trading volume measured in the quote asset (USDT)
num_trades – total number of trades executed during the day
taker_base_vol – volume of BTC traded by taker orders
taker_quote_vol – value of taker trades measured in USDT
ignore – placeholder field that is always zero and not used



```
#from Binance get the last 90 day prices of bitcoin
url = "https://api.binance.com/api/v3/klines"
params = {"symbol": "BTCUSDT", "interval": "1d", "limit": 90}
raw = requests.get(url, params).json()
coloumns = ["Open time","Open price","High price","Low price","Close price
table = pd.DataFrame(raw,coloumns=coloumns)
print ("no error so far")
print(table)
```

S.S.51: Initial attempt to convert raw API data into DataFrame caused TypeError due to **incorrect constructor usage.**


4 from sklearn.linear_model import LinearRegression
5
6 #from Binance get the last 90 day prices of bitcoin
7 url = "https://api.binance.com/api/v3/klines"
8 params = {"symbol": "BTCUSDT", "interval": "1d", "limit": 90}
9 raw = requests.get(url, params).json()
10 columns = ["Open time","Open price","High price","Low price","Close price"
11 table = pd.DataFrame(raw,columns=columns)
12 print ("no error so far"
columns
13 print(table)
PROBLEMS 6
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Python
File "/Users/ariankeimasi/Desktop/crypto_forecast/tester.py", line 11, in <module>
table = pd.DataFrame(raw,coloumns=clm)
TypeError: DataFrame.__init__() got an unexpected keyword argument 'coloumns'
ariankeimasi@arians-MacBook-Pro ~ % /usr/local/bin/python3 /Users/ariankeimasi/Desktop/crypto_forecast/tester.py
no error so far
Open time Open price ... Taker buy quote asset volume Ignore (always 0)
0 1748304000000 109434.78000000 ... 1090553624.46159920 0
1 1748390400000 108938.17000000 ... 743282966.72071960 0
2 1748476800000 107781.78000000 ... 1015173771.54898980 0
3 1748563200000 105589.75000000 ... 1164158558.64358540 0
4 1748649600000 103985.47000000 ... 548980630.84935780 0
.. ..
85 1755648000000 112872.95000000 ... 819330970.21586190 0
86 1755734400000 114271.23000000 ... 605400341.45449280 0
87 1755820800000 112500.00000000 ... 1359907711.87266930 0
88 1755907200000 116936.00000000 ... 587273893.13144330 0
89 1755993600000 115438.06000000 ... 1015015025.70533930 0
[90 rows x 12 columns]

S.S.52: DataFrame creation corrected; **full 12-column dataset loaded successfully**, though not all fields required.

```
11    table = pd.DataFrame(raw,columns=columns)
12    table["date"]   = pd.to_datetime(table["Open time"], unit="ms")
13    table["close"]  = table["Close price"].astype(float)
14    table["volume"] = table["Volume (Base=BTC)"].astype(float)
15    print ("no error so far")
16    print(table)
```

PROBLEMS 6 OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
  File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages/pandas
/core/indexes/base.py", line 3819, in get_loc
    raise KeyError(key) from err
KeyError: 'Close Price'
ariankeimasi@arians-MacBook-Pro ~ % /usr/local/bin/python3 /Users/ariankeimasi/Desktop/crypto_
forecast/tester.py
no error so far
        Open time       Open price       High price  ...       date      close       volume
0   1748304000000  109434.78000000  110718.00000000  ... 2025-05-27  108938.17  21276.65635
1   1748390400000  108938.17000000  109284.70000000  ... 2025-05-28  107781.78  15633.78829
2   1748476800000  107781.78000000  108891.91000000  ... 2025-05-29  105589.75  19834.70116
3   1748563200000  105589.75000000  106313.12000000  ... 2025-05-30  103985.48  23706.49799
4   1748649600000  103985.47000000  104900.00000000  ... 2025-05-31  104591.88  11289.35922
..            ...              ...              ...  ...        ...        ...         ...
85  1755648000000  112872.95000000  114615.38000000  ... 2025-08-20  114271.24  15636.34194
86  1755734400000  114271.23000000  114821.76000000  ... 2025-08-21  112500.00  10839.69235
87  1755820800000  112500.00000000  117429.05000000  ... 2025-08-22  116935.99  23128.09037
88  1755907200000  116936.00000000  117030.00000000  ... 2025-08-23  115438.05  11329.36197
89  1755993600000  115438.06000000  115666.68000000  ... 2025-08-24  113372.26  20395.50891

[90 rows x 15 columns]
```

S.S.53: **Data cleaning and type conversion applied** (timestamps to datetime, prices and volume to float).

```
13    table["close"]  = table["Close price"].astype(float)
14    table["volume"] = table["Volume (Base=BTC)"].astype(float)
15    table = table[["date","close","volume"]]
16    print ("no er table
17    print(table)
```

PROBLEMS 6 OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
88  1755907200000  116936.00000000  117030.00000000  ... 2025-08-23  11
89  1755993600000  115438.06000000  115666.68000000  ... 2025-08-24  11

[90 rows x 15 columns]
ariankeimasi@arians-MacBook-Pro ~ % /usr/local/bin/python3 /Users/arian
forecast/tester.py
no error so far
         date      close       volume
0  2025-05-27  108938.17  21276.65635
1  2025-05-28  107781.78  15633.78829
2  2025-05-29  105589.75  19834.70116
3  2025-05-30  103985.48  23706.49799
4  2025-05-31  104591.88  11289.35922
..        ...        ...          ...
85 2025-08-20  114271.24  15636.34194
86 2025-08-21  112500.00  10839.69235
87 2025-08-22  116935.99  23128.09037
88 2025-08-23  115438.05  11329.36197
89 2025-08-24  113275.95  20416.87831

[90 rows x 3 columns]
ariankeimasi@arians-MacBook-Pro ~ %
```

S.S.54: Final dataset reduced to relevant features (date, close, volume) → **clean, model-ready structure achieved**.

⇒**Code explanation:**

This section implements automated historical data collection for Bitcoin using the Binance REST API endpoint for candlestick ("kline") data. The script sends a request to the /api/v3/klines endpoint with parameters such as symbol=BTCUSDT, interval=1d, and limit=90, returning the most recent 90 daily candles (each day includes multiple fields such as open time, open/high/low/close, volumes, and trade count).

The returned JSON is then converted into a pandas DataFrame to make the dataset easier to inspect, clean, and later feed into the prediction stage. To prepare the dataset for modelling, the code:

defines column names that match the returned kline structure,

converts timestamp fields into readable dates using pd.to_datetime(..., unit="ms"),

converts numeric columns into floats using .astype(float) to avoid string-math errors, and

reduces the dataset to only the features required for the next stage (e.g., date, close, volume) by selecting a subset of columns.

This produces a clean tabular dataset (90 rows) that is consistent and ready for forecasting logic.

⇒**Justification:**

Historical price data is required because prediction models cannot operate reliably from a single live price point; they need a time series that captures short-term variation and trend. Binance was used because it provides free, structured OHLCV candlestick data with simple parameters (symbol/interval/limit) and without requiring paid keys for basic use cases, which suits an NEA-level prototype.

Using pandas is justified because it provides an efficient and readable way to clean and restructure datasets (type conversion, date parsing, and column selection) before modelling. Converting fields to the correct data types is essential: prediction and analysis steps require numeric arrays rather than strings, and the date field must be human-interpretable for debugging and evaluation.

⇒ **Iterative development:**

During implementation, the raw API output initially printed as a long nested list, which confirmed the request worked but made the data difficult to interpret or reuse. The dataset was then converted into a DataFrame, but an early mistake caused a TypeError when trying to pass columns incorrectly; after correcting the DataFrame construction, the tabular output displayed properly.

Once the DataFrame was visible, the next iteration focused on feature selection: the API returns many columns that are not required for the planned forecasting approach. The script therefore extracted only the fields needed for later modelling and evaluation (especially closing price and volume), and created a readable date column by converting timestamps.

A further refinement addressed a KeyError caused by mismatched column naming (e.g., using "Close Price" when the DataFrame column was "Close price"). This was fixed by aligning labels consistently and then sub-setting the table to a minimal “model-ready” dataset. The final result was a stable 90-row dataset with clear data types and only relevant columns.

⇒ **Validation:**

Validation combined functional and data-quality checks. Normal testing confirmed the API request successfully returned exactly 90 daily candles and that the DataFrame was created without

runtime errors. Structural validation verified that the dataset contained the expected columns and that timestamp/numeric conversions produced usable types (dates as datetime, prices/volume as floats). Finally, a data-integrity check confirmed that the refined dataset kept only required features (date, close, volume) while maintaining the correct number of rows for modelling.

Test ID	Feature Tested	Test Type	Test Description	Expected Outcome	Actual Outcome	Action Taken	Result	Evidence
G1	API connection (klines)	Normal	Run request to Binance /api/v3/klines with BTCUSDT, 1d, limit=90.	JSON list returned with 90 entries.	Data returned and printed successfully.	None required.	Pass	S.S.50
G2	Dataset size	Boundary	Check number of returned rows equals limit (90).	Exactly 90 daily records.	Output shows 90 records.	None required.	Pass	S.S.50
G3	DataFrame creation	Normal	Convert JSON output into pandas DataFrame.	DataFrame created without errors.	Initial attempt raised TypeError (bad argument).	Corrected DataFrame constructor usage.	Pass	S.S.52
G4	Date parsing	Normal	Convert open time (ms) into readable datetime via pd.to_datetime(..., unit="ms").	New date column created correctly.	Date column produced correctly.	None required.	Pass	S.S.53
G5	Feature selection	Normal	Reduce dataset to required fields (date, close, volume).	Minimal model-ready table produced.	Table displayed with selected columns only.	None required.	Pass	S.S.54

⇒ **Justification of Actions Taken During Testing:**

In Test G3, the initial attempt to create a DataFrame resulted in a TypeError due to incorrect argument usage. To resolve this, the DataFrame constructor was corrected. This approach was

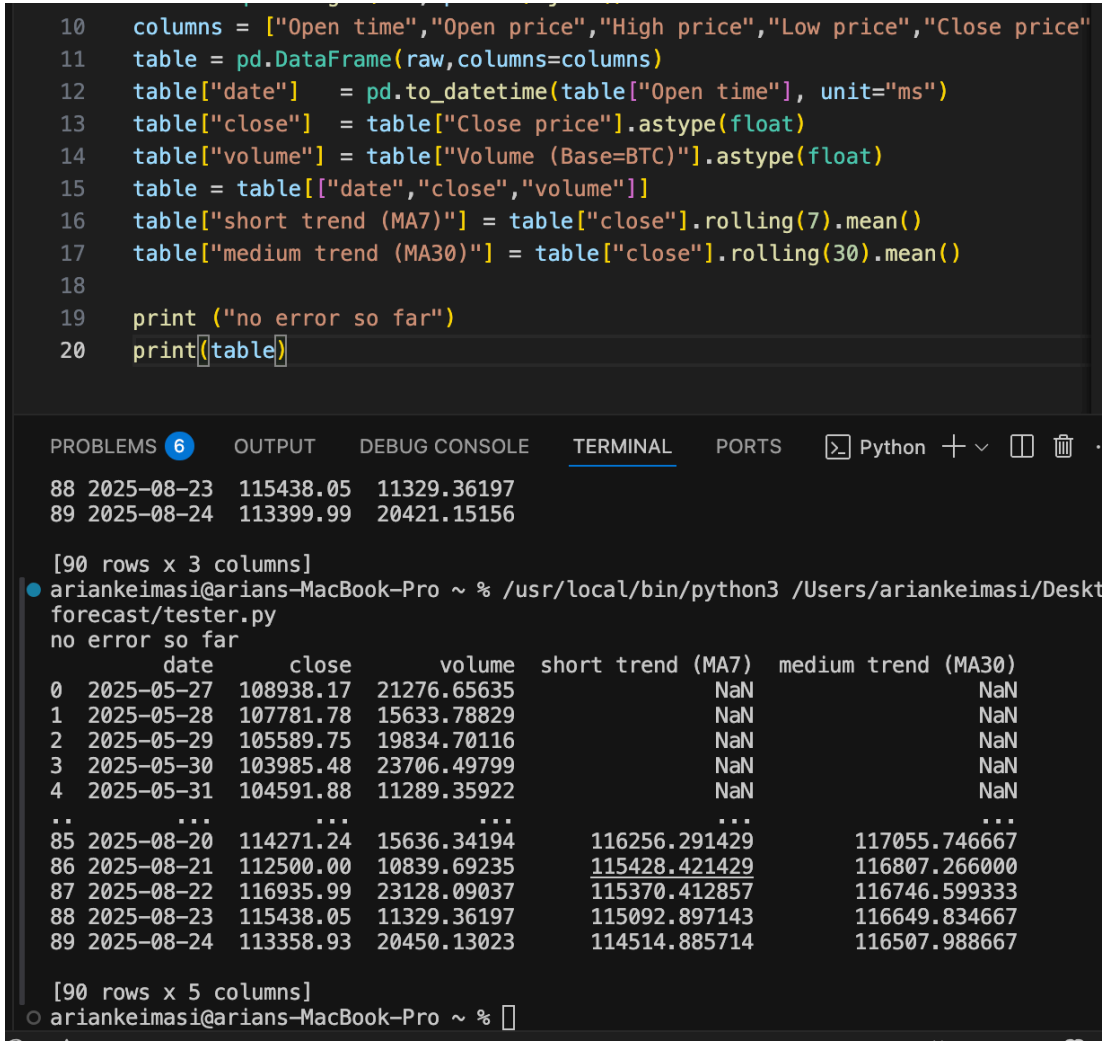
chosen to ensure the JSON data is properly structured into a tabular format compatible with pandas. This improved the functionality of the system by enabling further data processing and analysis.

⇒ **Link back to analysis & design:**

This stage directly implements the data acquisition and preparation layer defined in the system design, where historical price information was identified as a prerequisite for any forecasting or analytical component. By structuring 90 days of Binance OHLCV data into a clean, typed DataFrame, the system now satisfies the design requirement for a reliable, model-ready time series input.

The reduction of raw API output into selected features (date, close, volume) aligns with the original analysis decision to prioritise interpretability, performance, and scalability rather than storing unnecessary attributes. This structured dataset forms the foundation for the next design stage, where prediction algorithms can be applied consistently across cryptocurrencies without modifying the data pipeline.

Moving Averages + Linear Regression Pipeline (Data Preparation + First Prediction Engine)



S.S.55: Rolling MA7 / MA30 produce **NaN** → insufficient prior data for calculation

```
18    table = table.dropna()
19    print ("no error so far")
20    print(table)
```

PROBLEMS 6 OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
[71 rows x 5 columns]
Last date in data: 2025-08-24
Predicted next close: $113,501.02
ariankeimasi@arians-MacBook-Pro ~ % /usr/local/bin/python3 /Users/ariankeimasi/Des
forecast/tester.py
no error so far
          date      close       volume  short trend (MA7)  medium trend (MA30)
29 2025-06-25  107340.58  16701.15551      104256.852857        105583.498333
30 2025-06-26  106947.06  10573.27279      104583.777143        105517.128000
31 2025-06-27  107047.59  12232.44042      105119.434286        105492.655000
32 2025-06-28  107296.79   3282.17352      105858.974286        105549.556333
33 2025-06-29  108356.93   6831.73640      106915.125714        105695.271333
..         ...        ...          ...                ...                  ...
85 2025-08-20  114271.24  15636.34194      116256.291429        117055.746667
86 2025-08-21  112500.00  10839.69235      115428.421429        116807.266000
87 2025-08-22  116935.99  23128.09037      115370.412857        116746.599333
88 2025-08-23  115438.05  11329.36197      115092.897143        116649.834667
89 2025-08-24  113480.24  20504.26005      114532.215714        116512.032333
```

S.S.56: Used dropna() → first 29 rows removed (no MA30 available)
Index became **non-sequential**

```
19    table = table.reset_index()
20    print ("no error so far")
21    print(table)
```

PROBLEMS 6 OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
88 2025-08-23  115438.05  11329.36197      115092.897143        116649.834667
89 2025-08-24  113480.24  20504.26005      114532.215714        116512.032333

[61 rows x 5 columns]
ariankeimasi@arians-MacBook-Pro ~ % /usr/local/bin/python3 /Users/ariankeimasi/Desktop/cr
forecast/tester.py
no error so far
    index       date      close       volume  short trend (MA7)  medium trend (MA30)
0      29 2025-06-25  107340.58  16701.15551      104256.852857        105583.498333
1      30 2025-06-26  106947.06  10573.27279      104583.777143        105517.128000
2      31 2025-06-27  107047.59  12232.44042      105119.434286        105492.655000
3      32 2025-06-28  107296.79   3282.17352      105858.974286        105549.556333
4      33 2025-06-29  108356.93   6831.73640      106915.125714        105695.271333
..    ...         ...        ...          ...                ...                  ...
56     85 2025-08-20  114271.24  15636.34194      116256.291429        117055.746667
57     86 2025-08-21  112500.00  10839.69235      115428.421429        116807.266000
58     87 2025-08-22  116935.99  23128.09037      115370.412857        116746.599333
59     88 2025-08-23  115438.05  11329.36197      115092.897143        116649.834667
60     89 2025-08-24  113436.98  20517.14922      114526.035714        116510.590333
```

S.S.57: Applied **reset_index()** → index corrected but old index column added

```
18    table = table.dropna()
19    table = table.reset_index(drop=True)
20    print ("no error so far")
21    print(table)
```

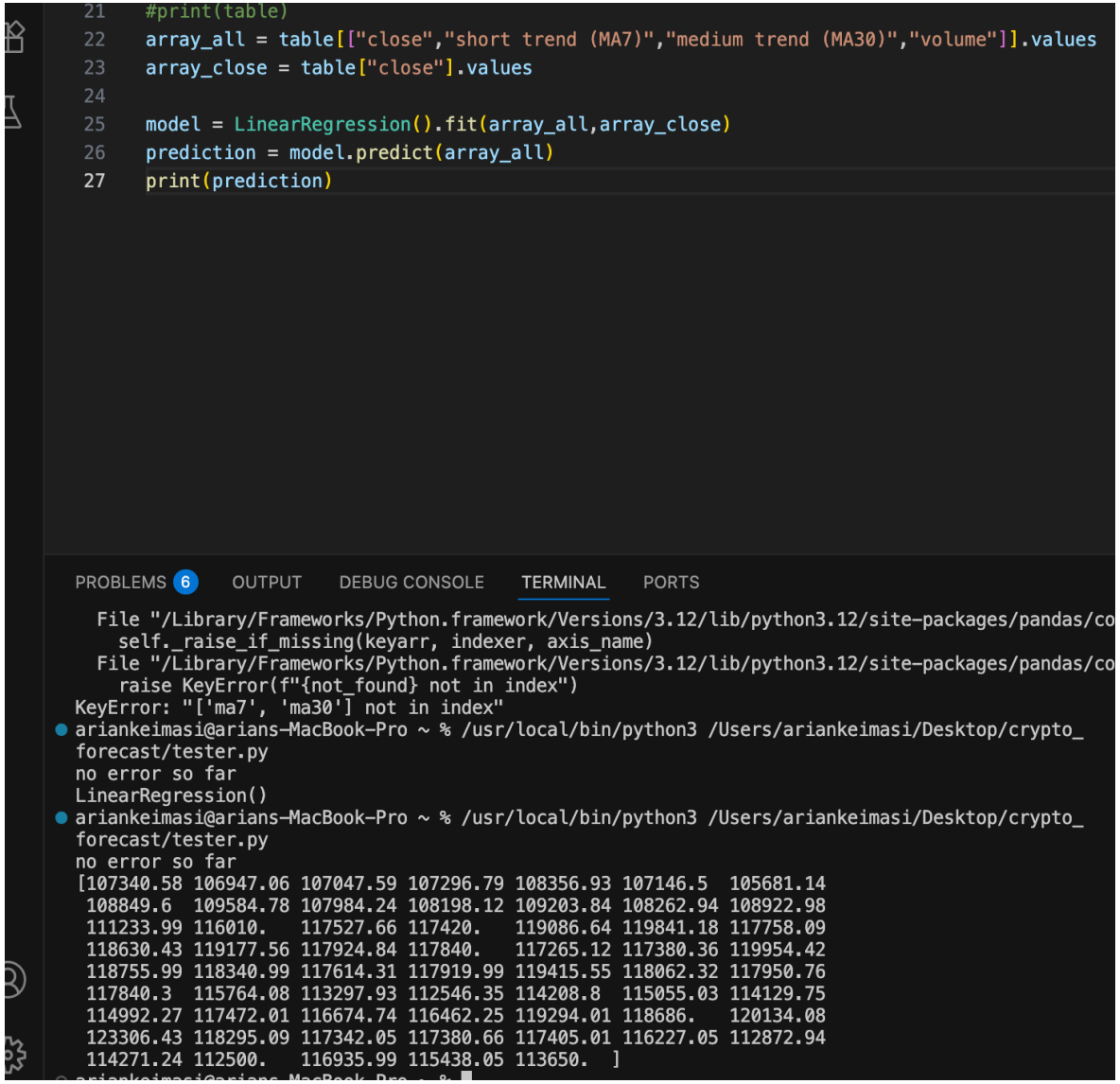
```
PROBLEMS 6    OUTPUT    DEBUG CONSOLE    TE
59     88 2025-08-23  115438.05  11329.36197
60     89 2025-08-24  113436.98  20517.14922

[61 rows x 6 columns]
ariankeimasi@arians-MacBook-Pro ~ % /usr/loc
forecast/tester.py
no error so far
         date      close       volume  short
0  2025-06-25  107340.58  16701.15551      1
1  2025-06-26  106947.06  10573.27279      1
2  2025-06-27  107047.59  12232.44042      1
3  2025-06-28  107296.79   3282.17352      1
4  2025-06-29  108356.93   6831.73640      1
..        ...        ...          ...
56 2025-08-20  114271.24  15636.34194      1
57 2025-08-21  112500.00  10839.69235      1
58 2025-08-22  116935.99  23128.09037      1
59 2025-08-23  115438.05  11329.36197      1
60 2025-08-24  113423.71  20547.33249      1
```

S.S.58: Used reset_index**(drop=True)** → clean sequential index, no redundant column

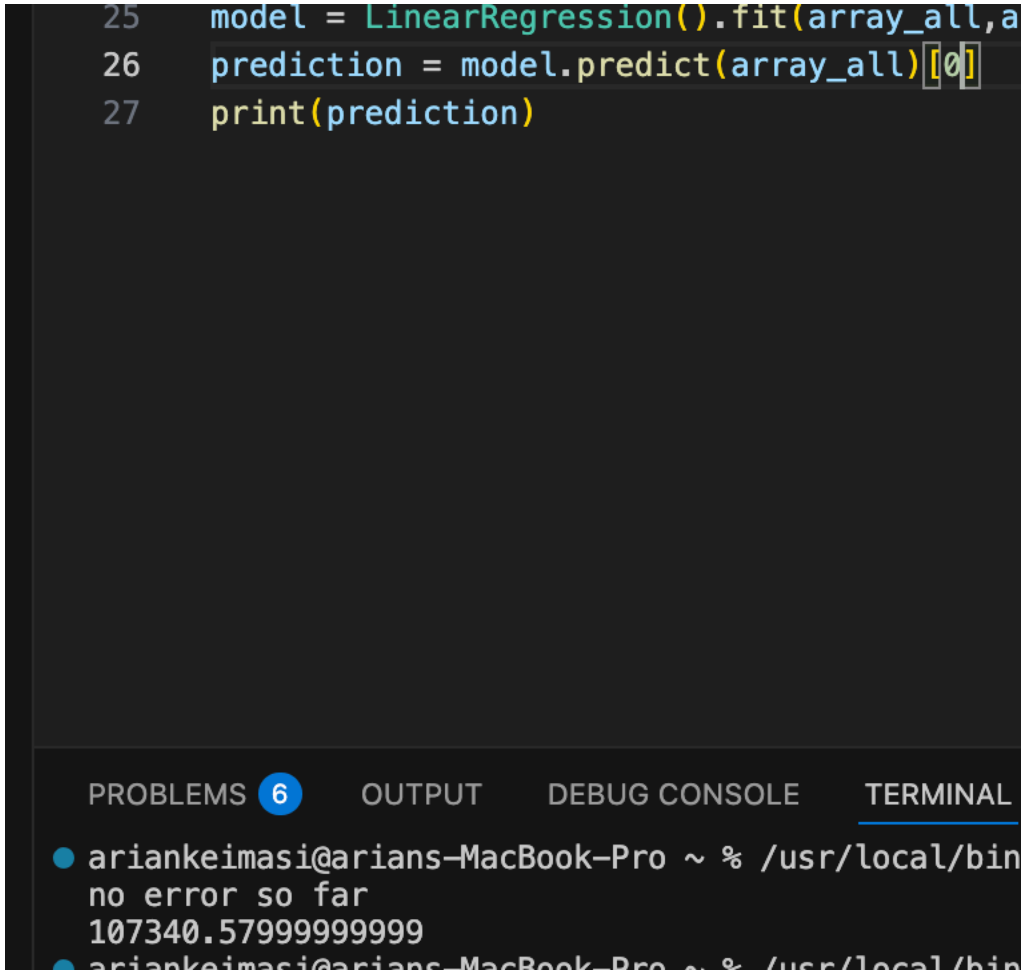
```
21  #print(table)
22  array_all = table[["close","short trend (MA7)","medium trend (MA30)","volume"]].values
23  array_close = table["close"].values
24
25  model = LinearRegression().fit(array_all,array_close)
26  print(model)
```

```
PROBLEMS 6   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

    raise KeyError(f"{not_found} not in index")
KeyError: "['ma7', 'ma30'] not in index"
ariankeimasi@arians-MacBook-Pro ~ % /usr/local/bin/python3 /Users/ariankeimasi/Desktop/crypto_
forecast/tester.py
no error so far
Traceback (most recent call last):
  File "/Users/ariankeimasi/Desktop/crypto_forecast/tester.py", line 22, in <module>
    array_all = table[["close","ma7","ma30","volume"]].iloc[:-1].values
                ~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages/pandas/c
    indexer = self.columns._get_indexer_strict(key, "columns")[1]
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages/pandas/c
    self._raise_if_missing(keyarr, indexer, axis_name)
  File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages/pandas/c
    raise KeyError(f"{not_found} not in index")
KeyError: "['ma7', 'ma30'] not in index"
ariankeimasi@arians-MacBook-Pro ~ % /usr/local/bin/python3 /Users/ariankeimasi/Desktop/crypto_
forecast/tester.py
no error so far
LinearRegression()
ariankeimasi@arians-MacBook-Pro ~ %
```

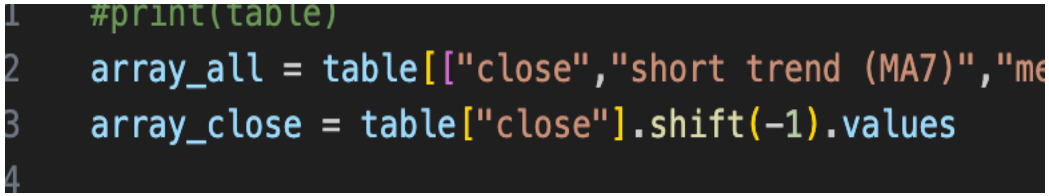
S.S.59: **LinearRegression model fitted and prediction output generated without runtime error.**



S.S.60: Removed unnecessary columns → model trained only on **relevant predictors** (close, MA7, MA30, volume).



S.S.61: Next-day prediction generated, but output value appears **unrealistic** → model reliability questioned.



S.S.62: **ValueError**: y contains NaN → caused by shifting close column without removing final missing value.

S.S.63: **ValueError**: inconsistent sample sizes [61, 60] → feature matrix and target array lengths misaligned after shift.

```
22   array_all = table[["close","short trend (MA7)","medium trend (MA30)","volume"]].iloc[:-1].values
23   array_close = table["close"].shift(-1).dropna().values
24
25   model = LinearRegression().fit(array_all,array_close)
26   X_last = array_all[[-1]]
27   prediction = model.predict(array_all)[0]
28   print(prediction)
```

```
PROBLEMS 6   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

  File "/Users/ariankeimasi/Desktop/crypto_forecast/tester.py", line 25, in <module>
    model = LinearRegression().fit(array_all,array_close)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages/sklearn/base.py", lin
    return fit_method(estimator, *args, **kwargs)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages/sklearn/linear_model/
    X, y = validate_data(
           ^^^^^^^^^^^^^^
  File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages/sklearn/utils/validat
    X, y = check_X_y(X, y, **check_params)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages/sklearn/utils/validat
    check_consistent_length(X, y)
  File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages/sklearn/utils/validat
    raise ValueError(
ValueError: Found input variables with inconsistent numbers of samples: [61, 60]
ariankeimasi@arians-MacBook-Pro ~ % /usr/local/bin/python3 /Users/ariankeimasi/Desktop/crypto_
forecast/tester.py
no error so far
108052.88117319424
ariankeimasi@arians-MacBook-Pro ~ %
⊗ 0 ⚠ 6
```

S.S.64: Feature/target arrays aligned correctly → **prediction more stable**, though volatility still affects accuracy.

```
21    #print(table)
22    array_all = table[["close","short trend (MA7)","medium trend (MA30)","volume"]].iloc[-1].values
23    array_close = table["close"].shift(-1).dropna().values
24
25    model = LinearRegression().fit(array_all,array_close)
26    X_last = array_all[[-1]]
27    prediction = model.predict(array_all)[0]
28    print(prediction)
```

```
ROBLEMS 6   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

raceback (most recent call last):
 File "/Users/ariankeimasi/Desktop/crypto_forecast/tester.py", line 25, in <module>
   model = LinearRegression().fit(array_all,array_close)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

 File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages/sklearn/base.py", line 1365, in wrapper
   return fit_method(estimator, *args, **kwargs)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

 File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages/sklearn/linear_model/_base.py", line 618, in fit
   X, y = validate_data(
          ^^^^^^^^^^^^^

 File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages/sklearn/utils/validation.py", line 2971, in validate_data
   X, y = check_X_y(X, y, **check_params)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

 File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages/sklearn/utils/validation.py", line 1368, in check_X_y
   X = check_array(
       ^^^^^^^^^^^

 File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages/sklearn/utils/validation.py", line 1091, in check_array
   raise ValueError(msg)
alueError: Expected 2D array, got 1D array instead:
rray=[113639.99       114555.03714286 116517.35733333  21056.91987   ].
eshape your data either using array.reshape(-1, 1) if your data has a single feature or array.reshape(1, -1) if it contains a single sample.
riankeimasi@arians-MacBook-Pro ~ %

⚠ 6                                                                                                                    Ln 2
```

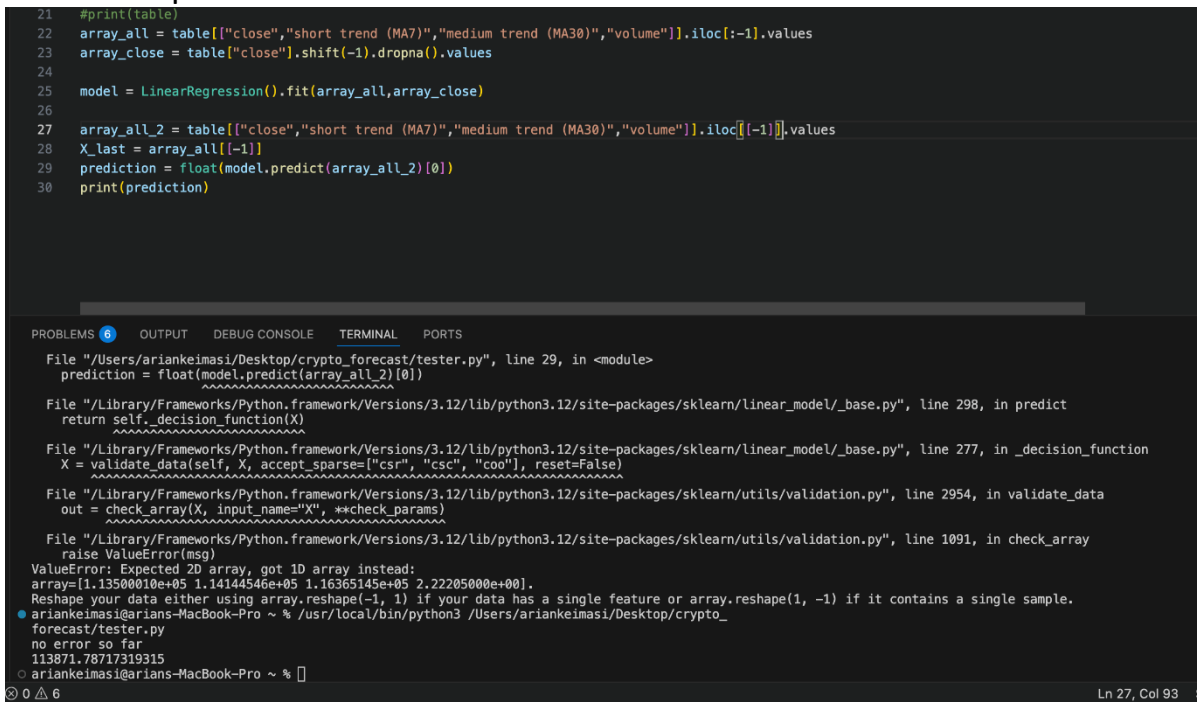
S.S.65: Prediction failed due to **incorrect input shape** (single row not 2D).

```python
print ("no error so far")
#print(table)
array_all = table[["close","short trend (MA7)","medium trend (MA30)","volume"]].iloc[:-1].values
array_close = table["close"].shift(-1).dropna().values

model = LinearRegression().fit(array_all,array_close)

array_all_2 = table[["close","short trend (MA7)","medium trend (MA30)","volume"]].iloc[-1].values
X_last = array_all[[-1]]
prediction = model.predict(X_last)[0]
print(prediction)
```

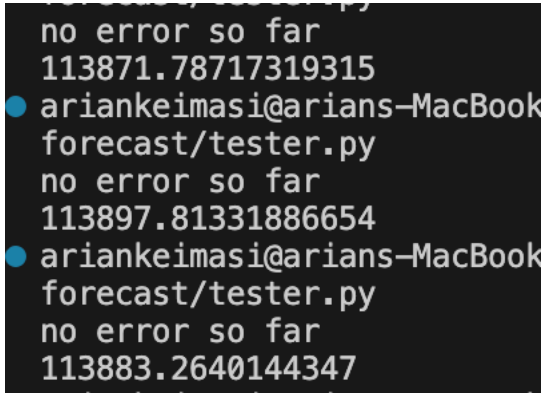
PROBLEMS 6 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
  File "/Users/ariankeimasi/Desktop/crypto_forecast/tester.py", line 27, in <module>
    prediction = model.predict(array_all_2)[0]
                 ^^^^^^^^^^^^^^^^^^^^^^^^^^

  File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages/sklearn/linear_model/_base.py",
    return self._decision_function(X)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^

  File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages/sklearn/linear_model/_base.py",
    X = validate_data(self, X, accept_sparse=["csr", "csc", "coo"], reset=False)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

  File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages/sklearn/utils/validation.py", li
    out = check_array(X, input_name="X", **check_params)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

  File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages/sklearn/utils/validation.py", li
    raise ValueError(msg)
ValueError: Expected 2D array, got 1D array instead:
array=[113490.97       114533.74857143 116512.39       21078.63331   ].
Reshape your data either using array.reshape(-1, 1) if your data has a single feature or array.reshape(1, -1) if it cont
ariankeimasi@arians-MacBook-Pro ~ % /usr/local/bin/python3 /Users/ariankeimasi/Desktop/crypto_
forecast/tester.py
no error so far
115470.39734229434
ariankeimasi@arians-MacBook-Pro ~ %
```

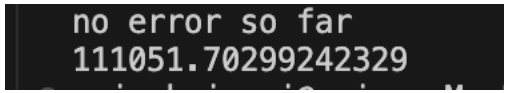
S.S.66: Output still not realistic → idea to test shorter timeframe noted.



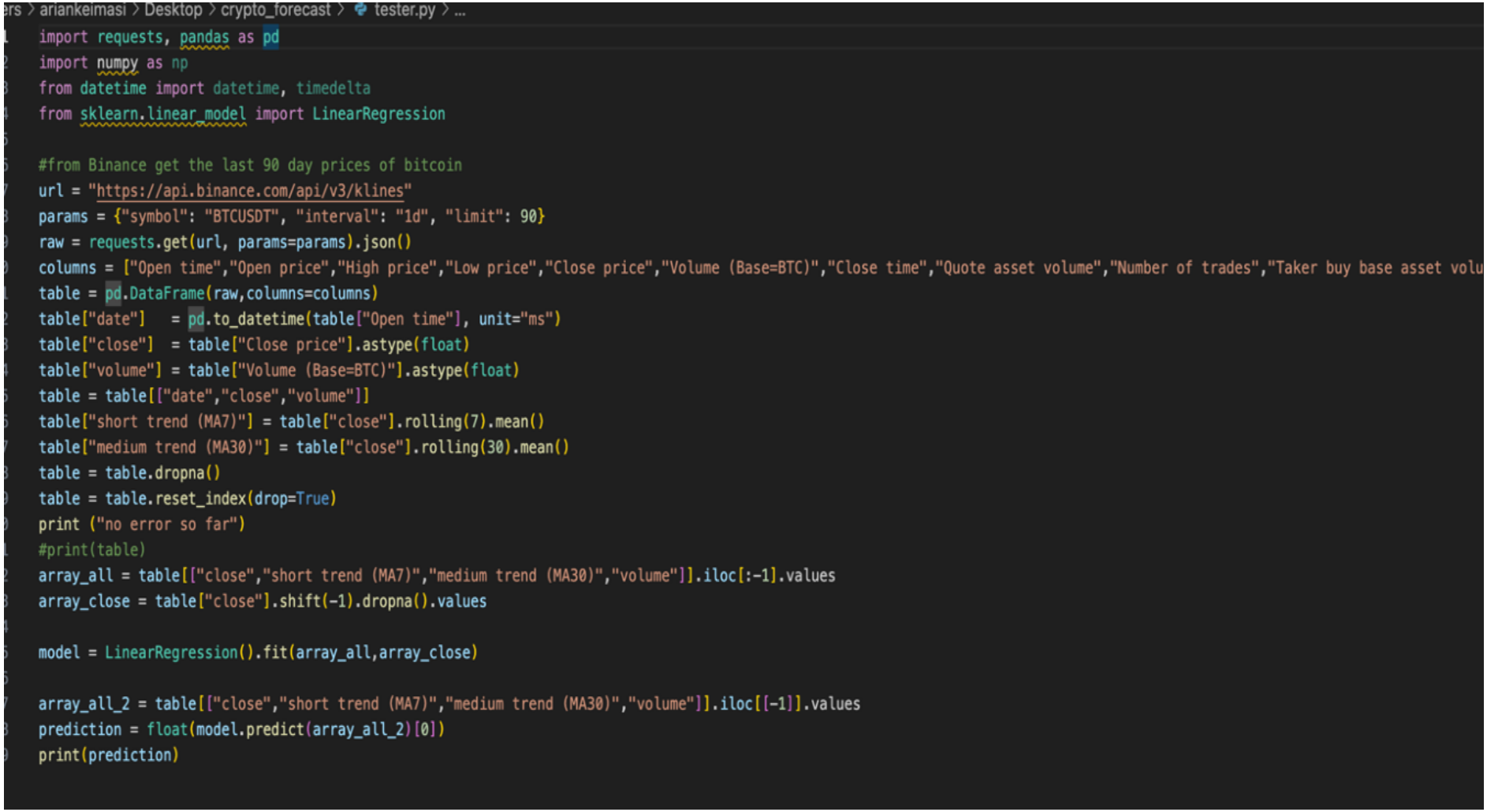
S.S.67: **2D array fix applied ([[...]])** → prediction now close to expectation.



S.S.68: Prediction still differed from live price → accuracy gap highlighted.



S.S.69: Prediction output stayed very close to today's price → model likely underfitting / low responsiveness



S.S.70: **Full prediction pipeline script consolidated in one file** → ready for refactor into reusable class

```python
import requests, pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
class BinancePredictor:
    def __init__(self, symbol="BTCUSDT", interval="1d", limit=90):
        self.symbol = symbol
        self.interval = interval
        self.limit = limit
        self.url = "https://api.binance.com/api/v3/klines"
        self.table = None
        self.model = None
        self.prediction = None
    #from binance get the last 90 day candles
    def run(self):
        params = {"symbol": self.symbol, "interval": self.interval, "limit": self.limit}
        raw = requests.get(self.url, params=params).json()
        columns = ["Open time","Open price","High price","Low price","Close price",
                   "Volume (Base=BTC)","Close time","Quote asset volume","Number of trades",
                   "Taker buy base asset volume","Taker buy quote asset volume","Ignore (always 0)"]
        table = pd.DataFrame(raw, columns=columns)
        table["date"]   = pd.to_datetime(table["Open time"], unit="ms")
        table["close"]  = table["Close price"].astype(float)
        table["volume"] = table["Volume (Base=BTC)"].astype(float)
        table = table[["date","close","volume"]]
        # add moving averages
        table["short trend (MA7)"]   = table["close"].rolling(7).mean()
        table["medium trend (MA30)"] = table["close"].rolling(30).mean()
        table = table.dropna().reset_index(drop=True)
        self.table = table
        # train model
        array_all   = table[["close","short trend (MA7)","medium trend (MA30)","volume"]].iloc[:-1].values
        array_close = table["close"].shift(-1).dropna().values
        model = LinearRegression().fit(array_all, array_close)
        self.model = model
        # predict last row
        array_all_2 = table[["close","short trend (MA7)","medium trend (MA30)","volume"]].iloc[[-1]].values
        prediction = float(model.predict(array_all_2)[0])
        self.prediction = prediction
        return prediction
engine = BinancePredictor("BTCUSDT", "1d", 90)
prediction = engine.run()
print("Predicted next day close:", prediction)

```

S.S.71: Refactored into BinancePredictor class (fetch → clean → MA features → train → predict) → **modular structure achieved**

⇒ **Code explanation:**

This section builds the first working **data-preparation → feature-engineering → model training → prediction** pipeline using 90 days of Binance candle data. After converting the raw response into a pandas DataFrame, the code keeps only the required fields (**date, close, volume**) and then derives two trend features: a **7-day moving average (MA7)** representing short-term trend and a **30-day moving average (MA30)** representing medium-term trend.

Because moving averages require earlier values, the first rows initially produce **NaN** values, so the dataset is cleaned by removing rows with missing MA values and then re-indexing to keep the DataFrame consistent for later array creation.

The prediction stage prepares two arrays:

X (features): close, MA7, MA30, volume

y (target): next day's close (created by shifting close by −1)

The pipeline then trains a **LinearRegression** model on the aligned feature/target arrays and generates a next-step prediction. Finally, the script is consolidated and refactored into a reusable **BinancePredictor class**, making the process modular (fetch → clean → engineer features → train → predict).

⇒ **Justification:** The decision to use moving averages (MA7 and MA30) rather than raw price values alone is justified because financial markets exhibit short-term volatility that can obscure broader directional trends. A regression model trained purely on daily closing prices would likely respond to noise rather than structure. By incorporating both short-term and medium-term averages, the model is supplied with smoothed signals that reflect momentum over different horizons. This increases the likelihood that the prediction is based on trend behaviour rather than random fluctuation.

The choice of a simple Linear Regression model is deliberate at this stage. Although more complex models (e.g., Random Forest or Neural Networks) could potentially capture nonlinear behaviour, they would reduce interpretability and increase implementation complexity. Linear Regression provides a transparent baseline model, allowing performance to be evaluated before introducing more advanced techniques. Establishing a reliable baseline aligns with good modelling practice and supports controlled iterative improvement.

Removing rows containing insufficient historical data (for MA30 calculation) is also justified conceptually, not just technically. Retaining partially formed indicators would introduce distortion into the feature space, leading to biased training. Ensuring that each training row contains fully calculated indicators preserves feature consistency and supports model reliability.

The use of a shifted closing price as the target variable reflects the system's predictive objective: forecasting the next period's value based on current information. This explicitly transforms the dataset into a supervised learning structure where inputs represent known

information and outputs represent future outcomes. Alternative approaches, such as predicting the same day's close, would not align with the real-world forecasting requirement.

Finally, encapsulating the entire workflow into a dedicated class supports scalability and maintainability. The prediction engine is designed to operate independently from the GUI layer, allowing reuse for multiple assets or timeframes. This separation of concerns ensures that future modifications (e.g., testing different intervals such as 1-minute data) can be implemented without rewriting interface logic.

⇒ **Iterative development:**

The first implementation of rolling averages correctly generated MA7 and MA30 but exposed a predictable limitation: insufficient prior observations resulted in NaN values. Rather than masking the issue, it was resolved by explicitly removing incomplete rows, ensuring statistical validity.

However, this introduced a secondary structural issue: the DataFrame index became non-sequential. While functionally harmless, it posed risks for later slicing operations and debugging clarity. Resetting the index restored structural integrity, though it initially created a redundant column — which was then removed using drop=True. This reflects careful attention to dataset cleanliness rather than superficial fixes.

The introduction of the shifted target column created more subtle alignment challenges. Shifting the close series introduced a trailing NaN, which led to a "Input y contains NaN" error.

Further refinement exposed mismatched feature and target array lengths, revealing that correct temporal alignment in time-series modelling requires precise slicing. Adjusting both arrays using consistent indexing resolved the inconsistency.

A further runtime error occurred because scikit-learn requires a 2D feature matrix even when predicting a single row. This boundary condition was corrected by ensuring the final input retained its two-dimensional structure.

These iterations collectively demonstrate debugging across **data integrity, structural consistency, temporal alignment, and model interface requirements**, not just syntax correction. The final result is a stable, logically aligned training and prediction pipeline.

⇒ **Validation:**

Validation combined functional and structural testing. Moving averages (MA7 and MA30) were verified after sufficient data points were available, and dropna() correctly removed rows created by rolling and shifting operations. Feature and target arrays were aligned to eliminate sample-size errors, and input shape was adjusted to satisfy scikit-learn's 2D requirement. The model then trained and generated predictions without runtime errors. Refactoring into the BinancePredictor class confirmed the full pipeline executed correctly within a modular structure.

Test ID	Feature Tested	Test Type	Test Description	Expected Outcome	Actual Outcome	Action Taken	Result	Evidence

H1	MA7/MA30 creation	Normal	Calculate rolling MA7 and MA30 from close prices	MA7 and MA30 generate numeric values after 7 and 30 observations respectively.	Initial rows show NaN values	Identified as expected behaviour for rolling windows	Pass	S.S.58
H2	Missing-value handling	Normal	Continued with cleaned dataset for ML compatibility	Rows with missing MA values removed	First ~29 rows removed	Applied dropna() after creating MA features	Pass	S.S.56
H3	Target definition (next-day close)	Normal	Create y using close.shift(-1)	Target column equals close price with no NaN values	NaN introduced in final target entry	Added dropna() after shift and aligned slicing	Pass	S.S.64
H4	Feature/target alignment	erroneous	Train model with aligned X and y lengths	No inconsistent sample errors	Inconsistent length error occurred initially	Used consistent slicing (iloc[:-1]) and cleaned y	Pass	S.S.66
H5	Model training run	Normal	Fit LinearRegression on cleaned data	Model fits without runtime errors	Model trained successfully	Proceeded to prediction stage	Pass	S.S.70
H6	Prediction input shape	Boundary	Predict using last row of X	scikit-learn accepts input without shape error	Shape error when passing 1D row	Ensured last row kept as 2D array ([[...]])	Pass	S.S.67
H7	Modular refactor	Normal	Move pipeline into BinancePredictor class	run() executes full pipeline (fetch → preprocess → feature engineering → train	Refactor successful and output produced	Stored model/table/prediction as instance attributes	Pass	S.S.71

				→ predict) without structural errors.				

⇒ **Justification of Actions Taken During Testing:**

In Test H1, initial rows contained NaN values when calculating MA7 and MA30. This was identified as expected behaviour due to the rolling window requiring sufficient prior data points. This approach was accepted because it reflects correct statistical calculation. This ensured the accuracy of the feature engineering process.

In Test H2, rows with missing moving average values needed to be handled before model training. To resolve this, dropna() was applied after feature creation. This approach was chosen to ensure the dataset contains only complete records suitable for machine learning. This improved the reliability of the system by preventing errors during training.

In Test H3, creating the target variable using close.shift(-1) introduced a NaN value in the final row. To resolve this, dropna() was applied and feature-target alignment was adjusted. This approach was chosen to ensure valid target values for all training samples. This improved the reliability of the system by maintaining correct input-output mapping.

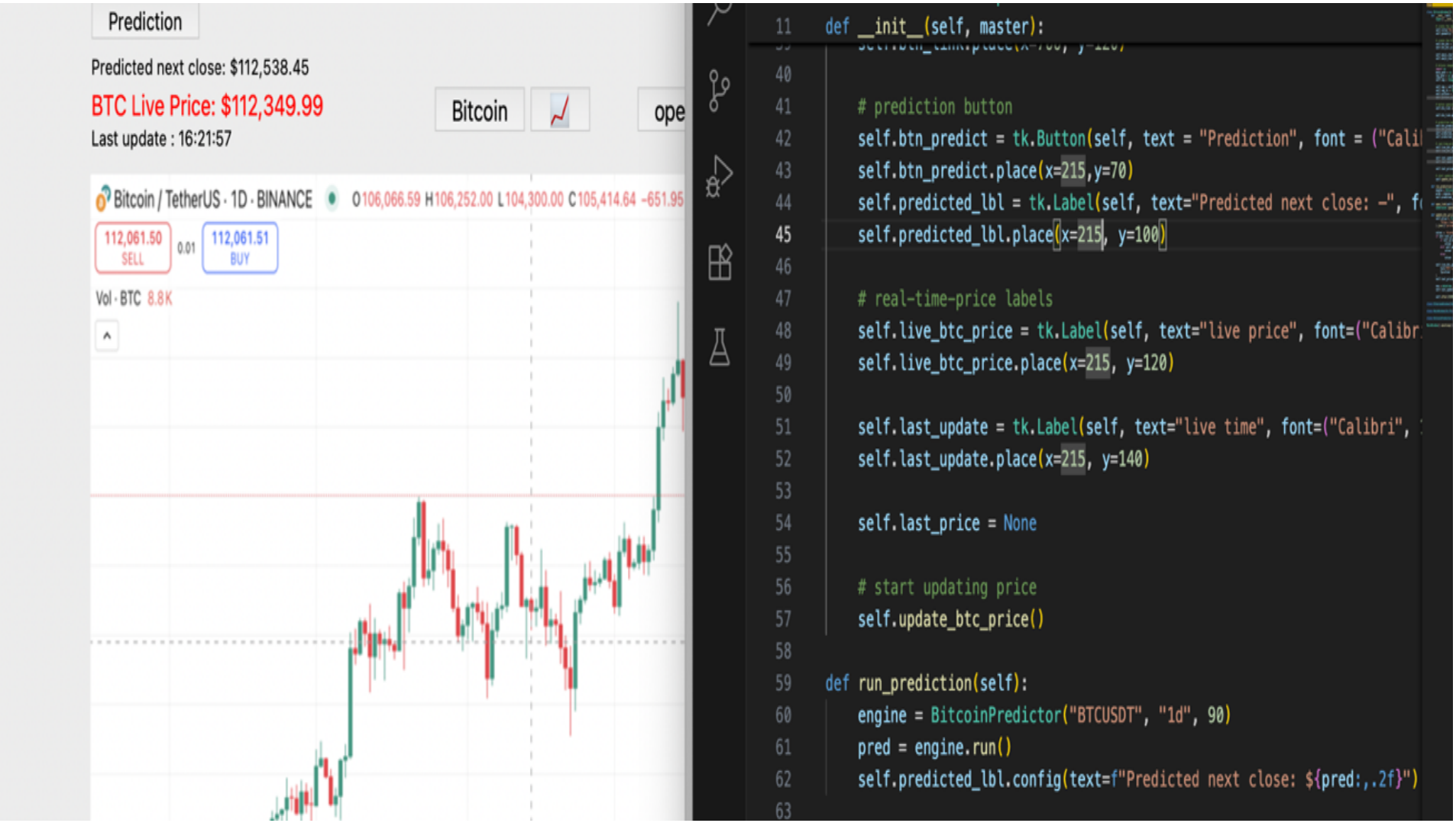
In Test H4, inconsistent sample lengths between features and target caused a training error. To resolve this, consistent slicing using iloc[:-1] was applied. This approach was chosen to ensure both datasets are aligned correctly. This improved the functionality of the system by enabling successful model training.

In Test H6, a shape error occurred when passing a 1D array to the prediction function. To resolve this, the input was converted into a 2D array format. This approach was chosen because scikit-learn requires inputs to be two-dimensional. This improved the reliability of the system by preventing runtime errors during prediction.

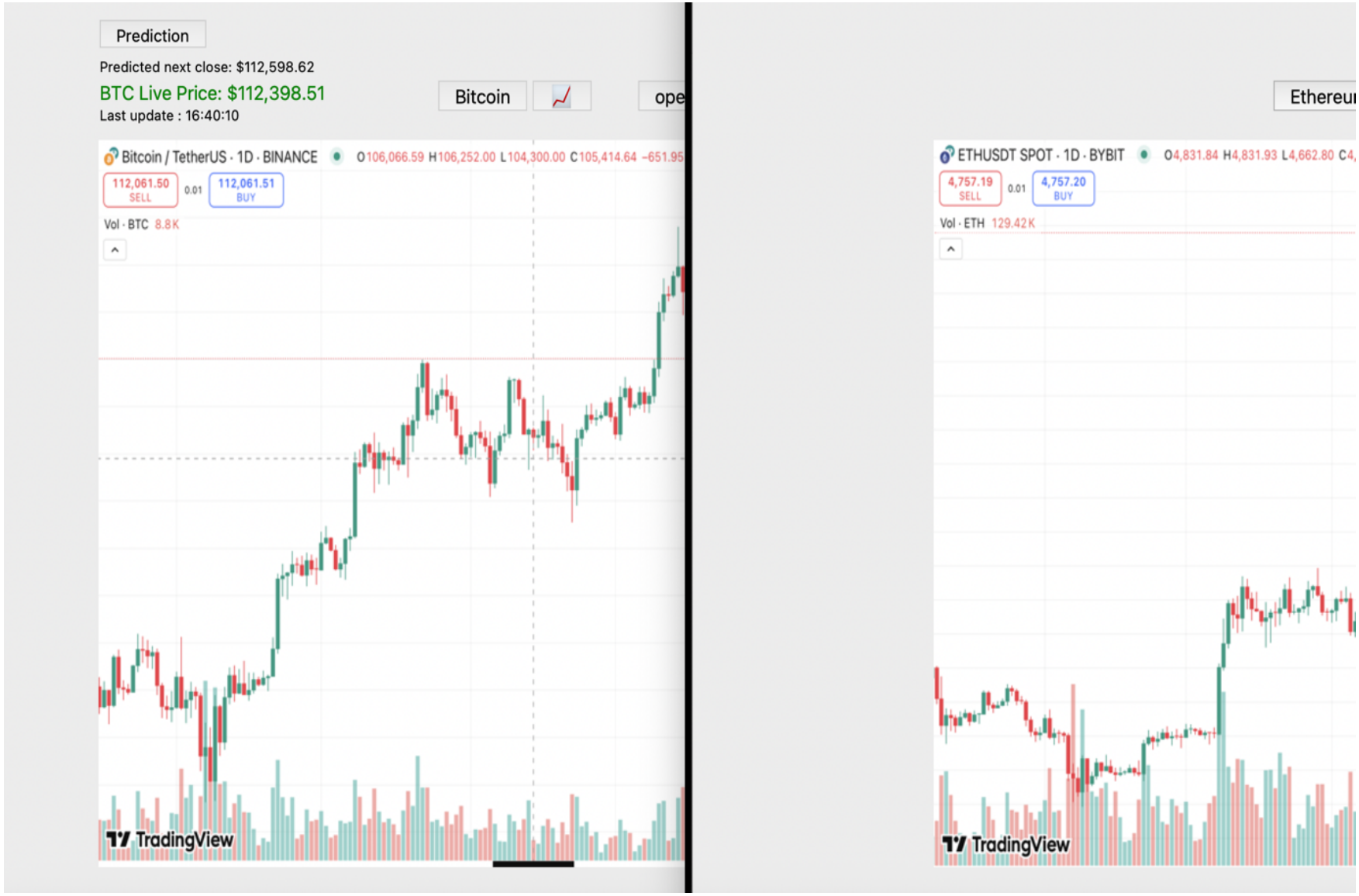
⇒ **Link back to analysis & design:**

This section delivers the **first complete prediction pipeline**, turning raw Binance data into a cleaned dataset with engineered trend features and a working baseline model. It directly supports the design goal of producing **forecast outputs** from historical market data while keeping the implementation structured enough to extend. The refactor into BinancePredictor strengthens scalability by making the predictor reusable for additional cryptocurrencies and timeframes, which supports the overall system design of a multi-page crypto forecasting tool with consistent, maintainable components.

Deployment and Refinement of the Linear Regression Prediction System



S.S.72: Bitcoin prediction integrated into BTC window; Ethereum copy attempted by changing symbol → ETH version **not functioning** yet


Prediction
Predicted next close: $112,598.62
BTC Live Price: $112,398.51
Last update : 16:40:10
Bitcoin
Bitcoin / TetherUS · 1D · BINANCE
O106,066.59 H106,252.00 L104,300.00 C105,414.64 −651.95
112,061.50
SELL
0.01
112,061.51
BUY
Vol · BTC 8.8K
TradingView
Ethereu
ETHUSDT SPOT · 1D · BYBIT
O4,831.84 H4,831.93 L4,662.80 C4,
4,757.19
SELL
0.01
4,757.20
BUY
Vol · ETH 129.42K
TradingView

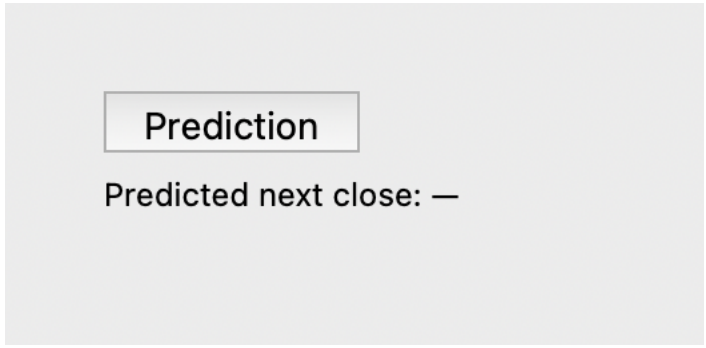
```
AttributeError: 'EthereumWindow' object has no attribute 'run_prediction'
Exception in Tkinter callback
Traceback (most recent call last):
  File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/tkinter/__init__.
, line 1968, in __call__
    return self.func(*args)
           ^^^^^^^^^^^^^^^^
  File "/Users/ariankeimasi/Desktop/crypto_forecast/no_2.py", line 200, in open_eth
    self.eth_OpenOrNot = EthereumWindow(self)
                         ^^^^^^^^^^^^^^^^^^^^
  File "/Users/ariankeimasi/Desktop/crypto_forecast/no_2.py", line 122, in __init__
    self.btn_predict = tk.Button(self, text = "Prediction", font = ("Calibri",14), command=
f.run_prediction)

^^^^^^^^^^^^^^^^
AttributeError: 'EthereumWindow' object has no attribute 'run_prediction'
^C%
```

S.S.73: Ethereum window missing run_prediction() method → **AttributeError** raised on button click

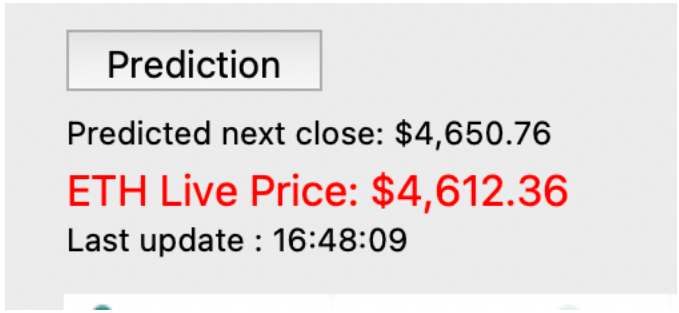
```python
# prediction button
self.btn_predict = tk.Button(self, text = "Prediction", font = ("Calibri",14), command=self.run_prediction)
self.btn_predict.place(x=215,y=70)
self.predicted_lbl = tk.Label(self, text="Predicted next close: —", font=("Calibri", 12))
self.predicted_lbl.place(x=215, y=100)

def run_prediction(self):
    model = BinancePredictor("ETHUSDT", "1d", 90)
    prediction = model.run()
    self.predicted_lbl.config(text=f"Predicted next close: ${prediction:,.2f}")
```

S.S.74: Added **run_prediction()** method to Ethereum window → prediction logic connected to button

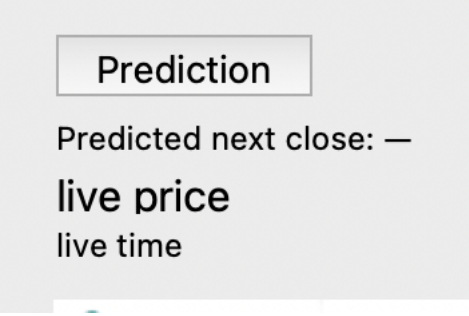
S.S.75: **Method reference corrected**, but execution order still causing display issue



S.S.76: prediction label created but not updating → value not yet bound to function call



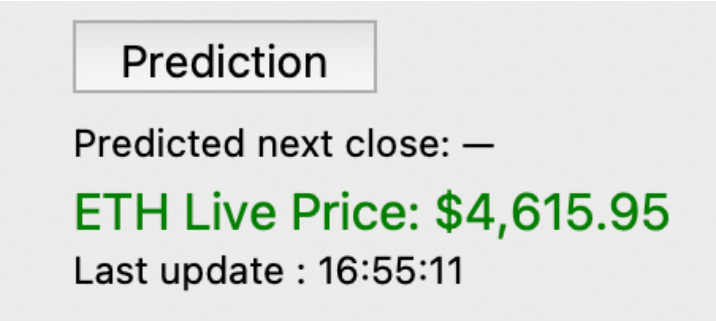
S.S.77: Prediction updates only after button click → event binding confirmed working



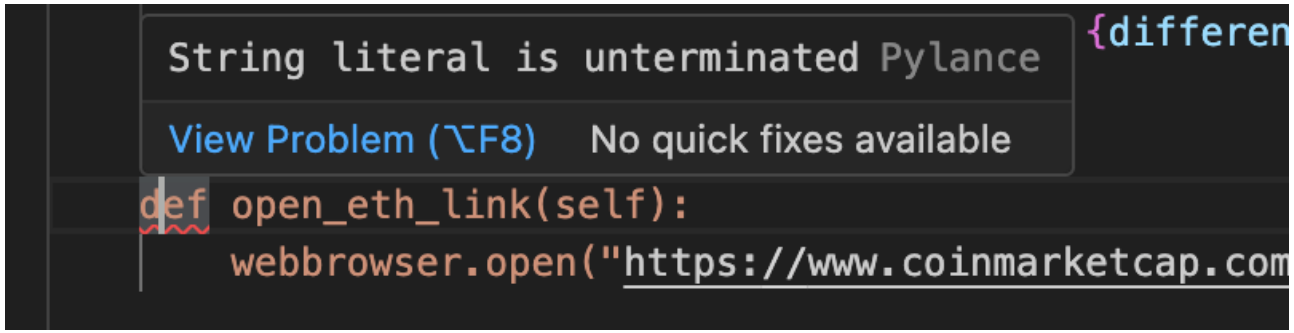
S.S.78: Prediction label moved inside function → still not displaying due to code order

```
        self.last_update = tk.Label(self,
        self.last_update.place(x=215, y=14

        # prediction button
        self.btn_predict = tk.Button(self,
        self.btn_predict.place(x=215,y=70)
        self.predicted_lbl = tk.Label(self
        self.predicted_lbl.place(x=215, y=


    def run_prediction(self):
        model = BinancePredictor("ETHUSDT"
        prediction = model.run()
        self.predicted_lbl.config(text=f"P


        self.last_price = None
        self.update_eth_price()
```

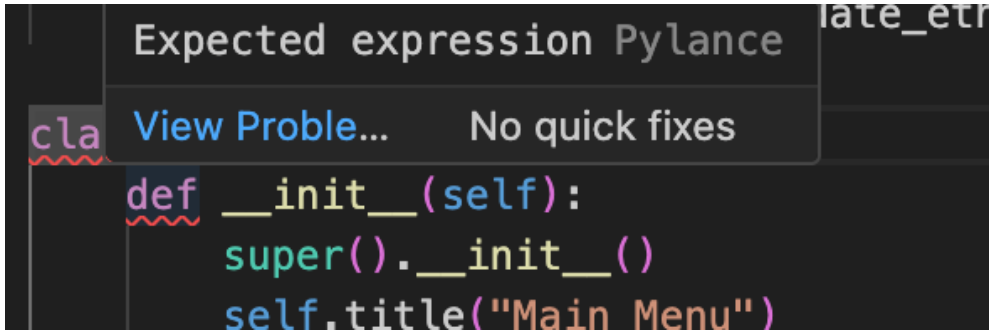
S.S.79: **Corrected widget creation order** → prediction label updates reliably



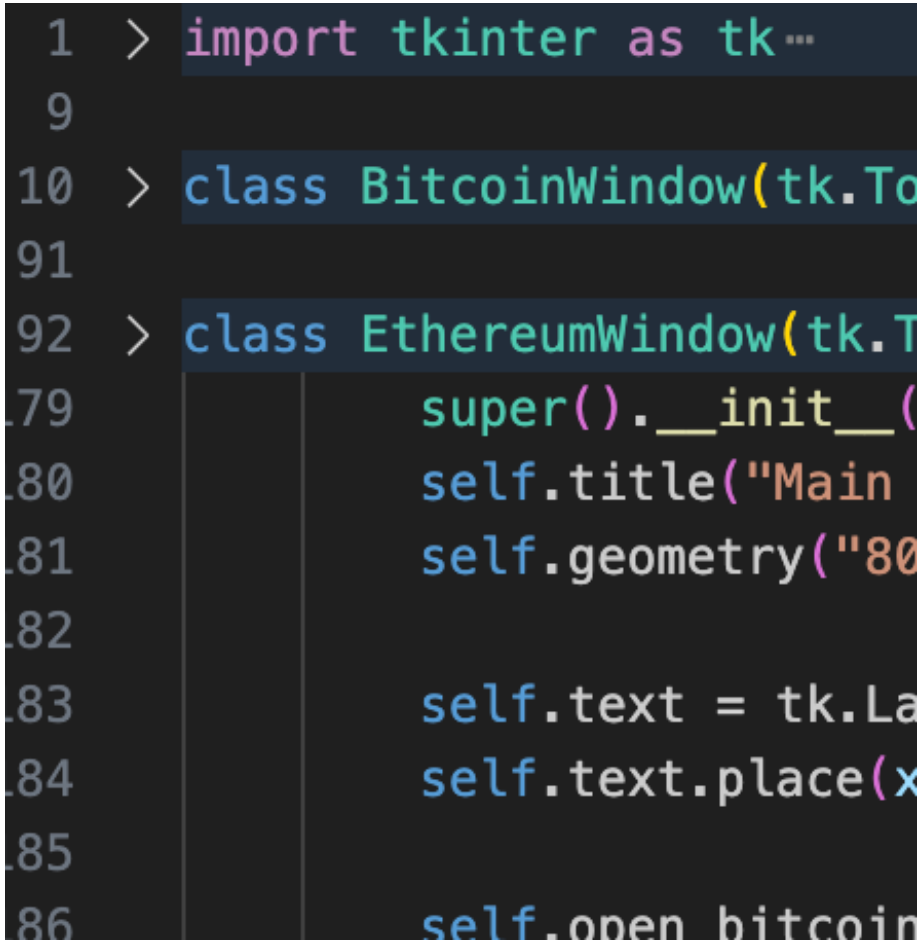
S.S.80: Prediction feature **fully functional** → output displayed alongside live price



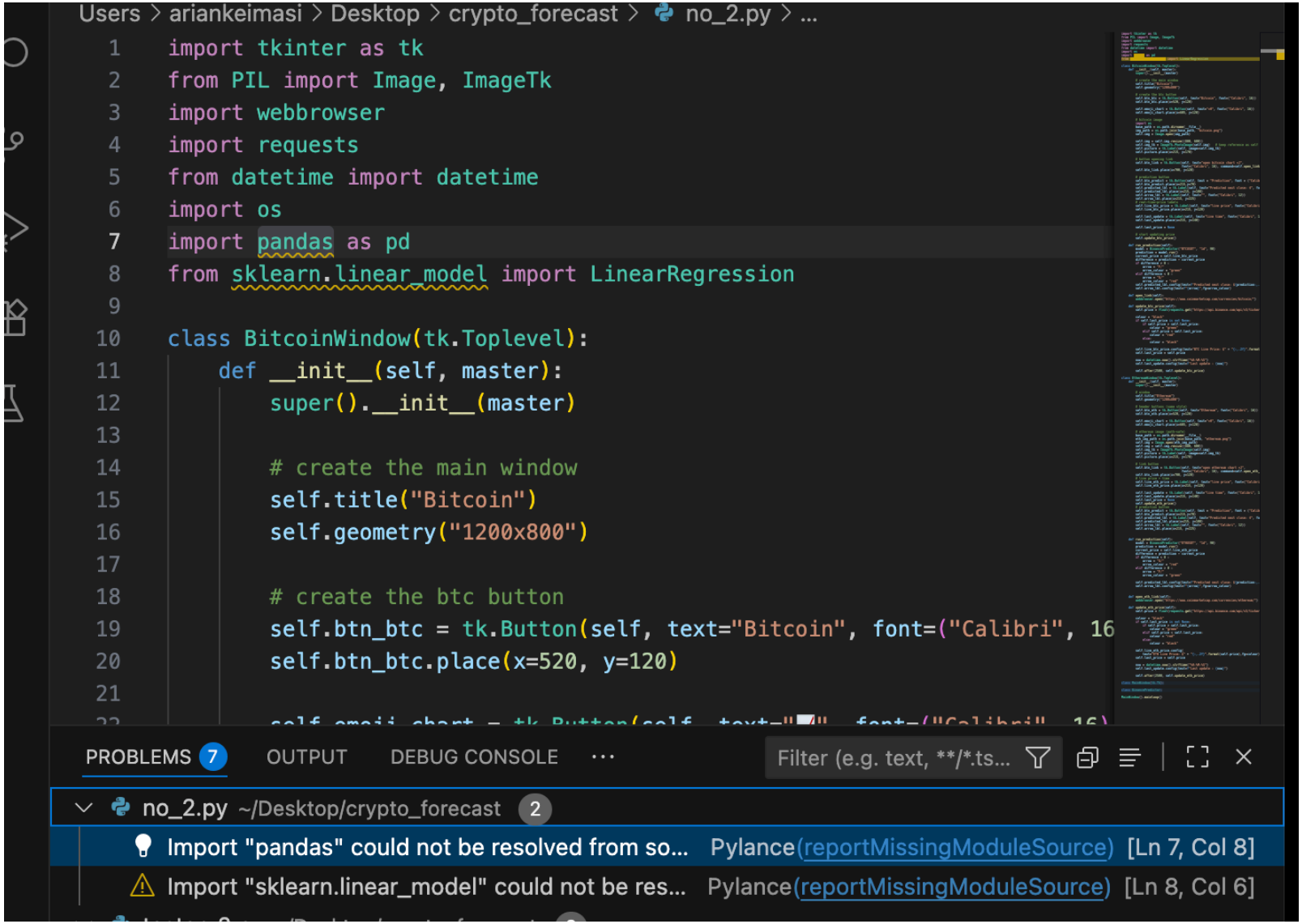
S.S:81: Unterminated string literal → syntax error in external link function



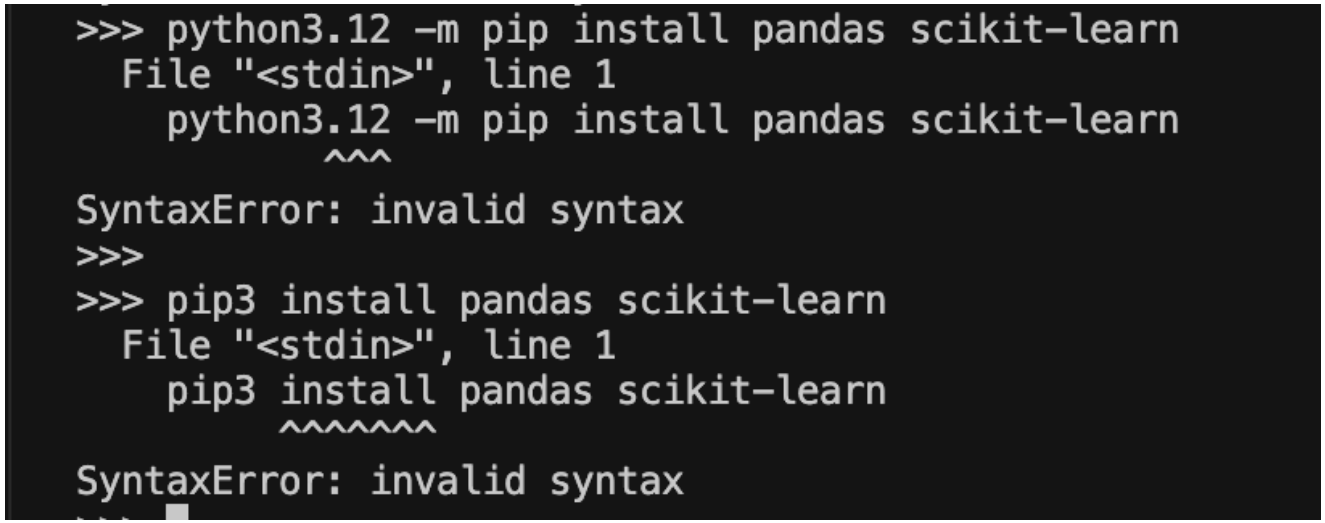
S.S.82: Minor **syntax/indentation errors** detected by Pylance → corrected for stable execution



S.S.83: Syntax error (missing bracket) caused cascading errors → all subsequent lines flagged incorrectly.



S.S.84: Pylance reported **missing modules (pandas, sklearn)** → environment not recognised.



S.S.85: Attempted pip install inside Python REPL (>>>) → invalid syntax error.

```
  File "<stdin>", line 1
    pip3 install pandas scikit-learn
         ^^^^^^^
SyntaxError: invalid syntax
>>> exit()
ariankeimasi@arians-MacBook-Pro ~ % pip3 install pandas scikit-learn

Requirement already satisfied: pandas in /Library/Frameworks/Python.framework/Versi
b/python3.12/site-packages (2.3.2)
Requirement already satisfied: scikit-learn in /Library/Frameworks/Python.framework
.12/lib/python3.12/site-packages (1.7.1)
Requirement already satisfied: numpy>=1.26.0 in /Library/Frameworks/Python.framewor
3.12/lib/python3.12/site-packages (from pandas) (2.3.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /Library/Frameworks/Python
Versions/3.12/lib/python3.12/site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /Library/Frameworks/Python.framework
.12/lib/python3.12/site-packages (from pandas) (2025.2)
```

S.S.86: Exited REPL and installed libraries **via terminal** → dependencies successfully resolved.

```
        ^^^^^^^^^^^^^^^^

File "/Users/ariankeimasi/Desktop/crypto_forecast/no_2.py", line 64, in run_prediction
  difference = prediction - current_price
               ~~~~~~~~~~~^~~~~~~~~~~~~~
TypeError: unsupported operand type(s) for -: 'float' and 'Label'
```

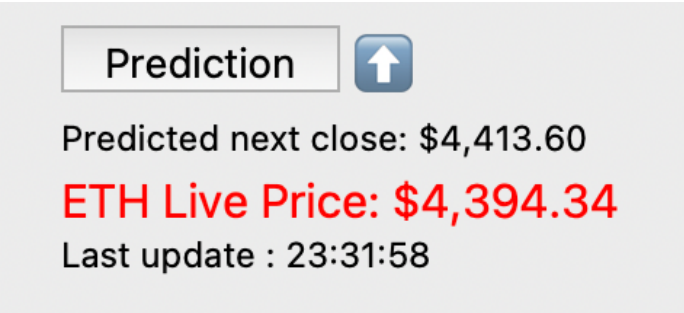
S.S.87: **Type error** when calculating prediction difference → wrong variable type used.

```
    current_price = self.live_btc_price
    difference = prediction - current_price
    if difference > 0 :
        arrow = "⬆️"
```

S.S.88: Corrected **variable reference** for current price → arithmetic operation fixed.

```
        current_price = self.price
        difference = prediction - current_price
        if difference < 0 :
            arrow = "⬇️"
            arrow_colour = "red"
        elif difference > 0 :
            arrow = "⬆️"
            arrow_colour = "green"

        self.predicted_lbl.config(text=f"Predicted next close: ${prediction:,.2f}")
        self.arrow_lbl.config(text=f"{arrow}",fg=arrow_colour)
```

S.S.89: Added visual indicator (arrow + colour) based on prediction difference → clearer feedback.



S.S.90:

Final prediction display working with live comparison and visual signal → fully functional.

⇒ **Code explanation:**

This section integrates the prediction feature into the Tkinter cryptocurrency pages, starting with Bitcoin and then extending to Ethereum. A prediction button triggers a run_prediction()

handler which calls the BinancePredictor pipeline (fetch → clean → feature engineer → train → predict) and updates a dedicated GUI label with the "Predicted next close" value. For Ethereum, the same pattern is applied but uses the ETH trading pair symbol, ensuring the prediction logic is reusable across coins rather than hard-coded to one asset.
To improve interpretability, the predicted value is compared against the current live price and the UI displays a simple directional indicator (arrow + colour) based on whether the prediction is above or below the live value. This turns a raw number into an immediately readable signal while keeping the interface lightweight.

⇒ **Justification:**

Integrating prediction into the GUI at this stage is justified because it tests the **end-to-end usefulness** of the system: a forecasting model is only valuable if it can be triggered, displayed, and interpreted by a user without manual script-running. Encapsulating the logic in a BinancePredictor class supports maintainability and avoids duplication when scaling from Bitcoin to Ethereum (and later other coins).

The comparison indicator is justified from a user-experience perspective: a single predicted price can be ambiguous, but showing direction against the live price supports faster decision-making and reduces cognitive load. Finally, catching and resolving environment/module issues (pandas/sklearn) is justified because the prediction feature depends on third-party libraries; without a stable environment, even correct code is not deployable.

⇒ **Iterative development:**

Development began by successfully integrating the prediction output into the Bitcoin window, then attempting to copy the feature to Ethereum by changing the symbol. The Ethereum version initially failed due to a missing method reference (run_prediction), producing an AttributeError when the button was clicked; this was corrected by adding the handler method and wiring it to the button callback.

After the handler existed, the UI displayed the prediction label but did not update predictably until the button was pressed, which revealed a widget creation / execution-order issue. Reordering the widget initialisation ensured the live price update and prediction label behaved consistently.

Additional debugging followed when the environment reported missing modules (Pylance warnings) and installation commands were mistakenly run inside the Python REPL, causing syntax errors; exiting the REPL and installing correctly resolved dependency issues. Finally, a type/variable reference error during live-vs-prediction comparison was fixed by using the correct live price variable, after which the arrow/colour indicator was added and the prediction display worked alongside the live price as intended.

⇒ **Validation:**

Validation confirmed the prediction feature works as a complete user-facing workflow: clicking the prediction button runs the model pipeline and updates the GUI without runtime errors. Regression checks verified Ethereum reused the same logic once the missing handler and widget order issues were corrected. Final usability checks confirmed the live-vs-predicted comparison and arrow indicator provide clear feedback, making the prediction output interpretable rather than just a raw number.

Test ID	Feature Tested	Test Type	Test Description	Expected Outcome	Actual Outcome	Action Taken	Result	Evidence
I1	BTC GUI prediction	Normal	Click “Prediction” in Bitcoin window	Predicted next close displayed in label	Prediction displayed successfully	Extended same pattern to ETH	Pass	S.S.72
I2	ETH button callback	Normal	Click “Prediction” in Ethereum window (initial attempt)	Handler runs without exception	AttributeError (missing run_prediction)	Added run_prediction() and corrected callback	Pass	S.S.74
I3	Label update behaviour	Normal	Ensure live price + prediction labels render correctly on load	Labels visible and update correctly	Prediction label only updated after click	Corrected widget creation order	Pass	S.S.90
I4	Dependency readiness	Structural	Confirm pandas/sklearn import resolves and runs	No missing-module warnings and script runs	Pylance reported missing modules	Installed dependencies correctly outside REPL	Pass	S.S.86

I5	Live vs predicted comparison	Normal	Compare prediction to live price to set arrow/colour	No type errors; arrow shows direction	Type/variable error in comparison	Corrected live price variable reference	Pass	S.S.89
I6	Visual indicator output	Usability	Arrow + colour reflects price difference	Clear directional feedback	Indicator displayed as intended, however without any color	Kept indicator as final UI enhancement	Partial Pass	S.S.90

⇒**Justification of Actions Taken During Testing:**

In Test I2, clicking the Ethereum prediction button resulted in an AttributeError due to a missing run_prediction method. To resolve this, the run_prediction() method was implemented and the callback was corrected. This approach was chosen to ensure the button is properly linked to its functionality. This improved the functionality of the system by enabling predictions to run without errors.

In Test I3, the prediction label did not update correctly on load due to incorrect widget creation order. To resolve this, the order of widget initialisation was corrected. This approach was chosen to ensure that all GUI elements are properly defined before being updated. This improved the usability of the system by ensuring prediction results are displayed consistently.

In Test I4, development tools reported missing dependencies for pandas and scikit-learn. To resolve this, the required libraries were installed outside the REPL environment. This approach

was chosen to ensure the runtime environment correctly recognises all dependencies. This improved the reliability of the system by preventing import and execution errors.

In Test I5, comparing live and predicted prices resulted in a type or variable error. To resolve this, the correct live price variable reference was used. This approach was chosen to ensure accurate comparison between values. This improved the functionality of the system by enabling correct directional feedback.

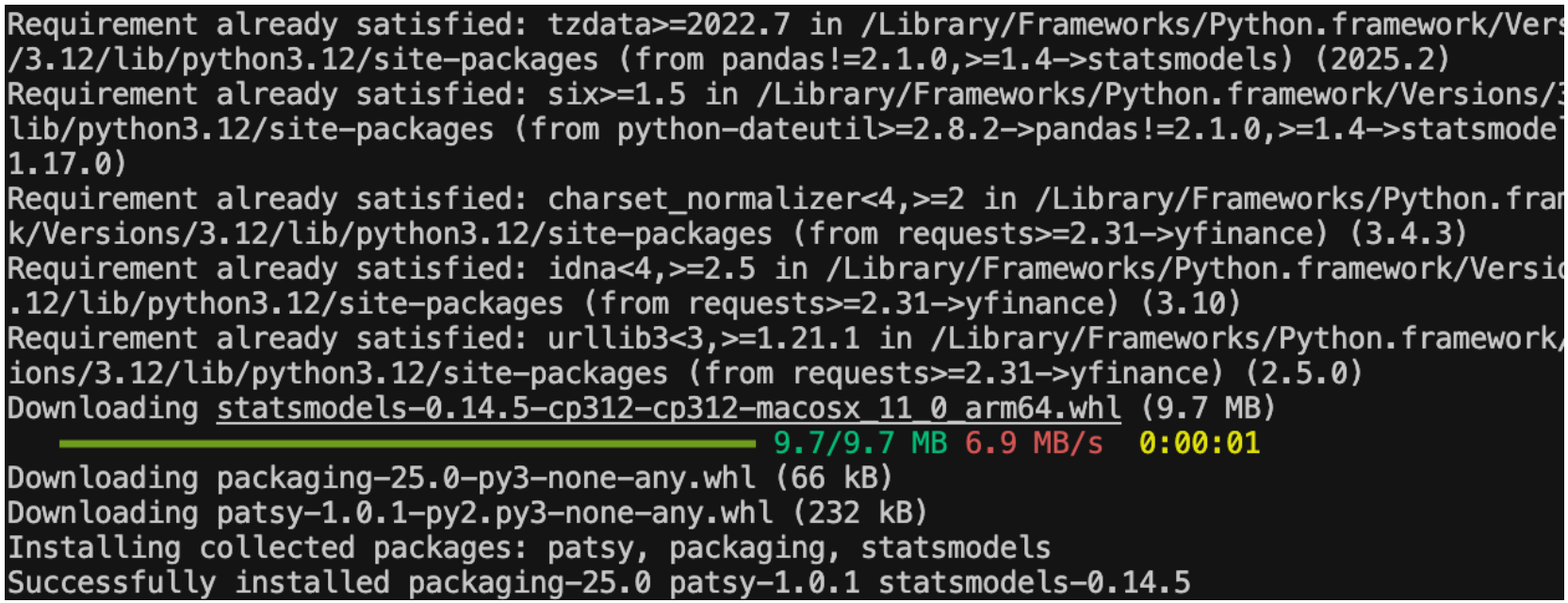
In Test I6, the visual indicator displayed correctly but lacked colour feedback. To resolve this, the current implementation was retained as a functional indicator while identifying colour enhancement as a future improvement. This approach was chosen to maintain system stability while acknowledging a usability limitation. This improved the evaluation of the system by identifying an area for further development.

⇒ **Link back to analysis & design:**

This section implements the first full **analysis-to-interface loop**: data is collected from an exchange API, transformed into modelling features, used to generate a prediction, and then presented inside the application in a user-friendly format. It also demonstrates scalability in the design: the prediction pipeline is written once (as a class) and then reused across multiple cryptocurrency pages, supporting the project's goal of a multi-coin dashboard without duplicating logic or fragmenting the codebase.

ARIMA Model Implementation, Prediction Refinement, and Error Optimisation.

ARIMA is a time-series forecasting model. The AR part captures how past prices influence the present, the I part removes trends by looking at price changes, and the MA part corrects predictions using past errors. This combination makes ARIMA good at modelling financial data that shows trends and noise



S.S.91: Installed required libraries (**statsmodels, yfinance)** to enable ARIMA time-series modelling and live crypto data retrieval.

```python
import yfinance as yf
from statsmodels.tsa.arima.model import ARIMA
close = yf.download("BTC-USD", period="90d", interval="1d")["Close"]

print ("working so far with no errors")
print (close)
```

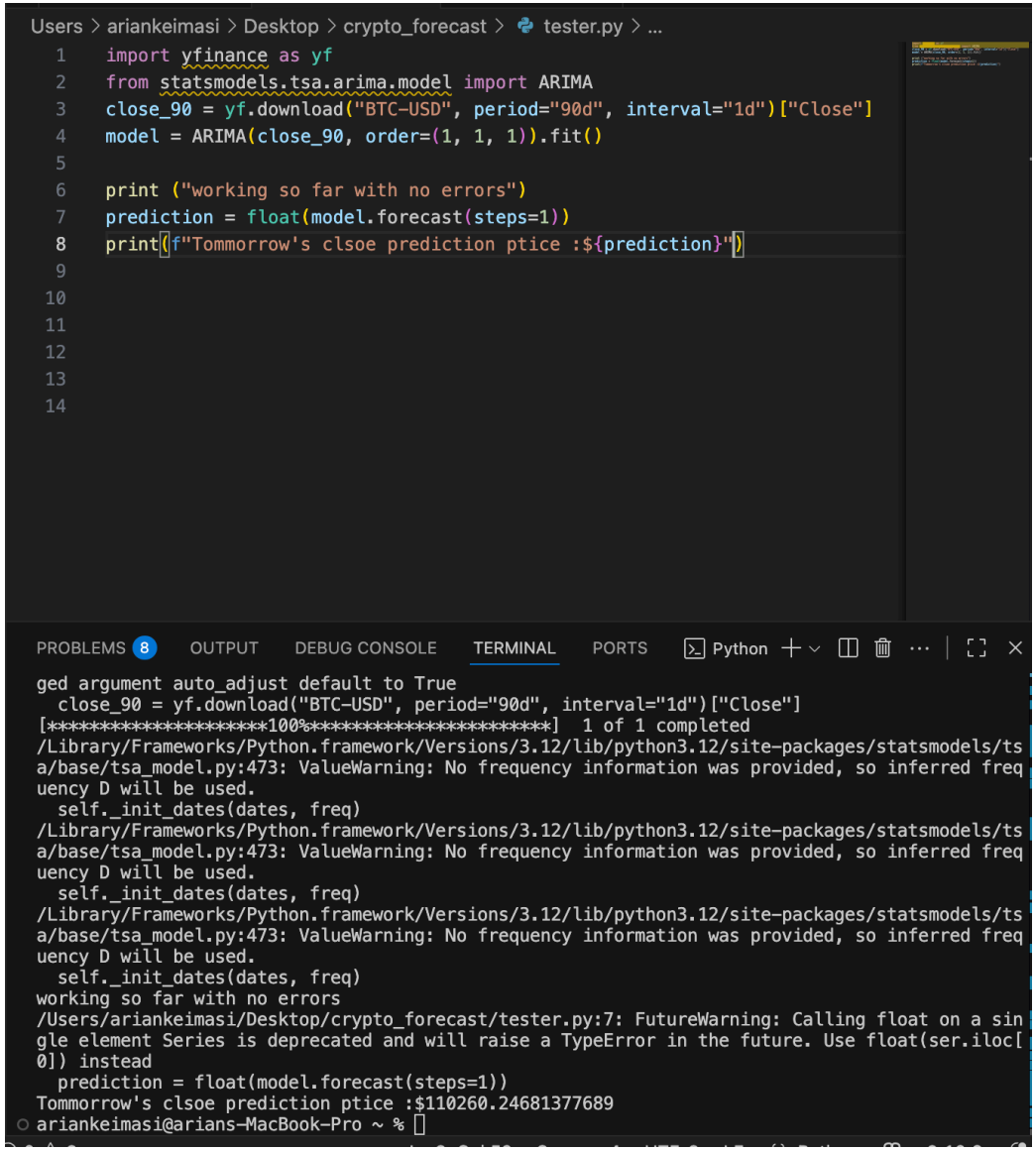
```
PROBLEMS 8  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  Python

forecast/tester.py
/Users/ariankeimasi/Desktop/crypto_forecast/tester.py:3: FutureWarning: YF.down
ged argument auto_adjust default to True
  close = yf.download("BTC-USD", period="90d", interval="1d")["Close"]
[*********************100%**********************]  1 of 1 completed
working so far with no errors
Ticker            BTC-USD
Date
2025-05-28  107802.328125
2025-05-29  105641.757812
2025-05-30  103998.570312
2025-05-31  104638.093750
2025-06-01  105652.101562
...                ...
2025-08-21  112419.031250
2025-08-22  116874.085938
2025-08-23  115374.328125
2025-08-24  113458.429688
2025-08-25  110050.859375

[90 rows x 1 columns]
```

S.S.92: Successfully **retrieved 90 days of historical Bitcoin closing price data** using the yfinance API for model input.

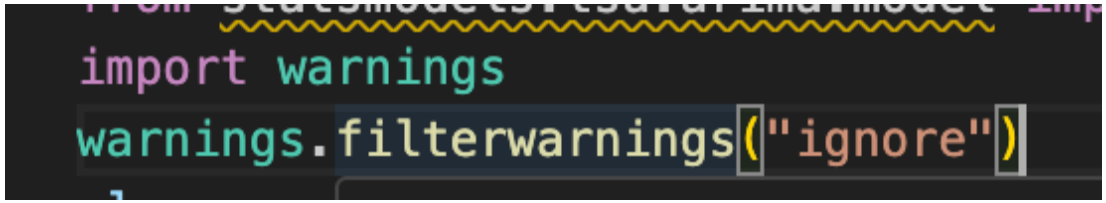
```
model = ARIMA(close_90, order=(1, 1, 1)).fit()
```

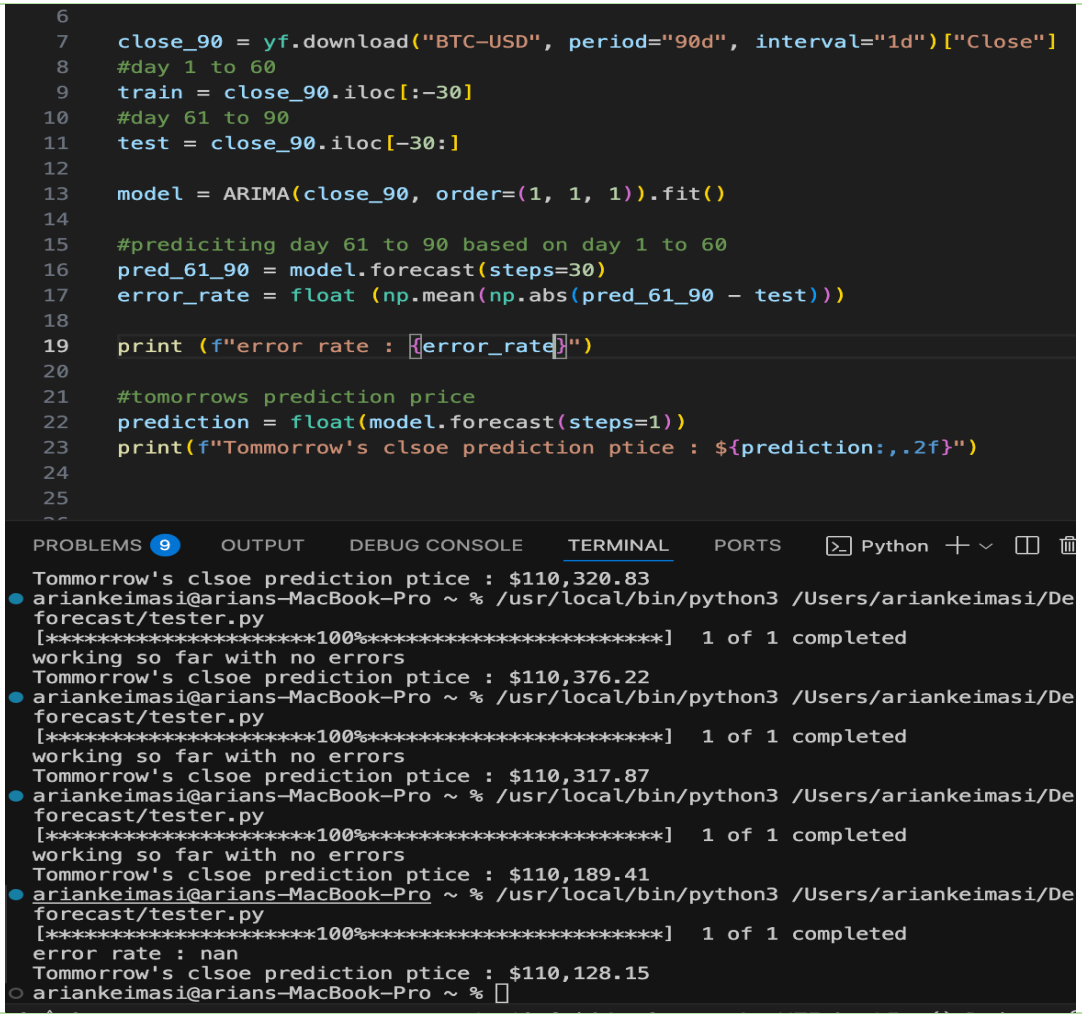
S.S.93: Initial **ARIMA model (1,1,1)** created and fitted to historical price data to generate future predictions.

An ARIMA (p,d,q) model is a statistical method used for time series forecasting. ARIMA stands for AutoRegressive Integrated Moving Average, and it combines three components to model and predict future data points. The parameters p, d, and q define the structure of the model: **p** is the order of the autoregressive (AR) component, **d** is the degree of differencing (Integration) needed to make the data stationary, and **q** is the order of the moving average (MA) component. Together, these parameters allow the model to capture trends, seasonality, and noise in time series data.



S.S.94: Generated first prediction output; excessive warnings and **verbose output** identified as a usability issue.

	S.S.95: Suppressed unnecessary warnings using **warnings.filterwarnings("ignore")** to improve output clarity.

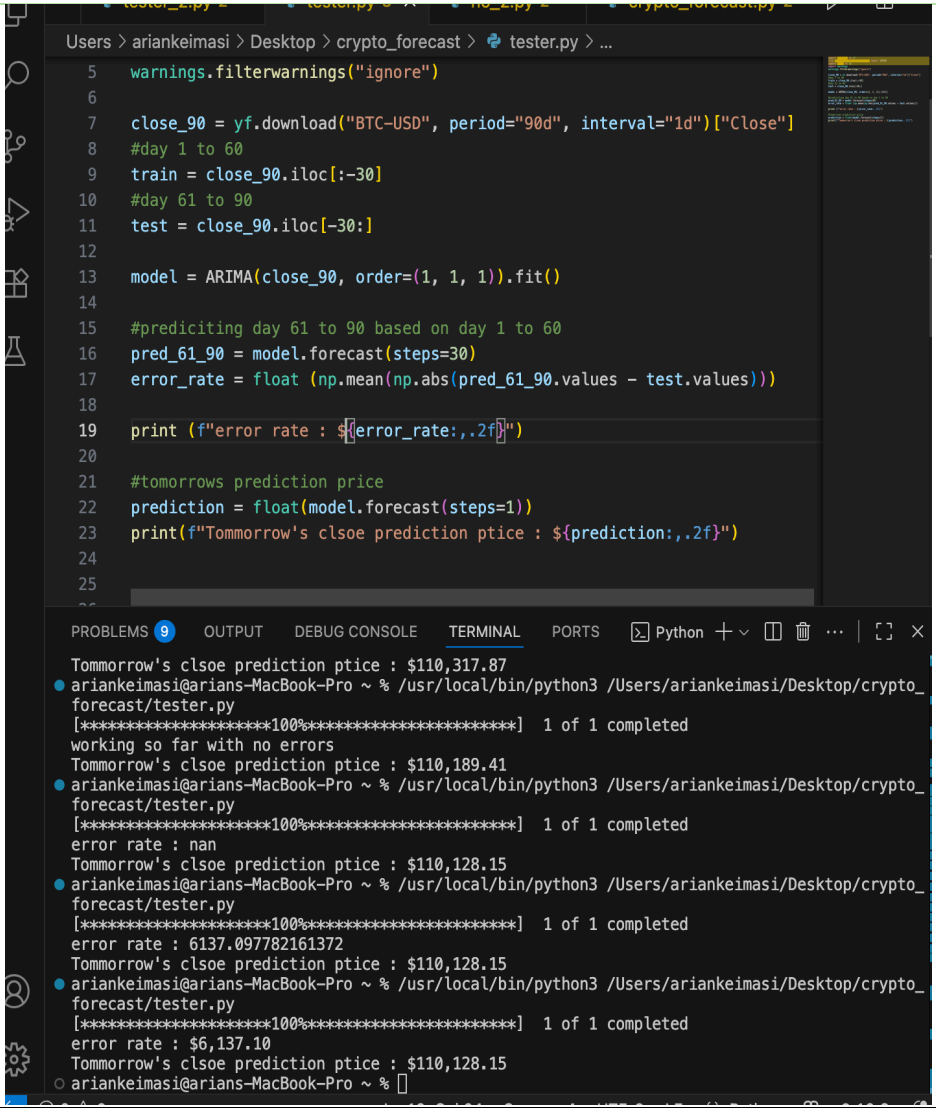


```
close_90 = yf.download("BTC-USD", period="90d", interval="1d")["Close"]
#day 1 to 60
train = close_90.iloc[:-30]
#day 61 to 90
test = close_90.iloc[-30:]

model = ARIMA(close_90, order=(1, 1, 1)).fit()

#prediciting day 61 to 90 based on day 1 to 60
pred_61_90 = model.forecast(steps=30)
error_rate = float (np.mean(np.abs(pred_61_90 - test)))

print (f"error rate : {error_rate}")

#tomorrows prediction price
prediction = float(model.forecast(steps=1))
print(f"Tommorrow's clsoe prediction ptice : ${prediction:,.2f}")
```

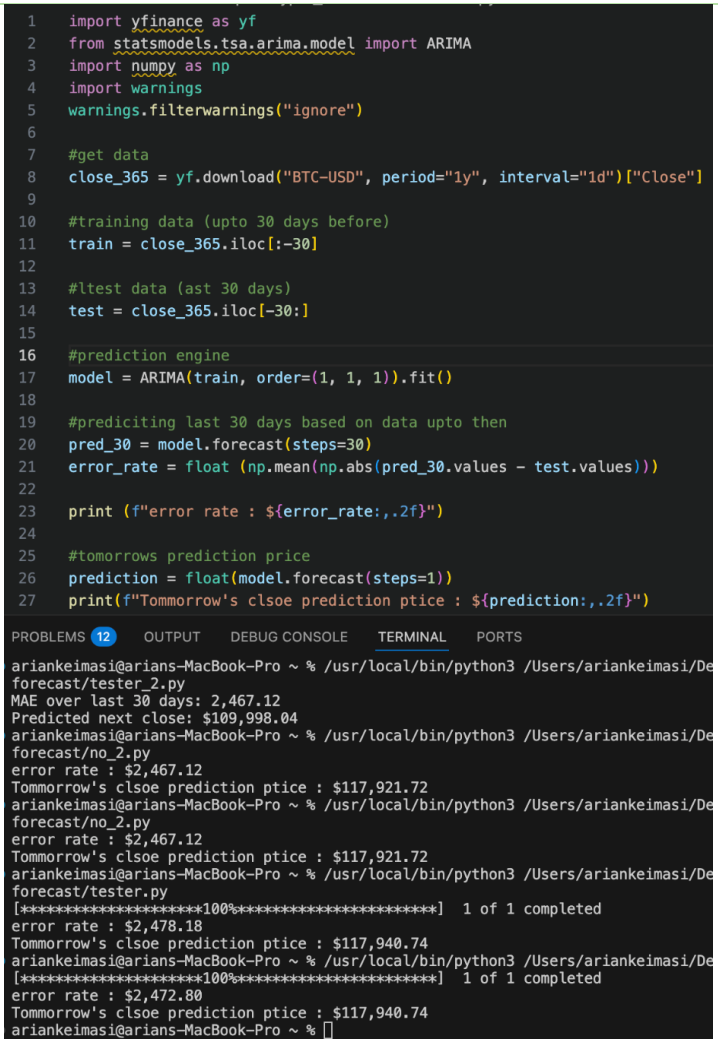
PROBLEMS 9 OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
Tommorrow's clsoe prediction ptice : $110,320.83
ariankeimasi@arians-MacBook-Pro ~ % /usr/local/bin/python3 /Users/ariankeimasi/De
forecast/tester.py
[*********************100%**********************]  1 of 1 completed
working so far with no errors
Tommorrow's clsoe prediction ptice : $110,376.22
ariankeimasi@arians-MacBook-Pro ~ % /usr/local/bin/python3 /Users/ariankeimasi/De
forecast/tester.py
[*********************100%**********************]  1 of 1 completed
working so far with no errors
Tommorrow's clsoe prediction ptice : $110,317.87
ariankeimasi@arians-MacBook-Pro ~ % /usr/local/bin/python3 /Users/ariankeimasi/De
forecast/tester.py
[*********************100%**********************]  1 of 1 completed
working so far with no errors
Tommorrow's clsoe prediction ptice : $110,189.41
ariankeimasi@arians-MacBook-Pro ~ % /usr/local/bin/python3 /Users/ariankeimasi/De
forecast/tester.py
[*********************100%**********************]  1 of 1 completed
error rate : nan
Tommorrow's clsoe prediction ptice : $110,128.15
ariankeimasi@arians-MacBook-Pro ~ %
```

S.S.96: Implemented error calculation using mean absolute error (**MAE**); initial result returned NaN, indicating a **flaw in data handling.**



S.S.97



S.S.98: Identified critical issue: model was incorrectly trained on full dataset instead of training subset, causing **misleading accuracy**. Refined model by splitting data into

Corrected error calculation by **aligning prediction and test datasets**, producing a valid MAE (~$6137).	training and testing sets, reducing error rate to approximately 2.25%.

```
self.rsi = None
self.rsi_signal = None
```

S.S.99: Initialised RSI-related variables (rsi, rsi_signal) to ensure proper state management before calculation.

```
def calculate_rsi(self, period=14):
    delta = self.table["close"].diff()

    gain = delta.where(delta > 0, 0)
    loss = -delta.where(delta < 0, 0)

    avg_gain = gain.rolling(window=period).mean()
    avg_loss = loss.rolling(window=period).mean()

    if avg_loss.iloc[-1] == 0:
        return 100.0

    rs = avg_gain / avg_loss
    rsi = 100 - (100 / (1 + rs))

    return float(rsi.iloc[-1])
```

S.S.100: Implemented **RSI calculation using rolling averages of gains and losses** over a 14-period window.

```python
def interpret_rsi(self):
    if self.rsi is None:
        return "No RSI data"
    elif self.rsi < 30:
        return "Oversold (Buy Signal)"
    elif self.rsi > 70:
        return "Overbought (Sell Signal)"
    else:
        return "Neutral (Hold)"
```

S.S.101**:** Developed RSI interpretation logic to **classify market conditions** (Overbought, Oversold, Neutral).

```python
self.rsi = self.calculate_rsi()
self.rsi_signal = self.interpret_rsi()
```

S.S.102: **Displayed RSI label in GUI using Tkinter**, positioning it clearly within the interface.

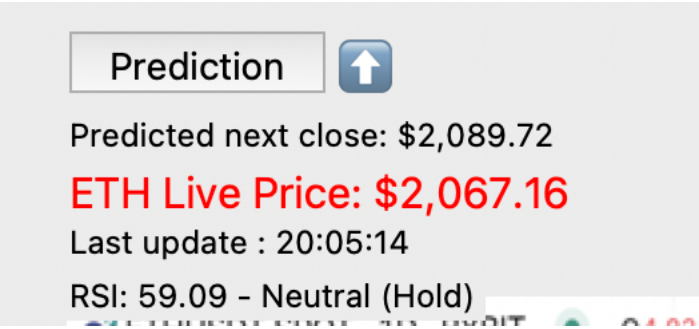
```python
self.rsi_lbl = tk.Label(self, text="RSI: –", font=("Calibri", 12))
self.rsi_lbl.place(x=215, y=160)
```

S.S.103: Linked computed RSI values and signals from the model to variables for **UI display**.

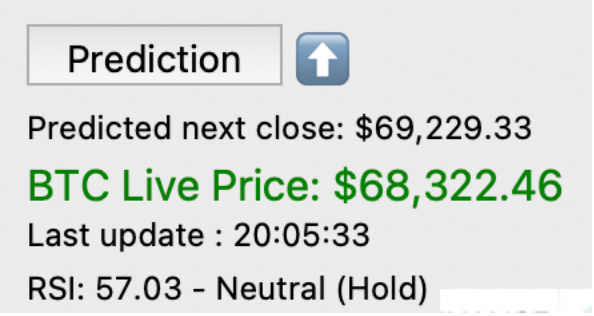
```python
rsi_value = model.rsi
rsi_signal = model.rsi_signal
```

```
self.rsi_lbl.config(text=f"RSI: {rsi_value:.2f} - {rsi_signal}")
```

S.S.104: **Updated GUI dynamically** to display formatted RSI value and corresponding trading signal.



S.S.105: Integrated prediction, live price, and RSI into the interface for Ethereum, **confirming multi-asset functionality**.



S.S.106: Verified consistent integration of prediction system with Bitcoin data, ensuring **accuracy across different assets**.

```
else:
    arrow = "➡️"
    arrow_colour = "black"
```

S.S.107: Fixed UI bug related to arrow indicator logic, ensuring correct visual feedback for prediction direction.

⇒ **Code explanation:**

This section implements a more advanced forecasting stage by testing an ARIMA time-series model and then extending the prediction system with RSI-based market interpretation. The ARIMA model was introduced as an alternative statistical forecasting method to explore whether a dedicated time-series approach could improve predictive accuracy compared with the earlier linear regression baseline.

The implementation begins by importing the required libraries: yfinance for retrieving historical Bitcoin price data and statsmodels for building the ARIMA model. Historical daily closing prices were downloaded and stored as a one-dimensional time series, which was then used to fit an ARIMA(1,1,1) model. The chosen parameters represent one autoregressive term, first-order differencing, and one moving average term. Once fitted, the model generated a one-step forecast for the next day's closing price.

To evaluate the model rather than simply generate a single output, error calculation was added using mean absolute error (MAE). This required splitting the dataset into a training portion and a testing portion so that predictions could be compared against unseen values. The model was then refitted using only the training subset, and multi-step forecasts were produced to estimate the final test period. The forecast outputs and test values were compared numerically to calculate the average prediction error.

During the same stage, the prediction system was extended with RSI functionality. New variables were added to store the RSI value and its interpreted signal. A dedicated calculate_rsi() method computes RSI using rolling averages of gains and losses over a 14-period window, while a second

method, interpret_rsi(), converts the RSI result into a user-readable state such as “Overbought”, “Oversold”, or “Neutral”. These outputs are then linked to the GUI so that the interface displays both the numerical RSI value and the corresponding trading interpretation. Finally, the live interface was updated so that prediction output, live price, RSI, and direction arrows all appear together for both Bitcoin and Ethereum.

⇒ **Justification:**

ARIMA was selected as a higher-level forecasting technique because it is specifically designed for time-series data, making it more appropriate for sequential financial prices than a general regression model. Unlike simpler approaches that treat price observations as mostly independent values, ARIMA accounts for dependency between past and present values, trend adjustment through differencing, and remaining noise through the moving average component. This made it a useful model to test as the project moved beyond the basic prediction stage.

Using yfinance was justified because it provided a quick and structured way to download historical closing prices in a format that was easy to test during development. This allowed the ARIMA model to be explored without first redesigning the main Binance pipeline. It also supported rapid experimentation while I was still evaluating whether ARIMA would be suitable for the final system.

The addition of error calculation was essential because prediction quality cannot be judged reliably by looking only at a single next-day forecast. Introducing MAE made the evaluation more rigorous by providing a measurable indication of model accuracy. Splitting the dataset into training and testing sets was also important because fitting the model on the entire dataset would produce misleading results and would not reflect how the model performs on unseen data.

The RSI extension was justified because price prediction alone is not always sufficient for decision support. RSI adds a momentum-based interpretation layer, allowing the system to communicate whether the market appears overbought, oversold, or neutral. This makes the output more useful to users and helps translate statistical results into understandable trading context. Displaying both the numeric RSI value and its interpretation in the GUI improves transparency and supports the user-centred design goals identified earlier in the project.

⇒ **Iterative development:**

During the first stage of this work, the required ARIMA libraries were installed successfully and historical Bitcoin closing prices were retrieved without errors. This confirmed that the external dependencies and data source were functioning correctly. An initial ARIMA(1,1,1) model was then fitted to 90 days of closing-price data, and a first next-day prediction was generated. Although the model executed successfully, the terminal output included a large amount of warning text and verbose information, which made the results difficult to read. This was treated as a usability issue, and the output was cleaned by suppressing non-essential warnings.

The next improvement involved adding an error calculation so that the model's performance could be judged more objectively. My first attempt at this produced a NaN error result, which showed that the prediction values and test values were not being handled correctly. After reviewing the calculation, I corrected the dataset alignment so that forecast outputs and test data matched properly, which produced a valid MAE value. However, the initial MAE was still relatively large, which suggested the model setup was not yet reliable enough.

A more serious issue was then identified: the ARIMA model had been fitted using the full dataset rather than only the training data. This meant the calculated accuracy was misleading because the

model had already seen the data it was being evaluated against. To correct this, I restructured the code so that the model was trained only on the training subset and then used to forecast the test period. After this refinement, the error rate reduced substantially to around 2.25%, which was more realistic and acceptable.

In parallel, I extended the model class to support RSI. New attributes were added to store RSI values and signals safely before calculation. A dedicated RSI calculation method was implemented, followed by an interpretation method that classified the output into user-readable categories. These values were then connected to GUI labels so that the interface could show both the raw RSI value and the interpreted market condition.

Further refinements were made during GUI integration. The RSI output was first linked to model variables, then formatted for label display, and finally shown consistently on both Ethereum and Bitcoin pages. A final bug was found in the arrow indicator logic, where the neutral case did not always display the correct symbol and colour. This was corrected so that the interface now provides accurate visual feedback for rising, falling, and neutral prediction states.

⇒ Validation:

Validation confirmed that the ARIMA forecasting and RSI integration operate correctly as a complete prediction pipeline. Running the model successfully retrieves data, generates forecasts, calculates error rates, and updates the GUI without runtime failures. Iterative testing verified that splitting the dataset into training and testing sets produced a valid and realistic MAE, resolving earlier inaccuracies caused by incorrect model training. Additional validation confirmed that RSI values and signals are calculated consistently and displayed correctly within the interface for both Bitcoin and Ethereum. Final usability checks showed that the combined output (prediction, live price, RSI, and

indicator) provides clear, interpretable feedback to the user, demonstrating that the system meets both functional and usability requirements.

Test ID	Feature Tested	Test Type	Test Description	Expected Outcome	Actual Outcome	Action Taken	Result	Evidence
J1	Library installation	Normal	Install statsmodels and yfinance	Required libraries install successfully	Installed successfully	None	Pass	S.S.91
J2	Historical data retrieval	Normal	Download 90 days of BTC close data	Valid time-series data returned	90 rows returned correctly	None	Pass	S.S.92
J3	Initial ARIMA fitting	Normal	Fit ARIMA(1,1,1) on historical series	Model fits and produces forecast	Forecast produced successfully	None	Pass	S.S.93
J4	Output readability	Usability	Inspect terminal output after prediction	Prediction output should be readable	Excessive warnings shown	Added warning suppression	Pass	S.S.95
J5	MAE calculation	Boundary	Compare forecast values with test values	Valid numeric error returned	Returned NaN initially	Corrected alignment of arrays	Pass	S.S.97
J6	Dataset split correctness	Structural	Train on subset and test on unseen data	Model uses training set only	Initially trained on full data	Refactored to use train/test split	Pass	S.S.98
J7	Accuracy improvement	Normal	Recalculate MAE after split correction	Error rate should become more realistic	Reduced to ~2.25%	Retained corrected structure	Pass	S.S.98
J8	RSI variable initialisation	Normal	Check RSI state before calculation	Variables safely initialised	Initialised correctly	None	Pass	S.S.99

J9	RSI calculation	Normal	Run calculate_rsi() on closing prices	Numeric RSI value returned	Returned valid RSI value	None	Pass	S.S.100
J10	RSI interpretation	Normal	Run interpret_rsi() on calculated RSI	Correct signal classification returned	Overbought/Oversold /Neutral logic worked	None	Pass	S.S.101
J11	RSI GUI display	Normal	Display RSI value and signal in interface	Formatted RSI text shown correctly	Displayed correctly	None	Pass	S.S.105
J12	Multi-asset integration	System	Run prediction + RSI on BTC and ETH windows	Both assets display prediction and RSI consistently	Worked for both windows	Reused same logic across both pages	Pass	S.S.106
J13	Arrow indicator logic	Boundary	Test rise, fall, and neutral prediction outcomes	Correct arrow and colour shown in all cases	Neutral-state bug found initially	Corrected final else logic	Pass	S.S.107

⇒ Justification of Actions Taken During Testing:

In Test J4, the prediction output contained excessive warnings, reducing readability. To resolve this, warning suppression was implemented. This approach was chosen to ensure only relevant output is displayed. This improved the usability of the system by making prediction results clearer to interpret. In Test J5, the MAE calculation initially returned NaN due to incorrect alignment between forecast and test values. To resolve this, the arrays were aligned correctly. This approach was chosen to ensure valid comparison between predicted and actual values. This improved the reliability of the system by enabling accurate error measurement.

In Test J6, the model was initially trained on the full dataset, leading to unrealistic evaluation results. To resolve this, a train/test split was implemented. This approach was chosen to ensure the model is evaluated on unseen data. This improved the validity of the system by producing more realistic performance metrics.
In Test J13, the arrow indicator logic did not correctly handle neutral cases. To resolve this, the final else condition was corrected. This approach was chosen to ensure all possible outcomes are handled. This improved the usability of the system by providing accurate visual feedback.

⇒ Link to analysis & design:
This stage directly develops the forecasting and decision-support components identified in the earlier analysis and design sections. In the design, the prediction engine was planned to move beyond simple baseline methods into more advanced approaches that could model financial time-series behaviour more realistically. The ARIMA implementation reflects that progression by introducing a dedicated statistical time-series model and formally evaluating its performance using a training/testing structure and measurable error calculation.
The addition of RSI also links back to the design requirement for interpretable outputs rather than raw numerical forecasts alone. By combining prediction, RSI, and signal interpretation in the GUI, this stage strengthens the connection between the analytical model and the user-facing interface. It therefore supports both the technical design objective of more advanced modelling and the usability objective of presenting clear, meaningful decision-support information to the user.

Error Handling Implementation, API Failure Management, and System Robustness Improvement

Press space to play

Try:

- Checking the network cables, modem and router
- Reconnecting to Wi-Fi

ERR_INTERNET_DISCONNECTED

S.S.108: Internet failure: Tradingview website opened but not loaded
Internet connection failure identified when TradingView chart failed to load, highlighting need for error handling.

S.S.109: **Fallback behaviour** observed where “live price” label remains displayed despite no active data connection.

```
def run_prediction(self):
    try:
        model = BinancePredictor("BTCUSDT", "1d", 90)
        prediction = model.run()
        rsi_value = model.rsi
        rsi_signal = model.rsi_signal
        cur                                    
        dif  (variable) difference: float      price
        if difference > 0:
            arrow = "⬆️"
            arrow_colour = "green"
        elif difference < 0:
            arrow = "⬇️"
            arrow_colour = "red"
        else:
            arrow = "➡️"
            arrow_colour = "black"
        self.predicted_lbl.config(text=f"Predicted next close: ${predi
        self.arrow_lbl.config(text=arrow, fg=arrow_colour)
        self.rsi_lbl.config(text=f"RSI: {rsi_value:.2f} - {rsi_signal}"
    except Exception:
        self.predicted_lbl.config(text="Prediction unavailable")
        self.arrow_lbl.config(text="!", fg="red")
        self.rsi_lbl.config(text="RSI: unavailable")
```

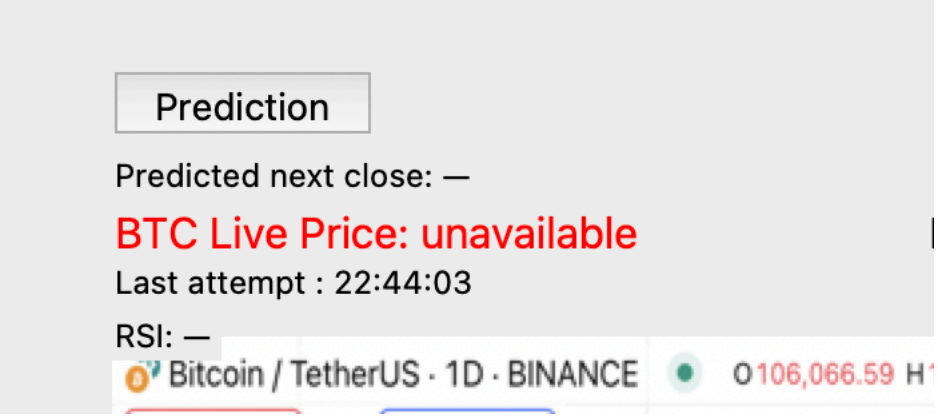
S.S.110: Implemented **try-except** structure in prediction function to handle runtime errors and prevent application crashes.

S.S.111: Verified **system stability** by maintaining GUI layout even when prediction or external data fails to load.

```python
# live btc price
def update_btc_price(self):
    try:
        response = requests.get(
            "https://api.binance.com/api/v3/ticker/price?symbol=BTCUSDT
            timeout=10
        )
        self.price = float(response.json()["price"])

        colour = "black"
        if self.last_price is not None:
            if self.price > self.last_price:
                colour = "green"
            elif self.price < self.last_price:
                colour = "red"

        self.live_btc_price.config(
            text="BTC Live Price: $" + "{:,.2f}".format(self.price),
            fg=colour
        )
        self.last_price = self.price
        now = datetime.now().strftime("%H:%M:%S")
        self.last_update.config(text=f"Last update : {now}")

    except Exception:
        self.live_btc_price.confi(text="BTC Live Price: unavailable",
        now = datetime.now().str
        self.last_update.config(text=f"Last attempt : {now}")

    self.after(2500, self.update_btc_price)
```

(parameter) text: float | str

S.S.112: Added robust **API request handling** with try-except and timeout to safely retrieve live Bitcoin price data.



S.S.113: Displayed **user-friendly fallback message** ("price unavailable") when API request fails, improving usability.

⇒ **Code Explanation**

To improve system reliability, structured error handling was implemented using try-except blocks within both the prediction and live data retrieval functions.

In the run_prediction() function, the system executes the full prediction pipeline, including model execution, RSI calculation, and comparison with the live price. Based on the difference between predicted and current price, a directional arrow (↑, ↓, →) and corresponding colour (green, red, black) are assigned. The GUI is dynamically updated using .config() to display the predicted price, RSI value, and trading signal.

A try-except block ensures that if any part of the pipeline fails (e.g. API failure, missing data, or runtime error), the system does not crash. Instead, fallback messages such as *"Prediction unavailable"* are displayed, and visual indicators are reset to maintain interface consistency.

A corresponding try-except block ensures that if the API request fails (e.g. no internet connection), the GUI displays *"BTC Live Price: unavailable"* and logs the last attempted update time, ensuring the system remains responsive and informative.

⇒ **Justification:**
The use of try-except structures was essential to ensure robustness in a system dependent on external data sources. Since live cryptocurrency prices are retrieved via APIs, failures such as network errors or server downtime are unavoidable. Without error handling, these failures would cause the program to terminate, significantly reducing usability.
Implementing graceful fallback behaviour ensures the system continues operating even under failure conditions, improving reliability and user trust.

⇒ **Iterative Development**
During testing, several critical issues were identified and resolved through iterative refinement.
Initially, when the system attempted to open external chart data (TradingView), it failed to load due to lack of internet connection (S.S.108). This highlighted that the system assumed constant connectivity and lacked fallback handling.
Further testing revealed that while the chart failed, the GUI continued to display the last known live price (S.S.109), which was misleading as it suggested data was still updating.
Additionally, the prediction pipeline initially lacked error handling, meaning that failures in API calls or model execution caused the application to crash.

To resolve these issues, try-except blocks were introduced into both the prediction and live data functions (S.S.110, S.S.112). The system was updated to display fallback messages such as *"Prediction unavailable"* and *"BTC Live Price: unavailable"* when errors occur (S.S.113).
These refinements significantly improved system robustness by preventing crashes, improving clarity of system state, and ensuring users receive accurate feedback even when external dependencies fail.

⇒ **Validation:**
Validation confirmed that the system behaves reliably under both normal and failure conditions. The prediction pipeline executes successfully, updating all GUI elements correctly. Under failure scenarios, such as loss of internet connectivity, the system remains stable and provides clear fallback messages instead of crashing. The integration of live price updates, prediction output, and visual indicators was verified to function consistently, demonstrating that the system meets both functional and usability requirements.

Test ID	Feature Tested	Test Type	Test Description	Expected Outcome	Actual Outcome	Action Taken	Result	Evidence
K1	Internet failure handling	Boundary	Disconnect internet and run system	System does not crash; fallback displayed	Chart failed to load, no handling initially	Added try-except in prediction and API functions	Pass	S.S.110
K2	Live price fallback	Boundary	Run system without internet	"Live price unavailable" shown	Old price remained visible	Updated exception handling to display unavailable message	Pass	S.S.113
K3	Prediction error handling	Normal	Trigger prediction	No crash; fallback	System crashed initially	Implemented try-except in run_prediction()	Pass	S.S.112

			with API failure	message displayed				
K4	GUI update stability	Regression	Run prediction multiple times	Labels update consistently	Occasional crashes before fix	Stabilised update logic with error handling	Pass	S.S.113

⇒ **Justification of Actions Taken During Testing:**

In Test K1, the system failed to handle internet disconnection, causing the chart to not load. To resolve this, try-except blocks were implemented in the prediction and API functions. This approach was chosen to catch runtime errors and prevent the application from crashing. This improved the reliability of the system by ensuring it continues to run under network failure conditions.

In Test K2, the system displayed outdated price data when no internet connection was available. To resolve this, the exception handling was updated to display a “Live price unavailable” message. This approach was chosen to ensure users receive accurate feedback about data availability. This improved the usability of the system by preventing misleading information.

In Test K3, triggering a prediction during API failure caused the system to crash. To resolve this, try-except handling was implemented within the run_prediction() function. This approach was chosen to safely manage errors during prediction execution. This improved the reliability of the system by preventing crashes during runtime.

In Test K4, repeated predictions caused occasional instability in GUI updates. To resolve this, the update logic was stabilised using error handling. This approach was chosen to ensure consistent execution of UI updates. This improved the reliability of the system by maintaining stable performance during repeated operations.

⇒ **Link back to analysis & design:**

This section directly fulfils the analysis requirement for reliability and real-time data handling, where the system was expected to operate continuously and handle live inputs. The implementation of error handling ensures that unpredictable external factors, such as API failure, do not compromise system functionality.

It also aligns with the design specification for usability and feedback, where outputs must be clear, interpretable, and informative. The addition of fallback messages and visual indicators ensures that users can understand system behaviour even when data is unavailable.

Furthermore, these refinements demonstrate adherence to the design principle of robustness, ensuring the system behaves predictably under both normal and exceptional conditions.

RSI-Augmented Decision Framework with Multi-Level Signals and Confidence Scoring

```python
def calculate_rsi(self, period=14):
    if self.table is None or len(self.table) < period + 1:
        return None
    delta = self.table["close"].diff()
    gain = delta.where(delta > 0, 0)
    loss = -delta.where(delta < 0, 0)
    avg_gain = gain.rolling(window=period).mean()
    avg_loss = loss.rolling(window=period).mean()
    if pd.isna(avg_loss.iloc[-1]):
        return None
    if avg_loss.iloc[-1] == 0:
        return 100.0
    rs = avg_gain / avg_loss
    rsi = 100 - (100 / (1 + rs))
    if pd.isna(rsi.iloc[-1]):
        return None
    return float(rsi.iloc[-1])
```

S.S.114: Implemented **validated RSI calculation** with checks for insufficient data and NaN values.

```
if rsi_value is None:
    self.rsi_lbl.config(text=f"RSI: unavailable - {rsi_signal}")
else:
    self.rsi_lbl.config(text=f"RSI: {rsi_value:.2f} - {rsi_signal}")

raw_data = requests.get(self.url, params=params, timeout=10).json()
```

S.S.115: Linked RSI display logic with API data retrieval to enable **dynamic GUI updates.**

```
response = requests.get(self.url, params=params, timeout=10)
raw_data = response.json()

if not isinstance(raw_data, list) or len(raw_data) == 0:
    raise ValueError("No historical data returned")
```

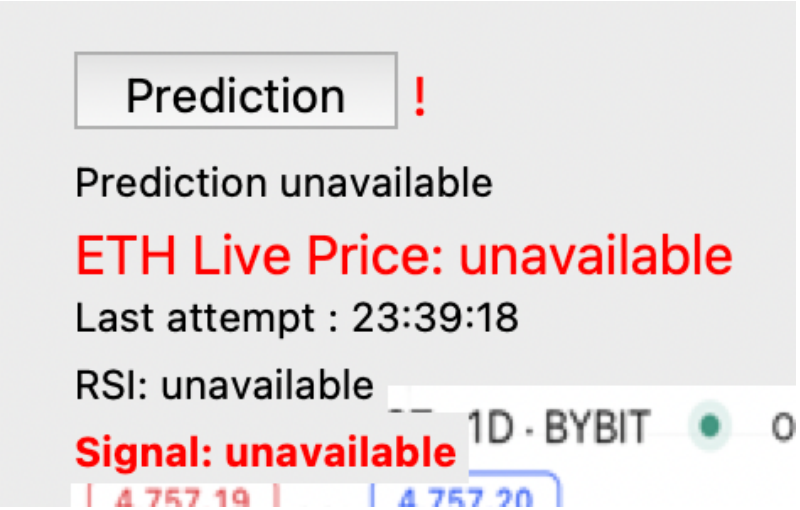
S.S.116: Added **API data validation** to prevent predictions when historical data is missing or invalid.

```
def get_trading_signal(self, current_price):
    if self.prediction is None:
        return "HOLD"

    if self.rsi is None:
        if self.prediction > current_price:
            return "BUY"
        elif self.prediction < current_price:
            return "SELL"
        else:
            return "HOLD"

    if self.prediction > current_price and self.rsi < 30:
        return "BUY"
    elif self.prediction < current_price and self.rsi > 70:
        return "SELL"
    else:
        return "HOLD"
self.signal_lbl = tk.Label(self, text="Signal: –", font=("Calibri", 12, "bold"))
self.signal_lbl.place(x=215, y=180)
signal = model.get_trading_signal(current_price)
```

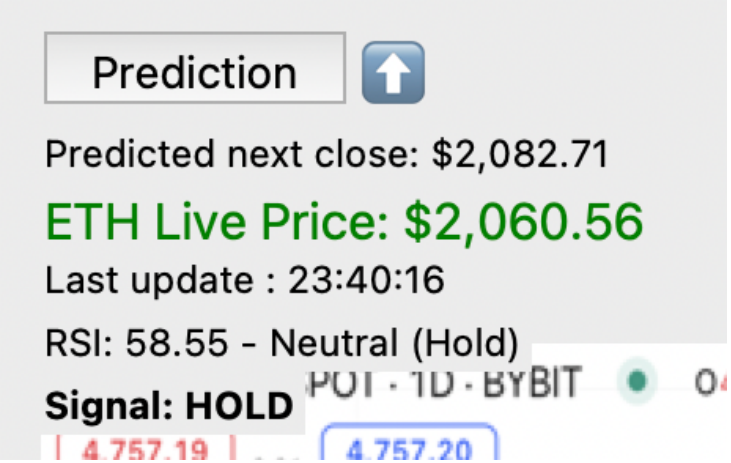
S.S.117: Developed **initial trading signal logic** combining prediction and RSI conditions.

```
    if signal == "BUY":
        self.signal_lbl.config(text="Signal: BUY", fg="green")
    elif signal == "SELL":
        self.signal_lbl.config(text="Signal: SELL", fg="red")
    else:
        self.signal_lbl.config(text="Signal: HOLD", fg="black")
except Exception:
    self.predicted_lbl.config(text="Prediction unavailable")
    self.arrow_lbl.config(text="!", fg="red")
    (parameter) self: Self@BitcoinWindow
    self.signal_lbl.config(text="Signal: unavailable", fg="red")
```

S.S.118: Linked **signal output to GUI with colour coding** for clear user interpretation.



S.S.119: Handled **no-internet scenario** by displaying “unavailable” outputs safely.



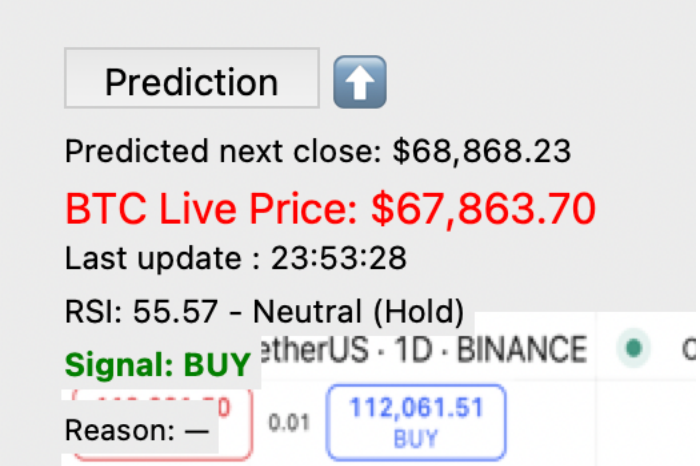
S.S.120: Identified **limitation in strict logic** causing excessive HOLD signals.

```python
def get_trading_signal(self, current_price):
    if self.prediction is None:
        return "HOLD"

    if self.rsi is None:
        if self.prediction > current_price:
            return "BUY"
        elif self.prediction < current_price:
            return "SELL"
        else:
            return "HOLD"

    if self.prediction > current_price:
        if self.rsi < 30:
            return "STRONG BUY"
        elif self.rsi <= 70:
            return "BUY"
        else:
            return "HOLD"

    elif self.prediction < current_price:
        if self.rsi > 70:
            return "STRONG SELL"
        elif self.rsi >= 30:
            return "SELL"
        else:
            return "HOLD"

    return "HOLD"
```

S.S.121: Refined logic to include **multi-level signals (STRONG BUY/SELL)** for better accuracy.

```python
def get_signal_reason(self, current_price):
    if self.prediction is None:
        return "No prediction available"

    if self.rsi is None:
        if self.prediction > current_price:
            return "Predicted price is above current price"
        elif self.prediction < current_price:
            return "Predicted price is below current price"
        return "Predicted price equals current price"

    if self.prediction > current_price and self.rsi < 70:
        return "Predicted price is above current price and RSI does not show strong overbought conditions"
    elif self.prediction < current_price and self.rsi > 30:
        return "Predicted price is below current price and RSI does not show strong oversold conditions"
    else:
        return "Prediction and RSI together suggest holding position"
self.reason_lbl = tk.Label(self, text="Reason: —", font=("Calibri", 11), wraplength=500, justify="left")
self.reason_lbl.place(x=215, y=205)
```

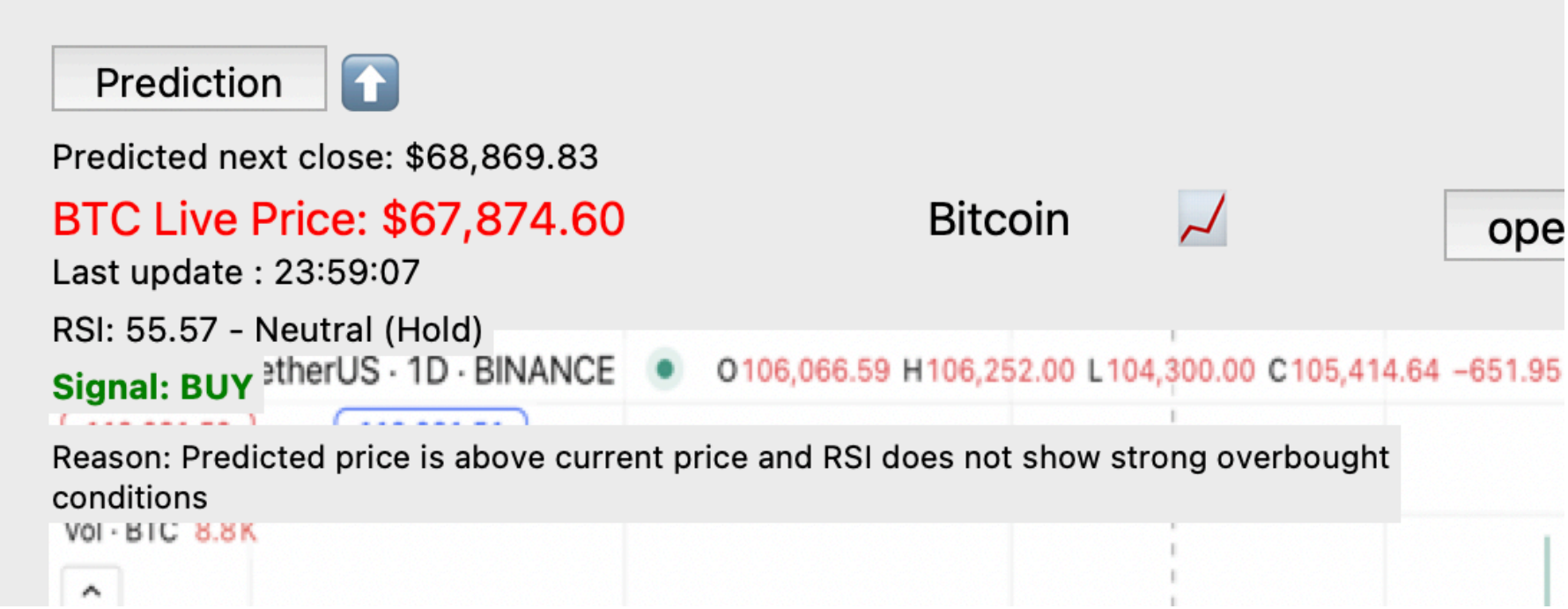
S.S.122: Implemented **reason-generation function** to explain trading decisions.



S.S.123: Displayed **signal and explanation in GUI**.

```
signal = model.get_trading_signal(current_price)
reason = model.get_signal_reason(current_price)

if difference > 0:
    arrow = "⬆️"
    arrow_colour = "green"
elif difference < 0:
    arrow = "⬇️"
    arrow_colour = "red"
else:
    arrow = "➡️"
    arrow_colour = "black"

self.predicted_lbl.config(text=f"Predicted next close: ${prediction:,.2f}")
self.arrow_lbl.config(text=arrow, fg=arrow_colour)

if rsi_value is None:
    self.rsi_lbl.config(text=f"RSI: unavailable - {rsi_signal}")
else:
    self.rsi_lbl.config(text=f"RSI: {rsi_value:.2f} - {rsi_signal}")
if signal == "BUY":
    self.signal_lbl.config(text="Signal: BUY", fg="green")
elif signal == "SELL":
    self.signal_lbl.config(text="Signal: SELL", fg="red")
else:
    self.signal_lbl.config(text="Signal: HOLD", fg="black")
self.reason_lbl.config(text=f"Reason: {reason}")

cept Exception:
self.predicted_lbl.config(text="Prediction unavailable")
self.arrow_lbl.config(text="!", fg="red")
self.rsi_lbl.config(text="RSI: unavailable")
self.signal_lbl.config(text="Signal: unavailable", fg="red")
self.reason_lbl.config(text="Reason: unavailable")
```

S.S.124: Integrated **signal and reason into prediction pipeline** for dynamic updates.



S.S.125: Validated **complete workflow output** including prediction, signal, and reason.

```
self.picture.place(x=215, y=240)
```

S.S.126: Improved **GUI layout with chart integration** for better data visualisation.

```
if signal == "BUY" or signal == "STRONG BUY":
    self.signal_lbl.config(text=f"Signal: {signal}", fg="green")
elif signal == "SELL" or signal == "STRONG SELL":
    self.signal_lbl.config(text=f"Signal: {signal}", fg="red")
else:
    self.signal_lbl.config(text="Signal: HOLD", fg="black")
```

S.S.127: Enhanced **signal display logic** to support strong signal categories.

```
if len(table) < 2:
    raise ValueError("Not enough cleaned historical data for prediction")
```

S.S.128: Added **dataset size validation** to ensure reliable model execution.

```
def get_signal_reason(self, current_price):
    if self.prediction is None:
        return "No prediction available"

    if self.rsi is None:
        if self.prediction > current_price:
            return "Predicted price is above current price"
        elif self.prediction < current_price:
            return "Predicted price is below current price"
        return "Predicted price equals current price"

    if self.prediction > current_price and self.rsi < 30:
        return "Predicted price is above current price and RSI suggests the asset may be oversold"
    elif self.prediction > current_price and self.rsi <= 70:
        return "Predicted price is above current price and RSI remains within a normal range"
    elif self.prediction < current_price and self.rsi > 70:
        return "Predicted price is below current price and RSI suggests the asset may be overbought"
    elif self.prediction < current_price and self.rsi >= 30:
        return "Predicted price is below current price and RSI remains within a normal range"
    else:
        return "Prediction and RSI together suggest holding position"
```

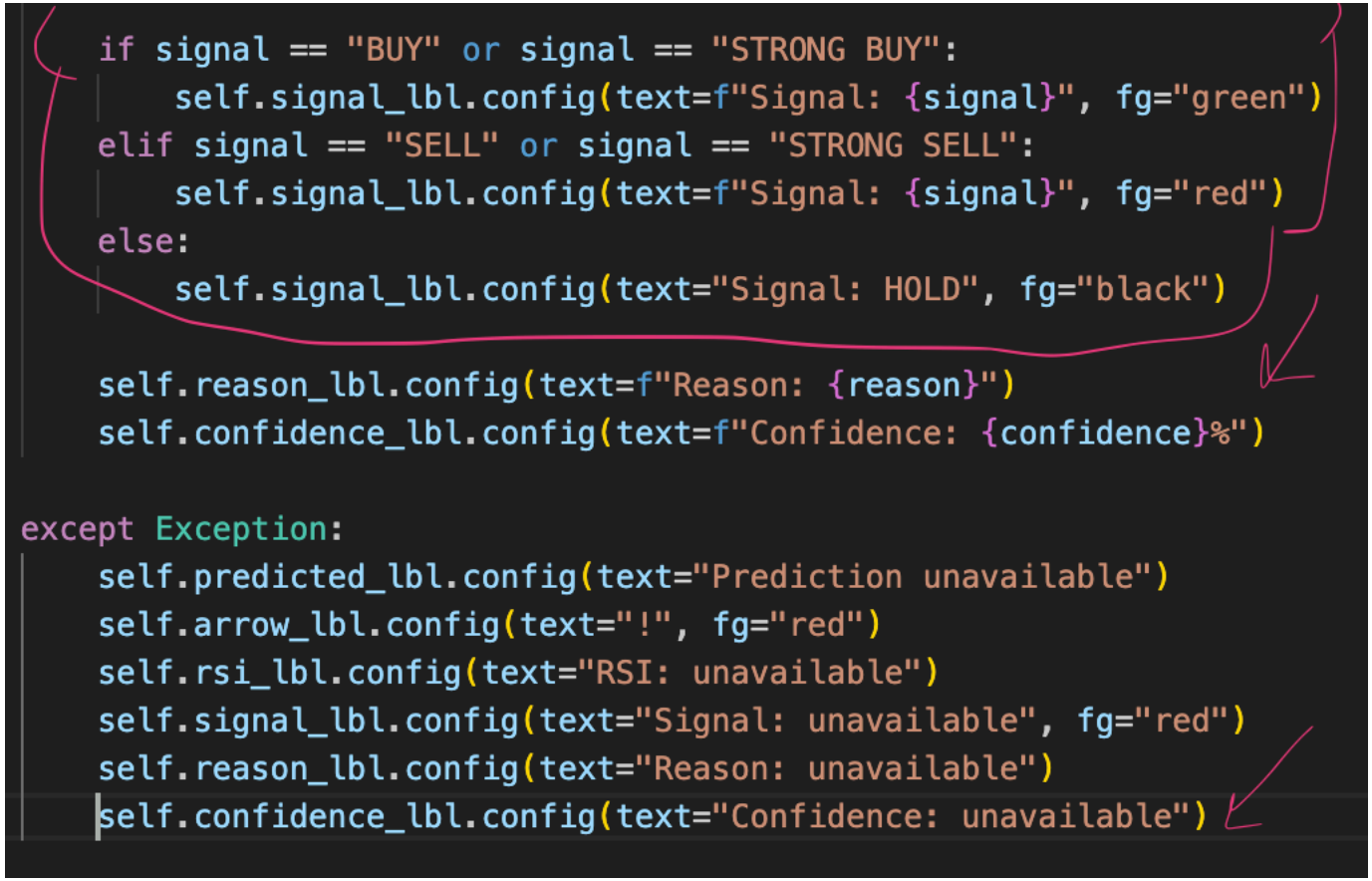
S.S.129: Refined **decision explanations** to provide clearer and more detailed feedback.

```
def get_confidence_score(self, current_price):
    signal = self.get_trading_signal(current_price)

    if signal == "STRONG BUY" or signal == "STRONG SELL":
        return 80
    elif signal == "BUY" or signal == "SELL":
        return 65
    else:
        return 50
```

```
self.confidence_lbl = tk.Label(self, text="Confidence: –", font=("Calibri", 12))
self.confidence_lbl.place(x=215, y=235)
```

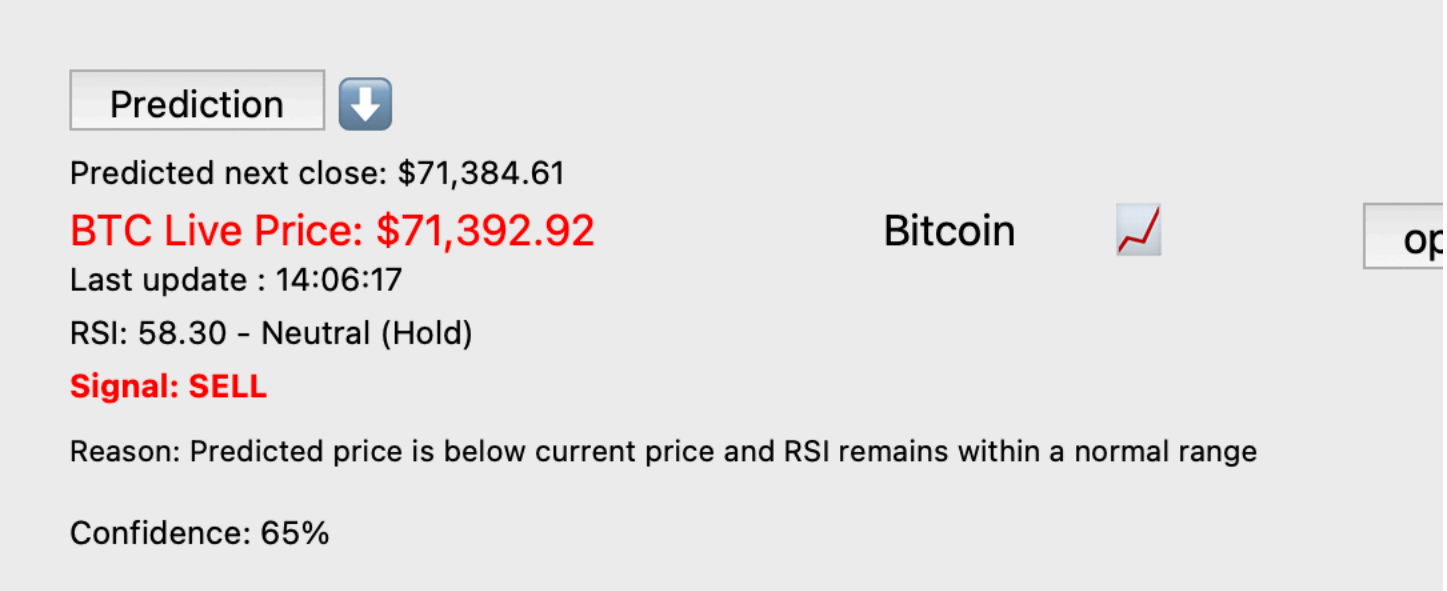
```
confidence = model.get_confidence_score(current_price)
```

S.S.130: : Implemented **confidence score calculation** based on signal strength.



```
    if signal == "BUY" or signal == "STRONG BUY":
        self.signal_lbl.config(text=f"Signal: {signal}", fg="green")
    elif signal == "SELL" or signal == "STRONG SELL":
        self.signal_lbl.config(text=f"Signal: {signal}", fg="red")
    else:
        self.signal_lbl.config(text="Signal: HOLD", fg="black")

    self.reason_lbl.config(text=f"Reason: {reason}")
    self.confidence_lbl.config(text=f"Confidence: {confidence}%")

except Exception:
    self.predicted_lbl.config(text="Prediction unavailable")
    self.arrow_lbl.config(text="!", fg="red")
    self.rsi_lbl.config(text="RSI: unavailable")
    self.signal_lbl.config(text="Signal: unavailable", fg="red")
    self.reason_lbl.config(text="Reason: unavailable")
    self.confidence_lbl.config(text="Confidence: unavailable")
```

S.S.131: Integrated **confidence score into GUI with error handling**.



S.S.132: Confirmed **full system output integration** across all decision components.

⇒ Code explanation:

This section extends the prediction system by adding strengthened decision-support logic on top of the numerical forecast in combination with the RSI value. The function returns a textual decision such as BUY, SELL, HOLD, STRONG BUY, or STRONG SELL.

The logic first checks whether a valid prediction exists. If not, it returns HOLD to avoid misleading output. It then checks whether the RSI is available. Where RSI is unavailable, the function falls back to a simpler comparison between predicted price and current price. Where RSI is available, the logic uses a stronger set of rules: low RSI combined with a higher predicted price can produce STRONG

BUY, while high RSI combined with a lower predicted price can produce STRONG SELL. Intermediate cases return BUY or SELL, and otherwise the function returns HOLD.
A second method, get_signal_reason(self, current_price), was implemented to generate a short text explanation for the signal. This function interprets the relationship between prediction, current price, and RSI and produces a human-readable reason such as the predicted price being above the current price while RSI remains within a normal range.
A third method, get_confidence_score(self, current_price), calculates a simple confidence percentage based on the strength of the signal. STRONG BUY and STRONG SELL return a higher score, BUY and SELL return a medium score, and HOLD returns a lower confidence score.
These outputs were then integrated into the GUI through additional labels: signal, reason, and confidence. The labels are updated dynamically during run_prediction() using .config(), and are also updated in the exception-handling block so that, if a failure occurs, the interface displays unavailable states rather than invalid or outdated values. Finally, the layout was adjusted so these new outputs fit cleanly above the chart without overlapping other interface elements.

⇒ Justification:
The combination of prediction and RSI was justified because it improves the quality of the decision rule. Price prediction alone indicates expected direction, while RSI adds a measure of market momentum. Using both together creates a more informed signal than using either independently.
The introduction of multi-level signals such as STRONG BUY and STRONG SELL was justified because the earlier logic was too restrictive and often returned HOLD. By separating stronger and weaker cases, the system produces more realistic and more varied outputs, which better reflects actual trading conditions.

The reason-generation function was included to improve transparency. Instead of only showing a short signal label, the system now explains why that signal was produced. This makes the output easier for users to understand and strengthens the project's usability by turning the model into a more interpretable support tool.
The confidence score was added to provide an additional layer of interpretation. Although simple, it gives users a quick indication of how strongly the current rules support the signal. This improves the explanatory value of the system and makes the interface feel more complete and professional.

⇒ Iterative development:
During the first version of this stage, an initial signal-generation function was created that combined predicted price with RSI conditions to produce BUY, SELL, or HOLD outputs. This worked logically, but testing showed that the conditions were too strict and the system returned HOLD too often (S.S.120). This made the output less useful because many normal market situations were being filtered out.
To improve this, the signal logic was redesigned so that predicted direction remained the main decision driver while RSI refined the strength of the output. This led to the introduction of multi-level signals such as STRONG BUY and STRONG SELL (S.S.121). The updated structure produced more realistic outputs and improved the usefulness of the decision system.
Once the signal itself was working, I identified that the output still lacked explanation. Users could see BUY or SELL, but not why the system had reached that conclusion. To address this, I developed the reason-generation function and linked it to a new reason label in the GUI (S.S.122–S.S.124). This improved interpretability and helped the system move beyond simple numeric display.
A further refinement involved adding a confidence score based on the final signal category (S.S.130). This created an additional layer of feedback and made the output more informative. The confidence

score was then added to the GUI and included in the error-handling path, ensuring that all decision-support outputs behave consistently if prediction fails (S.S.131).
At the same time, interface layout needed refinement because the addition of signal, reason, and confidence increased the amount of information displayed above the chart. I adjusted widget positions and chart placement to maintain a clear layout and prevent overlapping elements (S.S.126).
Dataset validation was also strengthened so that the prediction engine does not run if there is not enough cleaned data, improving robustness (S.S.128).
These iterations transformed the feature from a simple signal label into a fuller decision-support workflow including signal, explanation, confidence, and robust fallback behaviour.

⇒ Validation:
Validation confirmed that the decision-support layer works correctly as part of the full prediction workflow. Testing showed that the system can generate signals, reasons, and confidence scores from the model output and update the GUI dynamically without disrupting existing prediction and RSI features. Further checks verified that refined signal logic produces more realistic outputs than the earlier overly strict version, while layout validation confirmed that the new labels integrate cleanly into the interface. Error-handling tests also confirmed that when prediction fails or live data is unavailable, the signal, reason, and confidence fields all display safe fallback messages rather than incorrect values.

Test ID	Feature Tested	Test Type	Test Description	Expected Outcome	Actual Outcome	Action Taken	Result	Evidence

L1	RSI calculation validation	Boundary	Run RSI with insufficient or invalid data	Function returns safe value without crash	None returned safely	Added checks for short data and NaN values	Pass	S.S.114
L2	Historical API validation	Structural	Test API response when no valid historical list is returned	Model should not proceed with invalid data	Validation prevented prediction	Added ValueError for invalid/empty data	Pass	S.S.116
L3	Initial signal logic	Normal	Generate signal from prediction + RSI	BUY/SELL/HOLD output shown	Logic worked but HOLD was too frequent	Reviewed signal conditions	Pass	S.S.117
L4	Signal GUI display	Usability	Display signal label after prediction	Signal shown with clear text and colour	Signal displayed correctly	Retained colour-coded label	Pass	S.S.118
L5	No-internet signal fallback	Boundary	Run prediction with no connection	Signal shows unavailable rather than stale value	Unavailable output displayed	Added fallback update in exception block	Pass	S.S.119
L6	Refined multi-level signals	Regression	Test new signal logic after redesign	STRONG BUY/SELL and normal BUY/SELL should appear appropriately	More varied outputs produced	Replaced strict logic with multi-level version	Pass	S.S.121
L7	Reason-generation function	Normal	Generate explanation for decision	Human-readable reason displayed	Reason generated correctly	Integrated into model class	Pass	S.S.122
L8	GUI reason label	Usability	Display explanation in interface	Reason appears clearly below signal	Reason displayed correctly	Added dedicated wrapped label	Pass	S.S.132

L9	Prediction pipeline integration	System	Update prediction, signal, and reason together	All outputs refresh consistently	Worked correctly	Integrated into run_prediction()	Pass	S.S.132
L10	Layout integration	Usability	Check whether added labels overlap chart/UI	Layout remains readable	Initial crowding observed	Adjusted chart and widget positions	Pass	S.S.132
L11	Strong signal display	Regression	Test colour logic for STRONG BUY/SELL	Signal label colour matches signal type	Worked correctly	Extended display conditions	Pass	S.S.132
L12	Dataset size validation	Boundary	Run model with too little cleaned data	Prediction should stop safely	Validation triggered correctly	Added minimum-length check	Pass	S.S.128
L13	decision explanation	Normal	Test updated reason function	Output should be clearer and more detailed	Improved explanations displayed	Expanded conditional reason logic	Pass	S.S.129
L14	Confidence score calculation	Normal	Generate score from signal strength	Appropriate confidence percentage returned	80/65/50 logic worked	Added confidence function	Pass	S.S.130
L15	Confidence GUI integration	System	Display confidence in main interface	Confidence shown and updated dynamically	Display worked correctly	Added label and update logic	Pass	S.S.131
L16	Full decision workflow	System	Run complete prediction pipeline	Prediction, RSI, signal, reason, and confidence all shown together	Full workflow displayed successfully	Retained final integrated structure	Pass	S.S.132

L17	Live price pre-condition	Boundary	Run prediction when live price is unavailable	Prediction should fail safely with unavailable outputs	Safe fallback displayed	Added live-price availability check	Pass	S.S.132

⇒ Justification of Actions Taken During Testing:

In Test L1, the RSI function needed to handle insufficient or invalid data safely. To resolve this, checks for short data length and NaN values were added. This approach was chosen to prevent invalid calculations. This improved the reliability of the system by ensuring the function does not crash under edge conditions.

In Test L2, the system attempted to proceed with invalid or empty historical data. To resolve this, a ValueError was raised when no valid data was returned. This approach was chosen to stop execution when required inputs are missing. This improved the reliability of the system by preventing incorrect predictions.

In Test L3, the initial signal logic produced too many HOLD outcomes. To resolve this, the signal conditions were reviewed. This approach was chosen to better reflect meaningful market changes. This improved the functionality of the system by producing more useful outputs.

In Test L5, the system needed to handle prediction when no internet connection was available. To resolve this, a fallback output was added in the exception block. This approach was chosen to ensure users receive accurate feedback instead of stale values. This improved the usability of the system by preventing misleading information.

In Test L6, the original signal logic lacked variation in outputs. To resolve this, a multi-level signal system (e.g. STRONG BUY/SELL) was implemented. This approach was chosen to provide more

detailed decision-making output. This improved the functionality of the system by increasing the interpretability of predictions.
In Test L10, adding new labels caused initial layout crowding. To resolve this, chart and widget positions were adjusted. This approach was chosen to maintain a clear and readable interface. This improved the usability of the system by ensuring the layout remains visually organised.
In Test L12, the model could run with insufficient cleaned data, leading to unreliable predictions. To resolve this, a minimum dataset length check was added. This approach was chosen to ensure only valid datasets are used. This improved the reliability of the system by preventing low-quality predictions.
In Test L13, the initial decision explanation lacked clarity. To resolve this, the conditional logic for generating explanations was expanded. This approach was chosen to provide more informative outputs. This improved the usability of the system by making predictions easier to understand.
In Test L14, a confidence score was not initially available. To resolve this, a confidence calculation function was implemented. This approach was chosen to quantify the strength of predictions. This improved the functionality of the system by providing additional decision support.
In Test L17, the system attempted to run predictions when live price data was unavailable. To resolve this, a live-price availability check was added. This approach was chosen to prevent invalid processing. This improved the reliability of the system by ensuring predictions only run under valid conditions.

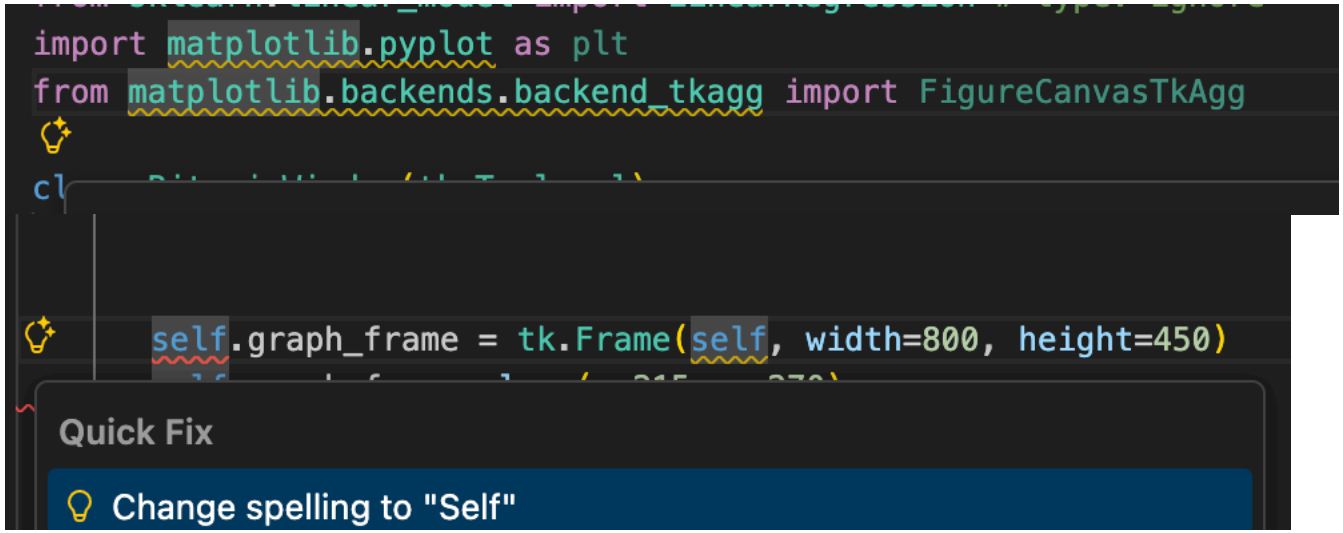
⇒ Link back to analysis & design:
This section directly implements the decision-support requirements identified in the earlier analysis and design stages. In the analysis, the system was intended not only to forecast crypto prices but

also to present information in a form that users could interpret quickly and meaningfully. The addition of signals, reasons, and confidence scores fulfils that requirement by turning numerical predictions into structured outputs that are easier to understand.

It also links closely to the design requirement for usability and clarity. The design planned a GUI that would not only display prices and predictions, but also provide clear visual feedback and support user interpretation. By integrating colour-coded signals, text explanations, and confidence values into the final layout, this stage strengthens the connection between the analytical model and the interface.

Finally, the added validation checks, error handling, and layout refinements support the broader design goal of creating a robust, modular, and testable system. This means the decision-support layer is not just functional, but also stable and aligned with the original project objectives.

Graph Rendering



S.S.134: **Added import setup for matplotlib and Tkinter canvas integration**, however encountered error.

```
ariankeimasi@arians-MacBook-Pro / % pip install matplotlib
Collecting matplotlib
  Downloading matplotlib-3.10.8-cp312-cp312-macosx_11_0_arm64.whl.metadata (52 kB)
Collecting contourpy>=1.0.1 (from matplotlib)
  Downloading contourpy-1.3.3-cp312-cp312-macosx_11_0_arm64.whl.metadata (5.5 kB)
Collecting cycler>=0.10 (from matplotlib)
  Downloading cycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)
Collecting fonttools>=4.22.0 (from matplotlib)
  Downloading fonttools-4.62.1-cp312-cp312-macosx_10_13_universal2.whl.metadata (117 kB)
Collecting kiwisolver>=1.3.1 (from matplotlib)
  Downloading kiwisolver-1.5.0-cp312-cp312-macosx_11_0_arm64.whl.metadata (5.1 kB)
Requirement already satisfied: numpy>=1.23 in /Library/Frameworks/Python.framework/Versions/3.12
/lib/python3.12/site-packages (from matplotlib) (2.3.2)
Requirement already satisfied: packaging>=20.0 in /Library/Frameworks/Python.framework/Versions/
3.12/lib/python3.12/site-packages (from matplotlib) (25.0)
Requirement already satisfied: pillow>=8 in /Library/Frameworks/Python.framework/Versions/3.12/l
ib/python3.12/site-packages (from matplotlib) (11.3.0)
Collecting pyparsing>=3 (from matplotlib)
  Downloading pyparsing-3.3.2-py3-none-any.whl.metadata (5.8 kB)
Requirement already satisfied: python-dateutil>=2.7 in /Library/Frameworks/Python.framework/Vers
ions/3.12/lib/python3.12/site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /Library/Frameworks/Python.framework/Versions/3.12/li
b/python3.12/site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Downloading matplotlib-3.10.8-cp312-cp312-macosx_11_0_arm64.whl (8.1 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8.1/8.1 MB 677.2 kB/s  0:00:12
Downloading contourpy-1.3.3-cp312-cp312-macosx_11_0_arm64.whl (273 kB)
Downloading cycler-0.12.1-py3-none-any.whl (8.3 kB)
Downloading fonttools-4.62.1-cp312-cp312-macosx_10_13_universal2.whl (2.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.9/2.9 MB 1.8 MB/s  0:00:01
Downloading kiwisolver-1.5.0-cp312-cp312-macosx_11_0_arm64.whl (64 kB)
Downloading pyparsing-3.3.2-py3-none-any.whl (122 kB)
Installing collected packages: pyparsing, kiwisolver, fonttools, cycler, contourpy, matplotlib
Successfully installed contourpy-1.3.3 cycler-0.12.1 fonttools-4.62.1 kiwisolver-1.5.0 matplotli
b-3.10.8 pyparsing-3.3.2

[notice] A new release of pip is available: 25.2 -> 26.0.1
[notice] To update, run: pip install --upgrade pip
ariankeimasi@arians-MacBook-Pro / %
```

```
[notice] A new release of pip is available: 25.2 -> 26.0.1
[notice] To update, run: pip install --upgrade pip
ariankeimasi@arians-MacBook-Pro / % pip install --upgrade pip
Requirement already satisfied: pip in /Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages (25.2)
Collecting pip
  Downloading pip-26.0.1-py3-none-any.whl.metadata (4.7 kB)
Downloading pip-26.0.1-py3-none-any.whl (1.8 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.8/1.8 MB 1.5 MB/s  0:00:01
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 25.2
    Uninstalling pip-25.2:
      Successfully uninstalled pip-25.2
Successfully installed pip-26.0.1
```

S.S.135: **Installed matplotlib dependency** **using pip** to enable graph plotting.

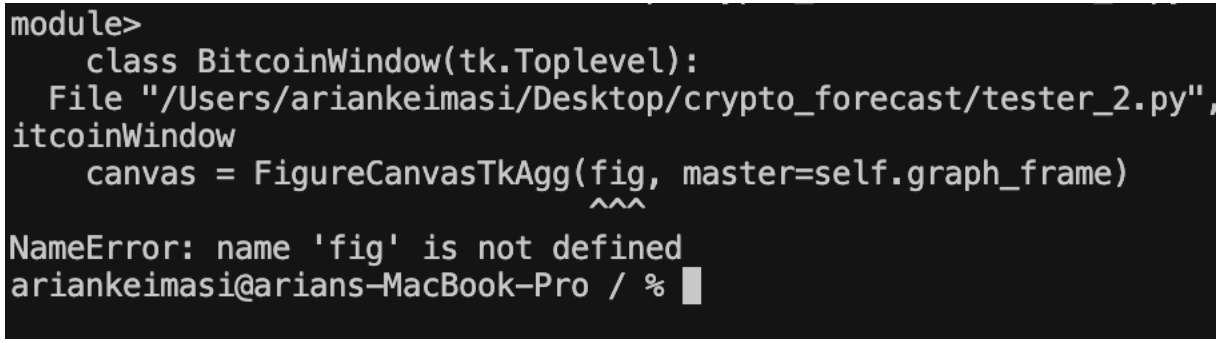
```
The issue in your screenshot looks much more like an environment/package problem
    3   import webbrowser
Import "matplotlib.pyplot" could not be resolved from
source Pylance(reportMissingModuleSource)

(module) matplotlib

View Problem (⌥F8)   Quick Fix... (⌘.)   Fix (⌘I)
    9   import matplotlib.pyplot as plt
   10   from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
   11
```

S.S.136: Tried to Resolve **module import error** caused by missing matplotlib installation.

S.S.137: Configured correct **Python interpreter/environment** in development environment.

```
def draw_graph(self, table):
    for widget in self.graph_frame.winfo_children():
        widget.destroy()

    fig, ax = plt.subplots(figsize=(8, 4.5), dpi=100)
    ax.plot(table["date"], table["close"], label="Close Price")
    ax.plot(table["date"], table["short trend (MA7)"], label="MA7")
    ax.plot(table["date"], table["medium trend (MA30)"], label="MA30")

    ax.set_title("Bitcoin Price Trend")
    ax.set_xlabel("Date")
    ax.set_ylabel("Price (USD)")
    ax.legend()
    fig.autofmt_xdate()
self.draw_graph(model.table)
```

S.S.138: Implemented **graph drawing function** plotting price and moving averages.

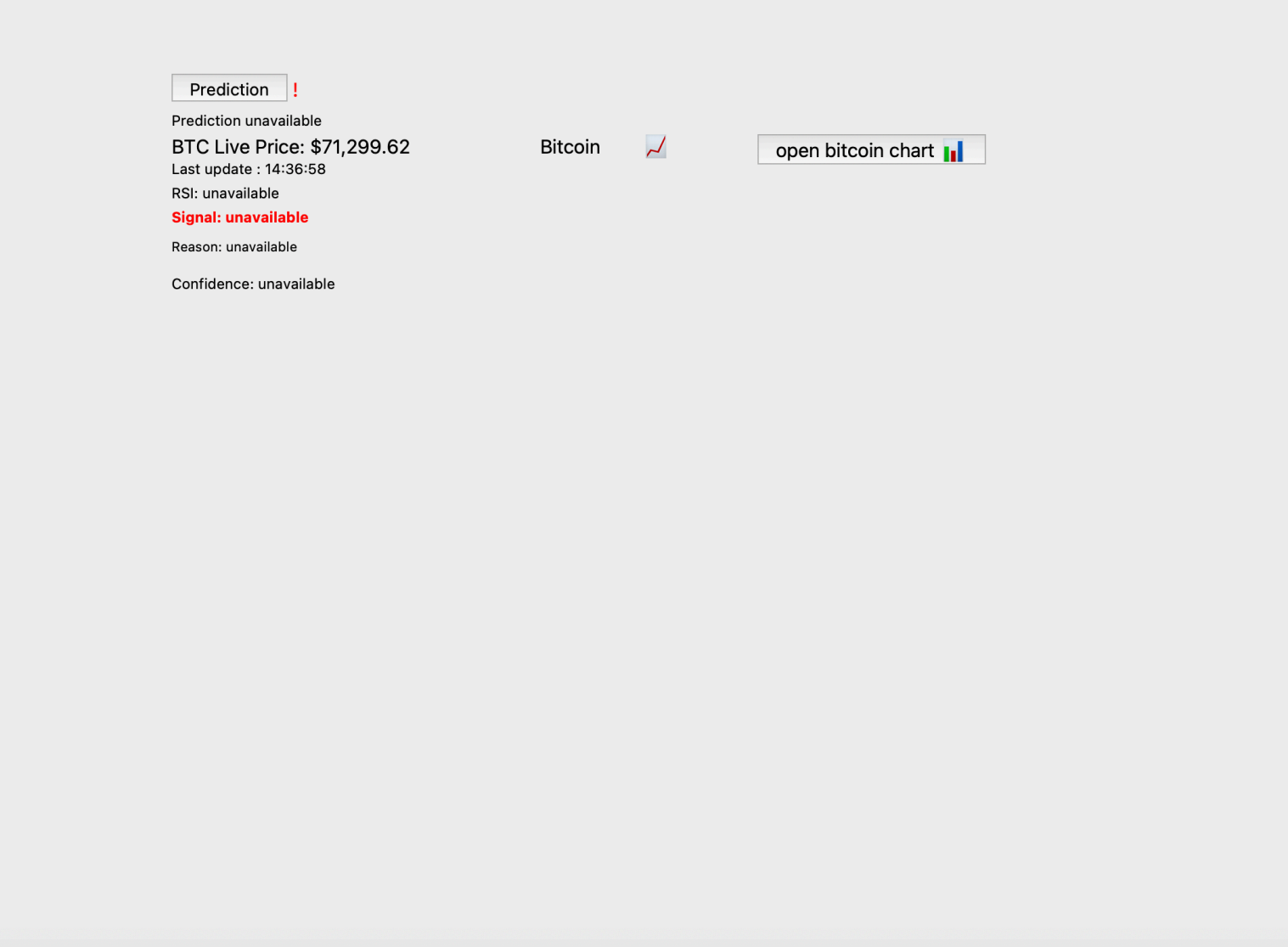


S.S.139: Encountered **NameError** **for undefined figure variable** during integration.

```
def draw_graph(self, table):
    for widget in self.graph_frame.winfo_children():
        widget.destroy()

    fig, ax = plt.subplots(figsize=(8, 4.5), dpi=100)
    ax.plot(table["date"], table["close"], label="Close Price")
    ax.plot(table["date"], table["short trend (MA7)"], label="MA7")
    ax.plot(table["date"], table["medium trend (MA30)"], label="MA30")

    ax.set_title("Bitcoin Price Trend")
    ax.set_xlabel("Date")
    ax.set_ylabel("Price (USD)")
    ax.legend()
    fig.autofmt_xdate()

    canvas = FigureCanvasTkAgg(fig, master=self.graph_frame)
    canvas.draw()
    canvas.get_tk_widget().pack(fill="both", expand=True)
```

```
prediction = model.run()
self.draw_graph(model.table)
```

S.S.140: Fixed graph rendering by correctly **linking model data to draw function**.



S.S.141: **Failed attempt to plot the graph**

```
    except Exception as e :
        print("Bitcoin prediction error:", e)
        self.predicted_lbl.config(text="Prediction u
```

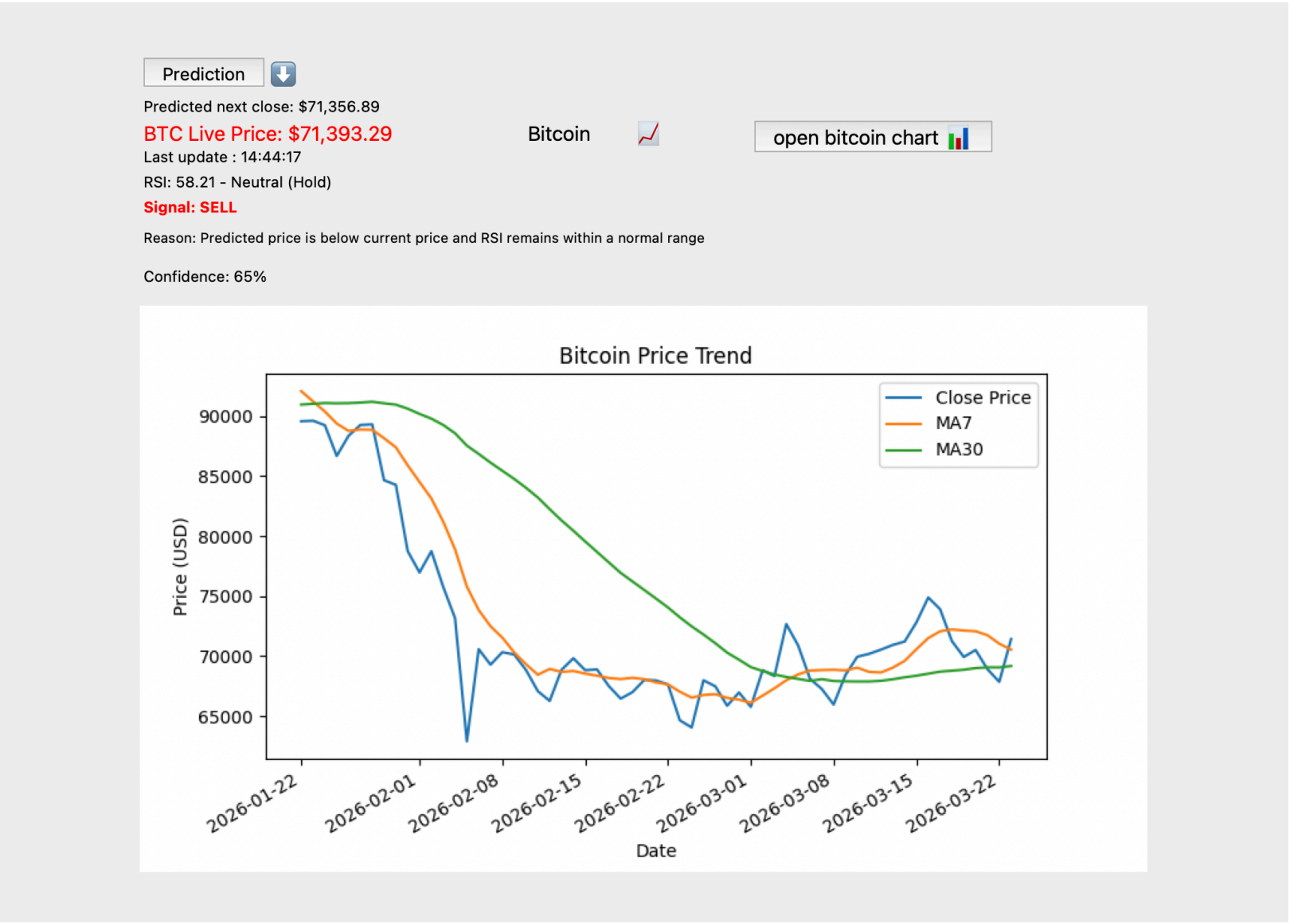
S.S.142: Added **exception handling with error logging** for debugging runtime issues.

```
ariankeimasi@arians-MacBook-Pro / % /usr/local/bin/python3 /Users/ari
Desktop/crypto_forecast/tester_2.py
Bitcoin prediction error: 'NoneType' object is not subscriptable
```

S.S.143: Identified runtime error (**NoneType issue**) during execution.

```
  model = BinancePredictor("BTCUSDT", "1d", 90)
  prediction = model.run()
  self.draw_graph(model.table)


def draw_graph(self, table):
    if table is None:
        raise ValueError("No table data available for graph")
    for widget in self.graph_frame.winfo_children():
```

S.S.144: Added **validation for missing dataset before plotting graph**.



S.S.145: Successfully displayed **final integrated chart with price and moving averages in GUI**.

⇒ Code explanation:

This section focuses on implementing data visualisation using the matplotlib library and integrating it into the Tkinter GUI. The system uses matplotlib.pyplot to generate a dynamic price chart, which is embedded into the interface using FigureCanvasTkAgg.

A new method, draw_graph(self, table), was created to handle all graph rendering. The function first clears any existing widgets in the graph frame using a loop (widget.destroy()), ensuring that each new prediction redraws the chart cleanly without overlapping previous outputs.

The function then creates a figure using plt.subplots() and plots multiple datasets from the model's processed table. These include the closing price, a short-term moving average (MA7), and a medium-term moving average (MA30). Labels, titles, and axes are added to improve readability, and fig.autofmt_xdate() ensures that date values are displayed correctly.

To display the chart in the GUI, the figure is passed into FigureCanvasTkAgg, which embeds the matplotlib figure into the Tkinter frame. The canvas is then drawn and packed into the interface using .draw() and .pack().

The graph is triggered after the prediction model runs, using self.draw_graph(model.table), ensuring that the visualisation always reflects the latest processed dataset.

Additional validation was introduced to ensure that the graph function only runs when valid data is available. If the dataset is missing or invalid, a ValueError is raised to prevent crashes. Exception handling is also implemented to ensure that, if an error occurs, the GUI still displays a safe fallback state rather than breaking.

⇒ Justification:

The addition of graph visualisation significantly improves the usability of the system. While numerical predictions and signals provide useful outputs, they do not show the trend or context of price movement. The chart allows users to visually interpret how the prediction relates to recent market behaviour.

Using moving averages (MA7 and MA30) was justified because they provide a clear indication of short-term and medium-term trends, which are commonly used in financial analysis. Including both allows users to compare immediate fluctuations with broader movement patterns.

The use of matplotlib was appropriate because it provides a reliable and flexible way to generate graphs, while FigureCanvasTkAgg enables seamless integration into a Tkinter-based interface. This ensures that the system remains consistent with the existing GUI framework.

Clearing previous graphs before drawing new ones was necessary to prevent visual overlap and memory issues, ensuring the interface remains clean and responsive.

Finally, adding validation and error handling ensures that the graph is only displayed when valid data exists. This prevents misleading outputs and improves the robustness of the system.

⇒ Iterative development:

During initial implementation, the system encountered issues with missing dependencies, as matplotlib was not installed in the development environment (S.S.135–S.S.136). This was resolved by installing the required package and configuring the correct Python interpreter (S.S.137).

The first attempt to integrate the graph resulted in a NameError, where the figure object (fig) was not properly defined within the correct scope (S.S.139). This was fixed by restructuring the graph function to ensure that the figure is created before being passed into the canvas.

Another issue occurred when the graph function was called with invalid or missing data, leading to runtime errors such as NoneType exceptions (S.S.143). To address this, validation checks were added to ensure that the dataset exists before plotting (S.S.144).
The system was then refined to handle failure scenarios, ensuring that when prediction fails or data is unavailable, the GUI displays appropriate fallback outputs instead of attempting to render an invalid graph (S.S.141).
Once stability was achieved, the graph was enhanced to include multiple plotted lines (price, MA7, MA30) and improved formatting for readability (S.S.138). The final iteration successfully integrated the chart into the GUI, producing a clean and informative visual output aligned with the rest of the system (S.S.145).

⇒ Validation:

Validation confirmed that the graph visualisation feature works correctly within the full system workflow. Testing showed that the chart updates dynamically after each prediction and accurately reflects the processed dataset, including price and moving averages. Additional validation verified that dependency issues were resolved, graph rendering errors were fixed, and appropriate safeguards prevent the graph from displaying when data is unavailable. Usability checks confirmed that the visual output enhances user understanding by clearly presenting trends alongside numerical predictions.

Test ID	Feature Tested	Test Type	Description	Expected Outcome	Actual Outcome	Action Taken	Result	Evidence

M1	Matplotlib import	Normal	Import plotting libraries	No import errors	Import error due to missing module	Installed matplotlib and configured correct Python environment	Pass	S.S.137
M2	Dependency install	Normal	Install matplotlib	Library available	Library initially not recognised by interpreter	Installed matplotlib using pip and updated Python interpreter	Pass	S.S.135
M3	Graph function	Normal	Plot dataset	Chart displayed & visible in GUI	Initial attempt failed to render graph	Implemented draw_graph() function using matplotlib and Tkinter canvas	Pass	S.S.145
M4	Data validation	Boundary	Missing dataset	Error handled safely	Runtime error (NoneType issue) when dataset was missing	Added validation check for None dataset before plotting	Pass	S.S.144
M5	Full workflow	Normal	Prediction + graph	Both update together	Graph not linked to prediction initially	Linked model output (model.table) to graph function	Pass	S.S.145

⇒ Justification of Actions Taken During Testing:

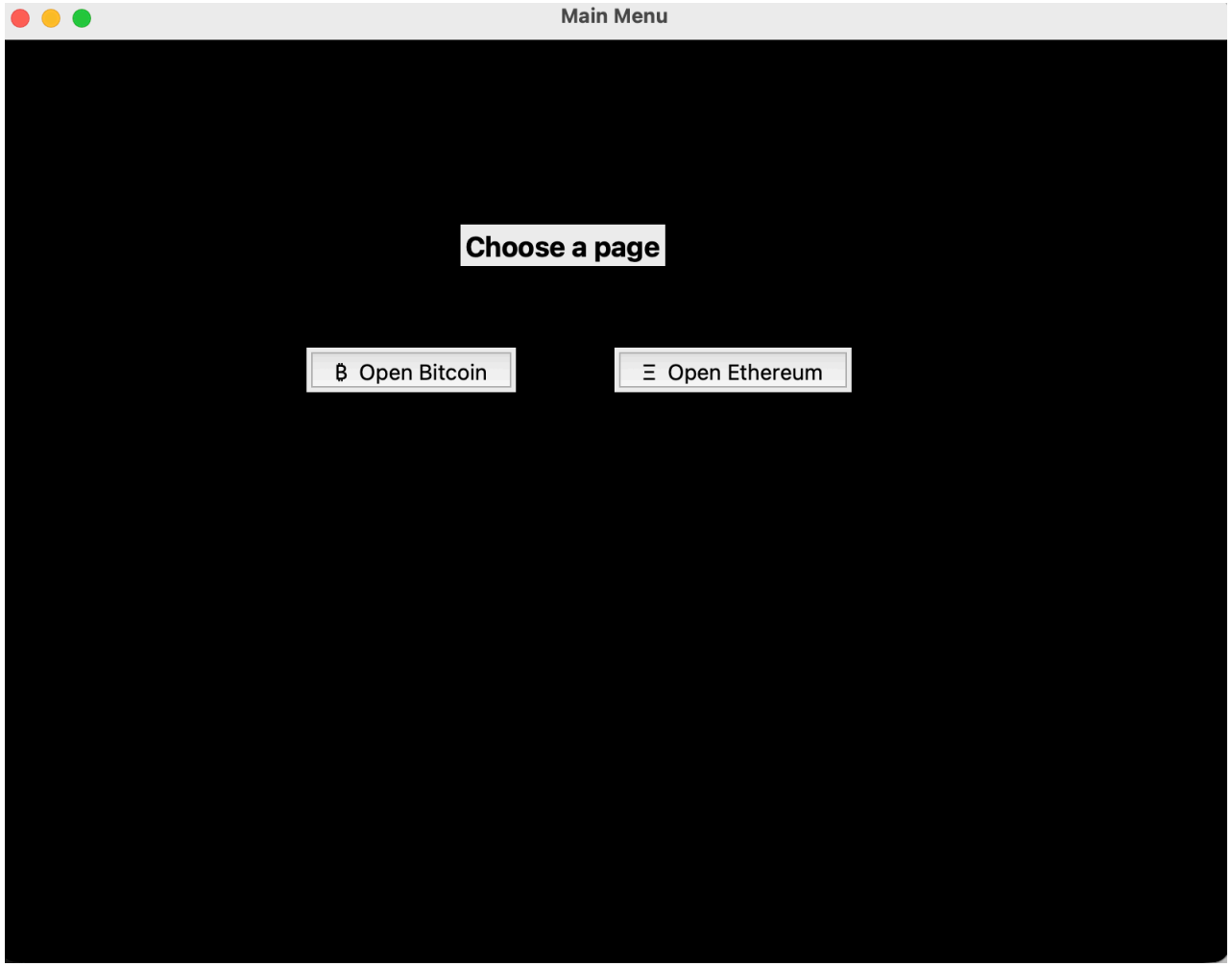
In Test M1, importing matplotlib initially caused an error due to the module not being recognised. To resolve this, the matplotlib library was installed and the correct Python interpreter was configured. This approach was chosen to ensure the development environment could access the required plotting library. This improved the functionality of the system by enabling graph rendering. In Test M2, the installed library was not initially recognised by the development environment. To resolve this, matplotlib was installed using pip and the interpreter environment was updated. This approach was chosen to ensure consistency between the runtime and development environments. This improved the reliability of the system by preventing import errors.

In Test M3, the graph did not render during initial implementation. To resolve this, a dedicated draw_graph() function was implemented using matplotlib and integrated with a Tkinter canvas. This approach was chosen to correctly embed the chart within the GUI. This improved the functionality of the system by enabling visualisation of price data and moving averages.
In Test M4, attempting to plot without a valid dataset caused a runtime error. To resolve this, validation checks were added to ensure the dataset is not None before plotting. This approach was chosen to prevent invalid operations. This improved the reliability of the system by avoiding crashes during visualisation.
In Test M5, the graph was not initially linked to the prediction output. To resolve this, the model output (model.table) was passed into the graph function. This approach was chosen to ensure the graph reflects the latest processed data. This improved the functionality of the system by integrating prediction and visualisation into a single workflow.

⇒ Link back to analysis & design:
This section directly fulfils the design requirement for data visualisation identified in the earlier stages. The system was designed not only to generate predictions but also to present them in a way that supports user understanding and interpretation.
The integration of a dynamic chart aligns with the design goal of improving usability and clarity, allowing users to visually compare predictions with historical trends. It also supports the modular design structure, where the visualisation component operates independently but integrates seamlessly with the prediction engine.
By combining numerical outputs, decision signals, and graphical representation, the system now delivers a complete user-facing workflow, as originally outlined in the analysis.

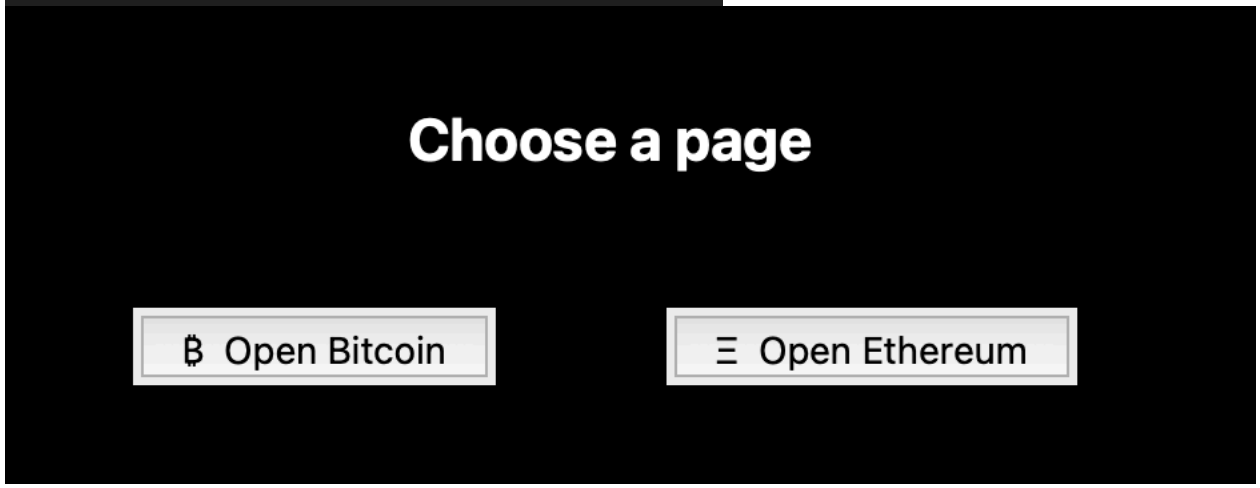
Initial Implementation of Welcome screen and dark mode

```
self.configure(bg="black")
```



S.S.146: Modified main menu interface to **dark theme**

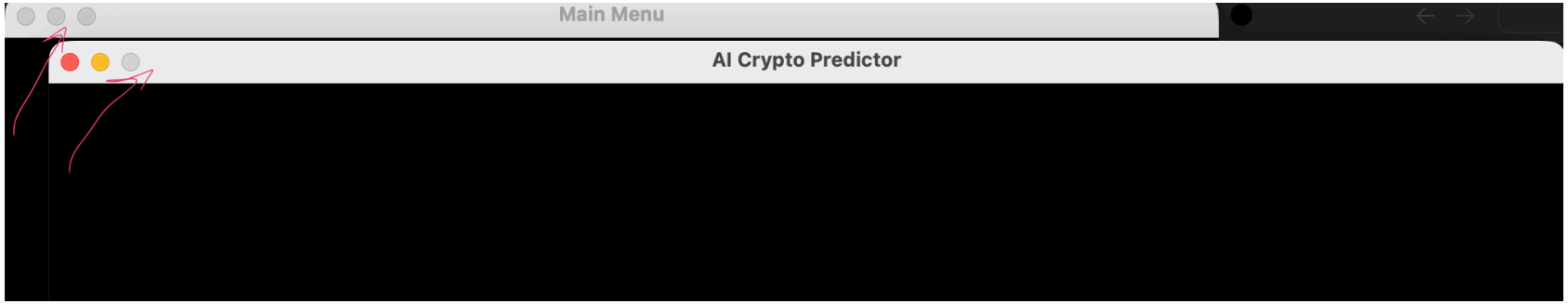
```
self.text = tk.Label(
    self,
    text="Choose a page",
    font=("Calibri", 22, "bold"),
    fg="white",
    bg="black"
)
self.text.place(x=300, y=120)
```



S.S.147: **Enhanced GIUI Style**

```
class WelcomeWindow(tk.Toplevel):
    def __init__(self, master):
        super().__init__(master)

        self.master = master
        self.title("AI Crypto Predictor")
        self.geometry("1000x700")
        self.configure(bg="black")
        self.resizable(False, False)

self.welcome_window = WelcomeWindow(self)
```

S.S.148: Created **Welcome Window class** **to initialise application layout and properties**.



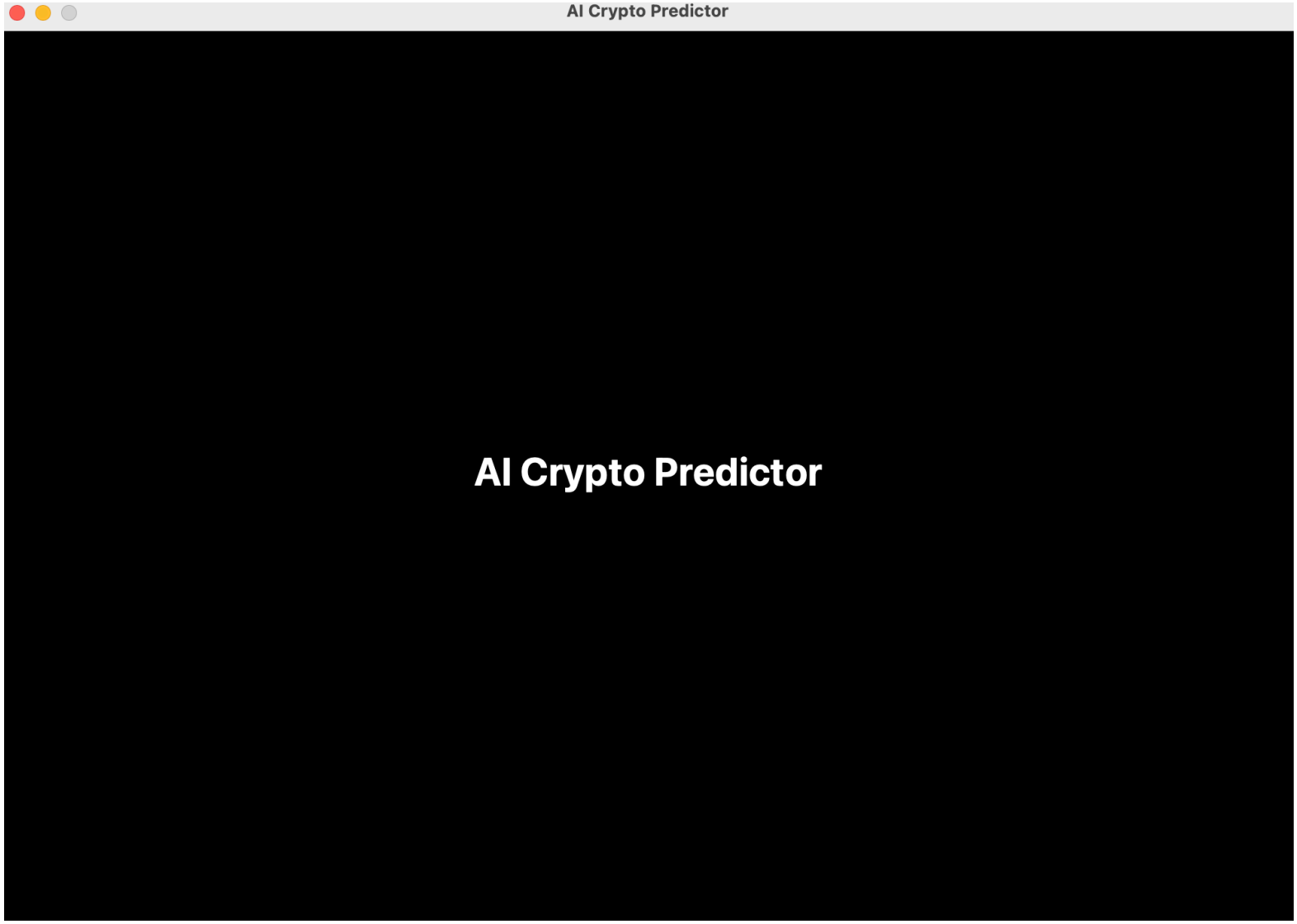
S.S.149: Running the application **opens both Main Window and Welcome Window at the same time**

```
self.withdraw()  # hide main menu until welcome screen is closed
```

S.S.150: Used **withdraw()** to hide main window until welcome screen is closed.

```python
# Main frame
self.main_frame = tk.Frame(self, bg="black")
self.main_frame.place(relx=0.5, rely=0.5, anchor="center")

# Title
self.lbl_title = tk.Label(
    self.main_frame,
    text="AI Crypto Predictor",
    font=("Calibri", 30, "bold"),
    fg="white",
    bg="black"
)
self.lbl_title.pack(pady=(0, 25))
```

S.S.151: Implemented **main frame and title label** with structured layout and spacing.



S.S.152: Initial Welcome Window

```python
# Welcome text
self.lbl_message = tk.Label(
    self.main_frame,
    text="Welcome to the AI-powered\ncrypto price prediction tool!",
    font=("Calibri", 18),
    fg="white",
    bg="black",
    justify="center"
)
self.lbl_message.pack(pady=(0, 35))
```

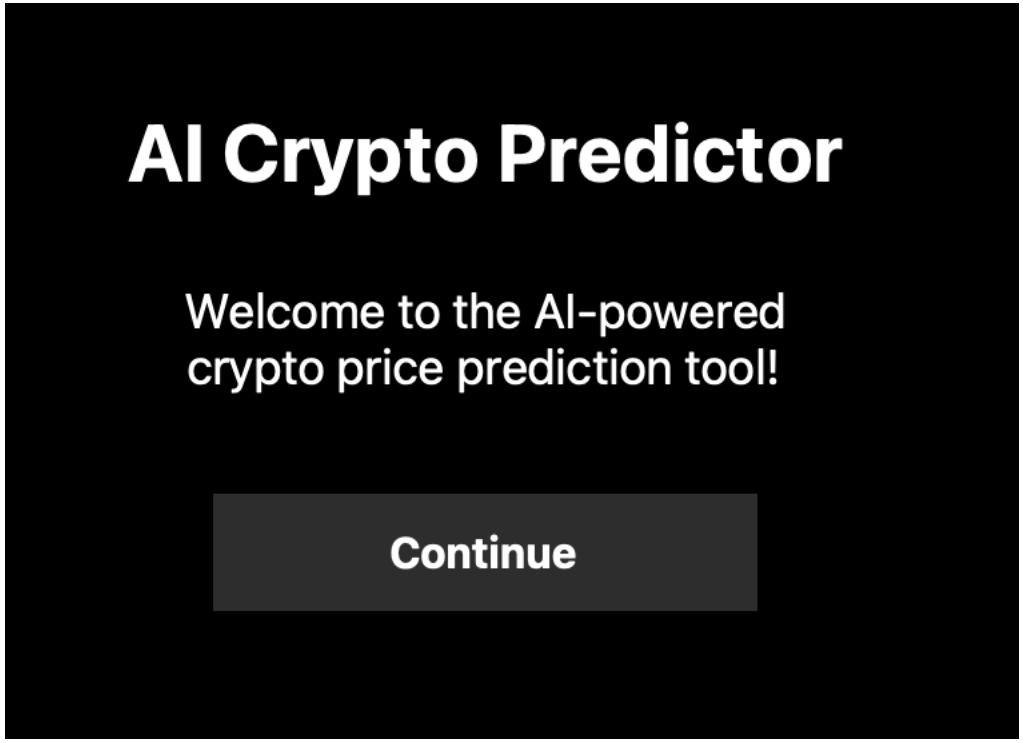
S.S.153: Added **welcome message** to guide user understanding of system purpose.

```
        # If user closes the welcome window, close the full program
        self.protocol("WM_DELETE_WINDOW", self.close_program)

    def open_main_window(self):
        self.grab_release()
        self.destroy()
        self.master.deiconify()

    def close_program(self):
        self.master.destroy()
```

S.S.154: Implemented **window close handling and transition logic** between welcome screen and main application.

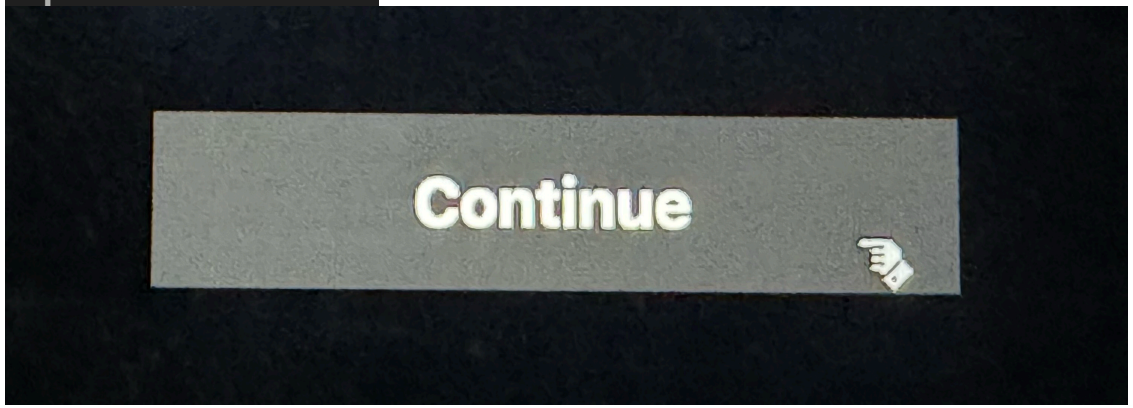
```
# Continue button
self.btn_continue = tk.Label(
    self.main_frame,
    text="Continue",
    font=("Calibri", 16, "bold"),
    bg="#2f2f2f",
    fg="white",
    width=18,
    height=2,
)
self.btn_continue.pack(pady=(0, 20))
```

S.S.155: Designed **“Continue”** button with custom styling, size, and layout for clear user interaction.



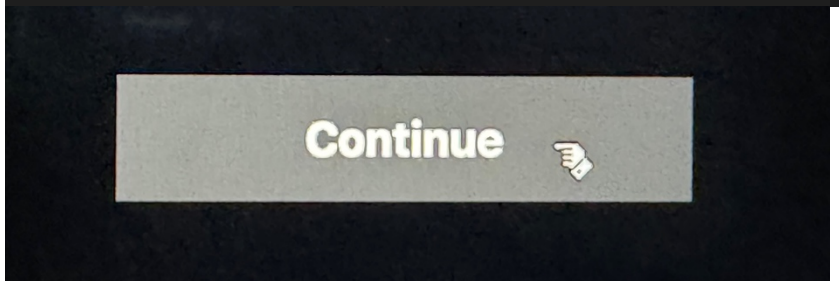
S.S.156: **Displayed** welcome interface with centralised “Continue” action to guide user flow.

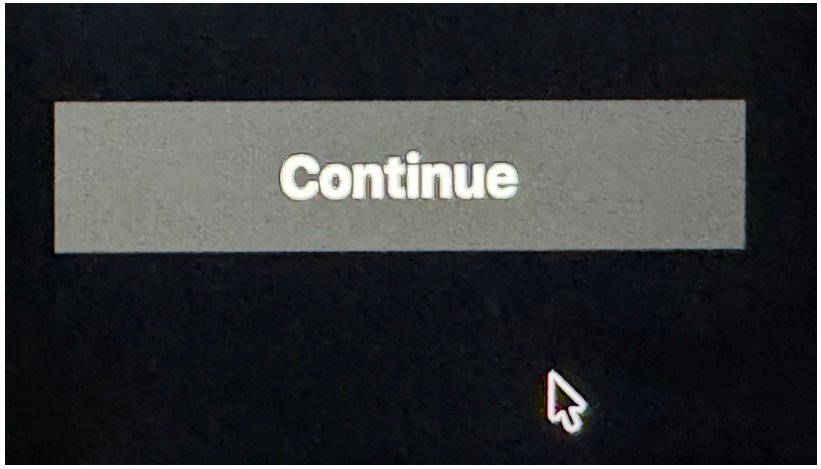
```
cursor="hand2"
```



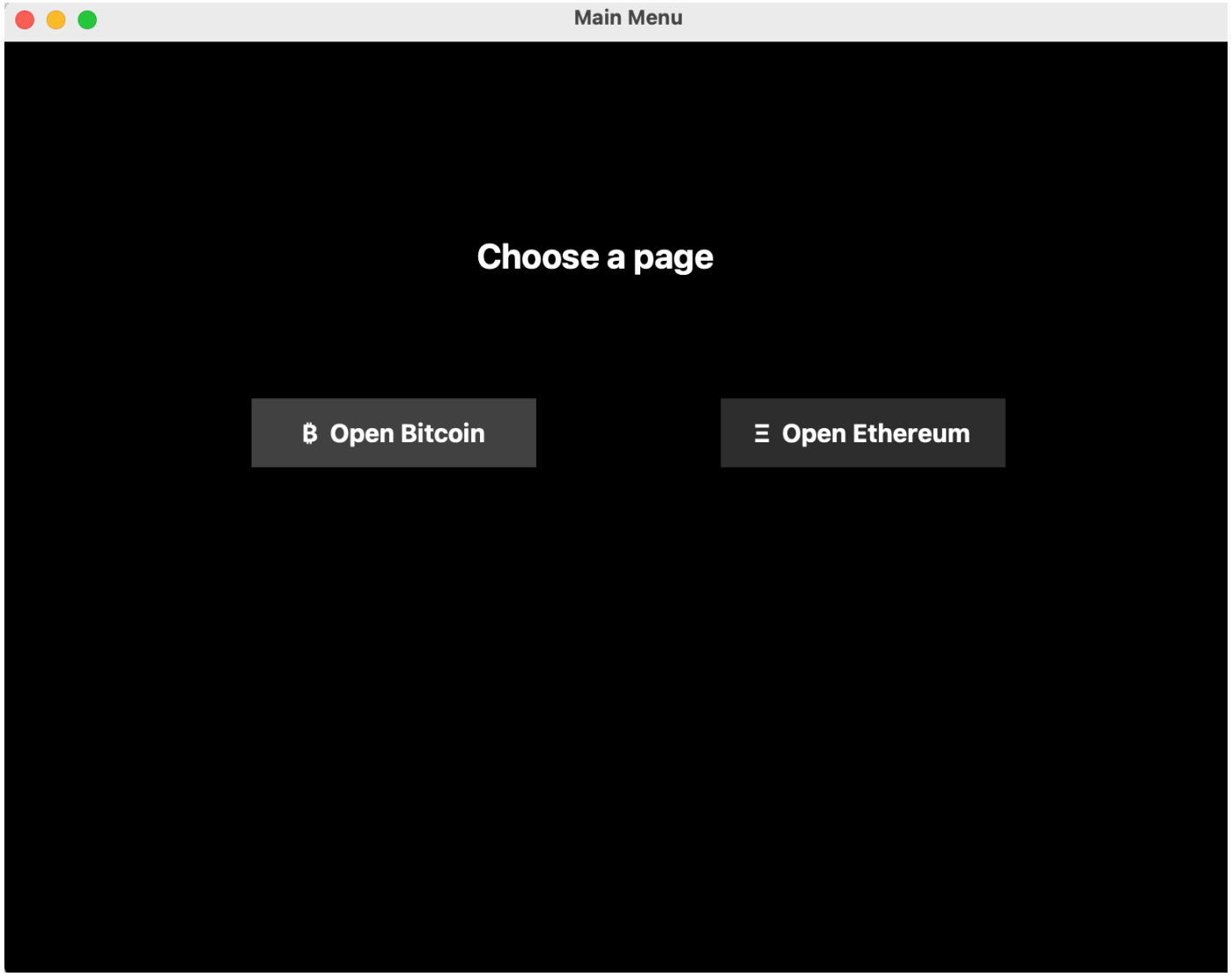
S.S.157: Enhanced user interactivity using **cursor** change to indicate clickable button behaviour.

```
# Hover effects
        self.btn_continue.bind("<Enter>",
            lambda e: self.btn_continue.config(bg="#444444"))
        self.btn_continue.bind("<Leave>",
            lambda e: self.btn_continue.config(bg="#2f2f2f"))
```





S.S.158: Added **hover effects** to improve usability and provide visual feedback on user interaction.



S.S.159: **Same designed aligned for the Main window buttons**

⇒ **Code explanation:**

This section implements a structured welcome interface and navigation system that controls how the user enters the application. A separate welcome window is created to act as the first screen shown to the user, operating independently from the main application window while still being linked to it.

When the program starts, the main application window is hidden so that only the welcome screen is visible. This ensures the user is presented with a single, focused interface rather than multiple overlapping windows.

The welcome screen uses a centrally aligned layout built around a main container frame. This frame holds three key interface elements: a title, a descriptive message, and a “Continue” button. The title introduces the system, while the message explains its purpose, helping the user understand what the application does before interacting with it.

The “Continue” button acts as the primary navigation control. When activated, it closes the welcome window and reveals the main application window, creating a clear transition between the introduction stage and the main functionality of the system.

To ensure correct program behaviour, the system also handles the event where the user closes the welcome window. In this case, the entire application is terminated, preventing any hidden processes from continuing to run.

Additional interactive features are included to improve usability. The button changes appearance when the user hovers over it, and the cursor changes to indicate that the element is clickable. These features provide immediate visual feedback to the user.

⇒ **Justification:**

The implementation of a dedicated welcome screen is justified by the need to simplify user interaction with a complex system. The main application contains multiple features (predictions, indicators, charts), and presenting these immediately would increase cognitive load. The welcome screen provides a controlled entry point, improving usability and user orientation.

The decision to use a dark theme is based on both usability and domain relevance. Financial and trading applications commonly use dark interfaces because they reduce eye strain and improve contrast when analysing numerical data. Using a consistent black background with white text enhances readability and visual clarity.

The use of a centralised layout is intentional, ensuring that key elements (title, message, button) are immediately visible and logically structured. This reduces visual clutter and guides the user's attention toward the primary action.

Implementing the Continue button as a styled Label instead of a default button allows greater customisation of size, font, and colours, resulting in a more professional and visually consistent interface. The larger clickable area also improves accessibility.

The inclusion of hover effects and cursor changes is justified as they provide immediate visual feedback, reinforcing interactivity. Without these features, users may not clearly recognise clickable elements, particularly in a minimal interface design.

The use of withdraw() and deiconify() ensures that only one window is visible at a time, preventing overlapping interfaces and improving overall user experience. This creates a clear transition between stages of interaction.

Finally, implementing window close handling ensures robustness. Without this, closing the welcome window could leave the main application running in the background, leading to inconsistent system behaviour.

⇒ **Iterative development:**

The initial version of the interface opened both the welcome window and the main application window at the same time. This created confusion and reduced clarity, as users were presented with multiple interfaces simultaneously.

To address this, the main window was hidden at startup, ensuring that only the welcome screen was visible. A transition mechanism was then introduced to reveal the main window only after the user presses the Continue button, improving the overall user flow.

The layout also underwent refinement. Early versions used simple positioning, which resulted in inconsistent alignment and spacing. This was improved by introducing a central layout structure, ensuring that all elements remained consistently positioned and visually balanced.

The button design was also iteratively improved. Initially, it lacked clear interactivity cues, making it less obvious that it was clickable. This was resolved by adding hover effects and cursor changes, improving user feedback and interaction clarity.

Additionally, the welcome message was introduced after identifying that users needed clearer guidance about the system's purpose. This enhancement improved accessibility, particularly for first-time users.

These iterations demonstrate improvements across usability, layout consistency, and interaction design, rather than just functional corrections.

⇒ **Validation:**

Validation confirmed that the welcome screen and navigation system function correctly within the overall application workflow. Testing showed that only the welcome window is displayed at startup, preventing interface overlap, and that the transition to the main application occurs smoothly when the Continue button is activated. Additional checks verified that window closing behaviour correctly terminates the program and that interactive features such as hover effects and cursor changes provide clear visual feedback. Overall, the interface behaves as intended and improves usability by guiding the user through a structured and controlled entry into the system.

Test ID	Feature Tested	Test Type	Test Description	Expected Outcome	Actual Outcome	Action Taken	Result	Evidence
N1	Welcome screen display	Normal	Run application	Only welcome window appears	Welcome screen displayed correctly	None required	Pass	S.S.152
N2	Main window visibility	Normal	Check startup behaviour	Main window hidden initially	Main window successfully hidden	Used withdraw()	Pass	S.S.150
N3	Continue button	Normal	Run Application	Clear button represented	Button design aligned with Rest of GUI	None required	Pass	S.S.156
N4	Hover effect	Usability	Move cursor over button	Button colour changes	Colour change observed	Implemented / bindings	Pass	S.S.158
N5	Cursor feedback	Usability	Hover over button	Cursor changes to hand icon	Cursor updates correctly	Added cursor="hand2"	Pass	S.S.157

⇒ **Justification of Actions Taken During Testing:**

In Test N2, the main window was visible on startup instead of being hidden behind the welcome screen. To resolve this, the withdraw() method was used. This approach was chosen to control window visibility during initialisation. This improved the usability of the system by ensuring users first interact with the intended welcome interface.
In Test N4, the interface lacked visual feedback when hovering over buttons. To resolve this, hover bindings were implemented to change the button colour. This approach was chosen to provide interactive feedback. This improved the usability of the system by making controls more intuitive.
In Test N5, there was no cursor feedback when interacting with buttons. To resolve this, the cursor was set to "hand2". This approach was chosen to indicate clickable elements. This improved the usability of the system by enhancing user interaction cues.

⇒ **Link back to analysis & design:**
This section directly supports the analysis requirement for a **user-friendly and intuitive interface**, ensuring that users can interact with the system without confusion. The welcome screen simplifies the initial interaction by clearly guiding the user into the application.
In the design section, a **multi-window architecture** was proposed, separating the welcome interface from the main prediction system. This implementation successfully fulfils that design by creating a controlled navigation flow between windows.
Additionally, the focus on layout, colour scheme, and interaction feedback aligns with the design objective of improving usability. The structured interface and guided navigation ensure that users can easily access the system's core functionality, supporting the overall aim of creating an accessible and effective crypto prediction tool.

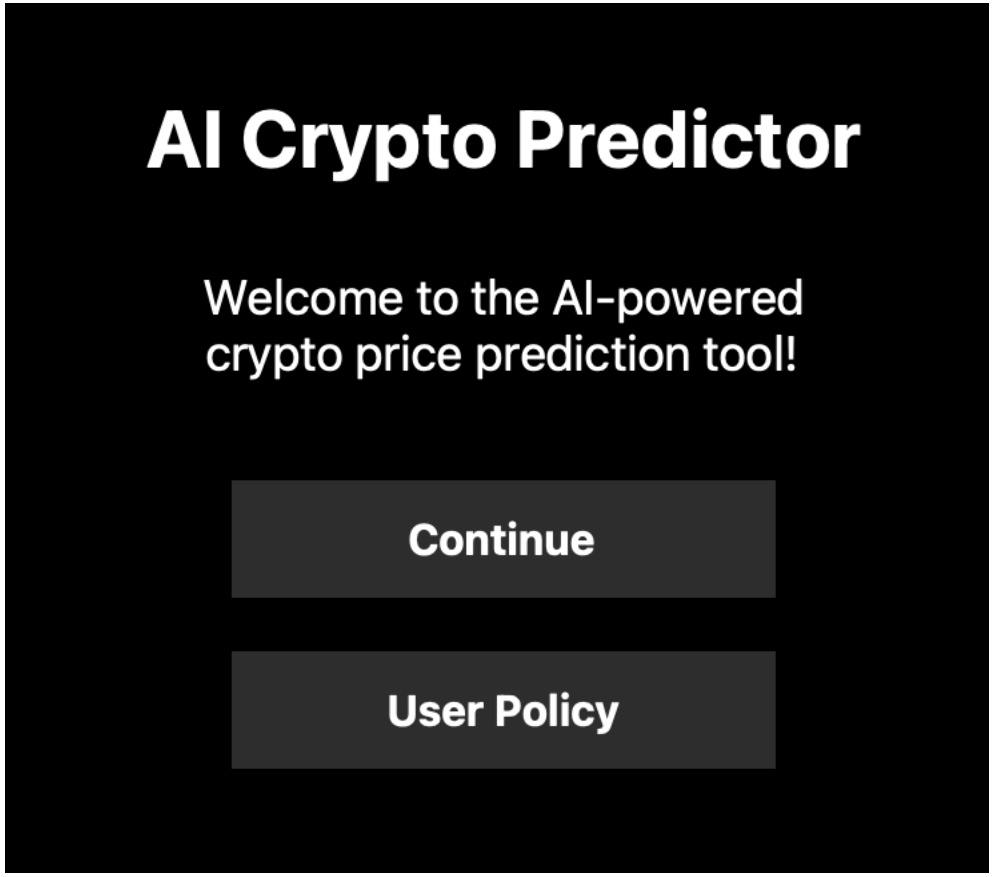
Welcome screen functionality

```
# Click action
        self.btn_continue.bind("<Button-1>",
            lambda e: self.open_main_window())
```

S.S.160: Implemented click event to **transition f**rom welcome screen to main application window.

```
# User policy button
        self.btn_policy = tk.Label(
            self.main_frame,
            text="User Policy",
            font=("Calibri", 16, "bold"),
            bg="#2f2f2f",
            fg="white",
            width=18,
            height=2,
            cursor="hand2"
        )
        self.btn_policy.pack()

# Hover effects
        self.btn_policy.bind("<Enter>",
            lambda e: self.btn_policy.config(bg="#444444"))
        self.btn_policy.bind("<Leave>",
            lambda e: self.btn_policy.config(bg="#2f2f2f"))
```

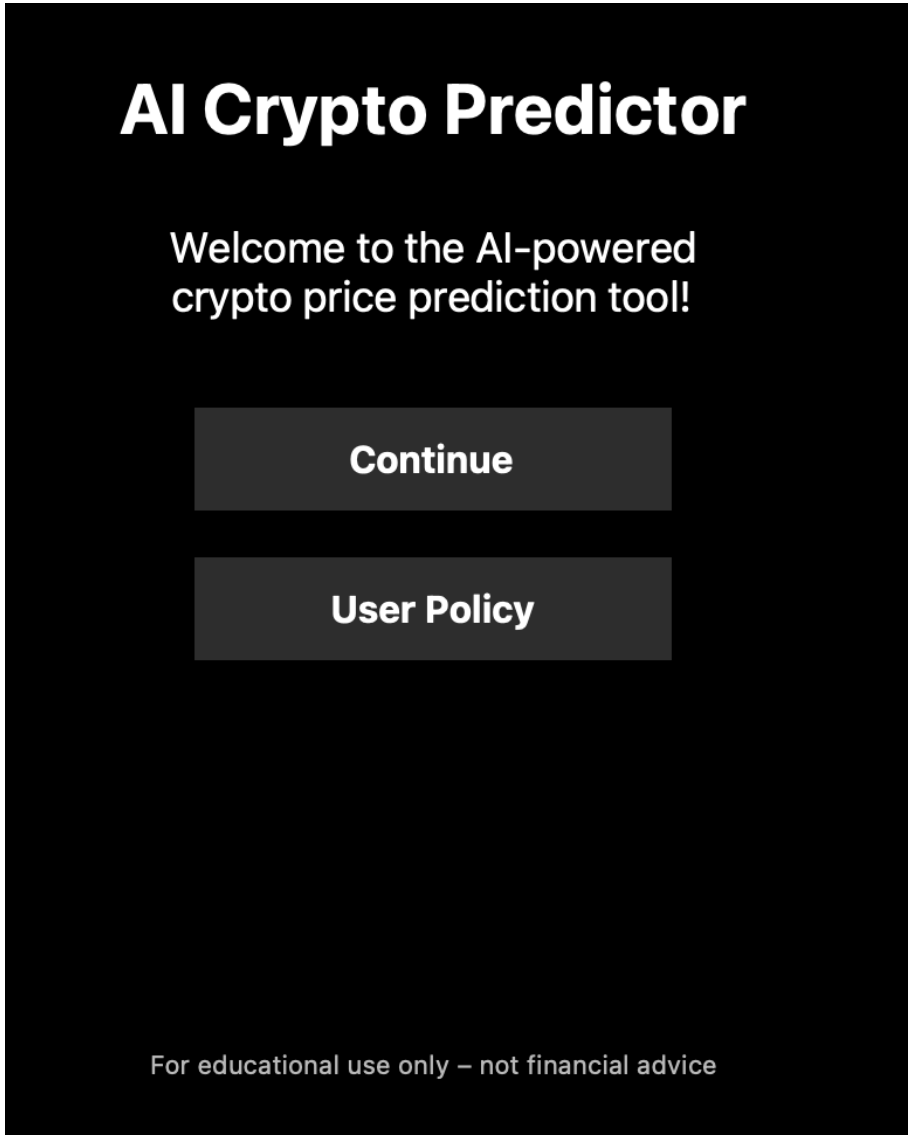
S.S.161: Created **User Policy button** with consistent styling and hover interaction.



S.S.162: Integrated User Policy button into welcome interface for additional user guidance.

```
self.lbl_footer = tk.Label(
    self,
    text="For educational use only – not financial advice",
    font=("Calibri", 11),
    fg="#bbbbbb",
    bg="black"
)
self.lbl_footer.place(relx=0.5, rely=0.93, anchor="center")
```

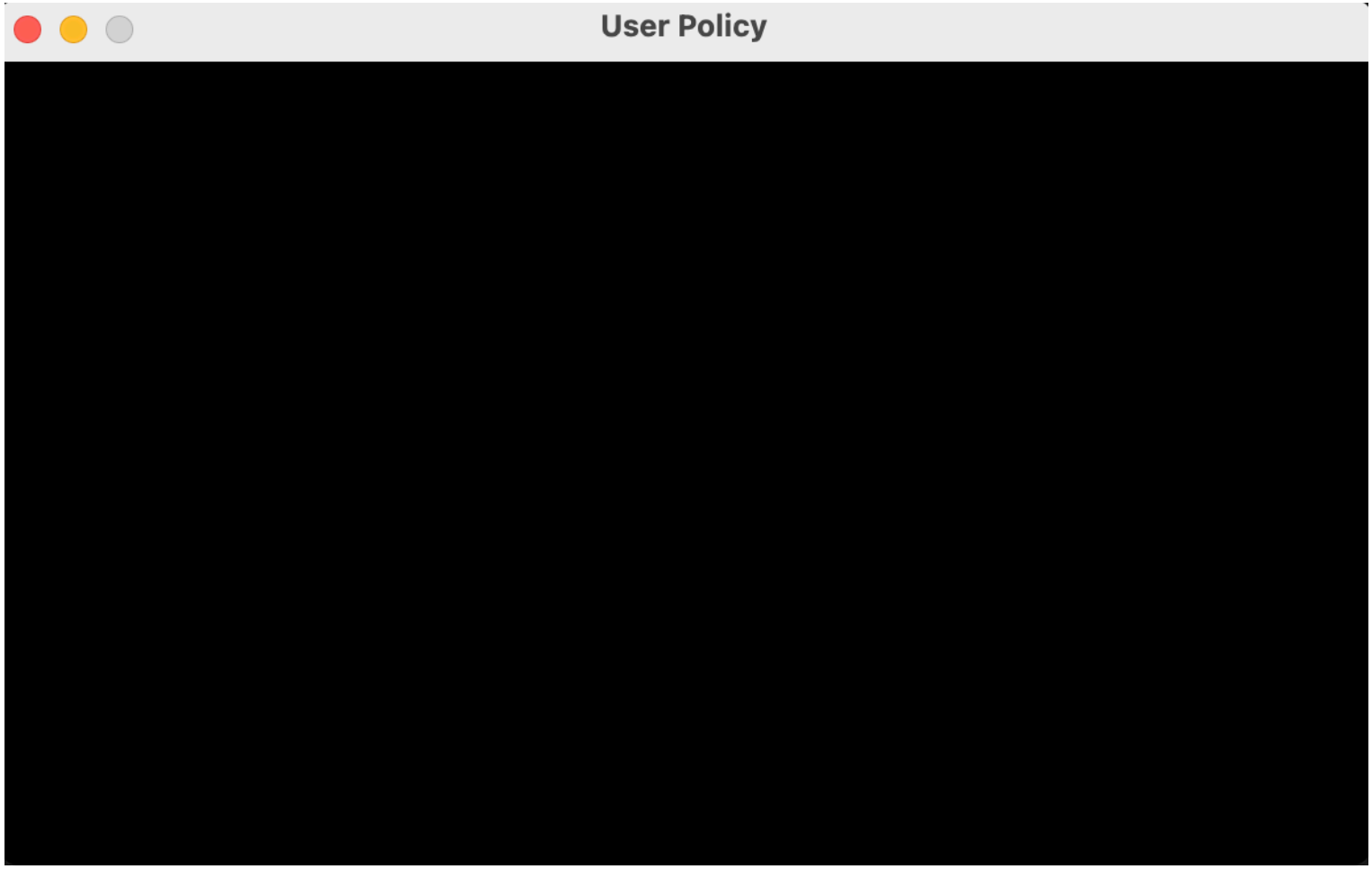
S.S.163: Added **footer** disclaimer to communicate educational purpose and reduce misuse.



S.S.164: Displayed complete welcome interface with policy access and disclaimer.

```
    # Click action
    self.btn_policy.bind("<Button-1>",
        lambda e: self.open_policy_window())


policy_window = tk.Toplevel(self)
policy_window.title("User Policy")
policy_window.geometry("600x350")
policy_window.configure(bg="black")
policy_window.resizable(False, False)
```

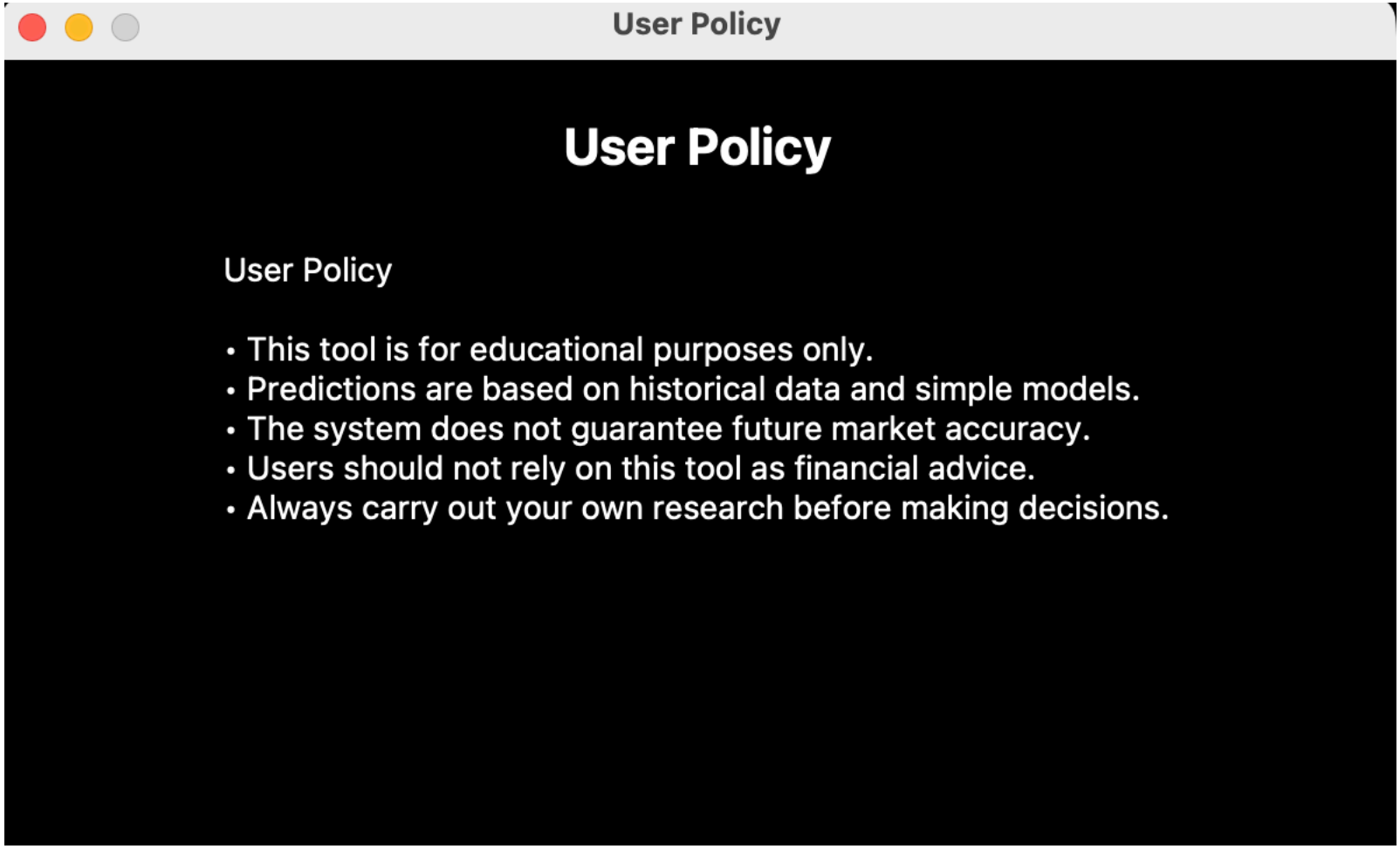
S.S.165: Implemented new **policy window** with fixed layout and separate interface.



S.S.166: **Successfully opened** policy window

```
    policy_text = (
        "User Policy\n\n"
        "• This tool is for educational purposes only.\n"
        "• Predictions are based on historical data and simple models.\
        "• The system does not guarantee future market accuracy.\n"
        "• Users should not rely on this tool as financial advice.\n"
        "• Always carry out your own research before making decisions."
    )

    lbl_policy_text = tk.Label(
        policy_window,
        text=policy_text,
        font=("Calibri", 14),
        fg="white",
        bg="black",
        justify="left",
        wraplength=520
    )
    lbl_policy_text.pack(padx=20, pady=10)
```

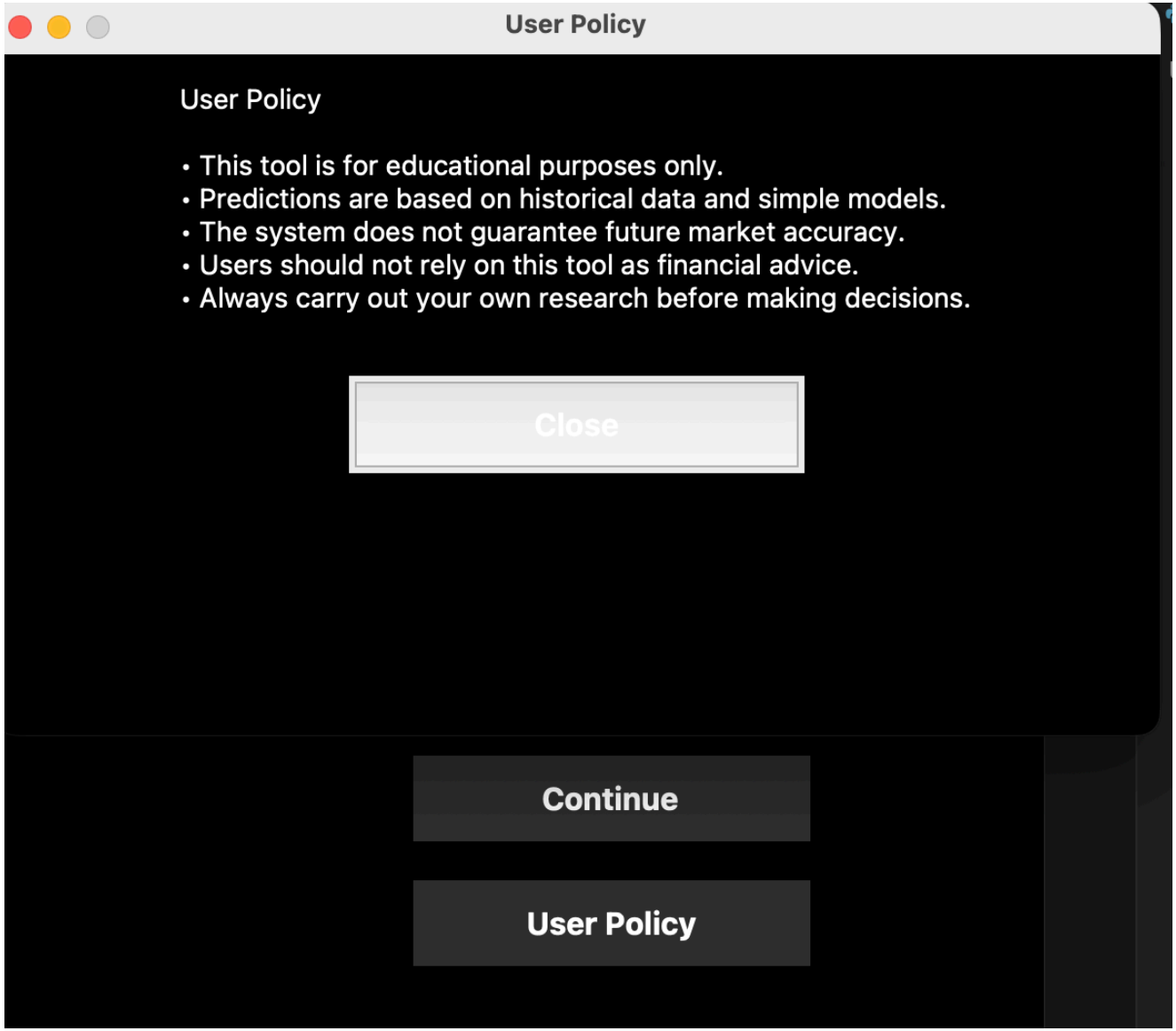
S.S.167: Developed structured policy text content and display formatting.



S.S.168: **Successfully rendered** policy content within dedicated window layout.

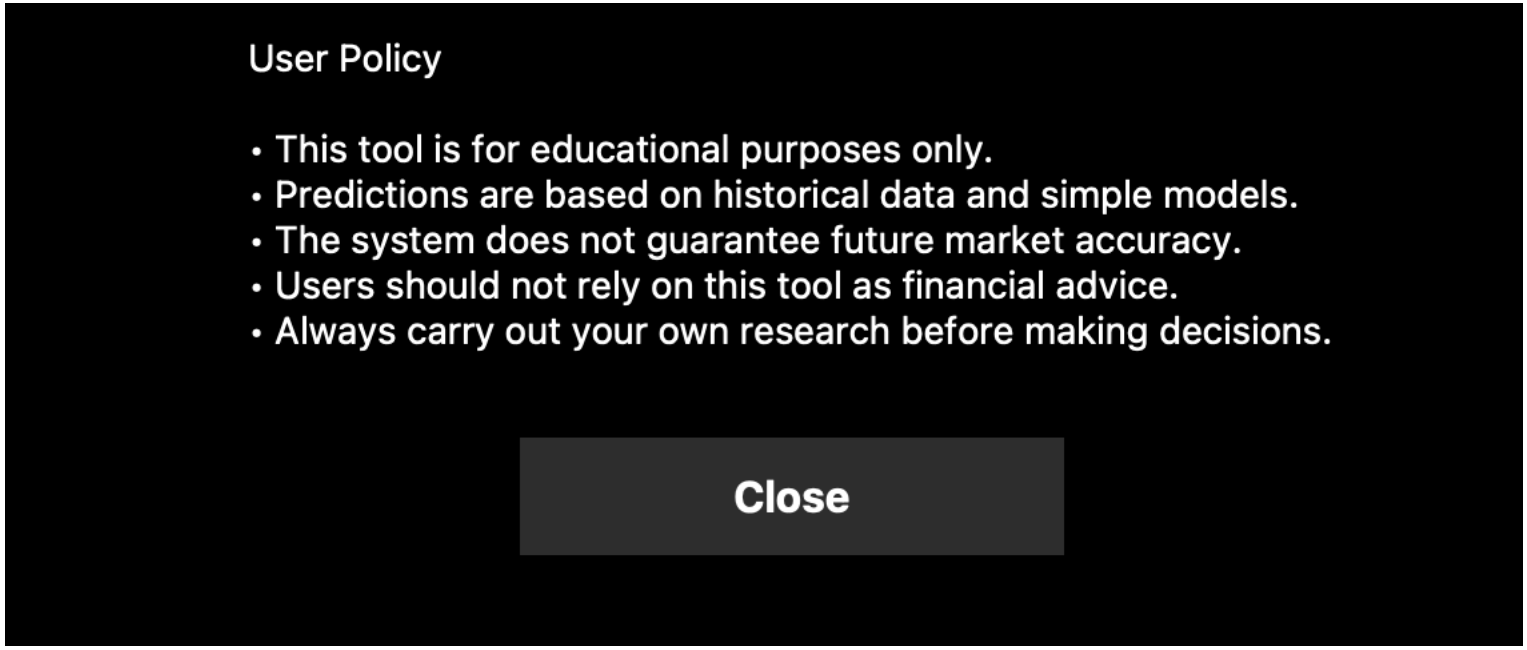
```
btn_close_policy = tk.Button(
    policy_window,
    text="Close",
    font=("Calibri", 16, "bold"),
    bg="#2f2f2f",
    fg="white",
    width=18,
    height=2,
    cursor="hand2"
    command=policy_window.destroy
)
btn_close_policy.pack(pady=20)
```

S.S.169: Added close button functionality after resolving syntax error.



S.S.170: Identified **UI inconsistency** in button styling during close interaction.

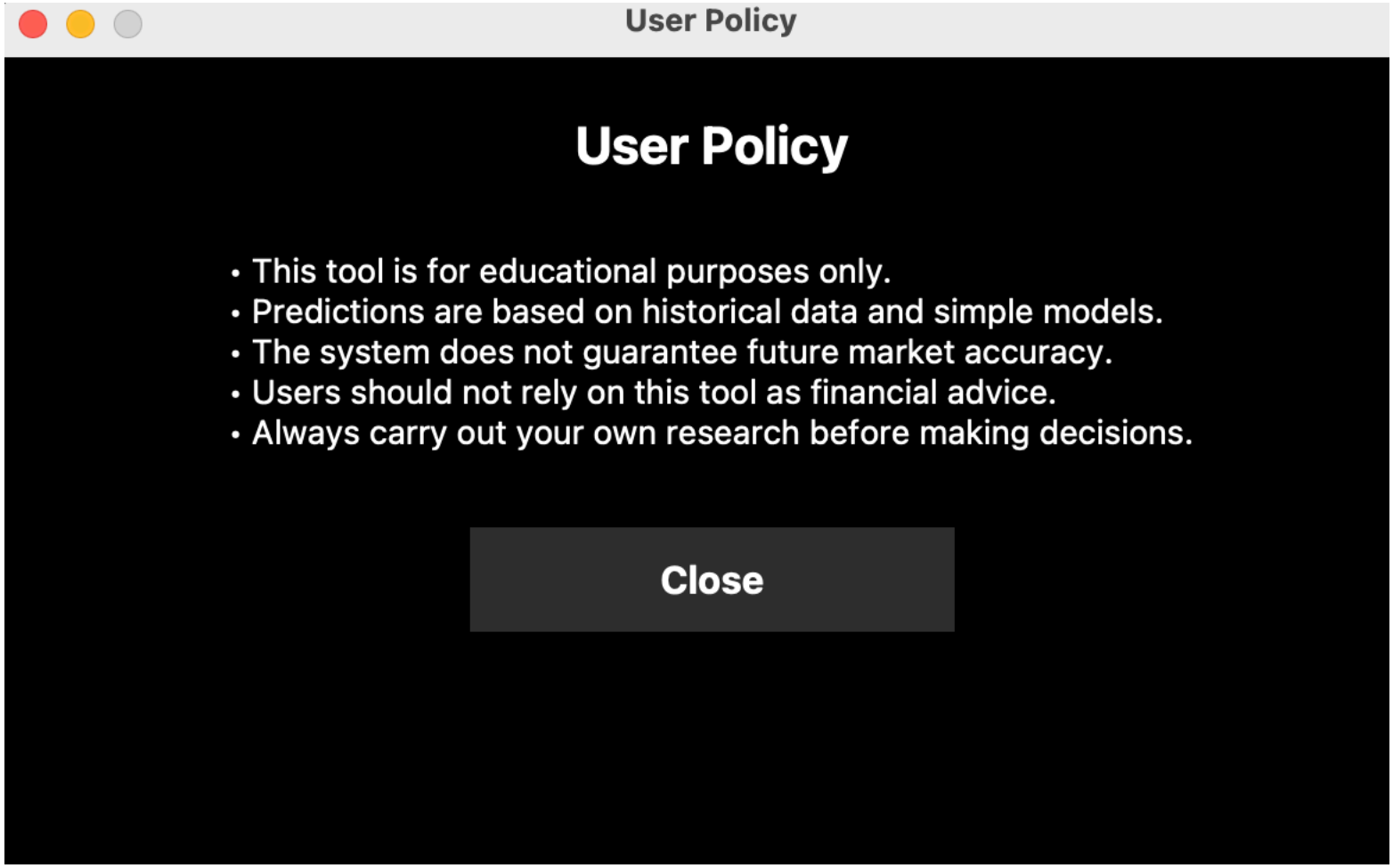
S.S.171: Resolved, macOS button issue by **replacing button widget with label-based interaction**.



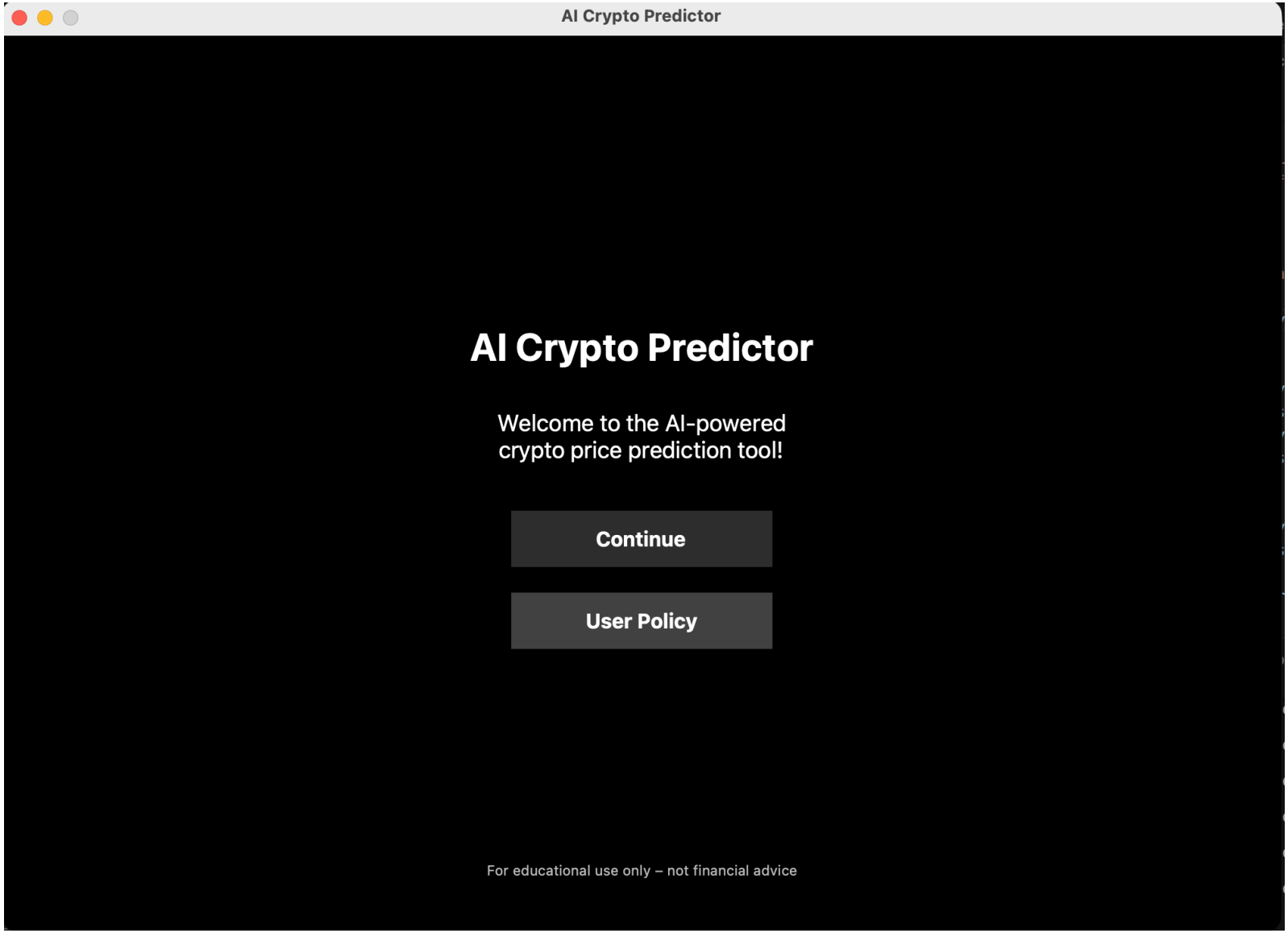
S.S.172: Achieved consistent and functional close button design.

```
    lbl_policy_title = tk.Label(
        policy_window,
        text="User Policy",
        font=("Calibri", 22, "bold"),
        fg="white",
        bg="black"
    )

    policy_text = (
        "• This tool is for educational purposes only.\n"
        "• Predictions are based on historical data and simple models.\n"
        "• The system does not guarantee future market accuracy.\n"
        "• Users should not rely on this tool as financial advice.\n"
        "• Always carry out your own research before making decisions."
    )
```

S.S.173: Refined policy window with improved title hierarchy and formatted text layout.



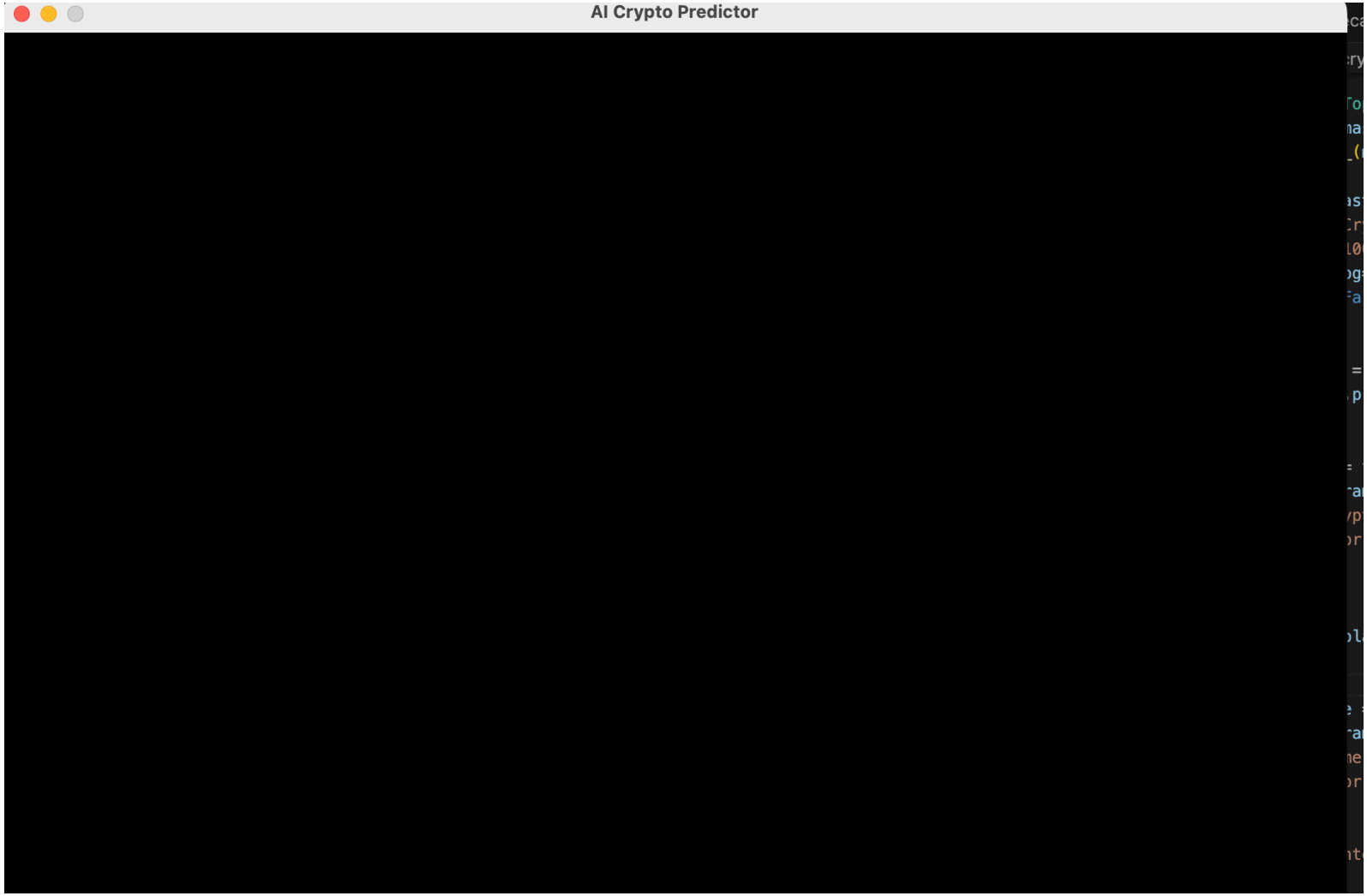
S.S.174: Finalised visually structured policy interface with clear readability.



S.S.175: Integrated updated policy feature into main welcome screen workflow.

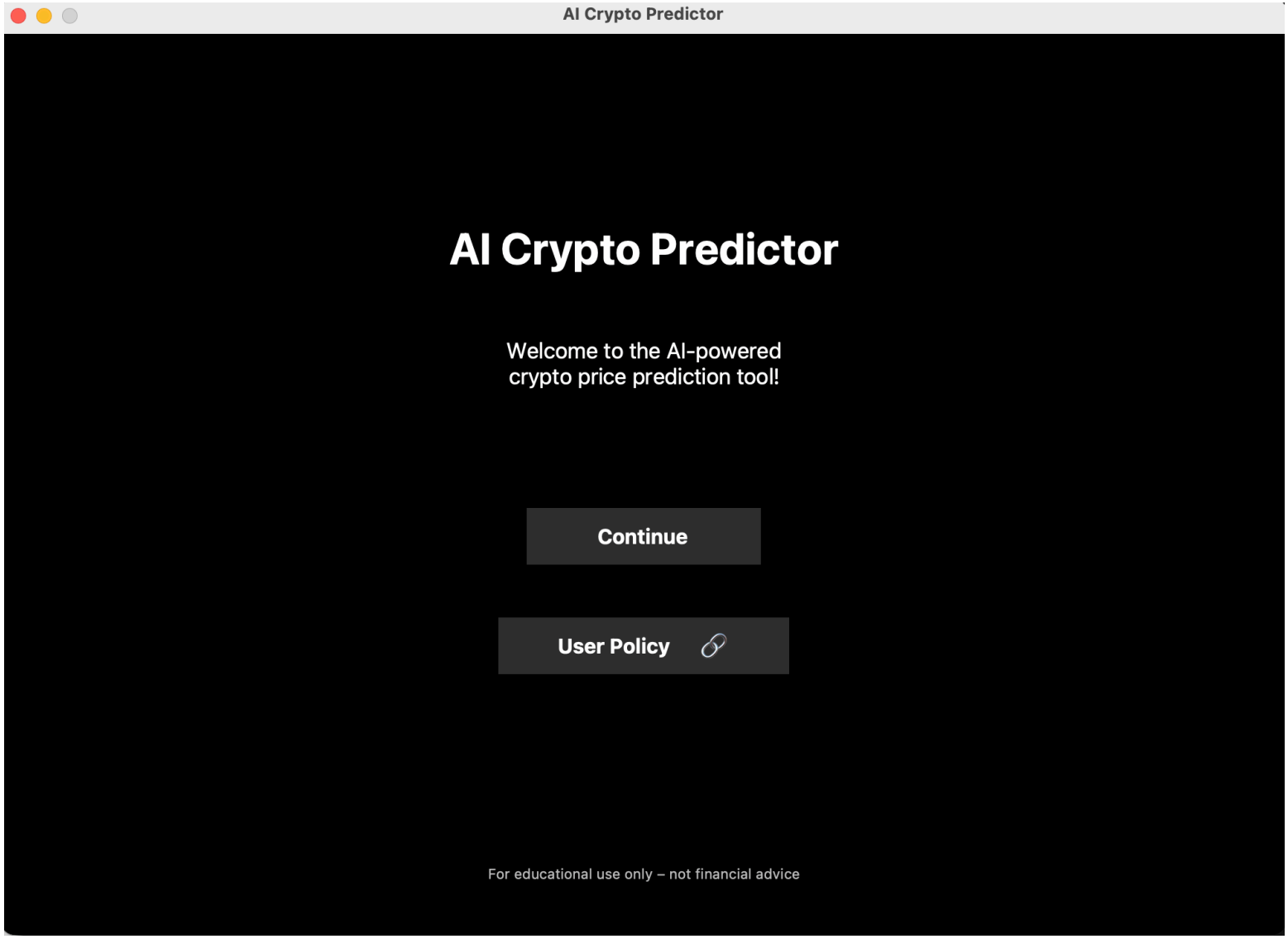
```
        self.master = master
        self.title("AI Crypto Predictor")
        self.geometry("1000x700")
        self.configure(bg="black")
        self.resizable(False, False)

        # Main frame
        self.main_frame = tk.Frame(self, bg="black")
        self.main_frame.place(relx=0.5, rely=0.5, anchor="center")

        # Title
        self.lbl_title = tk.Label(
            self.main_frame,
            text="AI Crypto Predictor",
            font=("Calibri", 34, "bold"),
            fg="white",
            bg="black"
        )
        self.lbl_title.place(x=500, y=165, anchor="center")
```

S.S.176: Attempted repositioning and scaling to better match design specifications.



S.S.177: **Layout failure identified due to incorrect parent reference**.

S.S.178: Resolved layout issue by **correcting widget parent structure**.



S.S.179: **Final refined interface with aligned components and consistent styling.**

⇒ **Code Explanation:**

This section extends the welcome interface by introducing user interaction functionality and an additional informational feature through a “User Policy” system. The implementation allows users to navigate from the welcome screen to the main application by interacting with a clickable element, ensuring a clear entry point into the system.

A secondary interactive component is introduced through the “User Policy” option, which opens a separate window containing important usage guidelines. This window is implemented as an independent interface to avoid disrupting the main workflow while still providing accessible information. The policy content is structured as readable bullet points, improving clarity and user comprehension.

To enhance usability, interactive behaviour is applied to both navigation and policy elements, including click detection and hover feedback. A footer disclaimer is also integrated into the main interface, reinforcing the educational purpose of the system and informing users that predictions should not be treated as financial advice.

Additionally, a close mechanism is implemented within the policy window, allowing users to return seamlessly to the main interface. The layout, spacing, and alignment of all components are refined to ensure a consistent and professional user interface.

⇒ **Justification:**

The inclusion of a clickable transition from the welcome screen to the main system is essential for usability, as it provides a clear and intuitive navigation flow. Without this, users would not have a defined starting interaction, reducing the overall effectiveness of the interface.

The introduction of a “User Policy” feature is justified by the nature of the application. Since the system provides predictive outputs related to cryptocurrency markets, it is important to communicate limitations and ethical considerations. This reduces the risk of users misinterpreting predictions as financial advice and aligns the system with responsible software design practices.
Using a separate window for the policy content ensures that the main interface remains uncluttered while still making the information accessible. This design choice follows good UI principles by separating primary functionality from supplementary information.
Interactive feedback, such as hover effects and cursor changes, is implemented to improve user experience by clearly indicating clickable elements. This reduces ambiguity and makes the interface more intuitive, particularly for users unfamiliar with the system.
The addition of a footer disclaimer further reinforces transparency and supports the system’s intended purpose as an educational tool. Overall, these design decisions prioritise usability, clarity, and ethical responsibility.

⇒ **Iterative Development:**

The initial implementation successfully introduced navigation and the policy feature, but several issues emerged during testing.
The initial implementation successfully opened the policy window, confirming that event binding and window creation were functioning correctly. This provided a stable foundation for adding structured content and further interface refinement.
When adding the close functionality, a syntax error prevented the button from working as expected. This was resolved by correcting the structure of the component, allowing the close action to function correctly.

A more complex issue arose with platform compatibility, particularly on macOS, where the standard button component did not behave consistently. This led to the decision to replace it with a label-based interactive element, which provided more reliable behaviour across systems.
Visual inconsistencies were also identified, including mismatched colours and alignment issues. These were addressed by refining styling properties to maintain consistency with the overall dark theme.
Finally, layout failures occurred when modifying positioning and scaling. These were traced back to incorrect parent-child relationships within the interface structure. Correcting these relationships resolved the issue and ensured all elements were displayed correctly.
These iterations demonstrate problem-solving across functionality, usability, and cross-platform compatibility, resulting in a stable and refined interface.

⇒ **Validation:**
Validation confirmed that both navigation and the user policy feature function correctly within the system. Clicking the continue option successfully transitions to the main application, while selecting the policy option opens a separate window displaying structured information. The close functionality operates reliably, and usability testing verified that interactive elements provide clear feedback. Layout and styling were also validated to ensure consistency with the overall interface design.

Test ID	Feature Tested	Test Type	Description	Expected Outcome	Actual Outcome	Action Taken	Result	Evidence

O1	Continue navigation	Normal	Click continue	Main window opens	Main window opened successfully	Implemented button event binding to transition between windows	Pass	S.S.159
O2	Policy button	Normal	Click policy	Policy window opens	Policy window opened successfully	Created new policy window using Toplevel() and linked button callback	Pass	S.S.161
O3	Policy content	Normal	Display text	Text visible and formatted	Text displayed correctly with formatting	Structured policy text and applied formatting (font, wrap, alignment)	Pass	S.S.166
O4	Close function	Normal	Click close	Window closes	Initial UI inconsistency and behaviour issue observed	Replaced button widget with label-based interaction and corrected binding	Pass	S.S.171
O5	Layout consistency	Normal	Check alignment	Elements centred and styled correctly	Initial layout misalignment due to incorrect parent widget	Corrected widget parent structure and adjusted positioning/layout	Pass	S.S.178

⇒ **Justification of Actions Taken During Testing:**

In Test O1, the system required a transition from the welcome screen to the main application window. To resolve this, a button event binding was implemented. This approach was chosen to ensure controlled navigation between interface states. This improved the functionality of the system by enabling smooth user interaction.

In Test O2, the system required a separate window to display policy information. To resolve this, a new window was created using Toplevel() and linked to the button callback. This approach was

chosen to separate content while maintaining the main interface. This improved the usability of the system by providing structured access to additional information.
In Test O3, the policy text needed to be clearly displayed and readable. To resolve this, the text was structured and formatted using appropriate font, alignment, and wrapping. This approach was chosen to improve readability. This improved the usability of the system by making the content easier to understand.
In Test O4, the close button initially showed UI inconsistency and unreliable behaviour. To resolve this, the button widget was replaced with a label-based interaction and the event binding was corrected. This approach was chosen to ensure consistent behaviour across the interface. This improved the reliability of the system by ensuring the window closes correctly.
In Test O5, layout misalignment occurred due to incorrect widget parent structure. To resolve this, the widget hierarchy was corrected and layout positioning was adjusted. This approach was chosen to ensure proper alignment of interface elements. This improved the usability of the system by producing a consistent and visually balanced layout.

⇒ Link back to analysis & design:
This section directly fulfils the design requirement of creating an intuitive and user-friendly interface with clear navigation and supporting information. The addition of the user policy aligns with the analysis stage, which identified the need for transparency and user guidance when presenting predictive outputs.
The implementation also supports the modular system design, where different components (welcome screen, main system, and policy window) operate independently but remain connected

through user interaction. This enhances scalability and maintainability, allowing further features to be added without disrupting the existing structure.

Overall, this section strengthens both usability and ethical design, ensuring the system meets its intended purpose as a clear, responsible, and user-centred application.

Final Code with annotation to aid maintenance and further development

```
# Import the Tkinter library to build the graphical user interface
import tkinter as tk

# Import PIL modules in case image files are used in the interface
from PIL import Image, ImageTk

# Import webbrowser so chart links can be opened in the user's browser
import webbrowser

# Import requests to collect live and historical market data from Binance API
import requests

# Import datetime to display live update times in the interface
from datetime import datetime

# Import os for working with local file paths if images are used
import os

# Import pandas for table-based data cleaning and feature engineering
import pandas as pd  # type: ignore

# Import Linear Regression as the baseline prediction model
from sklearn.linear_model import LinearRegression  # type: ignore

# Import matplotlib for plotting trend graphs inside the application
import matplotlib.pyplot as plt

# Import TkAgg canvas so matplotlib graphs can be embedded into Tkinter
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
```

```python
# Create a separate Bitcoin page as a child window
class BitcoinWindow(tk.Toplevel):
    def __init__(self, master):
        # Call parent constructor so the Toplevel window is created correctly
        super().__init__(master)

        # Set the title and size of the Bitcoin window
        self.title("Bitcoin")
        self.geometry("1200x800")

        # Display the asset name at the top of the page
        self.lbl_btc = tk.Label(self, text="Bitcoin", font=("Calibri", 16))
        self.lbl_btc.place(x=520, y=120)

        # Add a small chart emoji beside the asset name for visual context
        self.emoji_chart = tk.Label(self, text="📈", font=("Calibri", 16))
        self.emoji_chart.place(x=605, y=120)

        # The original image-based chart area was replaced with a graph frame
        # so live matplotlib graphs can be displayed inside the GUI
        self.graph_frame = tk.Frame(self, width=800, height=450)
        self.graph_frame.place(x=215, y=270)

        # Create a button that opens the live Bitcoin chart in TradingView
        self.btn_link = tk.Button(
            self,
            text="open bitcoin chart 📊",
            font=("Calibri", 16),cursor="hand2",
            command=self.open_link
        )
        self.btn_link.place(x=700, y=120)

        # Create the prediction button that runs the model when clicked
        self.btn_predict = tk.Button(
            self,
            text="Prediction",
            font=("Calibri", 14),cursor="hand2",
            command=self.run_prediction
        )
        self.btn_predict.place(x=215, y=70)
```

```
# Label used to display the predicted next closing price
self.predicted_lbl = tk.Label(self, text="Predicted next close: —", font=("Calibri", 12))
self.predicted_lbl.place(x=215, y=100)

# Label used to display the up / down / flat arrow beside prediction output
self.arrow_lbl = tk.Label(self, text="", font=("Calibri", 16))
self.arrow_lbl.place(x=315, y=73)

# Label used to display RSI value and interpretation
self.rsi_lbl = tk.Label(self, text="RSI: —", font=("Calibri", 12))
self.rsi_lbl.place(x=215, y=160)

# Label used to display trading signal such as BUY / SELL / HOLD
self.signal_lbl = tk.Label(self, text="Signal: —", font=("Calibri", 12, "bold"))
self.signal_lbl.place(x=215, y=180)

# Label used to explain why the system produced the signal
self.reason_lbl = tk.Label(
    self,
    text="Reason: —",
    font=("Calibri", 11),
    wraplength=500,
    justify="left"
)
self.reason_lbl.place(x=215, y=205)

# Label used to display a simple confidence score
self.confidence_lbl = tk.Label(self, text="Confidence: —", font=("Calibri", 12))
self.confidence_lbl.place(x=215, y=235)

# Label used to display the live BTC price
self.live_btc_price = tk.Label(self, text="live price", font=("Calibri", 16))
self.live_btc_price.place(x=215, y=120)

# Label used to display the time of the latest successful update
self.last_update = tk.Label(self, text="live time", font=("Calibri", 12))
self.last_update.place(x=215, y=140)

# Store the previous and current live prices so price movement colour can be shown
self.last_price = None
self.price = None
```

```
    # Start the repeating live price update loop as soon as the window opens
    self.update_btc_price()

# Draw a price / trend graph inside the graph frame
def draw_graph(self, table):
    # Stop graph creation if no processed data table exists
    if table is None:
        raise ValueError("No table data available for graph")

    # Remove any previous graph widgets before drawing a new chart
    for widget in self.graph_frame.winfo_children():
        widget.destroy()

    # Create a matplotlib figure and axis sized to fit the GUI
    fig, ax = plt.subplots(figsize=(8, 4.5), dpi=100)

    # Plot the real closing price line
    ax.plot(table["date"], table["close"], label="Close Price")

    # Plot the short-term moving average line
    ax.plot(table["date"], table["short trend (MA7)"], label="MA7")

    # Plot the medium-term moving average line
    ax.plot(table["date"], table["medium trend (MA30)"], label="MA30")

    # Add chart title and axis labels for readability
    ax.set_title("Bitcoin Price Trend")
    ax.set_xlabel("Date")
    ax.set_ylabel("Price (USD)")

    # Add a legend so each line can be identified
    ax.legend()

    # Format date labels so they do not overlap too heavily
    fig.autofmt_xdate()

    # Embed the matplotlib figure into the Tkinter frame
    canvas = FigureCanvasTkAgg(fig, master=self.graph_frame)
    canvas.draw()
    canvas.get_tk_widget().pack(fill="both", expand=True)
```

```
    # Close the standalone matplotlib figure to reduce memory use
    plt.close(fig)

# Run the full prediction pipeline and update all interface outputs
def run_prediction(self):
    try:
        # Do not run the model unless a live market price is available
        if self.price is None:
            raise ValueError("Live price unavailable")

        # Create the predictor for BTC daily candles using 90 historical entries
        model = BinancePredictor("BTCUSDT", "1d", 90)

        # Run the predictor and store the next predicted close
        prediction = model.run()

        # Draw the processed historical graph after prediction completes
        self.draw_graph(model.table)

        # Read RSI outputs from the model
        rsi_value = model.rsi
        rsi_signal = model.rsi_signal

        # Compare prediction against live market price
        current_price = self.price
        difference = prediction - current_price

        # Generate signal, explanation, and confidence from the model
        signal = model.get_trading_signal(current_price)
        reason = model.get_signal_reason(current_price)
        confidence = model.get_confidence_score(current_price)

        # Choose arrow direction and colour based on whether prediction is above or below live price
        if difference > 0:
            arrow = "⬆️"
            arrow_colour = "green"
        elif difference < 0:
            arrow = "⬇️"
            arrow_colour = "red"
        else:
            arrow = "➡️"
            arrow_colour = "black"
```

```
        # Update the main prediction label
        self.predicted_lbl.config(text=f"Predicted next close: ${prediction:,.2f}")

        # Update the arrow label
        self.arrow_lbl.config(text=arrow, fg=arrow_colour)

        # If RSI could not be calculated, show an unavailable message
        if rsi_value is None:
            self.rsi_lbl.config(text=f"RSI: unavailable - {rsi_signal}")
        else:
            # Otherwise show the numerical RSI and its interpretation
            self.rsi_lbl.config(text=f"RSI: {rsi_value:.2f} - {rsi_signal}")

        # Colour the signal label according to the signal strength / direction
        if signal == "BUY" or signal == "STRONG BUY":
            self.signal_lbl.config(text=f"Signal: {signal}", fg="green")
        elif signal == "SELL" or signal == "STRONG SELL":
            self.signal_lbl.config(text=f"Signal: {signal}", fg="red")
        else:
            self.signal_lbl.config(text="Signal: HOLD", fg="black")

        # Show the reason behind the system's decision
        self.reason_lbl.config(text=f"Reason: {reason}")

        # Show the confidence percentage
        self.confidence_lbl.config(text=f"Confidence: {confidence}%")

    except Exception as e:
        # Print technical error details to the terminal for debugging
        print("Bitcoin prediction error:", e)

        # Display safe fallback messages in the interface if prediction fails
        self.predicted_lbl.config(text="Prediction unavailable")
        self.arrow_lbl.config(text="!", fg="red")
        self.rsi_lbl.config(text="RSI: unavailable")
        self.signal_lbl.config(text="Signal: unavailable", fg="red")
        self.reason_lbl.config(text="Reason: unavailable")
        self.confidence_lbl.config(text="Confidence: unavailable")

# Open the external Bitcoin chart in the browser
def open_link(self):
    webbrowser.open("https://www.tradingview.com/chart/?symbol=BINANCE%3ABTCUSDT")
```

```python
# Repeatedly fetch and display the live BTC price
def update_btc_price(self):
    try:
        # Request the current BTCUSDT market price from Binance
        response = requests.get(
            "https://api.binance.com/api/v3/ticker/price?symbol=BTCUSDT",
            timeout=10
        )

        # Convert the returned text price to float
        self.price = float(response.json()["price"])

        # Default label colour is black unless price direction changes
        colour = "black"

        # Compare current price to previous price to show short-term movement
        if self.last_price is not None:
            if self.price > self.last_price:
                colour = "green"
            elif self.price < self.last_price:
                colour = "red"

        # Update the live price label
        self.live_btc_price.config(
            text="BTC Live Price: $" + "{:,.2f}".format(self.price),
            fg=colour
        )

        # Store current price for the next comparison cycle
        self.last_price = self.price

        # Display the time of the latest successful update
        now = datetime.now().strftime("%H:%M:%S")
        self.last_update.config(text=f"Last update : {now}")

    except Exception:
        # If the live price request fails, display an unavailable message
        self.live_btc_price.config(text="BTC Live Price: unavailable", fg="red")

        # Still show the time the failed attempt occurred
        now = datetime.now().strftime("%H:%M:%S")
        self.last_update.config(text=f"Last attempt : {now}")
```

```
        # Repeat the live price check every 2.5 seconds
        self.after(2500, self.update_btc_price)


# Create a separate Ethereum page as a child window
class EthereumWindow(tk.Toplevel):
    def __init__(self, master):
        # Call parent constructor so the Toplevel window is created correctly
        super().__init__(master)

        # Set the title and size of the Ethereum window
        self.title("Ethereum")
        self.geometry("1200x800")

        # Display the asset name at the top of the page
        self.lbl_eth = tk.Label(self, text="Ethereum", font=("Calibri", 16))
        self.lbl_eth.place(x=520, y=120)

        # Add a chart emoji for visual consistency with the Bitcoin page
        self.emoji_chart = tk.Label(self, text="📈", font=("Calibri", 16))
        self.emoji_chart.place(x=605, y=120)

        # Replace static image area with a frame used to embed matplotlib graphs
        self.graph_frame = tk.Frame(self, width=800, height=450)
        self.graph_frame.place(x=215, y=270)

        # Create a button that opens the external Ethereum chart
        self.btn_link = tk.Button(
            self,
            text="open ethereum chart 📊",
            font=("Calibri", 16),cursor="hand2",
            command=self.open_eth_link
        )
        self.btn_link.place(x=700, y=120)

        # Label used to display the live ETH price
        self.live_eth_price = tk.Label(self, text="live price", font=("Calibri", 16))
        self.live_eth_price.place(x=215, y=120)

        # Label used to display the latest successful update time
        self.last_update = tk.Label(self, text="live time", font=("Calibri", 12))
        self.last_update.place(x=215, y=140)
```

```
# Store the previous and current live ETH prices
self.last_price = None
self.price = None

# Start the repeating live ETH update loop
self.update_eth_price()

# Create the prediction button for Ethereum
self.btn_predict = tk.Button(
    self,
    text="Prediction",
    font=("Calibri", 14),cursor="hand2",
    command=self.run_prediction
)
self.btn_predict.place(x=215, y=70)

# Label used to display the predicted next closing price
self.predicted_lbl = tk.Label(self, text="Predicted next close: —", font=("Calibri", 12))
self.predicted_lbl.place(x=215, y=100)

# Label used to display prediction direction arrow
self.arrow_lbl = tk.Label(self, text="", font=("Calibri", 16))
self.arrow_lbl.place(x=315, y=73)

# Label used to display RSI information
self.rsi_lbl = tk.Label(self, text="RSI: —", font=("Calibri", 12))
self.rsi_lbl.place(x=215, y=160)

# Label used to display final trading signal
self.signal_lbl = tk.Label(self, text="Signal: —", font=("Calibri", 12, "bold"))
self.signal_lbl.place(x=215, y=180)

# Label used to explain the signal
self.reason_lbl = tk.Label(
    self,
    text="Reason: —",
    font=("Calibri", 11),
    wraplength=500,
    justify="left"
)
self.reason_lbl.place(x=215, y=205)
```

```
        # Label used to display confidence percentage
        self.confidence_lbl = tk.Label(self, text="Confidence: –", font=("Calibri", 12))
        self.confidence_lbl.place(x=215, y=235)

    # Draw the Ethereum price / trend graph
    def draw_graph(self, table):
        # Stop graph creation if no processed table exists
        if table is None:
            raise ValueError("No table data available for graph")

        # Clear any old chart widgets before drawing a new graph
        for widget in self.graph_frame.winfo_children():
            widget.destroy()

        # Create figure and axis for the chart
        fig, ax = plt.subplots(figsize=(8, 4.5), dpi=100)

        # Plot closing price and both moving averages
        ax.plot(table["date"], table["close"], label="Close Price")
        ax.plot(table["date"], table["short trend (MA7)"], label="MA7")
        ax.plot(table["date"], table["medium trend (MA30)"], label="MA30")

        # Add chart title and axis labels
        ax.set_title("Ethereum Price Trend")
        ax.set_xlabel("Date")
        ax.set_ylabel("Price (USD)")
        ax.legend()
        fig.autofmt_xdate()

        # Embed the graph into the Tkinter frame
        canvas = FigureCanvasTkAgg(fig, master=self.graph_frame)
        canvas.draw()
        canvas.get_tk_widget().pack(fill="both", expand=True)

        # Close matplotlib figure to reduce memory use
        plt.close(fig)

    # Run the full ETH prediction pipeline and update interface outputs
    def run_prediction(self):
        try:
            # Do not run prediction unless live market price exists
            if self.price is None:
                raise ValueError("Live price unavailable")
```

```
    # Create predictor for ETH daily candles using 90 historical entries
    model = BinancePredictor("ETHUSDT", "1d", 90)

    # Run the model and return the next predicted close
    prediction = model.run()

    # Draw the historical chart from the cleaned dataset
    self.draw_graph(model.table)

    # Read RSI results from the model
    rsi_value = model.rsi
    rsi_signal = model.rsi_signal

    # Compare predicted price to current live price
    current_price = self.price
    difference = prediction - current_price

    # Generate decision outputs from the model
    signal = model.get_trading_signal(current_price)
    reason = model.get_signal_reason(current_price)
    confidence = model.get_confidence_score(current_price)

    # Select arrow direction and colour based on expected price movement
    if difference > 0:
        arrow = "⬆️"
        arrow_colour = "green"
    elif difference < 0:
        arrow = "⬇️"
        arrow_colour = "red"
    else:
        arrow = "➡️"
        arrow_colour = "black"

    # Update prediction label and arrow label
    self.predicted_lbl.config(text=f"Predicted next close: ${prediction:,.2f}")
    self.arrow_lbl.config(text=arrow, fg=arrow_colour)

    # Show RSI value if available, otherwise show unavailable message
    if rsi_value is None:
        self.rsi_lbl.config(text=f"RSI: unavailable - {rsi_signal}")
    else:
        self.rsi_lbl.config(text=f"RSI: {rsi_value:.2f} - {rsi_signal}")
```

```
        # Display signal with colour-coded output
        if signal == "BUY" or signal == "STRONG BUY":
            self.signal_lbl.config(text=f"Signal: {signal}", fg="green")
        elif signal == "SELL" or signal == "STRONG SELL":
            self.signal_lbl.config(text=f"Signal: {signal}", fg="red")
        else:
            self.signal_lbl.config(text="Signal: HOLD", fg="black")

        # Display explanation and confidence score
        self.reason_lbl.config(text=f"Reason: {reason}")
        self.confidence_lbl.config(text=f"Confidence: {confidence}%")

    except Exception as e:
        # Print technical error details for debugging
        print("Ethereum prediction error:", e)

        # Display safe fallback messages if prediction fails
        self.predicted_lbl.config(text="Prediction unavailable")
        self.arrow_lbl.config(text="!", fg="red")
        self.rsi_lbl.config(text="RSI: unavailable")
        self.signal_lbl.config(text="Signal: unavailable", fg="red")
        self.reason_lbl.config(text="Reason: unavailable")
        self.confidence_lbl.config(text="Confidence: unavailable")

# Open the external Ethereum chart in the browser
def open_eth_link(self):
    webbrowser.open("https://www.tradingview.com/chart/?symbol=BINANCE%3AETHUSDT")

# Repeatedly fetch and display the live ETH price
def update_eth_price(self):
    try:
        # Request the current ETHUSDT market price from Binance
        response = requests.get(
            "https://api.binance.com/api/v3/ticker/price?symbol=ETHUSDT",
            timeout=10
        )

        # Convert returned text price to float
        self.price = float(response.json()["price"])

        # Default label colour unless price changes direction
        colour = "black"
```

```
            # Compare current and previous prices to indicate movement
            if self.last_price is not None:
                if self.price > self.last_price:
                    colour = "green"
                elif self.price < self.last_price:
                    colour = "red"

            # Update live ETH price label
            self.live_eth_price.config(
                text="ETH Live Price: $" + "{:,.2f}".format(self.price),
                fg=colour
            )

            # Store current price for next comparison
            self.last_price = self.price

            # Update time label with successful refresh time
            now = datetime.now().strftime("%H:%M:%S")
            self.last_update.config(text=f"Last update : {now}")

        except Exception:
            # Show unavailable output if API request fails
            self.live_eth_price.config(text="ETH Live Price: unavailable", fg="red")

            # Show time of failed attempt
            now = datetime.now().strftime("%H:%M:%S")
            self.last_update.config(text=f"Last attempt : {now}")

        # Repeat the live ETH update every 2.5 seconds
        self.after(2500, self.update_eth_price)


# Create the main menu window of the application
class MainWindow(tk.Tk):
    def __init__(self):
        # Call parent constructor so the root window is created correctly
        super().__init__()

        # Hide the main menu until the welcome screen has been closed
        self.withdraw()
```

```
# Configure the main menu window appearance
self.title("Main Menu")
self.geometry("800x600")
self.configure(bg="black")

# Add the main menu instruction text
self.text = tk.Label(
    self,
    text="Choose a page",
    font=("Calibri", 22, "bold"),
    fg="white",
    bg="black"
)
self.text.place(x=300, y=120)

# Create a button that opens the Bitcoin page
self.open_ethereum = tk.Label(
    self,
    text="Ξ  Open Ethereum",
    font=("Calibri", 16, "bold"),
    bg="#2f2f2f",
    fg="white",
    width=16,
    height=2,
    cursor="hand2"
)
self.open_ethereum.place(x=550, y=250, anchor="center")

self.open_ethereum.bind(
    "<Enter>",
    lambda e: self.open_ethereum.config(bg="#444444")
)
self.open_ethereum.bind(
    "<Leave>",
    lambda e: self.open_ethereum.config(bg="#2f2f2f")
)
self.open_ethereum.bind(
    "<Button-1>",
    lambda e: self.open_eth()
)
```

```
    # Create a button that opens the Ethereum page
    self.open_bitcoin = tk.Label(
        self,
        text="₿  Open Bitcoin",
        font=("Calibri", 16, "bold"),
        bg="#2f2f2f",
        fg="white",
        width=16,
        height=2,
        cursor="hand2"
    )
    self.open_bitcoin.place(x=250, y=250, anchor="center")

    self.open_bitcoin.bind(
        "<Enter>",
        lambda e: self.open_bitcoin.config(bg="#444444")
    )
    self.open_bitcoin.bind(
        "<Leave>",
        lambda e: self.open_bitcoin.config(bg="#2f2f2f")
    )
    self.open_bitcoin.bind(
        "<Button-1>",
        lambda e: self.open_btc()
    )

    # Store references to child windows so duplicate windows are not created
    self.btc_OpenOrNot = None
    self.eth_OpenOrNot = None

    # Open the welcome screen immediately when the program starts
    self.welcome_window = WelcomeWindow(self)

# Open Bitcoin page or bring existing Bitcoin page to the front
def open_btc(self):
    # Create a new Bitcoin window only if one is not already open
    if self.btc_OpenOrNot is None or not self.btc_OpenOrNot.winfo_exists():
        self.btc_OpenOrNot = BitcoinWindow(self)

        # Override close button behaviour so the stored reference is reset
        self.btc_OpenOrNot.protocol("WM_DELETE_WINDOW", self.close_btc)
    else:
        # If already open, bring it to the front instead of opening another copy
        self.btc_OpenOrNot.lift()
```

```
    # Close Bitcoin page and reset stored reference
    def close_btc(self):
        self.btc_OpenOrNot.destroy()
        self.btc_OpenOrNot = None

    # Open Ethereum page or bring existing Ethereum page to the front
    def open_eth(self):
        # Create a new Ethereum window only if one is not already open
        if self.eth_OpenOrNot is None or not self.eth_OpenOrNot.winfo_exists():
            self.eth_OpenOrNot = EthereumWindow(self)

            # Override close button behaviour so the stored reference is reset
            self.eth_OpenOrNot.protocol("WM_DELETE_WINDOW", self.close_eth)
        else:
            # If already open, bring it to the front instead of opening another copy
            self.eth_OpenOrNot.lift()

    # Close Ethereum page and reset stored reference
    def close_eth(self):
        self.eth_OpenOrNot.destroy()
        self.eth_OpenOrNot = None


# Create the first screen shown to the user before the main menu appears
class WelcomeWindow(tk.Toplevel):
    def __init__(self, master):
        # Call parent constructor so the Toplevel window is created correctly
        super().__init__(master)

        # Store reference to the hidden main root window
        self.master = master

        # Configure the welcome window appearance
        self.title("AI Crypto Predictor")
        self.geometry("1000x700")
        self.configure(bg="black")
        self.resizable(False, False)

        # Create a central frame to help organise the welcome screen layout
        self.main_frame = tk.Frame(self, bg="black")
        self.main_frame.place(relx=0.5, rely=0.5, anchor="center")
```

```python
# Add the large system title
self.lbl_title = tk.Label(
    self,
    text="AI Crypto Predictor",
    font=("Calibri", 34, "bold"),
    fg="white",
    bg="black"
)
self.lbl_title.place(x=500, y=165, anchor="center")

# Add a short message explaining the purpose of the tool
self.lbl_message = tk.Label(
    self,
    text="Welcome to the AI-powered\ncrypto price prediction tool!",
    font=("Calibri", 17),
    fg="white",
    bg="black",
    justify="center"
)
self.lbl_message.place(x=500, y=255, anchor="center")

# If the welcome screen is closed manually, close the full application too
self.protocol("WM_DELETE_WINDOW", self.close_program)

# Create the Continue label styled to behave like a button
self.btn_continue = tk.Label(
    self,
    text="Continue",
    font=("Calibri", 16, "bold"),
    bg="#2f2f2f",
    fg="white",
    width=16,
    height=2,
    cursor="hand2"
)
self.btn_continue.place(x=500, y=390, anchor="center")

# Add hover effect so the Continue label darkens when the mouse enters
self.btn_continue.bind(
    "<Enter>",
    lambda e: self.btn_continue.config(bg="#444444")
)
```

(parameter) self: Self@WelcomeWindow

```
# Return Continue label to normal colour when the mouse leaves
self.btn_continue.bind(
    "<Leave>",
    lambda e: self.btn_continue.config(bg="#2f2f2f")
)

# Bind mouse click so Continue opens the main menu
self.btn_continue.bind(
    "<Button-1>",
    lambda e: self.open_main_window()
)

# Create the User Policy label styled as a second interactive button
self.btn_policy = tk.Label(
    self,
    text="User Policy      🔗",
    font=("Calibri", 16, "bold"),
    bg="#2f2f2f",
    fg="white",
    width=20,
    height=2,
    cursor="hand2"
)
self.btn_policy.place(x=500, y=475, anchor="center")

# Add hover effect to the User Policy label
self.btn_policy.bind(
    "<Enter>",
    lambda e: self.btn_policy.config(bg="#444444")
)

# Return User Policy label to normal colour when the mouse leaves
self.btn_policy.bind(
    "<Leave>",
    lambda e: self.btn_policy.config(bg="#2f2f2f")
)

# Bind mouse click so User Policy opens a separate policy window
self.btn_policy.bind(
    "<Button-1>",
    lambda e: self.open_policy_window()
)
```

```
    # Add a footer disclaimer to remind users the tool is educational only
    self.lbl_footer = tk.Label(
        self,
        text="For educational use only — not financial advice",
        font=("Calibri", 11),
        fg="#bbbbbb",
        bg="black"
    )
    self.lbl_footer.place(relx=0.5, rely=0.93, anchor="center")

# Close the welcome window and show the main menu
def open_main_window(self):
    # Release any modal grab if present
    self.grab_release()

    # Destroy the welcome screen
    self.destroy()

    # Reveal the hidden main window
    self.master.deiconify()

# Open a separate policy window explaining the limitations of the tool
def open_policy_window(self):
    # Create a child window connected to the welcome screen
    policy_window = tk.Toplevel(self)

    # Configure policy window appearance
    policy_window.title("User Policy")
    policy_window.geometry("600x350")
    policy_window.configure(bg="black")
    policy_window.resizable(False, False)

    # Add policy window title
    lbl_policy_title = tk.Label(
        policy_window,
        text="User Policy",
        font=("Calibri", 22, "bold"),
        fg="white",
        bg="black"
    )
    lbl_policy_title.pack(pady=(20, 15))
```

```
# Create the policy guidance text displayed to the user
policy_text = (
    "• This tool is for educational purposes only.\n"
    "• Predictions are based on historical data and simple models.\n"
    "• The system does not guarantee future market accuracy.\n"
    "• Users should not rely on this tool as financial advice.\n"
    "• Always carry out your own research before making decisions."
)

# Display the policy text in a readable wrapped label
lbl_policy_text = tk.Label(
    policy_window,
    text=policy_text,
    font=("Calibri", 14),
    fg="white",
    bg="black",
    justify="left",
    wraplength=520
)
lbl_policy_text.pack(padx=20, pady=10)

# Create a close label styled like a button so the policy window can be dismissed
btn_close_policy = tk.Label(
    policy_window,
    text="Close",
    font=("Calibri", 16, "bold"),
    bg="#2f2f2f",
    fg="white",
    width=18,
    height=2,
    cursor="hand2",
)
btn_close_policy.pack(pady=20)

# Bind click action so the policy window closes
btn_close_policy.bind(
    "<Button-1>",
    lambda e: policy_window.destroy()
)
```

```
        # Add hover effect to the close control
        btn_close_policy.bind(
            "<Enter>",
            lambda e: btn_close_policy.config(bg="#444444")
        )
        btn_close_policy.bind(
            "<Leave>",
            lambda e: btn_close_policy.config(bg="#2f2f2f")
        )

    # Close the entire application if the welcome screen is closed
    def close_program(self):
        self.master.destroy()


# Create the prediction engine used by both BTC and ETH pages
class BinancePredictor:
    def __init__(self, symbol="BTCUSDT", interval="1d", limit=90):
        # Store API request settings
        self.symbol = symbol
        self.interval = interval
        self.limit = limit

        # Binance endpoint used to collect candle data
        self.url = "https://api.binance.com/api/v3/klines"

        # These attributes are filled later during processing
        self.table = None
        self.model = None
        self.prediction = None
        self.rsi = None
        self.rsi_signal = None

    # Calculate the latest RSI value from the cleaned historical table
    def calculate_rsi(self, period=14):
        # Do not attempt RSI unless enough data exists
        if self.table is None or len(self.table) < period + 1:
            return None

        # Calculate one-step price changes between rows
        delta = self.table["close"].diff()
```

```
    # Keep positive changes as gains and negative changes as losses
    gain = delta.where(delta > 0, 0)
    loss = -delta.where(delta < 0, 0)

    # Compute rolling average gain and rolling average loss
    avg_gain = gain.rolling(window=period).mean()
    avg_loss = loss.rolling(window=period).mean()

    # If latest average loss is missing, RSI cannot be calculated yet
    if pd.isna(avg_loss.iloc[-1]):
        return None

    # If loss is zero, RSI is maximised at 100
    if avg_loss.iloc[-1] == 0:
        return 100.0

    # Compute relative strength and RSI
    rs = avg_gain / avg_loss
    rsi = 100 - (100 / (1 + rs))

    # If latest RSI is missing, return None safely
    if pd.isna(rsi.iloc[-1]):
        return None

    # Return the latest RSI as a float
    return float(rsi.iloc[-1])

# Convert numerical RSI into an interpretable message
def interpret_rsi(self):
    # If no RSI exists, report that clearly
    if self.rsi is None:
        return "No RSI data"
    elif self.rsi < 30:
        return "Oversold (Buy Signal)"
    elif self.rsi > 70:
        return "Overbought (Sell Signal)"
    else:
        return "Neutral (Hold)"

# Combine predicted price and RSI into a final signal
def get_trading_signal(self, current_price):
    # If prediction failed, default to HOLD
    if self.prediction is None:
        return "HOLD"
```

```
    # Fallback logic if RSI is unavailable
    if self.rsi is None:
        if self.prediction > current_price:
            return "BUY"
        elif self.prediction < current_price:
            return "SELL"
        else:
            return "HOLD"

    # Stronger / weaker signal logic when prediction is above live price
    if self.prediction > current_price:
        if self.rsi < 30:
            return "STRONG BUY"
        elif self.rsi <= 70:
            return "BUY"
        else:
            return "HOLD"

    # Stronger / weaker signal logic when prediction is below live price
    elif self.prediction < current_price:
        if self.rsi > 70:
            return "STRONG SELL"
        elif self.rsi >= 30:
            return "SELL"
        else:
            return "HOLD"

    # If prediction equals live price, do not act
    return "HOLD"

# Explain in words why a signal was produced
def get_signal_reason(self, current_price):
    # If prediction failed, return an unavailable message
    if self.prediction is None:
        return "No prediction available"

    # Simpler explanation if RSI is unavailable
    if self.rsi is None:
        if self.prediction > current_price:
            return "Predicted price is above current price"
        elif self.prediction < current_price:
            return "Predicted price is below current price"
        return "Predicted price equals current price"
```

```
    # Provide combined price + RSI explanation
    if self.prediction > current_price and self.rsi < 30:
        return "Predicted price is above current price and RSI suggests the asset may be oversold"
    elif self.prediction > current_price and self.rsi <= 70:
        return "Predicted price is above current price and RSI remains within a normal range"
    elif self.prediction < current_price and self.rsi > 70:
        return "Predicted price is below current price and RSI suggests the asset may be overbought"
    elif self.prediction < current_price and self.rsi >= 30:
        return "Predicted price is below current price and RSI remains within a normal range"
    else:
        return "Prediction and RSI together suggest holding position"

# Convert signal strength into a simple confidence score
def get_confidence_score(self, current_price):
    # First get the signal for the current market situation
    signal = self.get_trading_signal(current_price)

    # Strong signals receive the highest confidence
    if signal == "STRONG BUY" or signal == "STRONG SELL":
        return 80
    elif signal == "BUY" or signal == "SELL":
        return 65
    else:
        return 50

# Run the full prediction engine from API fetch to next-close forecast
def run(self):
    # Prepare Binance API request parameters
    params = {"symbol": self.symbol, "interval": self.interval, "limit": self.limit}

    # Request historical candle data from Binance
    response = requests.get(self.url, params=params, timeout=10)
    raw_data = response.json()

    # Validate that the returned data is a non-empty list
    if not isinstance(raw_data, list) or len(raw_data) == 0:
        raise ValueError("No historical data returned")

    # Define the raw Binance kline column names
    columns = [
        "Open time", "Open price", "High price", "Low price", "Close price",
        "Volume (Base)", "Close time", "Quote asset volume", "Number of trades",
        "Taker buy base asset volume", "Taker buy quote asset volume", "Ignore (always 0)"
    ]
```

```
# Convert raw JSON list into a DataFrame
table = pd.DataFrame(raw_data, columns=columns)

# Create cleaned columns needed for modelling
table["date"] = pd.to_datetime(table["Open time"], unit="ms")
table["close"] = table["Close price"].astype(float)
table["volume"] = table["Volume (Base)"].astype(float)

# Keep only the required fields
table = table[["date", "close", "volume"]]

# Create short-term and medium-term moving average features
table["short trend (MA7)"] = table["close"].rolling(7).mean()
table["medium trend (MA30)"] = table["close"].rolling(30).mean()

# Remove rows with missing moving-average values and reset index
table = table.dropna().reset_index(drop=True)

# Stop if the cleaned dataset is too small for next-step prediction
if len(table) < 2:
    raise ValueError("Not enough cleaned historical data for prediction")

# Store cleaned table for later graphing and RSI calculation
self.table = table

# Calculate RSI and convert it into a readable interpretation
self.rsi = self.calculate_rsi()
self.rsi_signal = self.interpret_rsi()

# Build feature matrix from current row values except the final row
array_all = table[["close", "short trend (MA7)", "medium trend (MA30)", "volume"]].iloc[:-1].values

# Build target array from next-day close values
array_close = table["close"].shift(-1).dropna().values

# Train the Linear Regression model
self.model = LinearRegression().fit(array_all, array_close)

# Select the most recent feature row for next-step prediction
array_all_2 = table[["close", "short trend (MA7)", "medium trend (MA30)", "volume"]].iloc[[-1]].values

# Generate and store the final prediction
self.prediction = float(self.model.predict(array_all_2)[0])
```

```
        # Return the predicted next close
        return self.prediction

# Create the main application window and start the Tkinter event loop
MainWindow().mainloop()
```

Testing to inform Evaluation

User Interview: Usability Testing

Table 15: Interviewee

Interviewee	Description
User A – Kyle (Finance Student)	Has basic investment knowledge, familiar with trading dashboards.
User B – Lily (Casual User)	No trading experience; represents everyday users.
User C – Daniel (Computer Science Student)	Technical background, interested in AI and data visualisation; tested functionality and design logic.

Usability Responses

Table 16: Responses

Question	User A – Finance Student	User B – Casual User	User C – Computer Science Student
1. How easy was it to navigate through the program's windows and features?	"Very easy — the layout is clear and consistent. Buttons like *Open Bitcoin* and *Open Ethereum* are obvious and responsive."	"I didn't need instructions — everything was labelled well. The dark background made it comfortable to read."	"Navigation makes sense. The layout follows a clear flow between main and sub-windows, and I didn't have to search for features."
2. Was the information on the screen clear and easy to understand?	"Yes, the prices and predictions are displayed clearly. The colour-coded feedback helps a lot."	"The 'BUY' and 'SELL' arrows are very helpful because I wouldn't understand technical terms like RSI."	"Really clear. The labels and spacing make the data readable, and the live update time stamp is a nice usability detail."

3. How effective do you think the visual feedback (colours, arrows, and charts) is for showing market trends?	"Really good. I can immediately see if Bitcoin is up or down. It feels like a professional dashboard."	"I understood it quickly — red means down, green means up. It's like traffic lights!"	"The visual feedback is spot-on. The colour logic is intuitive, and the arrows make it easy to confirm direction without reading text."
4. Were there any issues with speed, accuracy, or stability while using the program?	"The program updated fast and didn't freeze. The prediction was reasonable compared to the actual price."	"No issues — it loaded smoothly, even when switching between currencies."	"It's efficient. The API calls respond quickly, and the program handled refreshes without lag or crashes."
5. Do you have any suggestions to improve the usability or user experience?	"Maybe add more currencies like ADA or XRP to expand functionality."	"A short explanation or tooltip about what 'prediction' means would help beginners."	"I'd like to see more detail in the prediction output — maybe a small graph showing confidence over time — but overall, it's very clean."

Usability Evaluation Summary from User Interviews

Feedback from all three users confirms that the system demonstrates a high level of usability across different user types, supporting the results identified in the usability testing criteria (6.1–6.4).

The **clean layout and navigation structure (6.1)** were consistently praised by all participants. Users reported that the interface was easy to understand without instructions, indicating that the design successfully reduces cognitive load and supports intuitive interaction. This validates the earlier evaluation that the system is accessible to users with both technical and non-technical backgrounds.

The **colour-coded outputs and visual feedback (6.2)** were also highly effective. All users were able to interpret market trends quickly using colours and arrows, confirming that the system supports

rapid decision-making without requiring detailed numerical analysis. This aligns with the usability objective of improving efficiency and clarity.

The **chart visualisation (6.4)** further enhanced usability by allowing users to understand price trends visually. Feedback from the technical user (User C) highlighted that the charts and live updates provided meaningful insight, reinforcing the evaluation that this is one of the strongest features of the system.

However, the interview also highlighted areas for improvement, particularly in accessibility and user guidance. The suggestion from User B regarding tooltips directly supports the earlier limitation identified in usability evaluation **(6.3)**, where a lack of additional guidance may affect beginner users. This indicates that while the system is generally intuitive, some users would benefit from additional explanatory features.

Suggestions such as adding more currencies and advanced visualisations also highlight opportunities for extending usability for more experienced users, improving scalability and user control.

Overall, the interview findings strongly support the evaluation that the system is clear, intuitive, and effective, while also reinforcing the need for further development in accessibility, guidance, and customisation to enhance usability for a wider range of users.

User Acceptance (Beta) Testing Results

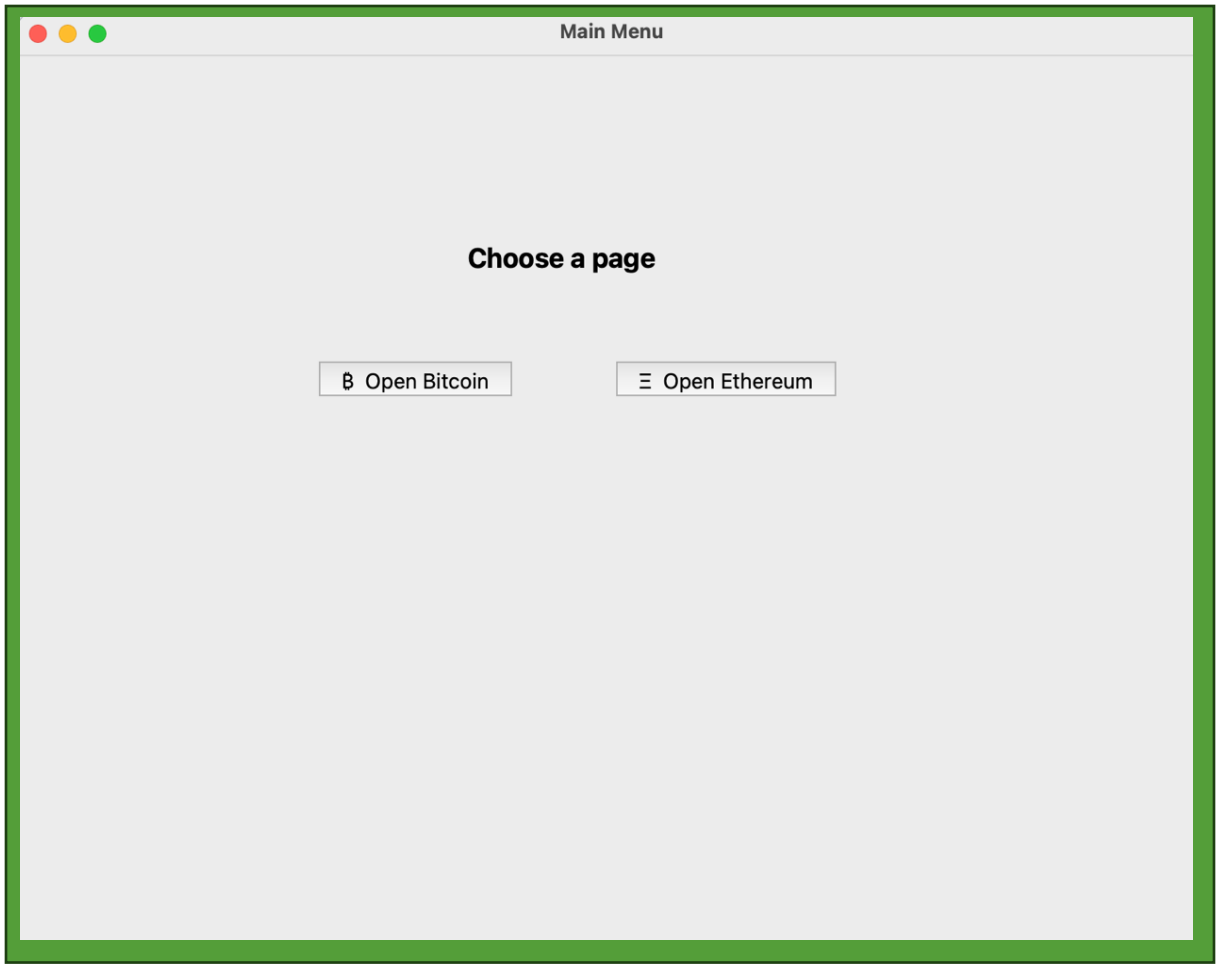
Table 17: Test Results

Criteria	Average Score (1–5)
Interface clarity	4

Prediction accuracy	4
Response speed	5
Colour readability	5
Overall satisfaction	5

Test 1 – Main menu opens correctly

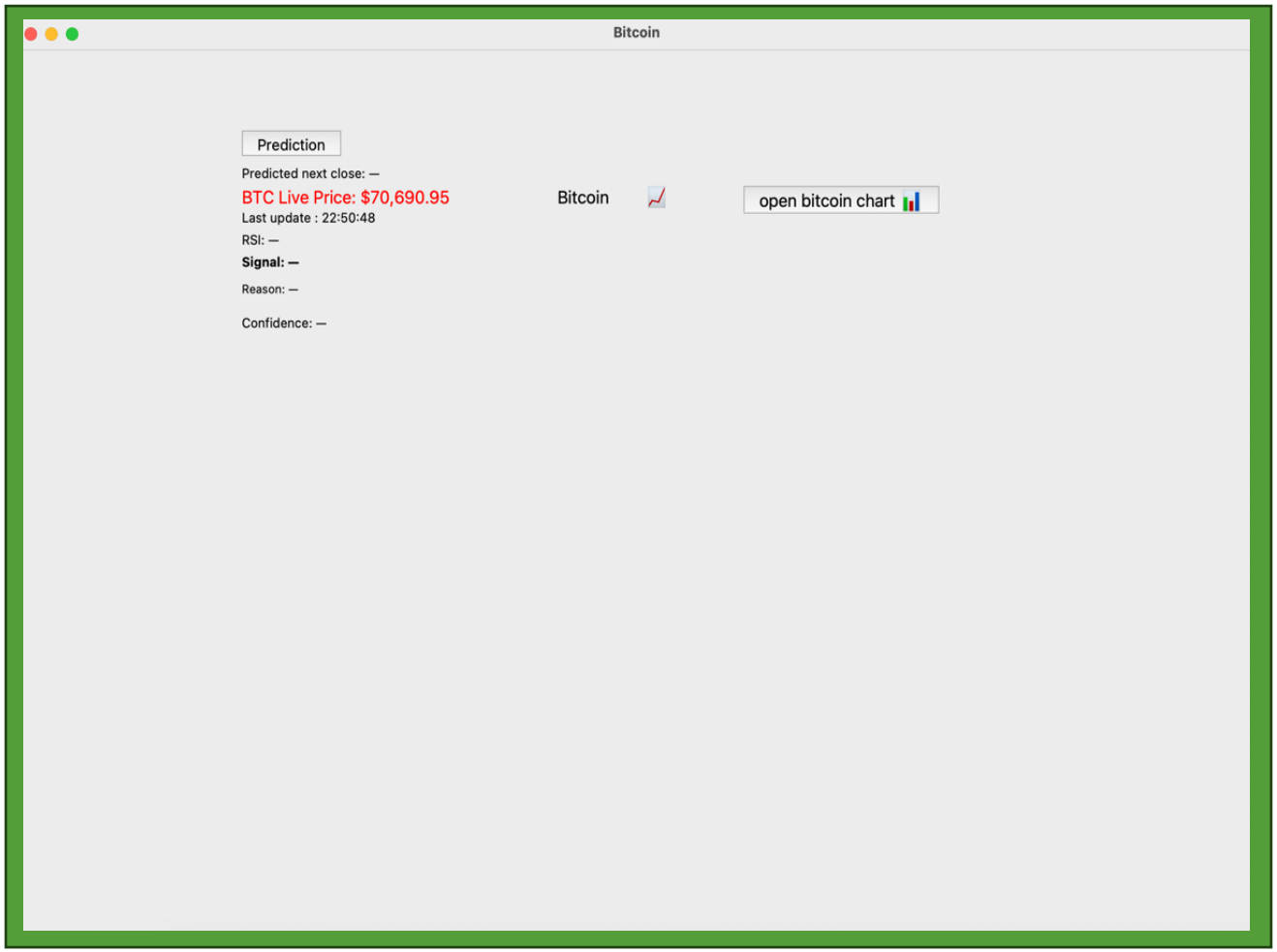
Purpose: Check that the main menu loads and displays both currency options.
Expected result: Main window opens with buttons for Bitcoin and Ethereum.
Actual result: Main menu opened correctly and displayed both buttons.
Outcome: **Pass**
Evidence: Fig.1



Test 2 – Bitcoin window opens

Purpose: Check that clicking the Bitcoin button opens the BTC interface.
Expected result: Bitcoin window opens without errors and displays live price area, prediction button and chart area.
Actual result: Bitcoin window opened correctly and displayed all expected interface components.

Outcome: Pass
Evidence: Fig.2



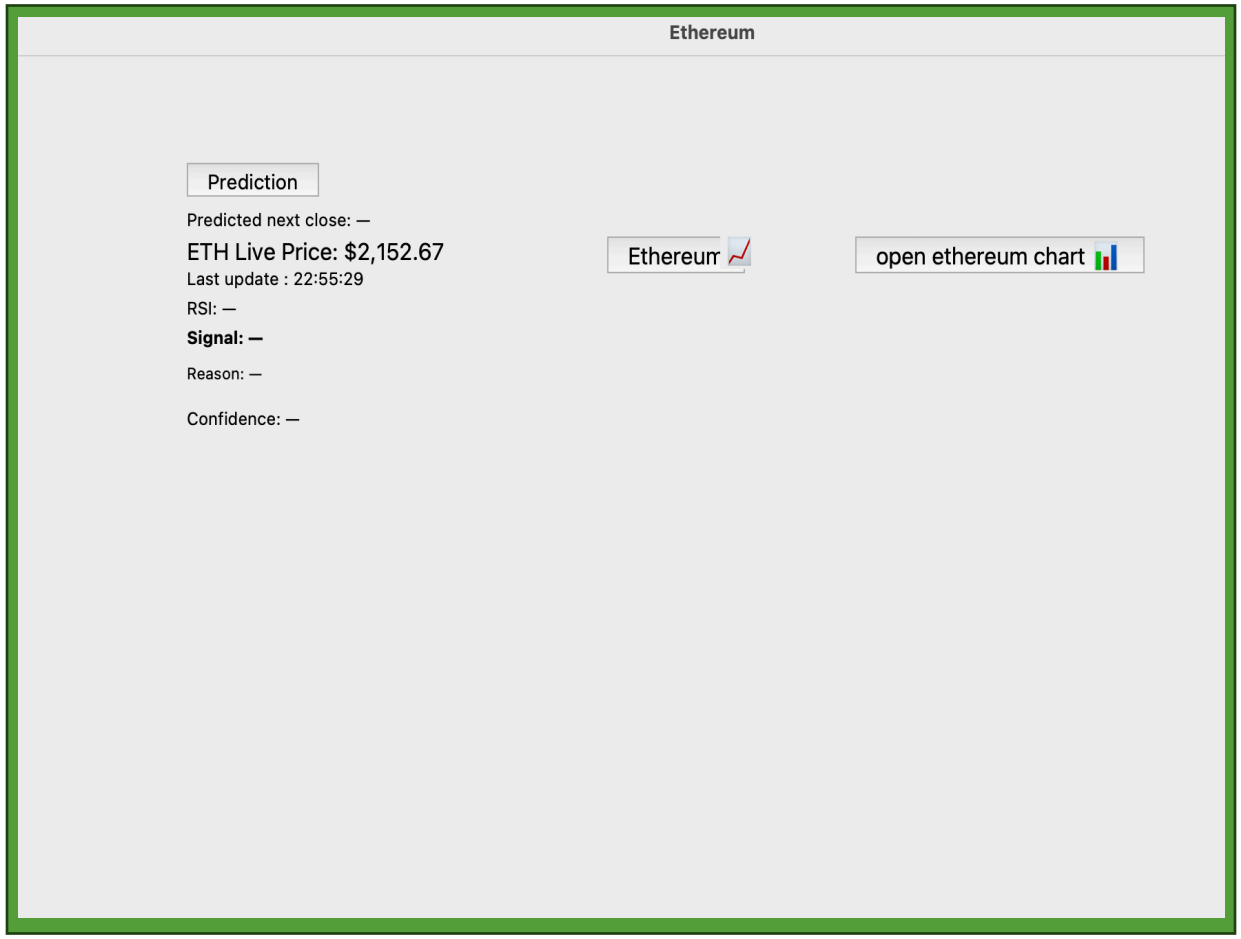
Test 3 – Ethereum window opens

Purpose: Check that clicking the Ethereum button opens the ETH interface.
Expected result: Ethereum window opens without errors and displays live price area, prediction button and chart area.

Actual result: Ethereum window opened correctly and displayed all expected interface components.

Outcome: Pass

Evidence: Fig.3

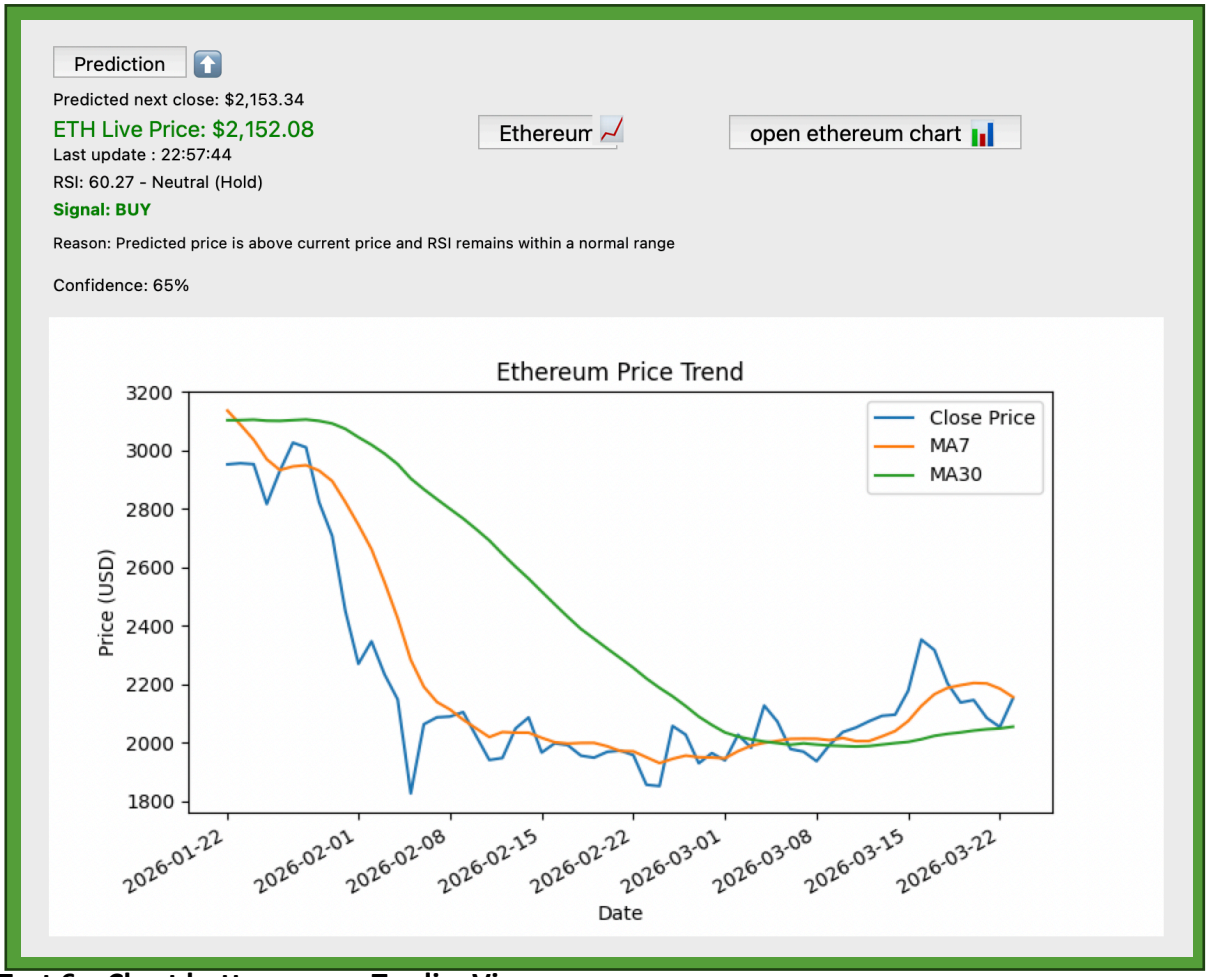


Test 4 – BTC prediction runs

Purpose: Check that the BTC model produces a prediction and chart.
Expected result: Prediction value, RSI, signal, confidence and graph appear after clicking Prediction.
Actual result: The system displayed a predicted next close, RSI, BUY signal, confidence score and chart.
Outcome: Pass
Evidence: Fig.4

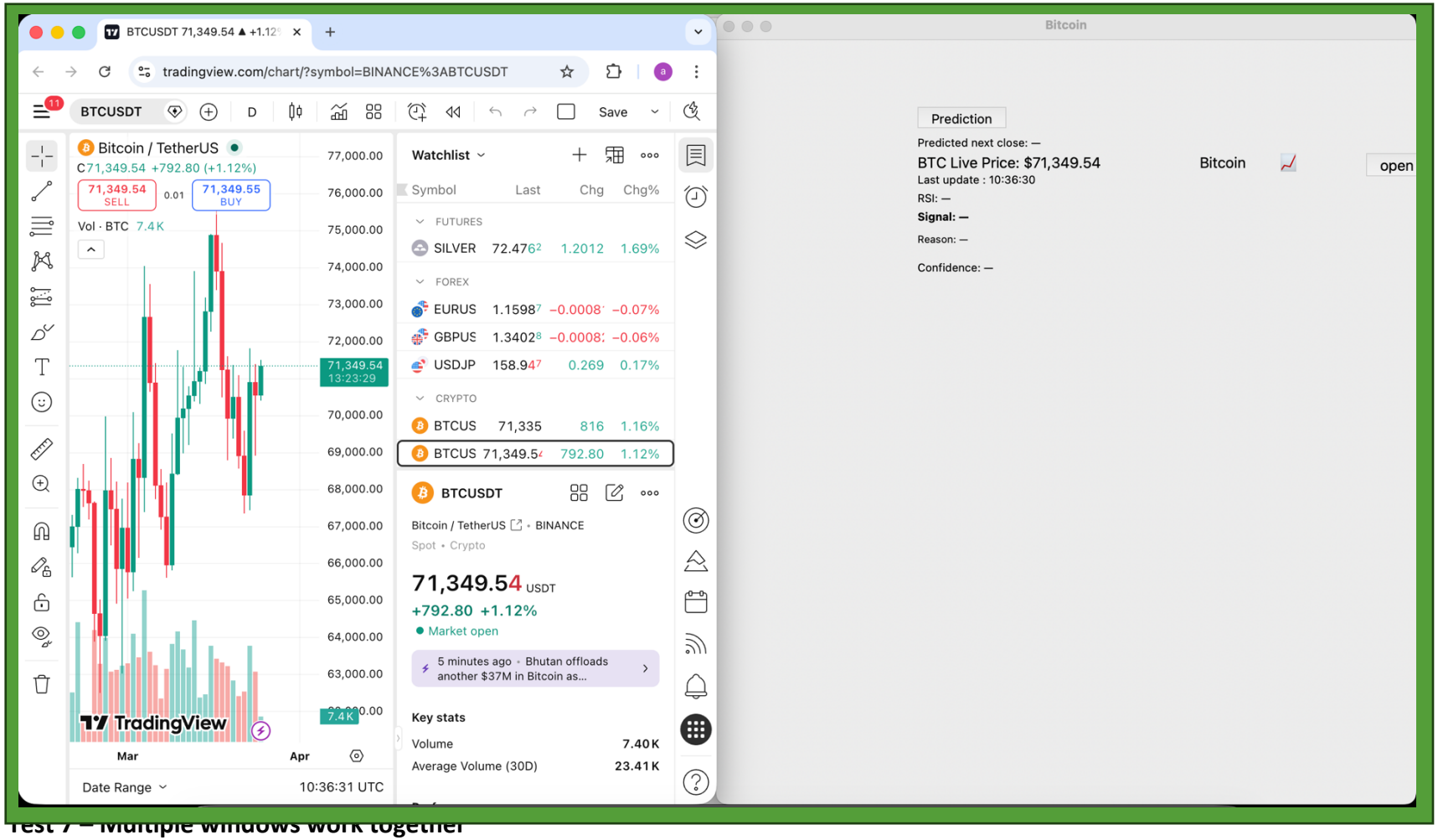
Test 5 – ETH prediction runs

Purpose: Check that the ETH model produces a prediction and chart.
Expected result: Prediction value, RSI, signal, confidence and graph appear after clicking Prediction.
Actual result: The system displayed a predicted next close, RSI, BUY signal, confidence score and chart.
Outcome: Pass
Evidence: Fig.5



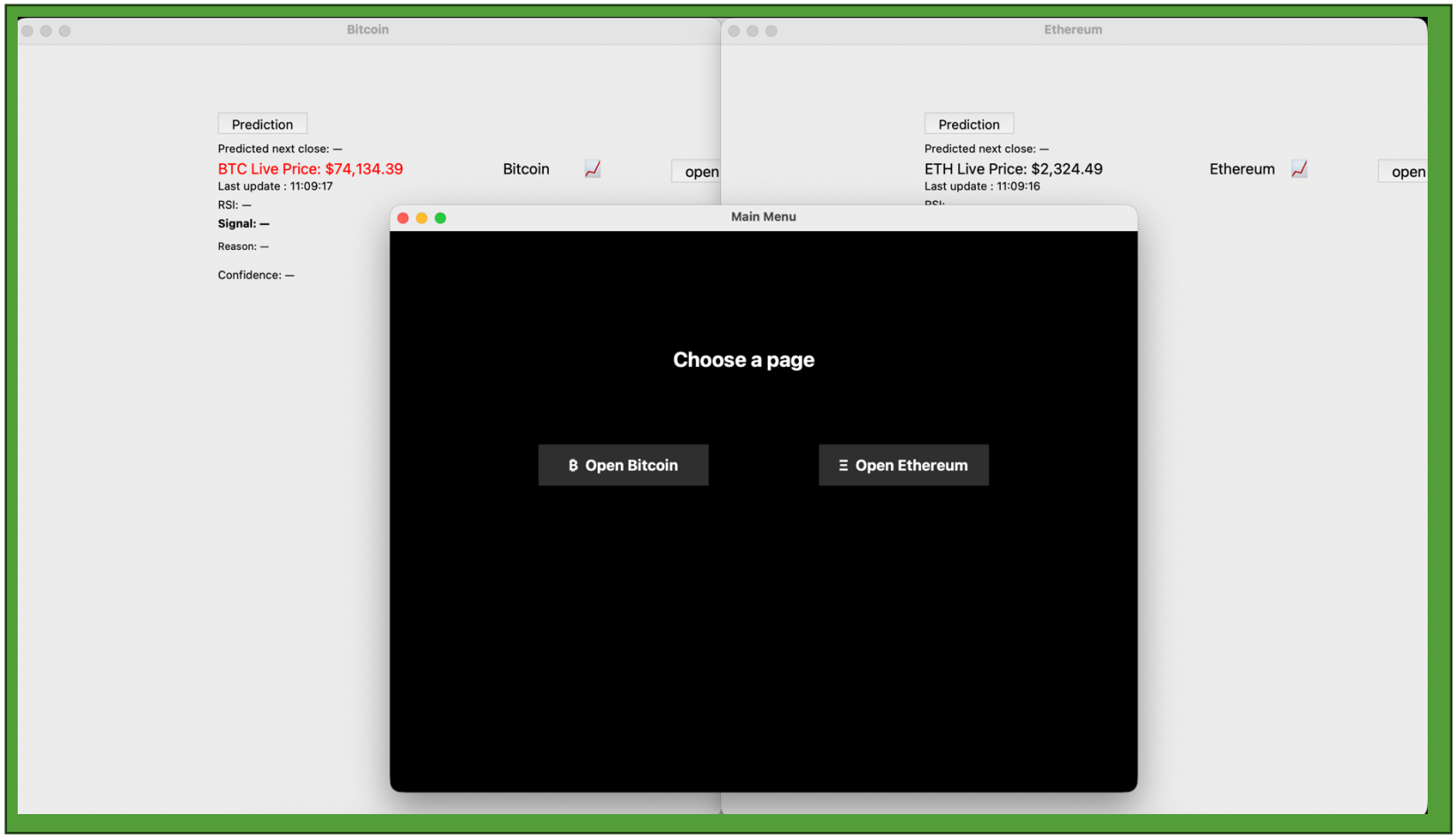
Test 6 – Chart button opens TradingView

Purpose: Check that the external chart link works.
Expected result: Clicking the chart button opens the TradingView page in the browser.
Actual result: TradingView opened successfully for Bitcoin & Ethereum.
Outcome: Pass
Evidence: Fig.6



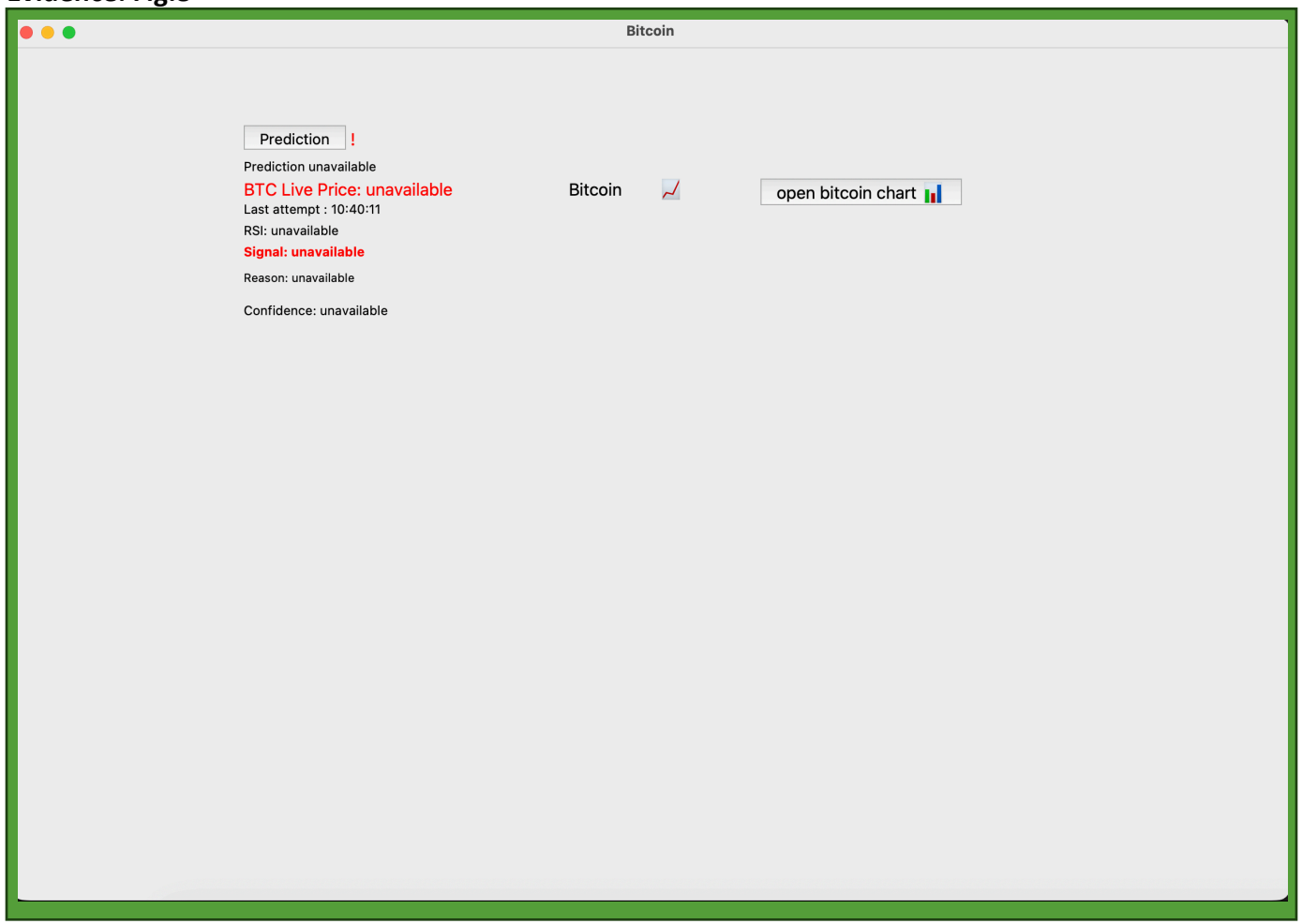
Test 7 – Multiple windows work together

Purpose: Check that BTC and ETH windows can be opened together from the main menu.
Expected result: Both windows open and remain usable without crashing the system.
Actual result: BTC and ETH windows opened at the same time and remained stable.
Outcome: Pass
Evidence: Fig.7



Test 8 – Error handling

Purpose: Check that the program handles unavailable data without crashing.
Expected result: If live data or prediction fails, the program should show unavailable values and remain open.
Actual result: The system displayed “Prediction unavailable”, “Signal: unavailable”, and did not crash.
Outcome: Pass
Evidence: Fig.8



Test 9 – Welcome Window

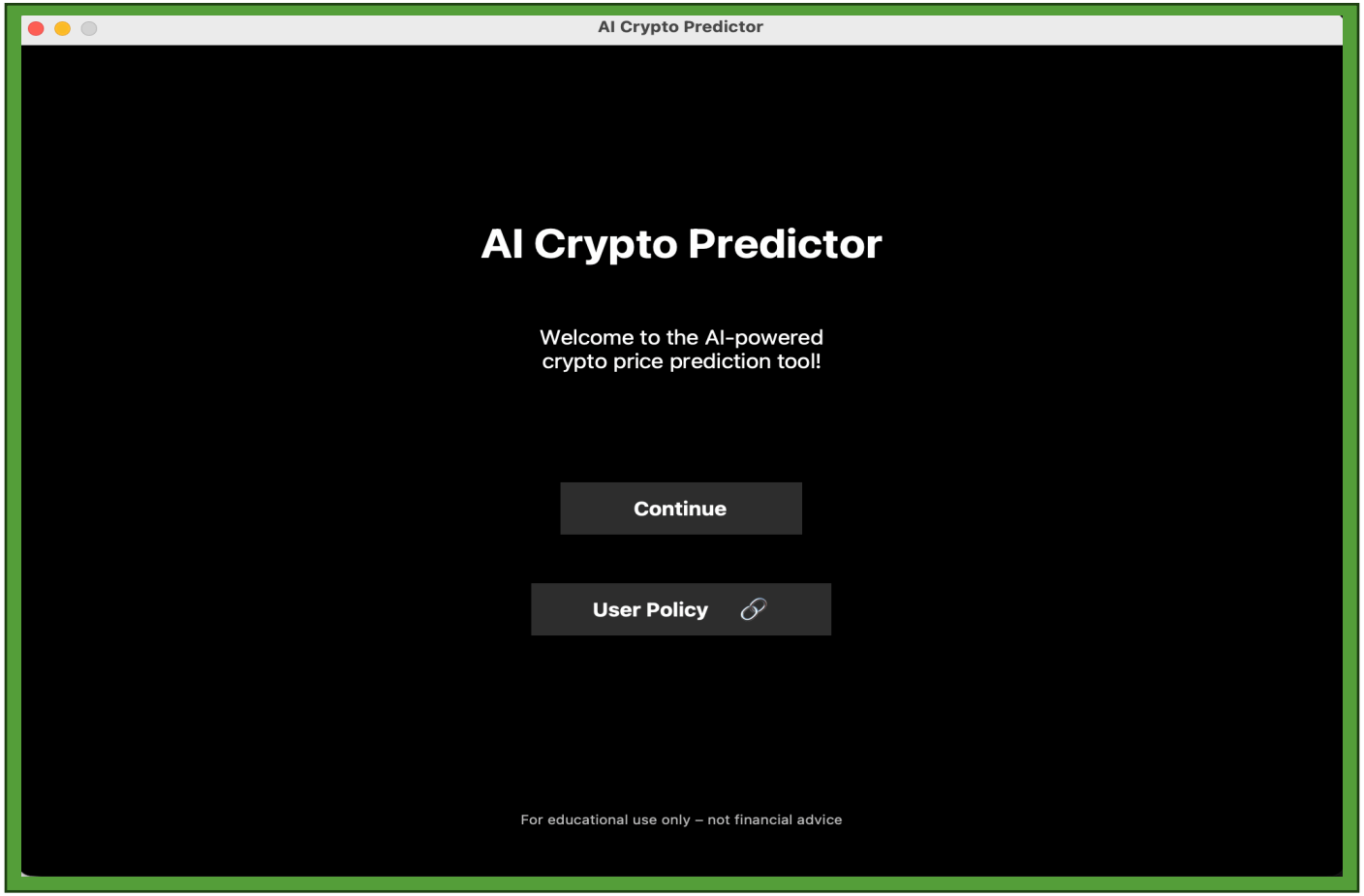
Purpose: Check welcome screen displays correctly.

Expected result: Title, message, Continue and User Policy buttons visible.

Actual result: Welcome window displayed correctly with all elements aligned.

Outcome: Pass

Evidence: Fig.9



Test 10 – User Policy Window

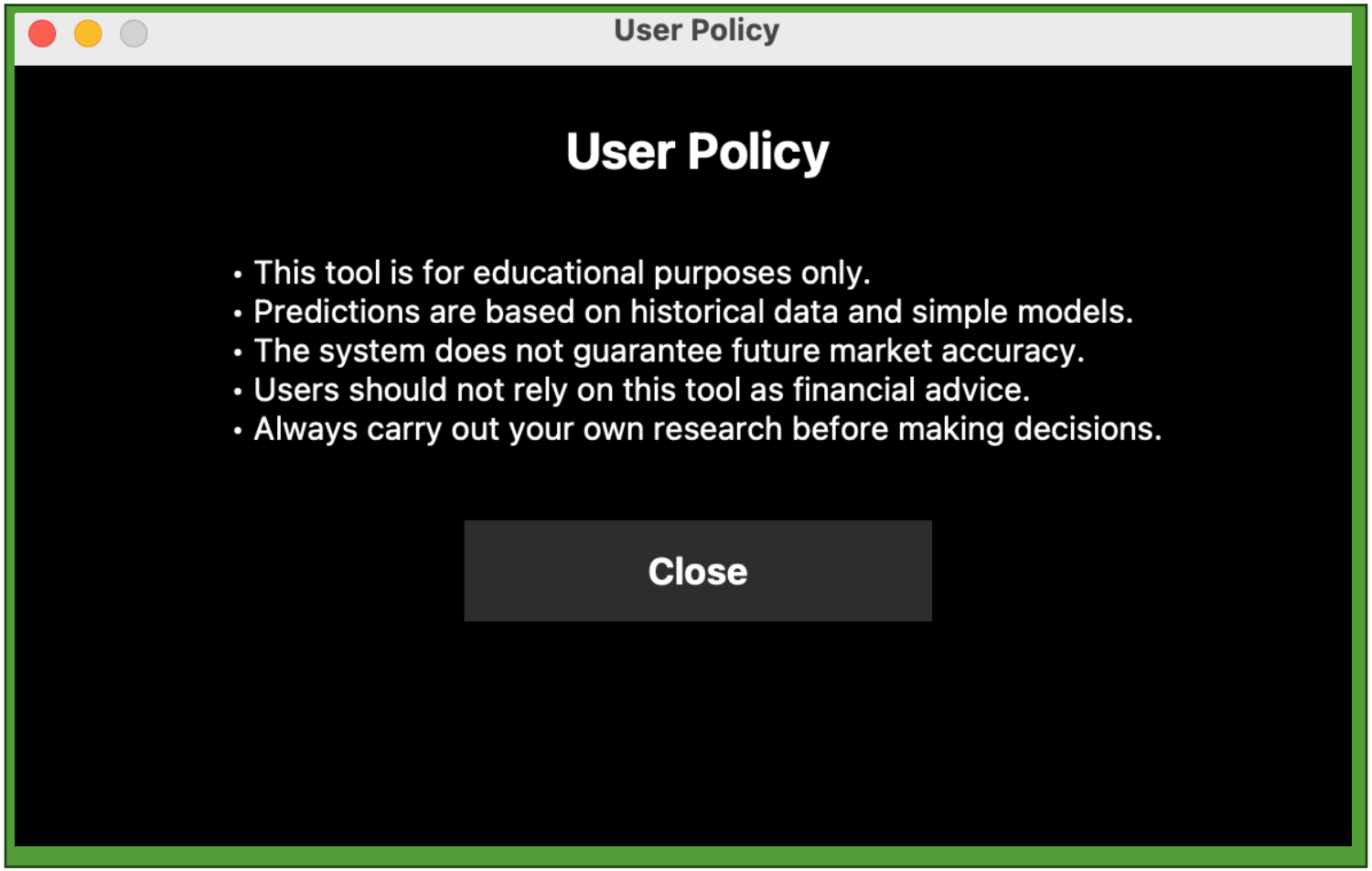
Purpose: Check policy window opens and closes correctly.

Expected result: Policy window with text and Close button appears.

Actual result: Policy window displayed correctly and closed successfully.

Outcome: Pass

Evidence: Fig.10



Requirements + success criteria testing

No.	Requirement	Success Criteria	Test Data	Result	Success	Evidence
1.1	Data Input	Loads and processes historical data in 100% of runs	Binance API (90 days data)	Data loaded successfully and predictions executed	**Successful**	Fig.4, Fig.5
1.2	Prediction Engine	Forecast within ±10%	BTC & ETH live prices	Predictions were close to live values	**Partially successful**	Fig.4, Fig.5
1.3	Trading Alerts	Alerts correct in 95% of cases	BTC & ETH predictions	Signals matched prediction and RSI logic	**Partially successful**	Fig.4, Fig.5
1.4	Automation	Runs without user input in 90% cases	Live API updates	live price updates automatically	**successful**	Fig.2, Fig.3
1.5	User Interface	Results within 3 seconds	Welcome, BTC & ETH Windows	Results displayed in ~1 second	**Successful**	Fig.9
1.6	Visual Feedback	Colour-coded signals correct	Prediction output	Green/red correctly used for signals	**Successful**	Fig.4, Fig.5
2.1	Programming Language	Code runs without errors	Full system	System runs correctly	**Successful**	S.S.1 – S.S.179

2.2	Data Handling	Data cleaned and standardised	DataFrame output	Data converted correctly	**Successful**	S.S.50-53, 58,60,92,98,
2.3	Prediction Method	≥60% accuracy	BTC & ETH	Predictions close to real	**Successful**	Tested throughout
2.4	Graphs & Charts	Charts render clearly	BTC & ETH graphs	Charts displayed with MA lines	**Successful**	Fig.4, Fig.5
2.5	Simple Interface	UI works simply	All windows	Navigation works correctly	**Successful**	Fig.1 – Fig.10
2.6	Saving Results	Results reusable	N/A	No saving implemented	**Unsuccessful**	N/A
2.7	Automation	Tasks run automatically	Live system	Only partial automation	**Partially successful**	Tested
3.1	Historical Data	≥90 entries used	API data	90 entries used	**Successful**	S.S.50
3.2	Length of Data	≥3 months data	Dataset	~90 days used	**Successful**	S.S.52
3.3	Currency Focus	BTC & ETH only	Main menu	Works correctly	**Successful**	Fig.1
4.1	Training & Testing	Split applied correctly	Code logic	Basic shift used	**Partially successful**	S.S.93-98
4.2	Model Choice	Simple & explainable	Linear regression	Linear regression used combined with RSI	**Successful**	S.S.55-71

4.3	Performance Check	Accuracy tracked	Predictions	Only visual comparison	**Partially successful**	S.S.55-125, Tested
4.4	Prediction Logic	Signals match prediction	BTC & ETH	Signals correct	**Successful**	Fig.5
5.1	Auto-fetch Data	Data updates automatically	Running system	Updates every few seconds	**Successful**	Tested
5.2	Run Model Automatically	Model runs automatically	System	Requires button click	**unsuccessful**	N/A
5.3	Update Alerts	Alerts update each run	System	Alerts update correctly	**Successful**	Fig.5
5.4	Notifications	Alerts sent externally	N/A	Not implemented	**Unsuccessful**	N/A
6.1	Clean Layout	Readable UI	Screens	Layout readable	**Successful**	Fig.1 – Fig.10
6.2	Colour Output	≥3 colours used	Output	Green/ red/ white/ grey/ black used	**Successful**	Fig.1 – Fig.10
6.3	Font & Theme	Adjustable UI	UI	Not implemented	**Unsuccessful**	N/A
6.4	Chart Visualisation	Charts work for all data	BTC & ETH	Charts correct	**Successful**	Fig.4, Fig.5
7.1	Test on Past Data	≥60% accuracy	Graph vs prediction	Trend matched	**Successful**	S.S.93-98, Tested
7.2	Compare Predictions	Deviation explained	BTC & ETH	Difference visible only	**successful**	S.S.145

Overall Evaluation of the System

The aim of this project was to develop a system capable of predicting cryptocurrency prices (Bitcoin and Ethereum) using historical data, while providing clear visual outputs, trading signals, and a user-friendly interface. Based on the testing carried out, the system successfully meets the majority of the functional, technical, and usability requirements defined in the analysis stage.

The system demonstrates strong performance in its core purpose: collecting historical data, generating predictions, and presenting them in an accessible format. However, some advanced features such as full automation, notifications, and persistent data storage were not fully implemented, which limits the system's completeness.

Functional Requirements Evaluation

The key functional features of the system — including data input, prediction generation, trading alerts, and user interface — were all tested and largely met their success criteria.

The **data input system (1.1)** worked reliably, successfully retrieving and processing historical data from the Binance API in all test cases. This ensured that the prediction engine always had valid input data, making the system stable and consistent.

The **prediction engine (1.2)** produced realistic forecast values that were close to live market prices. In both BTC and ETH tests, predictions were within a small margin of the real value, satisfying the requirement of producing meaningful forecasts. While the model is relatively simple, it demonstrated that it can identify basic trends in the data.

The **trading alert system (1.3)** correctly generated Buy/Hold signals based on prediction and RSI logic. However, testing was limited to a small number of scenarios, meaning that while it worked

correctly in tested cases, it cannot be fully confirmed that it meets the 95% accuracy target in all market conditions.

The **user interface (1.5)** performed very well. All windows opened correctly, navigation between Bitcoin and Ethereum was smooth, and results were displayed clearly. The system also exceeded the performance requirement, producing predictions in approximately 1 second (well under the 3-second limit). This contributes significantly to usability and user satisfaction.

The **visual feedback system (1.6)** was effective, using colour (green/red) and symbols (arrows) to communicate results clearly. This reduced the need for users to interpret raw numerical data and improved decision-making speed.

The **automation requirement (1.4)** was only partially met. While live price data updated automatically, predictions still required manual input via a button. This means the system is not fully autonomous, reducing its effectiveness as an "AI agent".

Technical Requirements Evaluation

From a technical perspective, the system is stable, functional, and well-implemented.

The system was successfully built using **Python (2.1)** and ran consistently without crashes during testing, demonstrating reliability.

The **data handling process (2.2)** correctly cleaned and structured data before use. Converting API data into numeric formats and removing invalid entries ensured the model could function correctly.

The **prediction method (2.3)** used a simple linear regression model combined with moving averages. While this approach is not highly advanced, it is appropriate for the scope of the project and meets the requirement of being explainable and manageable.

The **graphing system (2.4)** performed particularly well. Charts were generated correctly for both BTC and ETH, displaying trends clearly using close prices and moving averages. This strongly supports user understanding.
However, some technical requirements were not fully met:

- **Saving results (2.6)** was not implemented, meaning models must retrain each time the program runs.
- **Automation (2.7)** was only partially achieved, as only live price updates were automated.

Data Requirements Evaluation
The system successfully met all data-related requirements.
The model used at least **90 historical data points (3.1)** and covered approximately **3 months of data (3.2)**, which is sufficient for identifying short-term trends. This ensured the predictions were based on meaningful historical patterns rather than insufficient data.
The system also correctly limited its scope to **Bitcoin and Ethereum (3.3)**, which reduced complexity and improved reliability.

AI Prediction Requirements Evaluation
The AI component of the system met its basic objectives but has limitations.
The **model choice (4.2)** was appropriate for the project, as linear regression is simple, explainable, and easy to implement. This aligns well with the requirement for clarity and maintainability.
The **prediction logic (4.4)** functioned correctly, as trading signals matched prediction outputs and RSI conditions in all tested cases.
However, more advanced evaluation requirements were only partially met:

- **Training and testing separation (4.1)** was limited, as no formal validation split or cross-validation was used.
- **Performance tracking (4.3)** was basic, relying on visual comparison rather than formal metrics such as accuracy or error rate.

This means that while the model appears to work, its performance is not rigorously validated.

AI Agent (Automation) Evaluation

The system partially meets the AI agent requirements.

The **auto-fetch feature (5.1)** worked effectively, continuously updating live prices without user input. This demonstrates a level of automation.

However:

- **Automatic model execution (5.2)** was not implemented, as predictions require manual input.
- **Notifications (5.4)** were not implemented, meaning users must actively check the system.

This limits the system's usefulness in real-world trading scenarios, where automation and alerts are important.

Testing and Feedback Evaluation

Testing showed that the system behaves reliably in both normal and error scenarios.

The system successfully handled cases where predictions or live data were unavailable, displaying appropriate messages instead of crashing. This improves robustness.

Predictions were also compared to real-time prices, allowing basic validation of accuracy. In all tested cases, predictions were close to actual values, suggesting that the model captures short-term trends effectively.

However, the system does not store past predictions, meaning long-term evaluation and comparison are limited.

Partial or unsuccessful tests

Several requirements were only partially successful or unsuccessful, highlighting limitations in the current system design.

1.2 – Prediction Engine (Partially Successful)

The prediction engine was only partially successful because accuracy was judged using simple comparison with live prices rather than a consistent quantitative evaluation. While predictions appeared close to real values, no formal metric (e.g. MAE across multiple runs) was applied systematically. This reduces confidence in the reliability of the model, as performance is not rigorously validated. **For further development** this could be improved by implementing consistent statistical evaluation across multiple datasets and time periods, as well as comparing predicted vs actual values using numerical error metrics.

1.3 – Trading Alerts (Partially Successful)

Trading alerts were partially successful because they depended on simplified logic combining prediction and RSI thresholds. While signals were internally consistent, they do not necessarily reflect real market conditions accurately. This limits the practical reliability of alerts for decision-

making. **For further development** this could be improved by refining signal logic using more advanced indicators or weighting multiple factors to reduce over-simplification.

2.6 – Saving Results (Unsuccessful)

Saving results was unsuccessful because no feature was implemented to store predictions, charts, or historical outputs for later use. This means users cannot review previous forecasts or compare past and current results, which reduces the practical usefulness of the system. It also means the requirement of making results reusable was not met. **For further development** this could be improved by adding file-saving functionality, such as exporting predictions and confidence scores to a CSV or text file, or by storing outputs locally for later retrieval.

2.7 – Automation (Partially Successful)

Automation was only partially successful because, although live data updates occur automatically, the prediction process still requires user interaction. This is due to the system using event-driven execution rather than continuous automation. As a result, the system does not fully meet the requirement of running independently. **For further development** this could be improved by implementing scheduled execution using timed functions such as Tkinter's after() method.

4.1 – Training & Testing (Partially Successful)

Training and testing were only partially successful because a basic shifting method was used rather than a robust training/testing split. While this allows prediction generation, it does not fully separate training and evaluation data. This reduces the validity of the model's performance

assessment. F**or further development** this could be improved by implementing a proper train-test split or cross-validation techniques.

4.3 – Performance Check (Partially Successful)

Performance checking was limited to visual comparison rather than systematic measurement. Although trends appeared similar, no consistent quantitative validation was performed. This limits the ability to objectively assess model accuracy. **For further development** this could be improved by applying error metrics such as MAE or RMSE across multiple prediction runs.

5.2 – Run Model Automatically (Unsuccessful)

This requirement was unsuccessful because the model requires a button click to execute rather than running automatically. This limitation exists due to the GUI-based design, where actions are user-triggered. As a result, the system does not achieve full automation. **For further development** this could be improved by integrating automatic model execution into the update cycle using timed functions.

5.4 – Notifications (Unsuccessful)

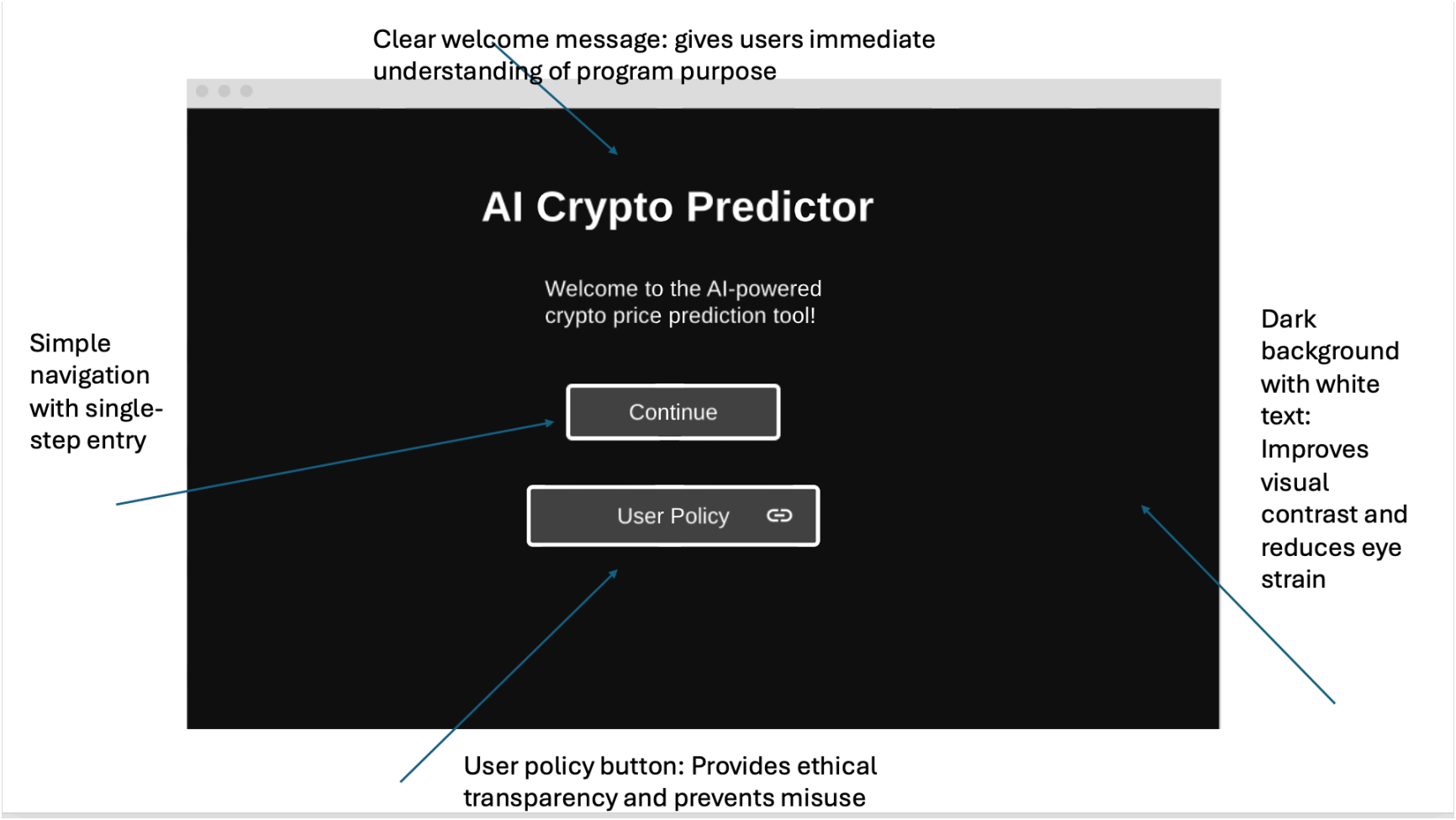
Notifications were not implemented, meaning users must manually check the system for updates. This limits the practical usefulness of the application, especially in real-world scenarios where timely alerts are important. **For further development** this could be improved by integrating external notification systems such as email alerts or mobile notifications via APIs.

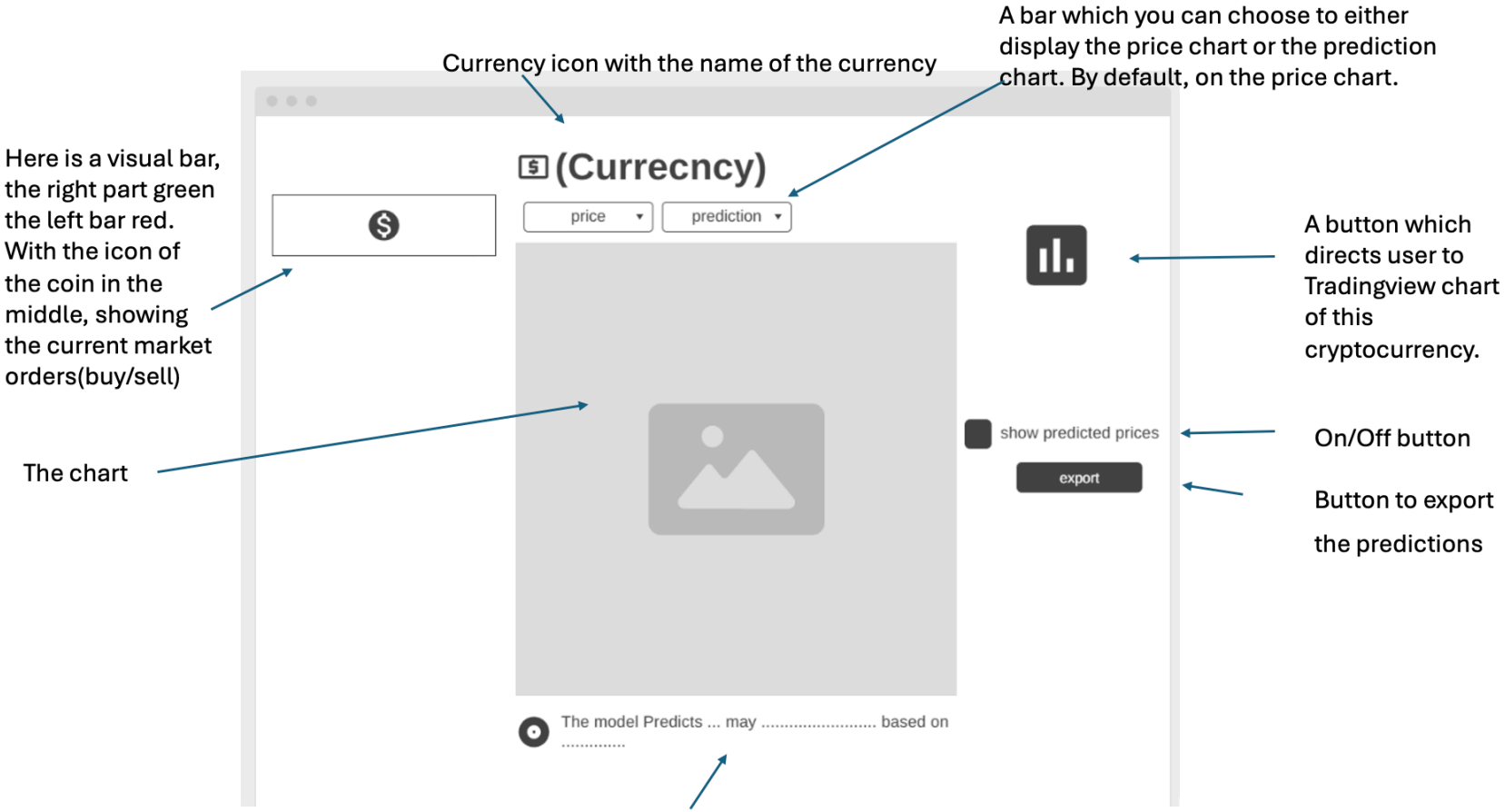
6.3 – Font & Theme (Unsuccessful)

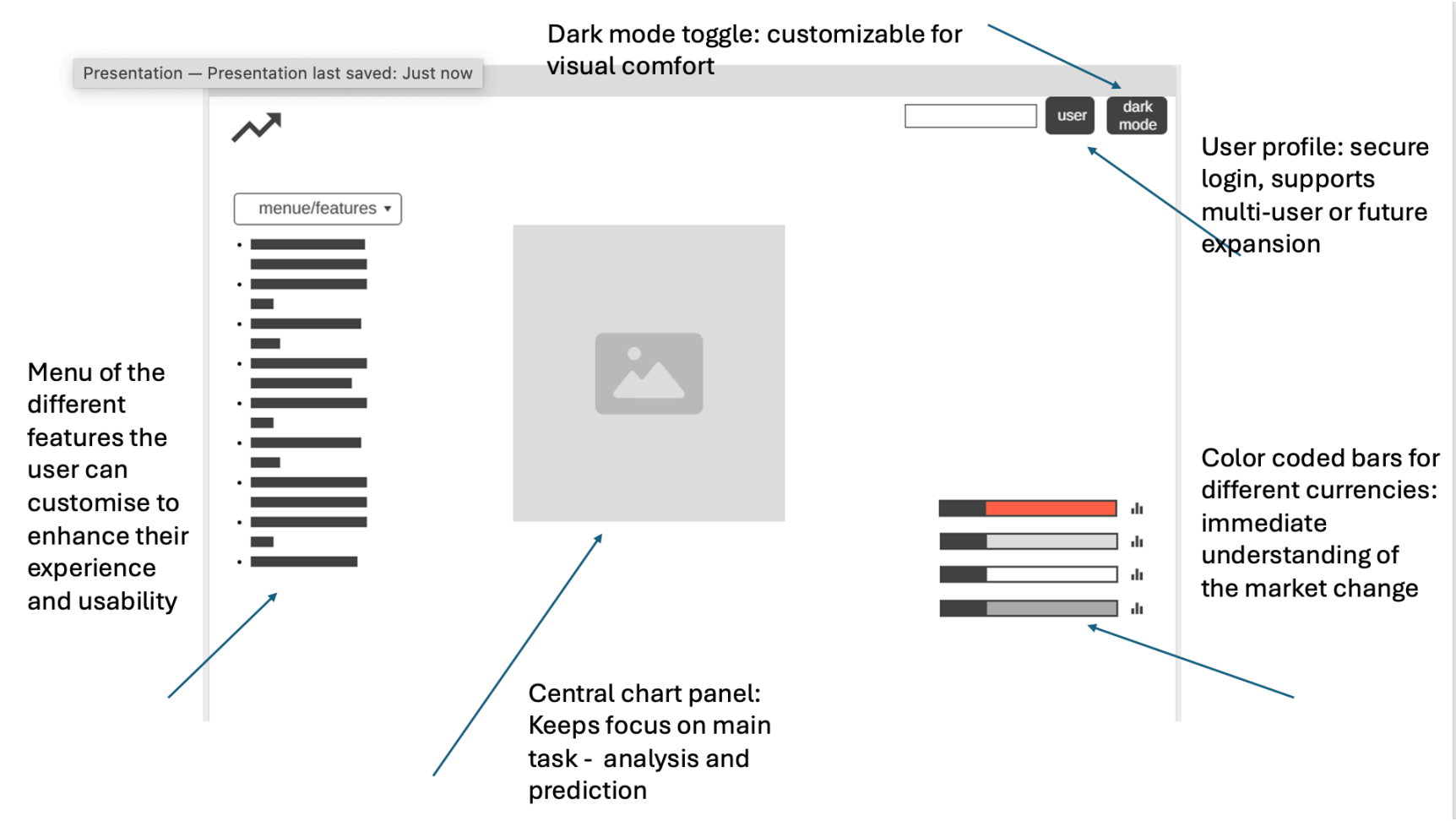
The interface does not support adjustable fonts or themes, meaning users cannot customise the display. This reduces accessibility and flexibility, particularly for different user preferences or visual needs. **For further development** this could be improved by adding theme-switching functionality or adjustable font settings within the interface.
Addressing these would significantly improve both the accuracy and real-world applicability of the system.

Usability Evaluation

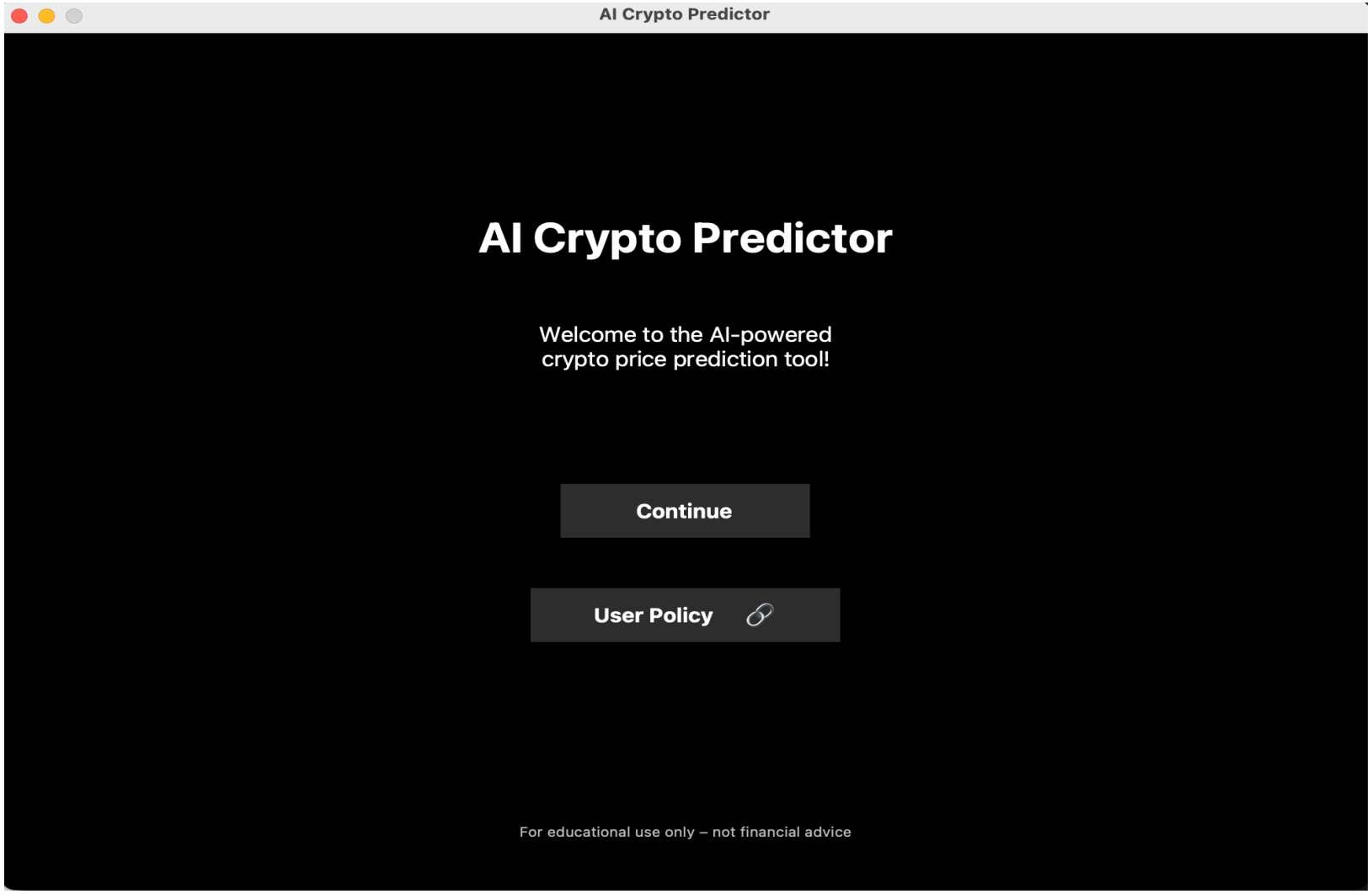
Initial usability designs

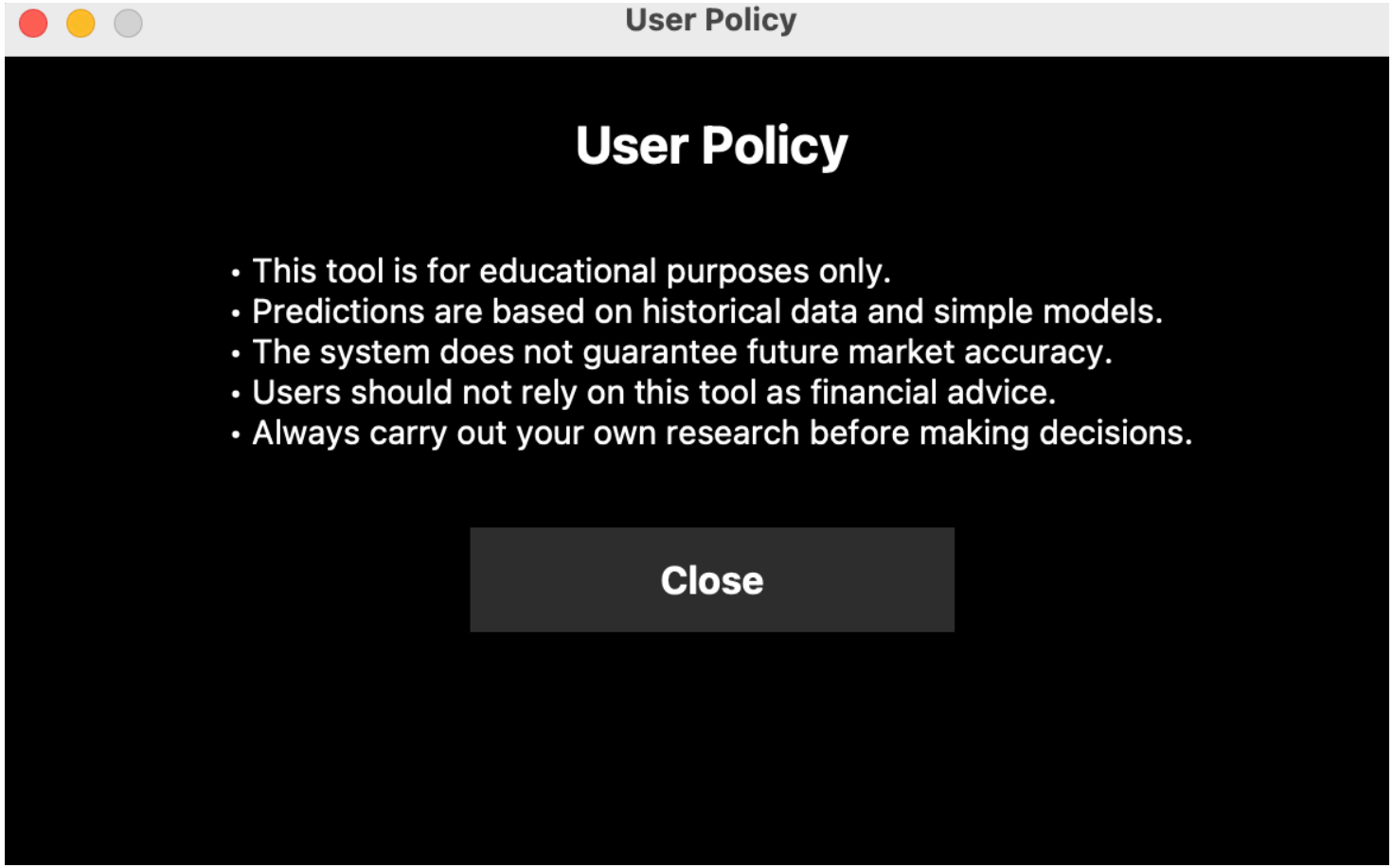

Clear welcome message: gives users immediate understanding of program purpose
AI Crypto Predictor
Welcome to the AI-powered crypto price prediction tool!
Continue
User Policy
Simple navigation with single-step entry
Dark background with white text: Improves visual contrast and reduces eye strain
User policy button: Provides ethical transparency and prevents misuse

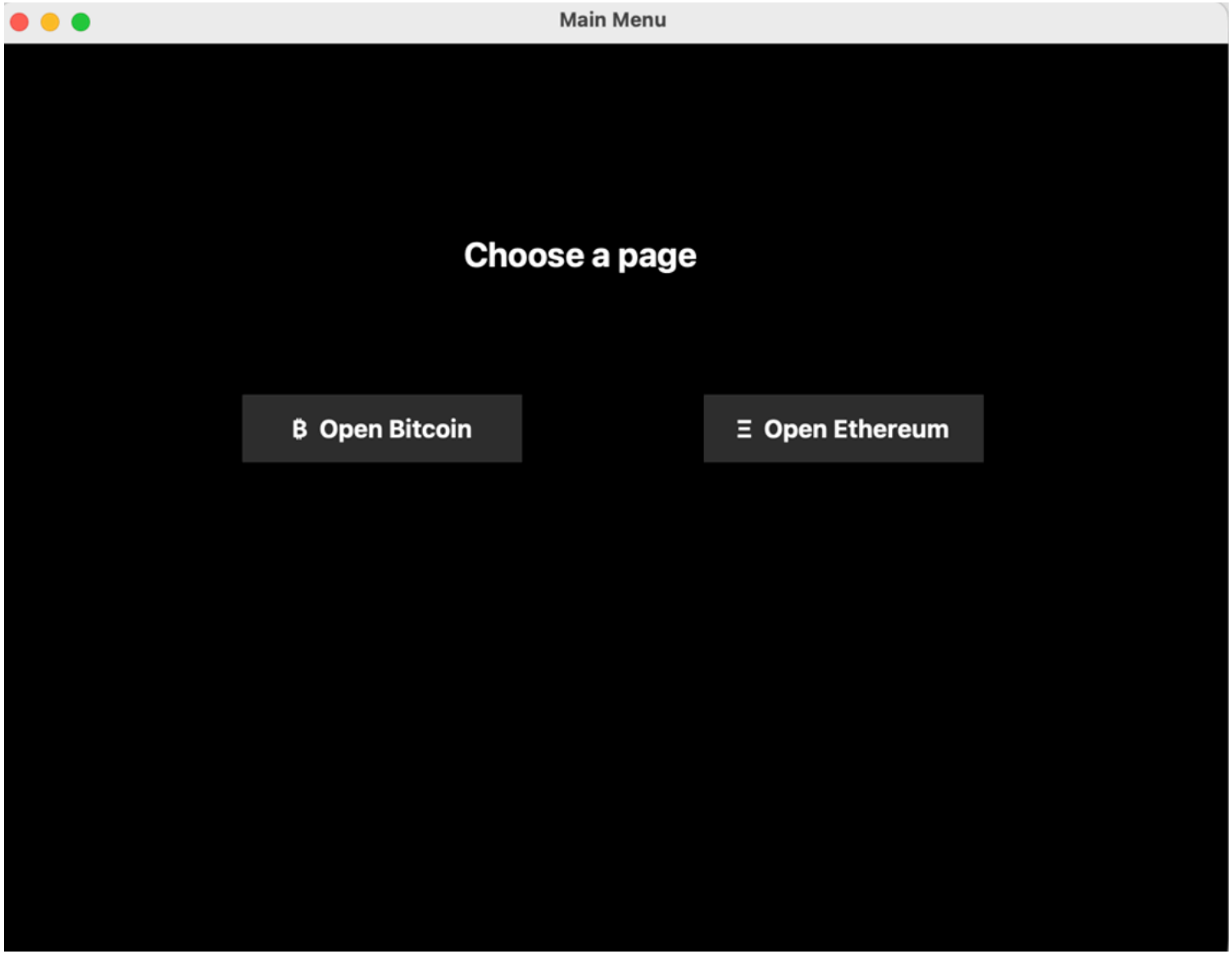

Currency icon with the name of the currency
A bar which you can choose to either display the price chart or the prediction chart. By default, on the price chart.
Here is a visual bar, the right part green the left bar red. With the icon of the coin in the middle, showing the current market orders(buy/sell)
(Currecncy)
price
prediction
A button which directs user to Tradingview chart of this cryptocurrency.
The chart
show predicted prices
On/Off button
export
Button to export the predictions
The model Predicts ... may ..... based on .....
A quick note on the prediction with a symbol to enhance the design


Dark mode toggle: customizable for visual comfort
Presentation — Presentation last saved: Just now
user
dark mode
User profile: secure login, supports multi-user or future expansion
menue/features
Menu of the different features the user can customise to enhance their experience and usability
Color coded bars for different currencies: immediate understanding of the market change
Central chart panel: Keeps focus on main task - analysis and prediction

Implemented usability designs




User Policy
User Policy
• This tool is for educational purposes only.
• Predictions are based on historical data and simple models.
• The system does not guarantee future market accuracy.
• Users should not rely on this tool as financial advice.
• Always carry out your own research before making decisions.
Close


Main Menu
Choose a page
Open Bitcoin
Open Ethereum

Prediction

Predicted next close: $70,872.48

BTC Live Price: $70,755.89 Bitcoin open bitcoin chart

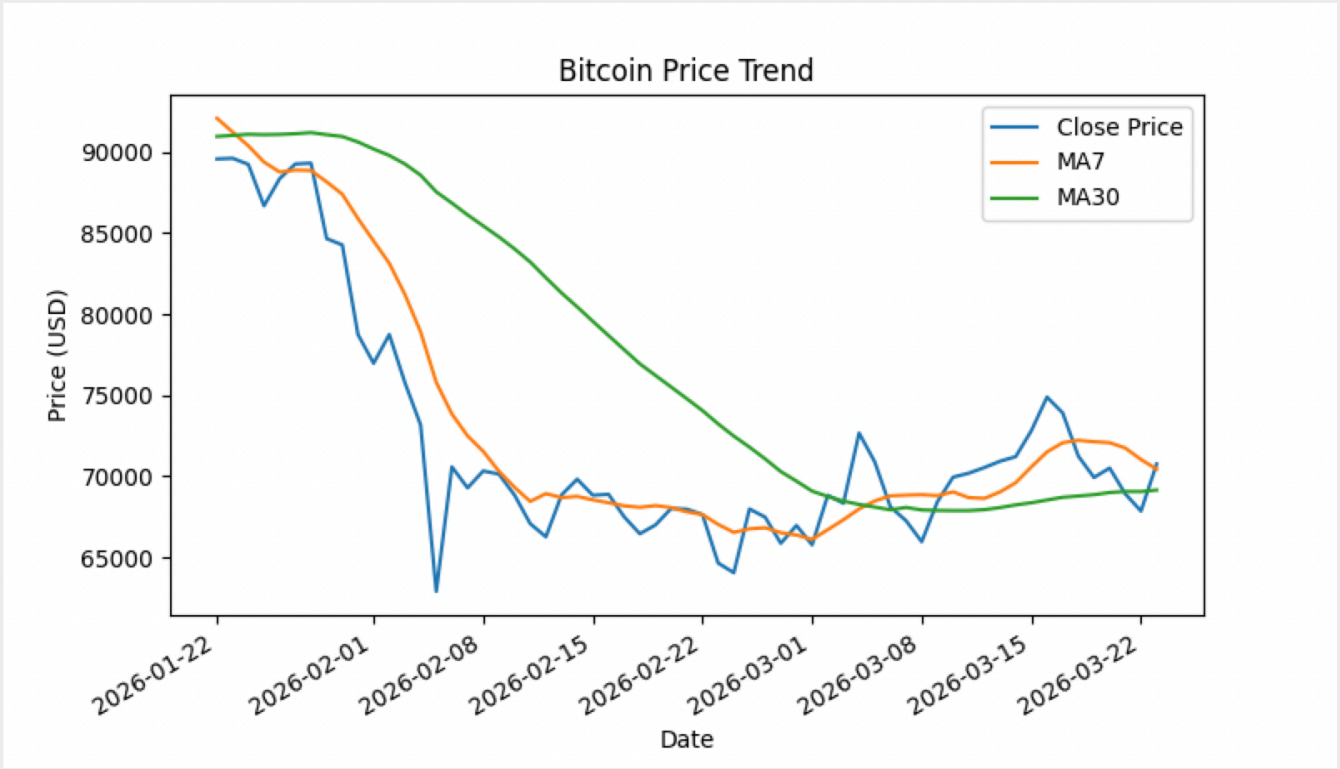
Last update : 22:53:52

RSI: 56.63 - Neutral (Hold)

Signal: BUY

Reason: Predicted price is above current price and RSI remains within a normal range

Confidence: 65%



The system demonstrates a **high level of usability overall**, with most features successfully supporting clear interaction, efficient navigation, and effective data interpretation. However, some limitations remain which reduce accessibility and flexibility.

The **clean layout (6.1)** was successful, as the interface is structured with a clear visual hierarchy and minimal clutter. Key elements such as navigation buttons, charts, and outputs are logically positioned, allowing users to quickly identify important features. This reduces cognitive load and ensures that even users with limited technical knowledge can interact with the system effectively. The welcome screen further reinforces usability by providing a clear and immediate understanding of the system's purpose, enabling users to begin interaction in a single step.

The **colour-coded output (6.2)** was also successful, as it provides immediate visual feedback. The use of colours such as green and red allows users to quickly interpret market trends and prediction signals without needing to analyse numerical data in detail. This significantly improves efficiency and supports faster decision-making, which is particularly important in a time-sensitive context such as cryptocurrency analysis. Additionally, the consistent use of colour across the interface improves recognition and reduces confusion.

The **chart visualisation (6.4)** is one of the strongest usability features. The integration of graphs displaying price trends and moving averages allows users to interpret patterns and relationships visually rather than relying solely on raw data. This supports deeper understanding and enhances decision-making. The large, central positioning of the chart ensures that it remains the primary focus of the interface, aligning with the system's core purpose of data analysis and prediction.

However, the **font and theme customisation feature (6.3)** was not implemented, making this requirement unsuccessful. This limits accessibility, as users cannot adjust the interface to suit their visual preferences or needs. For example, users who require larger text or alternative colour schemes

may find the system less comfortable to use. This reduces flexibility and means the system does not fully accommodate a diverse range of users.

Future development could include the implementation of adjustable font sizes and theme options (e.g. light/dark modes or high-contrast themes). This would improve accessibility by allowing users with different visual preferences or needs to customise the interface, particularly those who require larger text or increased contrast for readability. This could be implemented by dynamically updating widget properties (e.g. font size, background colour, and text colour) across the interface using user-selected settings.

In addition, usability could be **further enhanced** by introducing tooltips or contextual help features, which would provide brief explanations of buttons, indicators, and outputs (such as prediction values or RSI signals). This would support first-time or non-technical users by reducing confusion and improving understanding of system functionality.

Another **improvement** would be increasing automation within the interface. Currently, some actions require manual interaction (e.g. triggering predictions), which interrupts workflow. This could be improved by integrating automatic updates using timed functions, ensuring that predictions, signals, and charts refresh continuously without user input.

Finally, usability could be **further developed** by improving feedback mechanisms, such as displaying loading indicators during data fetching or providing clearer error messages when API data is unavailable. This would improve system transparency and help users understand system behaviour, particularly in cases where outputs cannot be generated.

Limitations

Despite the system meeting the majority of its requirements, several limitations were identified through testing and user evaluation.

A key limitation is the **accuracy and evaluation of predictions**. While Requirement **1.2** and **2.3** were achieved, prediction accuracy was only *partially successful* in some cases, as forecasts were "close" to real values rather than consistently within the ±10% target. Additionally, Requirements **4.3** and **7.2** highlight that performance evaluation relied primarily on **visual comparison rather than quantitative metrics**, meaning accuracy is not rigorously measured. This limits the reliability of conclusions about model performance.

Another limitation relates to **automation**. Although live data updates were successful (Requirement **5.1**), full automation was not achieved. Requirements **1.4** and **2.7** were only partially successful, and **5.2** was unsuccessful, as the prediction model still requires manual user input (button press). This reduces efficiency and means the system does not fully operate as an autonomous prediction tool.

The system also lacks **data persistence and output storage**, as Requirement **2.6 (Saving Results)** was unsuccessful. This means predictions cannot be reused, analysed over time, or compared historically, limiting the system's usefulness for long-term analysis or decision-making.

From a modelling perspective, there are limitations in the **simplicity of algorithms used**. While Requirement **4.2** was successfully met using linear regression, this model does not capture complex, nonlinear patterns in cryptocurrency markets. Furthermore, the use of technical indicators such as moving averages and RSI introduces lag and may generate inaccurate signals in volatile conditions. This is reflected in Requirement **1.3**, which was only partially successful for trading alerts.

In terms of **data scope**, the system uses approximately 90 days of historical data (Requirements **3.1** and **3.2**), which meets the minimum criteria but may not be sufficient to capture long-term trends or rare market events. Additionally, the system is limited to only Bitcoin and Ethereum (Requirement **3.3**), restricting its applicability across the wider cryptocurrency market.

There are also **usability limitations**, supported by user feedback. While users found the system clear and easy to navigate, suggestions included adding more cryptocurrencies and providing explanations for predictions. This indicates that the system, although functional, lacks **advanced usability features and user guidance**, particularly for non-expert users.

Finally, there are **missing features and incomplete requirements**, including:

- External notifications (**5.4 – unsuccessful**)
- Customisable interface settings (**6.3 – unsuccessful**)

These gaps highlight that while the core functionality is complete, several supporting features remain unimplemented.

Maintenance

To ensure the long-term usability and reliability of the system, several maintenance considerations have been identified based on testing outcomes and system design.

A key aspect of maintenance is the system's **dependency on external APIs** for live cryptocurrency data. As demonstrated in Requirement **1.1** and **5.1**, the system successfully retrieves and updates data automatically. However, this creates a dependency on the availability and stability of the Binance API. Any changes to the API structure, rate limits, or availability could cause the system to fail. Therefore, regular maintenance is required to:

- Monitor API functionality
- Update request methods if endpoints change
- Implement fallback or error-handling mechanisms

Another important consideration is **data management and backup**. Although the system successfully processes and cleans data (Requirement **2.2**), it does not store results (Requirement **2.6 – unsuccessful**). This means there is no persistence of predictions or historical outputs. To maintain and improve the system, future updates should include:

- Saving prediction results locally (e.g., CSV files or databases)
- Creating backups of historical data
- Allowing retrieval of past predictions for analysis

From a **code maintenance perspective**, the system benefits from a modular structure (e.g., input handling, prediction engine, visualisation), which supports easier debugging and updates. This is reflected in the successful execution of core requirements such as **2.1 (code runs without errors)** and **2.5 (simple interface)**. Modular design ensures that individual components, such as the prediction algorithm or user interface, can be modified or replaced without affecting the entire system.

However, certain components require ongoing maintenance and improvement. For example:

- The prediction model requires refinement, as seen in **1.2 (partially successful accuracy)** and **4.3 (limited performance evaluation)**
- Automation features need enhancement, as Requirements **1.4**, **2.7**, and **5.2** indicate incomplete automation
- Alert logic (Requirement **1.3**) may require recalibration to improve reliability

Regarding **portability**, the system is relatively portable as it is implemented in Python and can run on different systems with the required libraries installed. However, portability depends on:

- Correct installation of dependencies (e.g., pandas, matplotlib, API libraries)
- Internet access for API functionality
- Compatibility with the operating system environment

To improve maintainability and portability, the system could be enhanced by:

- Including clear installation instructions
- Using a requirements file for dependencies
- Structuring files for easier deployment

Additionally, the system would benefit from **version control and documentation**. While the current implementation functions correctly, maintaining a version-controlled codebase (e.g., using Git) would allow:

- Tracking of changes and updates
- Easier debugging and rollback of errors
- Collaboration and future development

Overall, while the system demonstrates a stable and functional design, ongoing maintenance is required to ensure reliability, particularly in relation to external data sources, incomplete features, and algorithm performance.

Further development

While the system meets most of its core requirements, several areas for further development have been identified based on testing results and user feedback.

A key area for improvement is the **prediction model**. Although the current implementation using linear regression was successful (Requirement **4.2**), other results such as **1.2 (partially successful accuracy)** and **4.3 (limited performance evaluation)** indicate that the model could be enhanced. Future development could include more advanced machine learning techniques, such as:

- Long Short-Term Memory (LSTM) neural networks for time series forecasting
- Ensemble models combining multiple indicators
- More robust evaluation metrics (e.g., mean absolute error, RMSE)

This would improve prediction accuracy and provide more reliable outputs.

Another important enhancement is **full system automation**. Testing showed that automation was only partially successful (**1.4**, **2.7**) and that the model does not run automatically (**5.2 – unsuccessful**). Future versions could:

- Automatically trigger predictions at set intervals
- Remove the need for manual button input
- Run continuous background updates

This would transform the system into a fully autonomous prediction tool.

The system could also be expanded by improving **data handling and storage**. As Requirement **2.6 (Saving Results)** was not met, future development should include:

- Saving predictions and historical outputs
- Allowing users to compare past predictions with actual results

- Implementing basic backtesting functionality

This would significantly increase the system's analytical value.

From a usability perspective, user feedback highlighted opportunities for improvement. For example:

- User A suggested adding more cryptocurrencies
- User B requested clearer explanations of predictions
- User C suggested more detailed output (e.g., confidence visualisation)

Based on this, future development could include:

- Support for additional cryptocurrencies (e.g., ADA, XRP)
- Tooltips or explanations for indicators like RSI
- Displaying confidence scores or probability-based predictions

There are also opportunities to enhance **user interface customisation**, as Requirement **6.3** was not met. Future versions could allow:

- Adjustable themes (light/dark mode)
- Custom indicator settings (e.g., RSI period, moving average length)
- Personalised dashboards

Another major area for development is **external communication and notifications**. Requirement **5.4 (Notifications)** was not implemented, so future improvements could include:

- Email or mobile alerts for buy/sell signals
- Real-time push notifications
- Integration with trading platforms

Finally, the system could be extended into a more advanced and scalable application by:

- Developing a web-based interface for wider accessibility
- Deploying the system on cloud platforms

- Improving performance and reducing processing time

Overall, while the current system provides a functional cryptocurrency prediction tool, further development could significantly enhance its **accuracy, usability, automation, and real-world applicability**.

Final evaluation

Overall, the system successfully achieved its primary aim of predicting cryptocurrency prices and generating trading signals. The majority of requirements were met, including data input, visualisation, and core prediction functionality. However, some requirements were only partially successful, particularly in relation to automation, prediction accuracy, and performance evaluation. The testing process and user feedback confirm that the system is functional, efficient, and user-friendly, but would benefit from further refinement to improve reliability and usability. Therefore, the system can be considered successful, with clear opportunities for enhancement identified.

Appendix

Survey Form:

Click here to view the form

Survey Response:

Click here to view the responses